

app\api_client.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      api_client.py
4  @brief     Client API pour les services oblosolutions.ch.
5  @details   Ce module fournit un client HTTP pour interroger les APIs
6             td25_param et td25_forecast. Il gère les timeouts, les retries
7             automatiques et les erreurs de réseau. Inclut des fonctions
8             utilitaires pour récupérer les données de simulation et envoyer
9             les mesures de température.
10 @author    Léo Mendes
11 @project   2510_RegulationsChauffage
12 @Mandant   Oblo_solution
13 @date      2025-09-18
14 @version   1.0.0
15 @copyright Copyright (c) 2025
16 """
17
18 from __future__ import annotations
19
20 # Client HTTP pour les requêtes REST
21 import requests
22 # Fonctions temporelles (délais, sleep)
23 import time
24 # Sérialisation/désérialisation JSON
25 import json
26 # Types pour l'annotation de code
27 from typing import Dict, Optional, Any
28
29 # -----
30 # Constantes du module
31 # -----
32
33 # Délai d'attente pour les requêtes HTTP [s]
34 REQUEST_TIMEOUT = 10
35 # Nombre maximum de tentatives en cas d'échec
36 MAX_RETRIES = 3
37 # Délai entre les tentatives [s]
38 RETRY_DELAY = 1
39
40 # -----
41 # Classes d'exception
42 # -----
43
44 class ApiError(Exception):
45     """
46     @brief   Exception pour les erreurs d'API.
47     @details Exception personnalisée levée en cas d'erreur lors des
48             communications avec les APIs oblosolutions.ch.
49     """
50     pass
51
52 # -----
53 # Classes principales
54 # -----
55
56 class ApiClient:
57     """
58     @brief   Client pour interroger les APIs oblosolutions.ch.
59     @details Fournit des méthodes pour récupérer les paramètres de simulation,
60             les prévisions météo et envoyer les mesures de température.
61             Gère automatiquement les retries et les timeouts.
62     """
63
64     # Pas de préfixe, endpoints à la racine
65     API_PATH_PREFIX = ""
66
67     def __init__(self, mac_address: str):
68         """
69         @brief   Initialise le client API.
70         @details Configure l'adresse MAC du dispositif et l'URL de base
71                 pour les communications avec les services oblosolutions.ch.
72
73         @param   mac_address Adresse MAC du dispositif (ex: "0030DEABCDE").
74         """
75         # Stockage de l'adresse MAC du dispositif
76         self.mac_address = mac_address
77         # URL de base pour tous les appels API
78         self.base_url = "https://dev.oblosolutions.ch"
79
80     def get_parameters(self) -> Dict[str, Any]:
81         """
82         @brief   Récupère les paramètres depuis l'API td25_param.
83         @details Effectue une requête GET vers l'endpoint td25_param avec
84                 l'adresse MAC comme paramètre. Parse la réponse JSON et

```

```

85         extrait les champs probe_type, n, k_m et temperature.
86         Inclut un système de retry automatique en cas d'échec.
87
88     @return Dictionnaire contenant probe_type, n, k_m, temperature.
89
90     @exception ApiError En cas d'erreur de récupération des données.
91     """
92     # Construction de l'URL pour l'endpoint des paramètres
93     url = f"{self.base_url}/td25_param"
94     # Paramètres de la requête GET
95     params = {"mac_address": self.mac_address}
96     # Affichage de la requête pour le debug
97     print(f"[API] GET {url} params={params}")
98
99     # Boucle de retry en cas d'échec
100    for attempt in range(MAX_RETRIES):
101        try:
102            # Exécution de la requête GET avec timeout
103            response = requests.get(url, params=params, timeout=REQUEST_TIMEOUT)
104
105            # Vérification du code de statut HTTP
106            if response.status_code != 200:
107                # Lever une exception en cas d'erreur HTTP
108                raise ApiError(f"Erreur HTTP {response.status_code}: {response.text}")
109
110            # Parsing de la réponse JSON
111            data = response.json()
112            # Vérification de la présence du champ 'param'
113            if "param" not in data or not data["param"]:
114                raise ApiError("Champ 'param' manquant ou vide")
115            # Extraction du premier élément des paramètres
116            entry = data["param"][0]
117            # Extraction des champs individuels
118            probe_type = entry.get("probe_type")
119            n = entry.get("n")
120            k_m = entry.get("k_m")
121            temperature = entry.get("temperature")
122            # Vérification que tous les champs requis sont présents
123            if None in (probe_type, n, k_m, temperature):
124                raise ApiError("Un ou plusieurs champs requis sont manquants dans la réponse API")
125            # Affichage des paramètres récupérés
126            print(f"[API] Paramètres récupérés: {entry}")
127            # Retour du dictionnaire formaté
128            return {
129                "probe_type": probe_type,
130                "n": n,
131                "k_m": k_m,
132                "temperature": temperature
133            }
134
135        except requests.exceptions.RequestException as e:
136            # Gestion des erreurs de réseau
137            print(f"[API] Tentative {attempt + 1}/{MAX_RETRIES} échouée: {e}")
138            # Attente avant retry si ce n'est pas la dernière tentative
139            if attempt < MAX_RETRIES - 1:
140                time.sleep(RETRY_DELAY)
141            else:
142                # Lever une exception après épuisement des tentatives
143                raise ApiError(f"Impossible de récupérer les paramètres après {MAX_RETRIES} tentatives: {e}")
144        except json.JSONDecodeError as e:
145            # Gestion des erreurs de parsing JSON
146            raise ApiError(f"Réponse JSON invalide: {e}")
147
148    def get_forecast(self) -> Dict[str, Any]:
149        """
150        @brief Récupère la température prévue depuis l'API td25_forecast.
151        @details Effectue une requête GET vers l'endpoint td25_forecast avec
152                 l'adresse MAC comme paramètre. Parse la réponse JSON et
153                 extrait la température de prévision météorologique.
154                 Inclut un système de retry automatique en cas d'échec.
155
156        @return Dictionnaire contenant la temperature prévue.
157
158        @exception ApiError En cas d'erreur de récupération des données.
159        """
160        # Construction de l'URL pour l'endpoint des prévisions
161        url = f"{self.base_url}/td25_forecast"
162        # Paramètres de la requête GET
163        params = {"mac_address": self.mac_address}
164        # Affichage de la requête pour le debug
165        print(f"[API] GET {url} params={params}")
166
167        # Boucle de retry en cas d'échec
168        for attempt in range(MAX_RETRIES):
169            try:
170                # Exécution de la requête GET avec timeout
171                response = requests.get(url, params=params, timeout=REQUEST_TIMEOUT)

```

```

172
173     # Vérification du code de statut HTTP
174     if response.status_code != 200:
175         # Lever une exception en cas d'erreur HTTP
176         raise ApiError(f"Erreur HTTP {response.status_code}: {response.text}")
177
178     # Parsing de la réponse JSON
179     data = response.json()
180     # Vérification de la présence du champ 'forecast'
181     if "forecast" not in data or not data["forecast"]:
182         raise ApiError("Champ 'forecast' manquant ou vide")
183     # Extraction du premier élément des prévisions
184     forecast = data["forecast"][0]
185     # Extraction de la température de prévision
186     temperature_forecast = forecast["temperature"]
187     # Affichage des prévisions récupérées
188     print(f"[API] Prévisions récupérées: {forecast}")
189     # Retour du dictionnaire avec la température
190     return {"temperature": temperature_forecast}
191
192 except requests.exceptions.RequestException as e:
193     # Gestion des erreurs de réseau
194     print(f"[API] Tentative {attempt + 1}/{MAX_RETRIES} échouée: {e}")
195     # Attente avant retry si ce n'est pas la dernière tentative
196     if attempt < MAX_RETRIES - 1:
197         time.sleep(RETRY_DELAY)
198     else:
199         # Lever une exception après épuisement des tentatives
200         raise ApiError(f"Impossible de récupérer les prévisions après {MAX_RETRIES} tentatives: {e}")
201 except json.JSONDecodeError as e:
202     # Gestion des erreurs de parsing JSON
203     raise ApiError(f"Réponse JSON invalide: {e}")
204
205 def post_measured_temperature(self, temperature: float, channel: int = None) -> bool:
206     """
207     @brief  Envoie la température mesurée vers l'API td25_param.
208     @details Effectue une requête POST vers l'endpoint td25_param avec
209             la température mesurée et optionnellement le numéro de canal.
210             Tous les paramètres sont envoyés dans l'URL comme paramètres
211             de requête. Inclut un système de retry automatique.
212
213     @param temperature Température mesurée en °C.
214     @param channel      Canal de mesure (optionnel).
215
216     @return True si succès, False sinon.
217     """
218     # Construction de l'URL pour l'endpoint des paramètres
219     url = f"{self.base_url}/td25_param"
220
221     # Préparation des paramètres URL (comme dans l'exemple fourni)
222     # Paramètres de base pour l'envoi
223     params = {
224         "mac_address": self.mac_address,
225         "measured_temp": temperature
226     }
227
228     # Ajout du canal si spécifié
229     if channel is not None:
230         params["channel"] = channel
231
232     # Affichage de la requête pour le debug
233     print(f"[API] POST {url} params={params}")
234
235     # Boucle de retry en cas d'échec
236     for attempt in range(MAX_RETRIES):
237         try:
238             # Exécution de la requête POST avec paramètres en URL
239             response = requests.post(
240                 url,
241                 # Tout en paramètres URL
242                 params=params,
243                 timeout=REQUEST_TIMEOUT
244             )
245
246             # Vérification du succès de la requête
247             if response.status_code in [200, 201]:
248                 print(f"[API] Température envoyée avec succès: {temperature}°C")
249                 return True
250             else:
251                 # Affichage de l'erreur HTTP
252                 print(f"[API] Erreur HTTP {response.status_code}: {response.text}")
253                 # Retry si ce n'est pas la dernière tentative
254                 if attempt < MAX_RETRIES - 1:
255                     time.sleep(RETRY_DELAY)
256                 continue
257             else:
258                 return False

```

```

259
260     except requests.exceptions.RequestException as e:
261         # Gestion des erreurs de réseau
262         print(f"[API] Tentative {attemp + 1}/{MAX_RETRIES} d'envoi échouée: {e}")
263         # Attente avant retry si ce n'est pas la dernière tentative
264         if attemp < MAX_RETRIES - 1:
265             time.sleep(RETRY_DELAY)
266         else:
267             print(f"[API] Impossible d'envoyer la température après {MAX_RETRIES} tentatives")
268             return False
269
270     return False
271
272 # -----
273 # Fonctions utilitaires
274 # -----
275
276 def get_simulation_data(mac_address: str) -> Dict[str, Any]:
277     """
278     @brief Récupère les données de simulation depuis les APIs.
279     @details Combine les appels vers td25_param et td25_forecast pour
280             récupérer tous les paramètres nécessaires à la simulation
281             de température. Utilise le client ApiClient pour gérer
282             les retries et les erreurs automatiquement.
283
284     @param mac_address Adresse MAC du dispositif.
285
286     @return Dictionnaire avec toutes les données nécessaires pour la simulation.
287
288     @exception ApiError Si aucune donnée valide n'est obtenue.
289     """
290     # Création d'une instance du client API
291     client = ApiClient(mac_address)
292
293     # Récupération automatique
294     print("[SIMULATION] Tentative de récupération automatique des données...")
295
296     # Récupération des paramètres
297     params = client.get_parameters()
298
299     # Récupération des prévisions
300     forecast = client.get_forecast()
301
302     # Combinaison des données
303     # Création du dictionnaire de données de simulation
304     simulation_data = {
305         "probe_type": params["probe_type"],
306         "n": float(params["n"]),
307         "k_m": float(params["k_m"]),
308         "temperature": float(params["temperature"]),
309         "forecast_temperature": float(forecast["temperature"])
310     }
311
312     # Confirmation du succès de la récupération
313     print("[SIMULATION] Données récupérées automatiquement avec succès")
314     return simulation_data
315
316 def send_temperature_measurement(mac_address: str, temperature: float, channel: int = None) -> bool:
317     """
318     @brief Envoie une mesure de température vers l'API.
319     @details Fonction utilitaire qui crée un client ApiClient et utilise
320             la méthode post_measured_temperature pour envoyer la température
321             mesurée. Gère les exceptions et retourne un booléen simple.
322
323     @param mac_address Adresse MAC du dispositif.
324     @param temperature Température mesurée en °C.
325     @param channel Canal de mesure (optionnel).
326
327     @return True si succès, False sinon.
328     """
329     # Création d'une instance du client API
330     client = ApiClient(mac_address)
331
332     try:
333         # Tentative d'envoi de la température
334         return client.post_measured_temperature(temperature, channel)
335     except Exception as e:
336         # Gestion des erreurs générales
337         print(f"[API] Erreur lors de l'envoi de la température: {e}")
338         return False
339
340 # -----
341 # Point d'entrée pour les tests
342 # -----
343
344 # Test du module (à des fins de développement)
345 if __name__ == "__main__":

```

```
346 # Adresse MAC de test
347 test_mac = "0030DEABCDEF"
348
349 try:
350     # Tentative de récupération des données de simulation
351     data = get_simulation_data(test_mac)
352     # Affichage des données récupérées
353     print(f"\nDonnées finales: {data}")
354 except ApiError as e:
355     # Gestion des erreurs spécifiques à l'API
356     print(f"Erreur: {e}")
```

config_module\cm5_config.py

```
1  # -*- coding: utf-8 -*-
2  """
3  @file      cm5_config.py
4  @brief     Configuration matérielle pour plateforme CM5.
5  @details   Ce module fournit la configuration des canaux ADC et la fonction
6             d'initialisation complète du matériel (LED, TMUX, ADC). Centralise
7             la création des instances matérielles avec câblage fixe CM5.
8  @author    Léo Mendes
9  @project   2510_RegulationsChauffage
10 @Mandant    Oblo_solution
11 @date      2025-09-18
12 @version   1.0.0
13 @copyright Copyright (c) 2025
14 """
15
16 # Import de la bibliothèque GPIO pour contrôle matériel
17 from periphery import GPIO
18 # Import des définitions de pins spécifiques CM5
19 from app.pins_cm5 import GPIO_CHIP_PATH, FRONT_LED
20 # Import du driver multiplexeur TMUX1204
21 from app.hw_tmux1204 import Tmux1204
22 # Import du driver ADC ADS124S08
23 from app.adc_ads124s08 import Ads124s08
24
25
26 # -----
27 # Constantes de configuration matérielle
28 # -----
29
30 # Liste des canaux ADC disponibles sur la plateforme CM5
31 ADC_CHANNELS = [1, 2, 3, 4]
32
33
34 # -----
35 # Fonctions d'initialisation matérielle
36 # -----
37
38 def build_hw():
39     """
40     @brief   Initialise et configure tout le matériel CM5.
41     @details Crée les instances des composants matériels avec configuration
42             fixe: LED frontale, multiplexeur TMUX et ADC. Configure la LED
43             en sortie et l'allume immédiatement pour indication système.
44
45     @return  Dictionnaire contenant toutes les instances matérielles:
46             - "led": Instance GPIO pour LED frontale
47             - "tmux": Instance Tmux1204 pour sélection canaux
48             - "adc": Instance Ads124s08 pour conversion analogique
49
50     @warning LED allumée automatiquement pour indiquer système actif.
51     """
52     # Création de l'instance GPIO pour LED frontale en mode sortie
53     led = GPIO(GPIO_CHIP_PATH, FRONT_LED, "out")
54     # Allumage immédiat de la LED pour indication système démarré
55     led.write(True)
56     # Création de l'instance multiplexeur pour sélection des canaux
57     tmux = Tmux1204()
58     # Création de l'instance ADC pour conversions analogique-numérique
59     adc = Ads124s08()
60     # Création du dictionnaire de retour avec toutes les instances
61     hw = {}
62     # Ajout de l'instance LED au dictionnaire matériel
63     hw["led"] = led
64     # Ajout de l'instance TMUX au dictionnaire matériel
65     hw["tmux"] = tmux
66     # Ajout de l'instance ADC au dictionnaire matériel
67     hw["adc"] = adc
68     # Retour du dictionnaire complet des composants initialisés
69     return hw
70
```

app\config_loader.py

```
1 # -*- coding: utf-8 -*-
2 """
3 @file      config_loader.py
4 @brief     Chargeur de configuration JSON pour le système de mesure.
5 @details   Ce module fournit une fonction simple pour charger et parser
6            un fichier de configuration JSON contenant les paramètres
7            des capteurs, timeouts et autres paramètres système.
8 @author    Léo Mendes
9 @project   2510_RegulationsChauffage
10 @Mandant   Oblo_solution
11 @date      2025-09-18
12 @version   1.0.0
13 @copyright Copyright (c) 2025
14 """
15
16 from __future__ import annotations
17
18 # Sérialisation/désérialisation JSON
19 import json
20 # Opérations sur les chemins et fichiers
21 import os
22
23 # -----
24 # Fonctions utilitaires
25 # -----
26
27 def load_config(path):
28     """
29     @brief   Charge un fichier de configuration JSON.
30     @details Vérifie l'existence du fichier, l'ouvre en UTF-8 et parse
31            le contenu JSON. Le format attendu inclut les timeouts,
32            délais entre mesures et configuration des canaux de capteurs.
33
34     @param   path Chemin vers le fichier de configuration JSON.
35
36     @return  Dictionnaire Python contenant les données de configuration.
37
38     @exception FileNotFoundError Si le fichier n'existe pas.
39     @exception json.JSONDecodeError Si le fichier n'est pas un JSON valide.
40
41     @example
42     Format JSON attendu:
43     {
44         "timeout_s": 180,
45         "inter_measure_sleep_s": 0.1,
46         "loop_sleep_s": 1.0,
47         "channels": [
48             { "channel": 1, "sensor": "Ni1000 TK5000" }
49         ]
50     }
51     """
52     # Vérification de l'existence du fichier
53     if os.path.exists(path) is False:
54         # Lever une exception si le fichier n'existe pas
55         raise FileNotFoundError("Config introuvable: " + str(path))
56     # Ouverture et lecture du fichier JSON
57     with open(path, "r", encoding="utf-8") as f:
58         # Parsing du contenu JSON
59         data = json.load(f)
60     # Retour des données parsées
61     return data
62
```

app\front_led_management.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      front_led_management.py
4  @brief     Gestionnaire de LED frontale avec patterns d'erreur.
5  @details   Ce module fournit une classe pour gérer les clignotements et états
6             de la LED frontale avec différents patterns d'erreur. Supporte
7             les modes continus, les clignotements d'erreur et la gestion
8             thread-safe des patterns LED.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 from __future__ import annotations
18
19 # Gestion multithread pour les patterns asynchrones
20 import threading
21 # Fonctions temporelles pour les délais et temporisations
22 import time
23 # Abstraction GPIO (bibliothèque periphery)
24 from periphery import GPIO
25
26 # -----
27 # Classes principales
28 # -----
29
30 class LedErrorIndicator:
31     """
32     @brief     Gestionnaire de LED avec patterns d'erreur.
33     @details   Classe pour contrôler une LED frontale avec différents patterns
34               de clignotement selon les états du système. Supporte les modes
35               normal, standby, séquence en cours et divers patterns d'erreur
36               avec clignotements codifiés.
37     """
38
39     # Définition des patterns d'erreur (temps de base pour clignotement)
40     # 200ms ON/OFF pour un clignotement plus rapide [s]
41     BASE_BLINK_TIME = 0.2
42     # Cycle de 10 secondes [s]
43     CYCLE_TIME = 10.0
44
45     # Dictionnaire des patterns disponibles
46     ERROR_PATTERNS = {
47         # LED normalement allumée en continu
48         'normal': 'solid_on',
49
50         # LED éteinte (standby)
51         'standby': 'solid_off',
52
53         # Clignotement continu pendant séquence de mesure
54         'sequence_running': 'continuous_blink',
55
56         # Patterns d'erreur : X clignotements sur 10 secondes puis répétition
57         'no_internet': 1,      # 1 clignotement = Pas de connexion Internet
58         'api_failed': 2,      # 2 clignotements = Erreur API (connexion ou réponse)
59         'measure_failed': 3,   # 3 clignotements = Erreur de mesure ADC/résistance
60         'critical_error': 4,   # 4 clignotements = Erreur critique système
61         'config_error': 5     # 5 clignotements = Erreur de configuration
62     }
63
64     def __init__(self, gpio_chip_path, led_pin):
65         """
66         @brief     Initialise le gestionnaire LED.
67         @details   Configure l'interface GPIO pour la LED, initialise les variables
68                   de thread et les mécanismes de synchronisation thread-safe.
69
70         @param     gpio_chip_path Chemin vers le périphérique GPIO.
71         @param     led_pin        Numéro de pin de la LED.
72         """
73         # Initialisation de l'interface GPIO pour la LED
74         self.led = GPIO(gpio_chip_path, led_pin, "out")
75         # Pattern actuel (par défaut normal)
76         self.current_pattern = 'normal'
77         # Thread pour le pattern en cours
78         self.pattern_thread = None
79         # Événement pour arrêter le pattern
80         self.pattern_stop = threading.Event()
81         # Verrou pour la synchronisation thread-safe
82         self.led_lock = threading.Lock()
83
84     def set_pattern(self, pattern_name):

```



```

85     """
86     @brief Change le pattern de clignotement.
87     @details Arrête le pattern précédent et démarre le nouveau selon
88             le nom fourni. Gère les modes continus, clignotements
89             d'erreur et états fixes de manière thread-safe.
90
91     @param pattern_name Nom du pattern à activer.
92     """
93     # Vérification de l'existence du pattern
94     if pattern_name not in self.ERROR_PATTERNS:
95         print(f"[LED] Pattern inconnu: {pattern_name}")
96         return
97
98     # Ne rien faire si c'est déjà le pattern actuel
99     if self.current_pattern == pattern_name:
100         return
101
102     # Section critique thread-safe
103     with self.led_lock:
104         # Arrêt du pattern précédent
105         # Vérification de l'existence et de l'activité du thread
106         if self.pattern_thread and self.pattern_thread.is_alive():
107             # Demande d'arrêt du thread
108             self.pattern_stop.set()
109             # Attente de la fin du thread avec timeout
110             self.pattern_thread.join(timeout=1)
111
112         # Démarrage du nouveau pattern
113         # Stockage du nouveau pattern courant
114         self.current_pattern = pattern_name
115         # Remise à zéro de l'événement d'arrêt
116         self.pattern_stop.clear()
117
118         # Récupération de la valeur du pattern
119         pattern_value = self.ERROR_PATTERNS[pattern_name]
120
121         # Gestion des différents types de patterns
122         if pattern_value == 'solid_on':
123             # LED allumée en continu
124             self.led.write(True)
125             print(f"[LED] Pattern activé: {pattern_name} (LED allumée)")
126         elif pattern_value == 'solid_off':
127             # LED éteinte en continu
128             self.led.write(False)
129             print(f"[LED] Pattern activé: {pattern_name} (LED éteinte)")
130         elif pattern_value == 'continuous_blink':
131             # Clignotement continu
132             # Création du thread pour clignotement continu
133             self.pattern_thread = threading.Thread(target=self._continuous_blink, daemon=True)
134             # Démarrage du thread
135             self.pattern_thread.start()
136             print(f"[LED] Pattern activé: {pattern_name} (clignotement continu)")
137         elif isinstance(pattern_value, int):
138             # Pattern d'erreur avec X clignotements
139             # Création du thread pour pattern d'erreur
140             self.pattern_thread = threading.Thread(target=self._error_pattern, args=(pattern_value,), daemon=True)
141             # Démarrage du thread
142             self.pattern_thread.start()
143             print(f"[LED] Pattern activé: {pattern_name} ({pattern_value} clignotement(s) sur 10s)")
144
145     def _continuous_blink(self):
146         """
147         @brief Clignotement continu pendant séquence.
148         @details Méthode privée exécutée dans un thread séparé pour gérer
149                 le clignotement continu de la LED. Alterne entre ON/OFF
150                 avec le temps de base défini jusqu'à réception d'un signal d'arrêt.
151         """
152         # Boucle de clignotement jusqu'à demande d'arrêt
153         while not self.pattern_stop.is_set():
154             try:
155                 # Allumage de la LED
156                 self.led.write(True)
157                 # Attente avec vérification d'arrêt
158                 if self.pattern_stop.wait(self.BASE_BLINK_TIME):
159                     break
160                 # Extinction de la LED
161                 self.led.write(False)
162                 # Attente avec vérification d'arrêt
163                 if self.pattern_stop.wait(self.BASE_BLINK_TIME):
164                     break
165             except Exception as e:
166                 # Gestion des erreurs de clignotement
167                 print(f"[LED] Erreur clignotement continu: {e}")
168                 break
169
170     def _error_pattern(self, blink_count):
171         """

```

```

172     @brief Pattern d'erreur avec nombre de clignotements défini.
173     @details Méthode privée pour gérer les patterns d'erreur. Effectue
174             un nombre spécifique de clignotements puis attend jusqu'à
175             compléter un cycle de 10 secondes avant de répéter.
176
177     @param blink_count Nombre de clignotements à effectuer par cycle.
178     """
179     # Boucle de pattern d'erreur jusqu'à demande d'arrêt
180     while not self.pattern_stop.is_set():
181         try:
182             # Phase clignotements
183             # Boucle pour le nombre de clignotements demandé
184             for i in range(blink_count):
185                 # Vérification d'arrêt
186                 if self.pattern_stop.is_set():
187                     break
188                 # ON
189                 # Allumage de la LED
190                 self.led.write(True)
191                 # Attente avec vérification d'arrêt
192                 if self.pattern_stop.wait(self.BASE_BLINK_TIME):
193                     break
194                 # OFF
195                 # Extinction de la LED
196                 self.led.write(False)
197                 # Attente avec vérification d'arrêt
198                 if self.pattern_stop.wait(self.BASE_BLINK_TIME):
199                     break
200
201             # Vérification d'arrêt avant la phase d'attente
202             if self.pattern_stop.is_set():
203                 break
204
205             # Phase d'attente (9 temps vides pour compléter les 10 secondes)
206             # Temps utilisé pour clignotements : blink_count * (0.5 + 0.5) = blink_count secondes
207             # Temps restant : 10 - blink_count secondes
208             # Calcul du temps restant pour compléter le cycle
209             remaining_time = self.CYCLE_TIME - (blink_count * 2 * self.BASE_BLINK_TIME)
210             # Attente seulement si du temps reste
211             if remaining_time > 0:
212                 # LED éteinte pendant l'attente
213                 self.led.write(False)
214                 # Attente du temps restant avec vérification d'arrêt
215                 if self.pattern_stop.wait(remaining_time):
216                     break
217
218         except Exception as e:
219             # Gestion des erreurs de pattern d'erreur
220             print(f"[LED] Erreur pattern d'erreur: {e}")
221             break
222
223     def set_solid(self, state):
224         """
225         @brief LED fixe ON ou OFF.
226         @details Force la LED dans un état fixe (allumée ou éteinte) en
227                 arrétant tout pattern en cours. Utilisé pour des états
228                 temporaires ou des tests.
229
230         @param state État demandé (True=ON, False=OFF).
231         """
232         # Section critique thread-safe
233         with self.led_lock:
234             # Arrêt du thread de pattern si actif
235             if self.pattern_thread and self.pattern_thread.is_alive():
236                 # Demande d'arrêt du thread
237                 self.pattern_stop.set()
238                 # Attente de la fin du thread avec timeout
239                 self.pattern_thread.join(timeout=1)
240             try:
241                 # Application de l'état demandé
242                 self.led.write(state)
243                 # Affichage de confirmation
244                 print(f"[LED] État fixe: {'ON' if state else 'OFF'}")
245             except Exception as e:
246                 # Gestion des erreurs d'écriture GPIO
247                 print(f"[LED] Erreur état fixe: {e}")
248
249     def close(self):
250         """
251         @brief Ferme proprement la LED.
252         @details Arrête tous les threads en cours, éteint la LED et ferme
253                 l'interface GPIO. À appeler avant la fin du programme pour
254                 un nettoyage propre des ressources.
255         """
256         # Section critique thread-safe
257         with self.led_lock:
258             # Arrêt du thread de pattern si actif

```

```
259         if self.pattern_thread and self.pattern_thread.is_alive():
260             # Demande d'arrêt du thread
261             self.pattern_stop.set()
262             # Attente de la fin du thread avec timeout
263             self.pattern_thread.join(timeout=1)
264         try:
265             # Extinction de la LED
266             self.led.write(False)
267             # Fermeture de l'interface GPIO
268             self.led.close()
269             # Confirmation de fermeture
270             print("[LED] LED fermée")
271         except Exception as e:
272             # Gestion des erreurs de fermeture
273             print(f"[LED] Erreur fermeture: {e}")
274
275
276 # -----
277 # Fonctions utilitaires
278 # -----
279
280 def create_led_indicator(gpio_chip_path, led_pin):
281     """
282     @brief  Factory function pour créer une instance LedErrorIndicator.
283     @details Fonction utilitaire qui encapsule la création et l'initialisation
284             d'un gestionnaire LED. Gère les erreurs d'initialisation et
285             configure automatiquement le pattern normal par défaut.
286
287     @param  gpio_chip_path Chemin vers le périphérique GPIO.
288     @param  led_pin        Numéro de pin de la LED.
289
290     @return Instance de LedErrorIndicator si succès, None sinon.
291     """
292     try:
293         # Création de l'instance du gestionnaire LED
294         led_indicator = LedErrorIndicator(gpio_chip_path, led_pin)
295         # LED allumée par défaut
296         led_indicator.set_pattern('normal')
297         # Confirmation de l'initialisation
298         print("[LED] Gestionnaire LED initialisé avec succès")
299         return led_indicator
300     except Exception as e:
301         # Gestion des erreurs d'initialisation
302         print(f"[LED] Erreur lors de l'initialisation: {e}")
303         return None
```

app\hw_mux_adg731.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      hw_mux_adg731.py
4  @brief     Pilote pour multiplexeur ADG731 via interface SPI.
5  @details   Ce module fournit un pilote pour contrôler les multiplexeurs
6             ADG731 via l'interface SPI0. Supporte jusqu'à 4 cartes avec
7             32 canaux chacune. Gère la configuration SPI et l'envoi des
8             commandes de contrôle pour la sélection des canaux.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 from __future__ import annotations
18
19 # Opérations sur les fichiers et chemins système
20 import os
21 # Interface SPI pour Linux
22 import spidev
23
24 # -----
25 # Fonctions utilitaires
26 # -----
27
28 def adg731_ctrl_byte(address, enable=True):
29     """
30     @brief    Génère le byte de contrôle pour l'ADG731.
31     @details   Construit l'octet de commande selon le protocole ADG731 avec
32                le bit d'activation (EN) en MSB et l'adresse sur 5 bits.
33                L'activation se fait avec EN=0 (logique inversée).
34
35     @param    address Adresse du canal (0-31).
36     @param    enable  État d'activation du canal (True=activé).
37
38     @return   Octet de contrôle formaté selon le protocole ADG731.
39
40     @exception ValueError Si l'adresse est hors de la plage valide.
41     """
42     # Vérification de la plage d'adresse valide
43     if address < 0 or address > 31:
44         # Lever une exception si l'adresse est invalide
45         raise ValueError("address out of range (0..31)")
46     # Logique inversée pour le bit d'activation
47     if enable is True:
48         # EN=0 pour activer (logique inversée)
49         en = 0
50     else:
51         # EN=1 pour désactiver
52         en = 1
53     # Construction de l'octet de contrôle (EN << 7) | (adresse & 0x1F)
54     ctrl = (en << 7) | (address & 0x1F)
55     return ctrl
56
57 # -----
58 # Classes principales
59 # -----
60
61 class Adg731MuxSpi:
62     """
63     @brief    Pilote SPI pour multiplexeurs ADG731.
64     @details   Classe pour contrôler jusqu'à 4 multiplexeurs ADG731 via
65                l'interface SPI0. Chaque multiplexeur peut gérer 32 canaux.
66                Configure automatiquement les paramètres SPI optimaux et
67                gère la communication avec les circuits.
68     """
69
70     # Chemins des périphériques SPI disponibles
71     DEV = ["/dev/spidev0.0", "/dev/spidev0.1", "/dev/spidev0.2", "/dev/spidev0.3"]
72
73     def __init__(self, speed_hz=100000):
74         """
75         @brief    Initialise le pilote pour les multiplexeurs ADG731.
76         @details   Configure l'interface SPI pour chaque canal disponible,
77                    définit les paramètres de communication et initialise
78                    les handles pour jusqu'à 4 cartes multiplexeurs.
79
80         @param    speed_hz Vitesse SPI en Hz (par défaut 100kHz).
81         """
82         # Stockage de la vitesse SPI configurée
83         self.speed = speed_hz
84         # Liste des handles SPI pour chaque carte

```

```

85     self.handles = []
86     # Index de boucle pour initialisation des cartes
87     index = 0
88     # Boucle d'initialisation pour les 4 cartes possibles
89     while index < 4:
90         # Récupération du chemin du périphérique SPI
91         path = self.DEV[index]
92         # Vérification de l'existence du périphérique
93         if os.path.exists(path) is True:
94             # Création d'une instance SpiDev
95             s = spidev.SpiDev()
96             # Ouverture du bus SPI (bus 0, device index)
97             s.open(0, index)
98             # Configuration du mode SPI (mode 1: CPOL=0, CPHA=1)
99             s.mode = 1
100            # Configuration de la vitesse maximale
101            s.max_speed_hz = self.speed
102            # Configuration du nombre de bits par mot
103            s.bits_per_word = 8
104            try:
105                # Configuration LSB first (False = MSB first)
106                s.lsbfirst = False
107            except Exception:
108                # Ignorer si non supporté par la plateforme
109                pass
110            try:
111                # Configuration CS high (False = CS actif bas)
112                s.cshigh = False
113            except Exception:
114                # Ignorer si non supporté par la plateforme
115                pass
116            try:
117                # Configuration no CS (False = utiliser CS)
118                s.no_cs = False
119            except Exception:
120                # Ignorer si non supporté par la plateforme
121                pass
122            try:
123                # Configuration three wire (False = mode 4 fils)
124                s.threewire = False
125            except Exception:
126                # Ignorer si non supporté par la plateforme
127                pass
128            # Ajout du handle configuré à la liste
129            self.handles.append(s)
130        else:
131            # Ajout d'un handle None si le périphérique n'existe pas
132            self.handles.append(None)
133        # Passage à l'index suivant
134        index = index + 1
135
136    def close(self):
137        """
138        @brief Ferme toutes les interfaces SPI.
139        @details Parcourt tous les handles SPI ouverts et les ferme
140        proprement. Gère les erreurs de fermeture pour éviter
141        les exceptions lors du nettoyage.
142        """
143        # Parcours de tous les handles SPI
144        for h in self.handles:
145            # Vérification que le handle existe
146            if h is not None:
147                try:
148                    # Fermeture du handle SPI
149                    h.close()
150                except Exception:
151                    # Ignorer les erreurs de fermeture
152                    pass
153
154    def set_channel(self, board_index, address):
155        """
156        @brief Active un canal spécifique sur une carte donnée.
157        @details Envoie la commande de sélection de canal vers la carte
158        multiplexeur spécifiée. Génère l'octet de contrôle et
159        l'envoi via SPI.
160
161        @param board_index Index de la carte (0-3).
162        @param address Adresse du canal à activer (0-31).
163
164        @return Octet de contrôle envoyé, ou None si erreur.
165
166        @exception ValueError Si l'index de carte est hors plage.
167        """
168        # Vérification de la plage d'index de carte
169        if board_index < 0 or board_index > 3:
170            # Lever une exception si l'index est invalide
171            raise ValueError("board_index out of range (0..3)")

```

```
172     # Récupération du handle SPI pour la carte
173     h = self.handles[board_index]
174     # Vérification de l'existence du handle
175     if h is None:
176         print("ADG731: board", board_index, "non présent, ignoré")
177         return None
178     # Génération de l'octet de contrôle pour activer le canal
179     ctrl = adg731_ctrl_byte(address, True)
180     # Envoi de la commande via SPI
181     h.xfer2([ctrl])
182     return ctrl
183
184 def set_output_channel(self, channel_index):
185     """
186     @brief    Utilitaire simple pour sélectionner un canal de sortie.
187     @details  Fonction de commodité qui utilise automatiquement la
188               carte 0 et mappe directement l'index de canal sur
189               l'adresse du multiplexeur.
190
191     @param   channel_index Index du canal de sortie (0-31).
192
193     @return  Octet de contrôle envoyé, ou None si erreur.
194
195     @exception ValueError Si l'index de canal est hors plage.
196     """
197     # Utilitaire simple si tu veux piloter par numéro de voie unique
198     # ici: board 0, address = channel_index
199     # Vérification de la plage d'index de canal
200     if channel_index < 0 or channel_index > 31:
201         # Lever une exception si l'index est invalide
202         raise ValueError("channel_index out of range (0..31)")
203     # Appel de la méthode générique avec la carte 0
204     return self.set_channel(0, channel_index)
205
```

app\hw_tmux1204.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      hw_tmux1204.py
4  @brief     Pilote pour multiplexeur TMUX1204 de sélection de résistance de référence.
5  @details   Ce module fournit un pilote pour contrôler le multiplexeur TMUX1204
6              via les signaux de contrôle ADC_A1 et ADC_A0. Permet la sélection
7              de quatre valeurs de résistance de référence (2.2k, 2.7k, 10k, 100k)
8              selon une table de vérité définie.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 from __future__ import annotations
18
19 # Bibliothèque pour la gestion des GPIO
20 from periphery import GPIO
21 # Importation des constantes matérielles
22 from pins_cm5 import GPIO_CHIP_PATH, ADC_A1, ADC_A0
23
24 # -----
25 # Classes principales
26 # -----
27
28 class Tmux1204:
29     """
30     @brief     Pilote pour le multiplexeur TMUX1204.
31     @details   Classe pour gérer la sélection de résistance de référence via
32                 le multiplexeur TMUX1204. Utilise deux signaux de contrôle
33                 (A1, A0) pour sélectionner parmi quatre valeurs de Rref selon
34                 la table de vérité :
35                 - A1=0, A0=0 : 2.2k ohms
36                 - A1=0, A0=1 : 2.7k ohms
37                 - A1=1, A0=0 : 10k ohms
38                 - A1=1, A0=1 : 100k ohms
39     """
40
41     def __init__(self):
42         """
43         @brief     Initialise le pilote TMUX1204.
44         @details   Configure les GPIO pour les signaux de contrôle A1 et A0
45                     en mode sortie. Utilise les constantes matérielles définies
46                     dans le module pins_cm5.
47         """
48         # Initialisation des GPIO pour les sorties A1 et A0
49         # GPIO pour A1
50         self.gpio_a1 = GPIO(GPIO_CHIP_PATH, ADC_A1, "out")
51         # GPIO pour A0
52         self.gpio_a0 = GPIO(GPIO_CHIP_PATH, ADC_A0, "out")
53
54     def close(self):
55         """
56         @brief     Ferme les interfaces GPIO.
57         @details   Ferme proprement les GPIO A1 et A0 pour libérer les
58                     ressources système. Gère les erreurs de fermeture pour
59                     éviter les exceptions lors du nettoyage.
60         """
61         # Fermeture des GPIO
62         try:
63             # Ferme le GPIO A1
64             self.gpio_a1.close()
65         except Exception:
66             # Ignore les erreurs
67             pass
68         try:
69             # Ferme le GPIO A0
70             self.gpio_a0.close()
71         except Exception:
72             # Ignore les erreurs
73             pass
74
75     def set_bits(self, bit_a1, bit_a0):
76         """
77         @brief     Définit les bits de contrôle A1 et A0.
78         @details   Applique les niveaux logiques sur les signaux de contrôle
79                     du multiplexeur. Convertit les valeurs numériques (0/1)
80                     en niveaux logiques GPIO (False/True).
81
82         @param     bit_a1 Valeur du bit A1 (0 ou 1).
83         @param     bit_a0 Valeur du bit A0 (0 ou 1).
84         """

```

```
85         # Définit les bits A1 et A0 (0 ou 1)
86         # Configuration du signal A1
87         if bit_a1 == 0:
88             # Écrit 0 sur A1
89             self.gpio_a1.write(False)
90         else:
91             # Écrit 1 sur A1
92             self.gpio_a1.write(True)
93         # Configuration du signal A0
94         if bit_a0 == 0:
95             # Écrit 0 sur A0
96             self.gpio_a0.write(False)
97         else:
98             # Écrit 1 sur A0
99             self.gpio_a0.write(True)
100
101     def select_rref_ohm(self, rref_ohm):
102         """
103         @brief Sélectionne la résistance de référence Rref.
104         @details Configure les signaux de contrôle selon la table de vérité
105                 pour sélectionner la résistance de référence demandée.
106                 Supporte quatre valeurs fixes : 2200, 2700, 10000, 100000 ohms.
107
108         @param rref_ohm Valeur de résistance de référence en ohms.
109
110         @exception ValueError Si la valeur de Rref n'est pas supportée.
111
112         @note Table de vérité TMUX1204 :
113             - 2200 ohms : A1=0, A0=0
114             - 2700 ohms : A1=0, A0=1
115             - 10000 ohms : A1=1, A0=0
116             - 100000 ohms : A1=1, A0=1
117         """
118         # Sélectionne la résistance de référence Rref
119         # Rref supportées: 2200, 2700, 10000, 100000
120         # Vérification pour 2.2k ohms
121         if rref_ohm == 2200:
122             # A1=0, A0=0 pour 2.2k
123             self.set_bits(0, 0)
124         else:
125             # Vérification pour 2.7k ohms
126             if rref_ohm == 2700:
127                 # A1=0, A0=1 pour 2.7k
128                 self.set_bits(0, 1)
129             else:
130                 # Vérification pour 10k ohms
131                 if rref_ohm == 10000:
132                     # A1=1, A0=0 pour 10k
133                     self.set_bits(1, 0)
134                 else:
135                     # Vérification pour 100k ohms
136                     if rref_ohm == 100000:
137                         # A1=1, A0=1 pour 100k
138                         self.set_bits(1, 1)
139                     else:
140                         # Erreur si Rref non valide
141                         raise ValueError("Rref non supportée: " + str(rref_ohm))
142
```


webui\index.html

```
1 <!--
2 @file          index.html
3 @brief         Interface Web de configuration et monitoring CM5.
4 @details       Cette page HTML fournit une interface utilisateur complète pour
5                la configuration du système de mesure de température et l'affichage
6                des données en temps réel. Inclut la gestion des capteurs, paramètres
7                de mesure et visualisation des résultats.
8 @author        Léo Mendes
9 @project        2510_RegulationsChauffage
10 @Mandant       Oblo_solution
11 @date          2025-09-18
12 @version       1.0.0
13 @copyright     Copyright (c) 2025
14 -->
15
16 <!-- Déclaration du type de document HTML5 -->
17 <!doctype html>
18 <!-- Élément racine HTML avec langue française -->
19 <html lang="fr">
20 <!-- En-tête du document contenant les métadonnées -->
21 <head>
22     <!-- Encodage de caractères UTF-8 pour support international -->
23     <meta charset="utf-8">
24     <!-- Titre affiché dans l'onglet du navigateur -->
25     <title>CMS WebUI - Config et Dernières valeurs</title>
26     <!-- Configuration responsive pour dispositifs mobiles -->
27     <meta name="viewport" content="width=device-width, initial-scale=1">
28
29     <!-- -----
30          Styles CSS intégrés pour l'interface utilisateur
31     ----- -->
32 <style>
33     /* Style général du corps de la page */
34     body { font-family: 'Verdana', sans-serif; margin: 20px; }
35
36     /* Conteneur pour grouper les paramètres de configuration */
37     .param-group { margin: 10px 0; padding: 8px; border: 1px solid #ddd; border-radius: 4px; }
38
39     /* Style des libellés de paramètres */
40     .param-label { font-weight: bold; margin-bottom: 4px; }
41
42     /* Style des descriptions de paramètres */
43     .param-desc { font-size: 0.9em; color: #666; margin-bottom: 6px; }
44
45     /* Style des champs de saisie de paramètres */
46     .param-input { width: 200px; padding: 4px; }
47
48     /* Styles pour les tableaux de données */
49     table { border-collapse: collapse; margin: 10px 0; }
50     th, td { padding: 8px; text-align: left; }
51
52     /* Style des boutons d'action */
53     button { padding: 8px 12px; margin: 4px; cursor: pointer; }
54
55     /* Boîte d'information avec bordure bleue */
56     .info-box { background: #f0f8ff; border-left: 4px solid #2196F3; padding: 10px; margin: 10px 0; }
57
58     /* Style pour les informations de canal */
59     .channel-info { background: #f9f9f9; padding: 8px; margin: 4px 0; border-radius: 4px; }
60
61     /* Style des logos dans l'en-tête */
62     .logo {
63         width: 50px;
64         height: 50px;
65         margin-right: 15px;
66         object-fit: contain;
67     }
68 </style>
69 </head>
70 <!-- Corps principal de la page -->
71 <body>
72     <!-- En-tête avec logos et titre principal -->
73     <header style="display: flex; align-items: center; margin-bottom: 20px; padding: 10px; border-bottom: 2px solid #ddd;">
74         <!-- Logo Oblo à gauche -->
75         
76         <!-- Titre principal de l'application -->
77         <h1 style="margin: 0;">Config 2510 Régulation Chauffage</h1>
78         <!-- Logo ETML ES à droite -->
79         
80     </header>
81
82     <!-- -----
83          Section de configuration du système de mesure
84     ----- -->
```

```
85 <section>
86 <!-- Titre de la section de configuration -->
87 <h2>Configuration du système de mesure</h2>
88
89 <!-- Boîte d'information pour l'utilisateur -->
90 <div class="info-box">
91   <strong>Information:</strong> Cette interface permet de configurer les paramètres de mesure et simulation de température.
92   Les modifications sont appliquées automatiquement lors de la prochaine séquence de mesure.
93 </div>
94
95 <!-- Groupe de paramètres : Adresse MAC -->
96 <div class="param-group">
97   <!-- Libellé du paramètre adresse MAC -->
98   <div class="param-label">Adresse MAC du dispositif</div>
99   <!-- Description détaillée du paramètre -->
100   <div class="param-desc">Identifiant unique du Raspberry Pi CMS pour l'API. Format: 6 octets en hexadécimal.</div>
101   <!-- Champ de saisie pour l'adresse MAC -->
102   <input id="mac_address" type="text" class="param-input" placeholder="ex: 0030DEABCDEF">
103 </div>
104
105 <!-- Groupe de paramètres : Timeout de mesure -->
106 <div class="param-group">
107   <!-- Libellé du paramètre timeout -->
108   <div class="param-label">Timeout de mesure (secondes)</div>
109   <!-- Description du timeout de mesure -->
110   <div class="param-desc">Temps maximum d'attente pour une mesure ADC avant abandon. Valeur recommandée: 180s.</div>
111   <!-- Champ numérique pour le timeout -->
112   <input id="timeout_s" type="number" step="1" min="1" max="3600" class="param-input">
113 </div>
114
115 <!-- Groupe de paramètres : Délai entre mesures -->
116 <div class="param-group">
117   <!-- Libellé du délai entre mesures -->
118   <div class="param-label">Délai entre mesures (secondes)</div>
119   <!-- Description du délai de stabilisation -->
120   <div class="param-desc">Pause entre chaque échantillonnage ADC pour stabilisation. Valeur recommandée: 0.1s.</div>
121   <!-- Champ numérique pour le délai inter-mesure -->
122   <input id="inter_measure_sleep_s" type="number" step="0.001" min="0" max="10" class="param-input">
123 </div>
124
125 <!-- Groupe de paramètres : Intervalle des séquences -->
126 <div class="param-group">
127   <!-- Libellé de l'intervalle des séquences automatiques -->
128   <div class="param-label">Intervalle des séquences automatiques (minutes)</div>
129   <!-- Description de la fréquence d'exécution -->
130   <div class="param-desc">Fréquence d'exécution automatique des mesures complètes. 0 = désactivé. Valeur recommandée: 5 min.</div>
131   <!-- Champ numérique pour l'intervalle en minutes -->
132   <input id="sequence_interval_minutes" type="number" step="1" min="0" max="1440" class="param-input">
133 </div>
134
135 <!-- Groupe de paramètres : Mode automatique -->
136 <div class="param-group">
137   <!-- Libellé du mode automatique -->
138   <div class="param-label">Mode automatique</div>
139   <!-- Description de l'activation automatique -->
140   <div class="param-desc">Active/désactive l'exécution automatique des séquences de mesure selon l'intervalle défini.</div>
141   <!-- Case à cocher pour le mode automatique -->
142   <input id="auto_sequence" type="checkbox">
143 </div>
144
145 <!-- Sous-titre pour la configuration des canaux -->
146 <h3>Configuration des canaux de mesure</h3>
147
148 <!-- Tableau de configuration des canaux de mesure -->
149 <table id="channels_table" border="1" cellpadding="4">
150   <!-- En-tête du tableau des canaux -->
151   <thead>
152     <tr>
153       <th>Canal</th>
154       <th>Type de capteur</th>
155       <th>État</th>
156     </tr>
157   </thead>
158   <!-- Corps du tableau généré dynamiquement -->
159   <tbody id="channels_body"></tbody>
160 </table>
161
162 <!-- Zone de boutons d'action pour la configuration -->
163 <div style="margin-top:8px;">
164   <!-- Bouton pour recharger la configuration -->
165   <button id="reload_btn">Recharger</button>
166   <!-- Bouton pour enregistrer les modifications -->
167   <button id="save_btn">Enregistrer</button>
168   <!-- Bouton pour forcer le rechargement complet -->
169   <button id="force_reload_btn">Forcer rechargement config</button>
170   <!-- Zone d'affichage du statut de sauvegarde -->
171   <span id="save_status"></span>
```

```
172     </div>
173 </section>
174
175 <!-- Séparateur horizontal entre les sections -->
176 <hr>
177
178 <!-- -----
179      Section des données en temps réel
180 ----- -->
181 <section>
182     <!-- Titre de la section données temps réel -->
183     <h2>Données en temps réel</h2>
184     <!-- Container pour les données par canal généré dynamiquement -->
185     <div id="channels_data">
186         <!-- Les données par canal seront affichées ici -->
187     </div>
188
189     <!-- Sous-section pour la dernière mesure globale -->
190     <h3>Dernière mesure globale</h3>
191     <!-- Boîte d'information pour la dernière mesure -->
192     <div class="channel-info">
193         <!-- Affichage du timestamp de la dernière mesure -->
194         <div><strong>Timestamp:</strong> <span id="ts">-</span></div>
195         <!-- Affichage du canal mesuré -->
196         <div><strong>Canal mesuré:</strong> <span id="last_channel">-</span></div>
197         <!-- Affichage de la résistance mesurée -->
198         <div><strong>Résistance mesurée:</strong> <span id="last_res">-</span> Ω</div>
199         <!-- Affichage de la température calculée -->
200         <div><strong>Température calculée:</strong> <span id="last_temp">-</span> °C</div>
201         <!-- Affichage de la température simulée -->
202         <div><strong>Température simulée:</strong> <span id="last_tsim">-</span> °C</div>
203         <!-- Affichage des erreurs éventuelles -->
204         <div><strong>Erreur:</strong> <span id="last_err">-</span></div>
205         <!-- Affichage de l'âge de la mesure -->
206         <div><strong>Âge de la mesure:</strong> <span id="age_s">-</span> secondes</div>
207     </div>
208 </section>
209
210 <!-- -----
211      Script JavaScript pour la logique de l'interface
212 ----- -->
213 <script>
214     /**
215      * @brief      Retourne les options de capteurs disponibles.
216      * @details    Liste des types de capteurs de température supportés par le système.
217      * @return     Tableau des noms de capteurs.
218      */
219     function sensorOptions() {
220         return [
221             "De Dietrich AF60",
222             "Siemens QAC32",
223             "PT1000",
224             "Ni1000 TK5000",
225             "Ni1000 TK6180",
226             "NTC 1k 3528",
227             "KTY81-210",
228             "NTC 2k 3390",
229             "NTC 2.2k 3528",
230             "NTC 10k 3977"
231         ];
232     }
233
234     /* Variables globales pour stocker les données */
235     var currentConfig = null; // Configuration actuelle chargée
236     var lastData = null;     // Dernières données de mesure
237
238     /**
239      * @brief      Raccourci pour getElementById.
240      * @param id   Identifiant de l'élément DOM.
241      * @return     Élément DOM correspondant.
242      */
243     function el(id) {
244         return document.getElementById(id);
245     }
246
247     /**
248      * @brief      Convertit une valeur en nombre ou retourne null.
249      * @details    Gère la conversion sécurisée des valeurs utilisateur.
250      * @param value Valeur à convertir.
251      * @return     Nombre valide ou null si conversion impossible.
252      */
253     function numberOrNull(value) {
254         if (value === null) {
255             return null;
256         }
257         var n = Number(value);
258         if (isNaN(n)) {
```

```
259     return null;
260 }
261 return n;
262 }
263
264 /**
265  * @brief      Charge la configuration depuis l'API.
266  * @details    Récupère la configuration actuelle et met à jour l'interface.
267  *            Peuple les champs de formulaire et génère le tableau des canaux.
268  */
269 function loadConfig() {
270     /* Requête GET vers l'API de configuration */
271     fetch("/api/config").then(function(resp) {
272         return resp.json();
273     }).then(function(data) {
274         /* Stockage de la configuration pour usage ultérieur */
275         currentConfig = data;
276
277         /* Mise à jour des champs de configuration générale */
278         el("mac_address").value = data.mac_address || "";
279         el("timeout_s").value = data.timeout_s;
280         el("inter_measure_sleep_s").value = data.inter_measure_sleep_s;
281         el("sequence_interval_minutes").value = data.sequence_interval_minutes;
282         el("auto_sequence").checked = !!data.auto_sequence;
283
284         /* Référence au corps du tableau des canaux */
285         var body = el("channels_body");
286         /* Effacement du contenu existant */
287         body.innerHTML = "";
288
289         /* Génération des lignes de canaux si disponibles */
290         if (Array.isArray(data.channels)) {
291             var i = 0;
292             while (i < data.channels.length) {
293                 var ch = data.channels[i];
294
295                 /* Création d'une nouvelle ligne de tableau */
296                 var tr = document.createElement("tr");
297                 /* Coloration selon l'état d'activation */
298                 if (ch.enabled) {
299                     tr.style.backgroundColor = "#e8f5e8"; // Vert pour actif
300                 } else {
301                     tr.style.backgroundColor = "#f5f5f5"; // Gris pour inactif
302                 }
303
304                 /* Colonne numéro de canal */
305                 var tdChan = document.createElement("td");
306                 tdChan.innerHTML = "<strong>Canal " + ch.channel + "</strong>";
307                 tr.appendChild(tdChan);
308
309                 /* Colonne sélecteur de type de capteur */
310                 var tdSensor = document.createElement("td");
311                 var sel = document.createElement("select");
312                 sel.style.width = "100%";
313
314                 /* Ajout des options de capteurs standards */
315                 var opts = sensorOptions();
316                 var j = 0;
317                 while (j < opts.length) {
318                     var o = document.createElement("option");
319                     o.value = opts[j];
320                     o.textContent = opts[j];
321                     sel.appendChild(o);
322                     j = j + 1;
323                 }
324
325                 /* Ajout d'option non-standard si nécessaire */
326                 if (opts.indexOf(ch.sensor) === -1) {
327                     var o2 = document.createElement("option");
328                     o2.value = ch.sensor;
329                     o2.textContent = ch.sensor + " (non standard)";
330                     sel.appendChild(o2);
331                 }
332
333                 /* Sélection de la valeur actuelle */
334                 sel.value = ch.sensor;
335                 sel.setAttribute("data-index", String(i));
336                 tdSensor.appendChild(sel);
337                 tr.appendChild(tdSensor);
338
339                 /* Colonne état avec case à cocher */
340                 var tdEn = document.createElement("td");
341                 var cb = document.createElement("input");
342                 cb.type = "checkbox";
343                 cb.checked = !!ch.enabled;
344                 cb.setAttribute("data-index", String(i));
345
```

state\last_state.json

```
1 {
2   "_comment_file": "last_state.json",
3   "_comment_brief": "État de la dernière mesure de température effectuée.",
4   "_comment_details": "Ce fichier JSON stocke les résultats de la dernière mesure de température effectuée par le système. Il contient les données brutes (résistance), les valeurs calculées (température réelle et simulée) ainsi que les métadonnées temporelles et d'erreur. Utilisé pour la persistance d'état entre les redémarrages.",
5   "_comment_author": "Léo Mendes",
6   "_comment_project": "2510_RegulationsChauffage",
7   "_comment_mandant": "Oblo_solution",
8   "_comment_date": "2025-09-18",
9   "_comment_version": "1.0.0",
10  "_comment_copyright": "Copyright (c) 2025",
11
12  "_comment_section_temporal": "--- Informations temporelles et de canal ---",
13  "timestamp": null,
14  "_comment_timestamp": "Horodatage ISO 8601 de la dernière mesure effectuée. null = aucune mesure disponible.",
15
16  "channel": null,
17  "_comment_channel": "Numéro du canal de mesure utilisé pour la dernière acquisition. null = aucun canal mesuré.",
18
19  "_comment_section_measurement": "--- Données de mesure brutes et calculées ---",
20  "resistance_ohm": null,
21  "_comment_resistance_ohm": "Valeur de résistance mesurée par l'ADC en ohms. null = aucune mesure disponible. Base du calcul de température.",
22
23  "temperature_c": null,
24  "_comment_temperature_c": "Température calculée à partir de la résistance mesurée en degrés Celsius. null = calcul impossible ou aucune mesure.",
25
26  "temperature_sim_c": null,
27  "_comment_temperature_sim_c": "Température simulée générée par l'algorithme de simulation en degrés Celsius. null = simulation désactivée.",
28
29  "_comment_section_error": "--- Gestion d'erreur et statut ---",
30  "error": "no data yet",
31  "_comment_error": "Message d'erreur ou de statut de la dernière opération de mesure. 'no data yet' = système initialisé mais aucune mesure effectuée. null ou '' = pas d'erreur."
32 }
```

app\main.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      main.py
4  @brief     Système de mesure et simulation de température.
5  @details   Ce module exécute des cycles de mesure ADC, convertit les valeurs
6             en résistances puis en températures, récupère des paramètres via
7             une API distante, calcule une température simulée, convertit
8             cette température en résistance cible et applique la simulation
9             via un multiplexeur matériel. Le module gère également un petit
10            serveur web optionnel et un indicateur LED.
11  @author    Léo Mendes
12  @project   2510_RegulationsChauffage
13  @Mandant   Oblo_solution
14  @date      2025-09-18
15  @version   1.0.0
16  @copyright Copyright (c) 2025
17  """
18
19  from __future__ import annotations
20
21  # Opérations système et manipulation de chemins
22  import os
23  # Accès à la table des chemins d'import et arguments
24  import sys
25
26  # Calculer le répertoire courant du module et le répertoire source parent
27  # Dossier contenant ce fichier
28  CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
29  # Dossier parent, racine du paquet local
30  SRC_DIR = os.path.dirname(CURRENT_DIR)
31  # S'assurer que le dossier source est dans sys.path pour permettre les imports relatifs locaux
32  if SRC_DIR not in sys.path:
33      sys.path.append(SRC_DIR)
34
35  # Fonctions temporelles (sleep, timestamps)
36  import time
37  # Parsing des arguments de la ligne de commande
38  import argparse
39  # Fonctions mathématiques utiles
40  import math
41  # Gestion multithread (input monitor, webui thread)
42  import threading
43  # Gestion des signaux système (Ctrl+C)
44  import signal
45  # Surveillance d'entrées/sockets si nécessaire
46  import select
47  # Client HTTP pour l'API distante
48  import requests
49  # Sérialisation/désérialisation JSON
50  import json
51  # Manipulation de dates et intervalles
52  from datetime import datetime, timedelta
53  # Abstraction GPIO (bibliothèque periphery)
54  from periphery import GPIO
55  # Constantes matérielles pour le CM5
56  from pins_cm5 import GPIO_CHIP_PATH, FRONT_LED, CMD_RELAY
57  # Utilitaire de chargement de configuration JSON
58  from app.config_loader import load_config
59  # Profils de capteurs et tables associées
60  from app.sensor_profiles import get_profile, SENSOR_TABLES
61  # Pilote du multiplexeur matériel TMUX1204
62  from hw_tmux1204 import Tmux1204
63  # Pilote ADC ADS124S08
64  from app.adc_ads124s08 import Ads124s08, request_stop as request_adc_stop, reset_stop_flag as reset_adc_stop_flag, is_stop_requested as is_adc_stop_requested
65  # Conversion T -> R pour sondes
66  from app.temperature_conversion import convert_temperature_to_resistance
67  # Client API (fonctions d'accès réseau)
68  from api_client import ApiClient
69  # Moteur de simulation de température
70  from temperature_simulation import run_temperature_simulation
71  # Simulation/application de résistance via MUX
72  from resistance_simulation import ResistanceSimulator
73  # Création/gestion de l'indicateur LED frontal
74  from front_led_management import create_led_indicator
75  # Test des entrées 24V (contacts secs)
76  from mesure_24v_dry_contact import test_24v_dry_contact, get_dry_contact_status
77
78  def start_webui_server():
79      """
80      @brief   Lance le serveur web local dans un thread séparé.
81      @details Import dynamique du module `webui.webui_server` et démarrage de
82              l'application Flask/WSGI. Les exceptions sont interceptées et
83              converties en messages sur la sortie standard pour ne pas

```

```

84         interrompre le thread principal.
85     @return None
86     """
87     try:
88         # Import retardé pour permettre l'exécution sans webui
89         from webui import webui_server
90         # Démarrage du serveur Flask avec les paramètres spécifiés
91         webui_server.app.run(host="192.168.1.109", port=8080, debug=False, use_reloader=False)
92     except ImportError:
93         # Gestion du cas où le module webui_server n'est pas disponible
94         print("[WEBUI] Module webui_server non trouvé")
95     except Exception as e:
96         # Gestion des autres erreurs du serveur web
97         print(f"[WEBUI] Erreur serveur web: {e}")
98
99 # -----
100 # Variables globales pour le contrôle du séquenceur
101 # -----
102
103 # Drapeau global indiquant que le séquenceur tourne
104 sequencer_running = False
105 # Événement pour demander l'arrêt propre
106 stop_event = threading.Event()
107 # Événement pour forcer une exécution manuelle
108 force_sequence = threading.Event()
109
110 # Instance globale de gestion LED
111 # Instance globale du gestionnaire LED (ou None si non disponible)
112 led_indicator = None
113
114 def save_last_state(channel=None, resistance_ohm=None, temperature_c=None, temperature_sim_c=None, error=None):
115     """
116     @brief Sauvegarde l'état courant et historique par canal dans un fichier JSON.
117     @details Crée le dossier `state` si nécessaire, lit le fichier `last_state.json`
118             existant, met à jour les informations du canal fourni et écrit le
119             document au format JSON. Les erreurs internes sont capturées et
120             affichées sur la sortie standard.
121
122     @param channel Numéro du canal (ou None).
123     @param resistance_ohm Résistance mesurée (ou None).
124     @param temperature_c Température mesurée [°C] (ou None).
125     @param temperature_sim_c Température simulée [°C] (ou None).
126     @param error Message d'erreur s'il y en a un (ou None).
127     @return None
128     """
129     try:
130         # Répertoire de stockage
131         state_dir = os.path.join(os.path.dirname(CURRENT_DIR), "state")
132         # Crée le répertoire si absent
133         os.makedirs(state_dir, exist_ok=True)
134         # Chemin complet du fichier
135         state_path = os.path.join(state_dir, "last_state.json")
136
137         # Charger l'état existant depuis le fichier JSON
138         try:
139             # Lecture du fichier JSON existant
140             with open(state_path, "r", encoding="utf-8") as f:
141                 existing_data = json.load(f)
142         except (FileNotFoundError, json.JSONDecodeError):
143             # Structure par défaut si inexistant
144             existing_data = {"channels": {}}
145
146         # S'assurer de la présence de la clé 'channels'
147         if "channels" not in existing_data:
148             existing_data["channels"] = {}
149
150         # Mettre à jour les données du canal spécifique si fourni
151         if channel is not None:
152             existing_data["channels"][str(channel)] = {
153                 "timestamp": datetime.now().isoformat(),
154                 "resistance_ohm": resistance_ohm,
155                 "temperature_c": temperature_c,
156                 "temperature_sim_c": temperature_sim_c,
157                 "error": error,
158             }
159         # Affichage de confirmation de sauvegarde
160         print(f"[STATE] Canal {channel} sauvegardé: {temperature_c}°C")
161
162         # Mettre à jour les méta-données globales (dernière mesure)
163         existing_data.update({
164             "last_timestamp": datetime.now().isoformat(),
165             "last_channel": channel,
166             "last_resistance_ohm": resistance_ohm,
167             "last_temperature_c": temperature_c,
168             "last_temperature_sim_c": temperature_sim_c,
169             "last_error": error,
170         })

```

```

171
172     # Écriture finale du fichier JSON
173     with open(state_path, "w", encoding="utf-8") as f:
174         json.dump(existing_data, f, ensure_ascii=False, indent=2)
175 except Exception as e:
176     # Gestion des erreurs de sauvegarde
177     print(f"[STATE] Erreur sauvegarde état: {e}")
178
179 def signal_handler(sig, frame):
180     """
181     @brief   Gestionnaire POSIX pour les signaux (Ctrl+C).
182     @details Met le drapeau `sequencer_running` à False et déclenche
183             l'événement `stop_event` pour demander un arrêt propre.
184             Demande aussi l'arrêt des opérations ADC en cours.
185     @param   sig   Signal reçu (int).
186     @param   frame Contexte d'exécution (non utilisé).
187     @return  None
188     """
189     global sequencer_running
190     # Affichage du message d'arrêt
191     print("\n[SEQUENCER] Arrêt demandé...")
192     # Arrêt du séquenceur
193     sequencer_running = False
194     # Déclenchement de l'événement d'arrêt
195     stop_event.set()
196     # Demande d'arrêt des opérations ADC
197     request_adc_stop()
198     print("[SEQUENCER] Arrêt des opérations ADC demandé...")
199
200 def input_monitor():
201     """
202     @brief   Thread d'écoute des commandes utilisateur depuis stdin.
203     @details Permet de forcer une séquence (touche Entrée ou 's') ou de
204             demander l'arrêt ('q'). Les exceptions d'E/S sont gérées pour
205             permettre la terminaison propre du thread.
206     @return  None
207     """
208     # Boucle de surveillance des entrées utilisateur
209     while sequencer_running and not stop_event.is_set():
210         try:
211             # Lit la ligne utilisateur
212             user_input = input()
213             # Vérification des commandes de déclenchement manuel
214             if user_input.strip().lower() in ['', 's', 'sequence', 'start']:
215                 print("[MANUAL] Séquence manuelle déclenchée")
216                 # Déclenchement manuel
217                 force_sequence.set()
218             # Vérification des commandes d'arrêt
219             elif user_input.strip().lower() in ['q', 'quit', 'exit']:
220                 print("[MANUAL] Arrêt demandé par l'utilisateur")
221                 # Simule Ctrl+C
222                 signal_handler(signal.SIGINT, None)
223                 break
224             except (EOFError, KeyboardInterrupt):
225                 break
226             except Exception:
227                 # Ignorer les erreurs d'entrée pour rester résilient
228                 pass
229
230 def show_error_and_continue(error_pattern, sleep_time=2):
231     """
232     @brief   Affiche un motif LED d'erreur pendant un temps donné puis
233             remet l'indicateur en mode 'sequence_running'.
234     @param   error_pattern Nom du motif LED à afficher.
235     @param   sleep_time    Durée d'affichage en secondes (par défaut 2s).
236     @return  None
237     """
238     global led_indicator
239     # Afficher l'erreur si l'indicateur LED est disponible
240     if led_indicator:
241         # Afficher l'erreur
242         led_indicator.set_pattern(error_pattern)
243     # Temporisation pour permettre la lecture
244     time.sleep(sleep_time)
245     # Revenir en mode séquence si l'indicateur LED est disponible
246     if led_indicator:
247         # Revenir en mode séquence
248         led_indicator.set_pattern('sequence_running')
249
250 def run_measurement_sequence(cfg, mac_address, adc, tmux):
251     """
252     @brief   Exécute une séquence complète de mesures pour tous les canaux
253             activés selon la configuration fournie.
254     @details Pour chaque canal actif : sélection du rref, configuration ADC,
255             lecture des valeurs, conversion en température, envoi API, calcul
256             de la température simulée et application via le simulateur de
257             résistance. Les motifs LED sont ajustés en fonction des erreurs.

```



```
258
259 @param cfg Dictionnaire de configuration chargé depuis JSON.
260 @param mac_address Adresse MAC ou identifiant pour les appels API.
261 @param adc Instance de l'ADC (Ads124s08).
262 @param tmux Instance du multiplexeur (Tmux1204).
263 @return bool True si la séquence est globalement considérée comme
264 réussie, False sinon.
265
266 """
267 global led_indicator
268 # Affichage du début de séquence avec horodatage
269 print(f"\n[SEQUENCER] Nouvelle séquence - {datetime.now().strftime('%H:%M:%S')}")
270
271 # Rechargement de la configuration depuis le fichier JSON
272 print(f"[CONFIG] Rechargement configuration...")
273 # Chemin vers le fichier de configuration
274 CONFIG_FILE = os.path.join(SRC_DIR, "config_module", "sensors.json")
275 try:
276     # Lecture et parsing du JSON - RECHARGEMENT COMPLET
277     cfg = load_config(CONFIG_FILE)
278     # Filtrage des canaux activés
279     enabled_channels = [ch for ch in cfg.get("channels", []) if ch.get("enabled")]
280     # Affichage du nombre de canaux actifs
281     print(f"[CONFIG] Config rechargée: {len(enabled_channels)} canaux actifs")
282     # Affichage du détail des canaux actifs
283     for ch_info in enabled_channels:
284         print(f"[CONFIG] Canal {ch_info['channel']}: {ch_info['sensor']}")
285
286     # Affichage des paramètres de temporisation rechargés
287     print(f"[CONFIG] Timeout ADC: {cfg.get('timeout_s', 180)}s")
288     print(f"[CONFIG] Délai inter-mesure: {cfg.get('inter_measure_sleep_s', 0.1)}s")
289     print(f"[CONFIG] Intervalle séquences: {cfg.get('sequence_interval_minutes', 5)} min")
290     print(f"[CONFIG] Mode automatique: {cfg.get('auto_sequence', True)}")
291
292 except Exception as e:
293     # Gestion des erreurs de rechargement
294     print(f"[CONFIG] Erreur rechargement: {e}")
295     return False
296
297 # Indiquer au gestionnaire LED que la séquence est en cours
298 if led_indicator:
299     led_indicator.set_pattern('sequence_running')
300
301 # Valeur de retour agrégée
302 sequence_success = True
303
304 # Récupération des paramètres de simulation via l'API
305 from app.api_client import get_simulation_data
306 try:
307     # Appel API pour récupérer les données de simulation
308     simulation_data = get_simulation_data(mac_address)
309     # Vérification de la validité des données
310     if simulation_data is None:
311         print("[SIMULATION] Erreur: impossible de récupérer les paramètres API")
312         # Indication LED d'erreur API
313         if led_indicator:
314             led_indicator.set_pattern('api_failed')
315         return False
316 except requests.exceptions.ConnectionError:
317     # Gestion des erreurs de connexion Internet
318     print("[SIMULATION] Erreur: Pas de connexion Internet")
319     # Indication LED de problème de connexion
320     if led_indicator:
321         led_indicator.set_pattern('no_internet')
322     return False
323
324 except Exception as e:
325     # Gestion des autres erreurs API
326     print(f"[SIMULATION] Erreur API critique: {e}")
327     # Indication LED d'erreur API
328     if led_indicator:
329         led_indicator.set_pattern('api_failed')
330     return False
331
332 # Vérification des entrées 24V (contacts secs)
333 test_24v_dry_contact()
334
335 # Boucle de traitement des canaux
336 # Compteur de canaux traités
337 channels_processed = 0
338 # Compteur de canaux en erreur
339 channels_failed = 0
340
341 # Boucle sur tous les canaux configurés
342 for entry in cfg["channels"]:
343     # Vérification du flag d'arrêt avant chaque canal
344     if is_adc_stop_requested():
345         print("[SEQUENCER] Arrêt demandé - interruption de la séquence")
346         return False
```

```
345
346 # Numéro du canal à traiter
347 ch = entry["channel"]
348 # Type de capteur configuré
349 sensor_name = entry["sensor"]
350 # État d'activation du canal
351 enabled = entry.get("enabled", True)
352
353 # Ignorer les canaux désactivés
354 if not enabled:
355     print(f"[CONFIG] Canal {ch} désactivé - ignoré")
356     continue
357
358 # Récupération du profil sonde
359 profile = get_profile(sensor_name)
360 # Incrément du compteur de canaux traités
361 channels_processed += 1
362
363 # Affichage du début de traitement du canal
364 print(f"\n[MESURE] Canal {ch} - {sensor_name}")
365 try:
366     # Sélection de la résistance de référence et configuration ADC
367     # Sélection de la résistance de référence via le multiplexeur
368     tmux.select_rref_ohm(profile["rref_ohm"])
369     # Configuration du canal ADC avec les paramètres du profil
370     adc.configure_channel(ch, profile["pga_gain"], profile["idac_uA"])
371     # Lecture et affichage du gain configuré
372     adc.read_gain()
373     # Lecture et affichage de la référence
374     adc.read_ref()
375     # Lecture et affichage du multiplexeur d'entrée
376     adc.read_inpmux()
377
378     # Mesure de la résistance
379     # Mesure de la résistance avec les paramètres du profil
380     r = adc.measure_resistance(profile["rref_ohm"], profile["pga_gain"], cfg["timeout_s"])
381     # Vérification de la validité de la mesure
382     if r is None:
383         print("[MESURE] Résistance: NaN - Erreur de mesure")
384         # Incrément du compteur d'erreurs
385         channels_failed += 1
386         # Affichage d'erreur LED et continuation
387         show_error_and_continue('measure_failed')
388         continue
389     else:
390         # Affichage de la résistance mesurée
391         print(f"[MESURE] Résistance: {r:.6f} ohms")
392
393     # Mesure de la température à partir de l'ADC
394     # Mesure de la température avec conversion selon le type de sonde
395     temperature = adc.measure_temperature(sensor_name, profile["rref_ohm"], profile["pga_gain"], cfg["timeout_s"])
396     # Vérification de la validité de la mesure de température
397     if temperature is None:
398         print("[MESURE] Température non mesurable ou saturation détectée")
399         # Incrément du compteur d'erreurs
400         channels_failed += 1
401         # Affichage d'erreur LED et continuation
402         show_error_and_continue('measure_failed')
403         continue
404     else:
405         # Affichage de la température mesurée
406         print(f"[MESURE] Température mesurée: {temperature:.2f}°C")
407
408     # Sauvegarde locale de l'état
409     save_last_state(channel=ch, resistance_ohm=r, temperature_c=temperature, error=None)
410
411     # Envoi de la mesure vers l'API distante
412     from app.api_client import send_temperature_measurement
413     try:
414         # Tentative d'envoi de la température vers l'API
415         if send_temperature_measurement(mac_address, temperature, ch):
416             print(f"[API] Température {temperature:.2f}°C envoyée avec succès pour le canal {ch}")
417         else:
418             print(f"[API] Erreur lors de l'envoi de la température pour le canal {ch}")
419             # Affichage court d'erreur API
420             show_error_and_continue('api_failed', 1)
421     except requests.exceptions.ConnectionError:
422         # Gestion spécifique des erreurs de connexion
423         print(f"[API] Erreur connexion pour canal {ch}")
424         # Affichage court d'erreur de connexion
425         show_error_and_continue('no_internet', 1)
426     except Exception as e:
427         # Gestion des autres erreurs d'envoi
428         print(f"[API] Erreur envoi canal {ch}: {e}")
429         # Affichage court d'erreur API
430         show_error_and_continue('api_failed', 1)
431
```

```
432         # Calcul et application de la simulation
433         print("[SIMULATION] Calcul et application...")
434
435         # TODO: Vérification de cohérence entre sonde API et sonde canal (temporairement désactivée)
436         # api_probe_type = simulation_data.get("probe_type")
437         # if api_probe_type != sensor_name:
438         #     print(f"[SIMULATION] ERREUR: Incohérence type de sonde!")
439         #     print(f"[SIMULATION] Sonde API: '{api_probe_type}' vs Sonde Canal {ch}: '{sensor_name}'")
440         #     print(f"[SIMULATION] La simulation est ignorée pour ce canal")
441         #     channels_failed += 1
442         #     show_error_and_continue('config_error')
443         #     continue
444
445         # Injection de la température mesurée dans les données de simulation
446         simulation_data["temperature"] = temperature
447         # Injection du type de sonde (temporaire - remplace le type API)
448         simulation_data["probe_type"] = sensor_name
449         # Affichage des paramètres de simulation
450         print(f"[SIMULATION] Paramètres: n={simulation_data['n']}, k_m={simulation_data['k_m']}")
451         # Affichage des températures utilisées
452         print(f"[SIMULATION] T_mes={temperature:.2f}°C, T_prev={simulation_data['forecast_temperature']:.2f}°C")
453
454         # Calcul de la température simulée
455         t_sim = run_temperature_simulation(simulation_data)
456         # Vérification de la validité du calcul
457         if t_sim is None:
458             print("[SIMULATION] Erreur lors du calcul de T_sim")
459             # Incrément du compteur d'erreurs
460             channels_failed += 1
461             # Affichage d'erreur LED et continuation
462             show_error_and_continue('measure_failed')
463             continue
464         # Affichage de la température simulée
465         print(f"[SIMULATION] Température simulée: {t_sim:.2f}°C")
466
467         # Mise à jour de l'état avec la température simulée
468         save_last_state(channel=ch, resistance_ohm=r, temperature_c=temperature, temperature_sim_c=t_sim, error=None)
469
470         # Conversion de la température simulée en résistance cible
471         resistance_target = convert_temperature_to_resistance(t_sim, sensor_name)
472         # Vérification de la validité de la conversion
473         if resistance_target is None:
474             print(f"[SIMULATION] Erreur conversion T_sim → résistance pour {sensor_name}")
475             # Incrément du compteur d'erreurs
476             channels_failed += 1
477             # Affichage d'erreur LED et continuation
478             show_error_and_continue('measure_failed')
479             continue
480         # Affichage de la résistance cible calculée
481         print(f"[SIMULATION] Résistance cible: {resistance_target:.2f} ohms")
482
483         # Application de la résistance simulée via le multiplexeur
484         resistance_sim = ResistanceSimulator()
485         # Tentative d'application de la résistance via le MUX
486         if resistance_sim.apply_resistance_simulation(resistance_target, ch, sensor_name):
487             print(f"[SIMULATION] Résistance appliquée via MUX sur canal {ch} (sonde: {sensor_name})")
488         else:
489             print(f"[SIMULATION] Erreur lors de l'application MUX sur canal {ch}")
490             # Incrément du compteur d'erreurs
491             channels_failed += 1
492             # Affichage d'erreur LED et continuation
493             show_error_and_continue('measure_failed')
494
495         # Pause entre canaux
496         time.sleep(cfg["inter_measure_sleep_s"])
497
498     except Exception as e:
499         # Gestion des erreurs générales de traitement du canal
500         print(f"[ERREUR] Canal {ch}: {e}")
501         # Incrément du compteur d'erreurs
502         channels_failed += 1
503         # Affichage d'erreur critique LED et continuation
504         show_error_and_continue('critical_error')
505         continue
506
507 # Évaluation finale de la séquence
508 # Cas où aucun canal n'a échoué
509 if channels_failed == 0:
510     print(f"[SEQUENCER] Séquence réussie - {channels_processed} canaux traités")
511     # Motif LED normal après succès
512     if led_indicator:
513         # Motif LED normal après succès
514         led_indicator.set_pattern('normal')
515     sequence_success = True
516 # Cas où certains canaux ont échoué mais pas tous
517 elif channels_failed < channels_processed:
518     print(f"[SEQUENCER] Séquence partielle - {channels_processed - channels_failed}/{channels_processed} canaux réussis")
```

```

519         # Indication LED d'erreur de mesure partielle
520         if led_indicator:
521             led_indicator.set_pattern('measure_failed')
522             sequence_success = True
523     # Cas où tous les canaux ont échoué
524     else:
525         print(f"[SEQUENCER] Séquence échouée - Tous les canaux ont échoué")
526         # Indication LED d'erreur critique
527         if led_indicator:
528             led_indicator.set_pattern('critical_error')
529             sequence_success = False
530
531     # Affichage de fin de séquence avec horodatage
532     print(f"[SEQUENCER] Séquence terminée - {datetime.now().strftime('%H:%M:%S')}")
533     return sequence_success
534
535
536 # -----
537 # Fonctions utilitaires de rechargement dynamique
538 # -----
539
540 def reload_timing_config():
541     """
542     @brief   Recharge les paramètres de temporisation depuis le fichier de configuration.
543     @details Lit uniquement les paramètres de temporisation pour mise à jour dynamique
544             sans perturber le reste du système. Utilisé pour permettre la modification
545             des intervalles de séquence et du mode automatique via l'interface web.
546
547     @return  Tuple (auto_sequence, sequence_interval_minutes, success)
548             où success indique si le rechargement a réussi.
549     """
550     try:
551         # Chemin vers le fichier de configuration
552         CONFIG_FILE = os.path.join(SRC_DIR, "config_module", "sensors.json")
553         # Rechargement de la configuration complète
554         cfg = load_config(CONFIG_FILE)
555
556         # Extraction des paramètres de temporisation
557         auto_sequence = cfg.get("auto_sequence", True)
558         sequence_interval_minutes = cfg.get("sequence_interval_minutes", 5)
559
560         # Log des paramètres rechargés
561         print(f"[CONFIG] Paramètres rechargés - Auto: {auto_sequence}, Intervalle: {sequence_interval_minutes}min")
562
563         return auto_sequence, sequence_interval_minutes, True
564     except Exception as e:
565         # Log d'erreur en cas d'échec
566         print(f"[CONFIG] Erreur rechargement paramètres: {e}")
567         # Retour avec valeurs par défaut
568         return True, 5, False
569
570
571 # -----
572 # Point d'entrée principal du programme
573 # -----
574
575 def main():
576     """
577     @brief   Point d'entrée principal du programme.
578     @details Parse les arguments de la ligne de commande, initialise le
579             matériel (GPIO, ADC, MUX), le gestionnaire LED, et exécute le
580             séquenceur en mode automatique ou manuel selon la configuration.
581     @return  None
582     """
583     global sequencer_running, led_indicator
584
585     # Analyse des options en ligne de commande
586     # Création du parser d'arguments
587     parser = argparse.ArgumentParser(description="Système de mesure et simulation de température")
588     # Option pour le mode manuel
589     parser.add_argument("--manual", action="store_true", help="Mode manuel (demande confirmation à chaque séquence)")
590     # Option pour une exécution unique
591     parser.add_argument("--once", action="store_true", help="Exécute une seule séquence puis s'arrête")
592     # Option pour le serveur web
593     parser.add_argument("--webui", action="store_true", help="Lance le serveur web sur le port 8080")
594     # Parsing des arguments fournis
595     args = parser.parse_args()
596
597     # Affichage du titre du programme
598     print("=== SYSTÈME DE MESURE ET SIMULATION ===")
599
600     # Chargement de la configuration
601     CONFIG_FILE = os.path.join(SRC_DIR, "config_module", "sensors.json")
602     # Chargement initial de la configuration
603     cfg = load_config(CONFIG_FILE)
604
605     # Paramètres de séquence automatique

```

```
606 # Récupération du mode automatique depuis la config
607 auto_sequence = cfg.get("auto_sequence", True)
608 # Récupération de l'intervalle entre séquences
609 sequence_interval_minutes = cfg.get("sequence_interval_minutes", 5)
610
611 # Présentation des canaux configurés
612 # Filtrage des canaux activés
613 enabled_channels = [entry for entry in cfg["channels"] if entry.get("enabled", True)]
614 # Filtrage des canaux désactivés
615 disabled_channels = [entry for entry in cfg["channels"] if not entry.get("enabled", True)]
616
617 # Affichage du nombre de canaux activés
618 print(f"[CONFIG] Canaux activés: {len(enabled_channels)}")
619 # Affichage du détail des canaux activés
620 for entry in enabled_channels:
621     print(f"[CONFIG] Canal {entry['channel']}: {entry['sensor']}")
622
623 # Affichage des canaux désactivés s'il y en a
624 if disabled_channels:
625     print(f"[CONFIG] Canaux désactivés: {len(disabled_channels)}")
626     # Affichage du détail des canaux désactivés
627     for entry in disabled_channels:
628         print(f"[CONFIG] Canal {entry['channel']}: {entry['sensor']} (OFF)")
629
630 # Affichage du mode choisi
631 # Mode manuel demandé en ligne de commande
632 if args.manual:
633     print(f"[CONFIG] Mode: MANUEL (demande confirmation)")
634 # Mode exécution unique
635 elif args.once:
636     print(f"[CONFIG] Mode: UNE SEULE FOIS")
637 # Mode automatique configuré
638 elif auto_sequence:
639     print(f"[CONFIG] Mode: AUTOMATIQUE (toutes les {sequence_interval_minutes} minutes)")
640     print(f"[CONFIG] Commandes disponibles pendant l'exécution:")
641     print(f"[CONFIG] - Appuyez sur ENTRÉE ou tapez 's' pour forcer une séquence")
642     print(f"[CONFIG] - Tapez 'q' pour quitter")
643 # Mode manuel par configuration
644 else:
645     print(f"[CONFIG] Mode: MANUEL (auto_sequence=false dans config)")
646
647 # Démarrage optionnel du serveur web en arrière-plan
648 webui_thread = None
649 # Vérification si le serveur web est demandé
650 if args.webui:
651     print(f"[WEBUI] Démarrage du serveur web sur http://localhost:8080")
652     # Création du thread pour le serveur web
653     webui_thread = threading.Thread(target=start_webui_server, daemon=True)
654     # Démarrage du thread serveur web
655     webui_thread.start()
656     # Laisser le temps au serveur de monter
657     time.sleep(2)
658
659 # Affichage des motifs LED disponibles (référence)
660 print(f"\n[LED] Patterns d'erreur disponibles:")
661 print(f"[LED] - NORMAL: LED allumée en continu")
662 print(f"[LED] - SEQUENCE: Clignotement continu pendant mesure")
663 print(f"[LED] - NO_INTERNET: 1 clignotement sur 10s (pas de connexion Internet)")
664 print(f"[LED] - API_FAILED: 2 clignotements sur 10s (erreur API/connexion)")
665 print(f"[LED] - MEASURE_FAILED: 3 clignotements sur 10s (erreur mesure ADC/résistance)")
666 print(f"[LED] - CRITICAL_ERROR: 4 clignotements sur 10s (erreur critique système)")
667 print(f"[LED] - CONFIG_ERROR: 5 clignotements sur 10s (erreur de configuration)")
668
669 # Ligne vide pour la lisibilité
670 print()
671
672 # Installation du handler pour Ctrl+C
673 signal.signal(signal.SIGINT, signal_handler)
674
675 # Remise à zéro du flag d'arrêt ADC au démarrage
676 reset_adc_stop_flag()
677 print("[ADC] Système d'interruption ADC initialisé")
678
679 # Initialisation du gestionnaire LED
680 # Création de l'instance de gestion LED
681 led_indicator = create_led_indicator(GPIO_CHIP_PATH, FRONT_LED)
682 # Vérification de l'initialisation LED
683 if led_indicator:
684     print("[LED] Système LED initialisé - LED normale ON")
685 else:
686     print("[LED] Erreur : Impossible d'initialiser la LED")
687
688 # Initialisation du matériel GPIO/ADC/MUX
689 # Initialisation du relais de commande principal
690 relay = GPIO(GPIO_CHIP_PATH, CMD_RELAY, "out")
691 # Activer le relais principal
692 relay.write(True)
```

```
693 # Instance du multiplexeur
694 tmux = Tmux1204()
695 # Instance de l'ADC
696 adc = Ads124s08()
697 # Récupération de l'adresse MAC depuis la configuration
698 mac_address = cfg.get("mac_address", "0030DEABCDEF")
699
700 try:
701     # Activation du séquenceur principal
702     sequencer_running = True
703     # Calcul du temps de la première séquence
704     next_sequence_time = datetime.now()
705
706     # Thread d'écoute utilisateur pour le mode automatique
707     input_thread = None
708     # Vérification si le thread d'écoute est nécessaire
709     if auto_sequence and not args.manual and not args.once:
710         # Création du thread de surveillance des entrées
711         input_thread = threading.Thread(target=input_monitor, daemon=True)
712         # Démarrage du thread de surveillance
713         input_thread.start()
714         print("[MANUAL] Thread de surveillance des entrées démarré")
715
716     # Boucle principale du séquenceur
717     # Boucle tant que le séquenceur est actif et pas d'arrêt demandé
718     while sequencer_running and not stop_event.is_set():
719         # Récupération du temps actuel
720         current_time = datetime.now()
721
722         # Initialisation des variables de contrôle
723         should_run = False
724         sequence_reason = ""
725
726         # Déclenchement manuel depuis input_monitor
727         # Vérification d'un déclenchement manuel
728         if force_sequence.is_set():
729             should_run = True
730             sequence_reason = "MANUELLE"
731             # Remise à zéro du flag de déclenchement manuel
732             force_sequence.clear()
733             # Rechargement des paramètres avant reprogrammation du cycle
734             auto_sequence_updated, sequence_interval_minutes_updated, reload_success = reload_timing_config()
735             if reload_success:
736                 auto_sequence = auto_sequence_updated
737                 sequence_interval_minutes = sequence_interval_minutes_updated
738             # Reprogrammation du cycle automatique si applicable
739             if auto_sequence and not args.manual:
740                 # Recalcule le temps de prochaine séquence avec le nouvel intervalle
741                 next_sequence_time = current_time + timedelta(minutes=sequence_interval_minutes)
742                 print(f"[SEQUENCER] Cycle automatique redémarré - Prochaine séquence à: {next_sequence_time.strftime('%H:%M:%S')}")
743             # Mode exécution unique
744             elif args.once:
745                 should_run = True
746                 sequence_reason = "UNIQUE"
747                 # Arrêt après une exécution
748                 sequencer_running = False
749             # Mode manuel interactif
750             elif args.manual or not auto_sequence:
751                 print(f"[SEQUENCER] Appuyez sur Entrée pour lancer une séquence (Ctrl+C pour arrêter)...")
752                 try:
753                     # Attente d'une entrée utilisateur
754                     input()
755                     should_run = True
756                     sequence_reason = "MANUELLE"
757                 except EOFError:
758                     break
759             # Mode automatique basé sur l'horloge
760             else:
761                 # Vérification si c'est l'heure d'une nouvelle séquence
762                 if current_time >= next_sequence_time:
763                     should_run = True
764                     sequence_reason = "AUTOMATIQUE"
765                     # Programmation de la prochaine séquence
766                     next_sequence_time = current_time + timedelta(minutes=sequence_interval_minutes)
767                     print(f"[SEQUENCER] Prochaine séquence automatique programmée à: {next_sequence_time.strftime('%H:%M:%S')}")
768
769         # Exécution de la séquence si déclenchée
770         if should_run:
771             # Rechargement des paramètres de temporisation avant séquence (si pas déjà fait)
772             if sequence_reason != "MANUELLE": # Pour les manuelles, déjà fait plus haut
773                 print(f"[SEQUENCER] Rechargement paramètres temporisation...")
774                 auto_sequence_updated, sequence_interval_minutes_updated, reload_success = reload_timing_config()
775                 if reload_success:
776                     # Sauvegarde de l'ancien intervalle pour comparaison
777                     old_interval = sequence_interval_minutes
778                     # Mise à jour des paramètres pour les prochains cycles
779                     auto_sequence = auto_sequence_updated
```

```

780         sequence_interval_minutes = sequence_interval_minutes_updated
781         # Recalcul du prochain cycle si en mode automatique et intervalle modifié
782         if sequence_reason == "AUTOMATIQUE" and sequence_interval_minutes_updated != old_interval:
783             next_sequence_time = current_time + timedelta(minutes=sequence_interval_minutes)
784             print(f"[SEQUENCER] Intervalle mis à jour: {old_interval}min → {sequence_interval_minutes}min")
785
786         print(f"[SEQUENCER] Démarrage séquence {sequence_reason} - {current_time.strftime('%H:%M:%S')}")
787         # Appel de la fonction de séquence de mesure
788         success = run_measurement_sequence(cfg, mac_address, adc, tmux)
789         # Vérification du succès de la séquence
790         if success:
791             print(f"[SEQUENCER] Séquence {sequence_reason} terminée avec succès")
792         else:
793             print("[SEQUENCER] Erreur lors de la séquence")
794
795         # Pause courte pour permettre la réactivité aux entrées
796         # Pause seulement en mode automatique continu
797         if auto_sequence and not args.manual and not args.once and sequencer_running:
798             time.sleep(1)
799
800     except KeyboardInterrupt:
801         # Gestion de l'interruption par Ctrl+C
802         print("\n[SEQUENCER] Arrêt demandé par l'utilisateur...")
803     finally:
804         # Arrêt du séquenceur
805         sequencer_running = False
806         # Déclenchement de l'événement d'arrêt
807         stop_event.set()
808
809         # Arrêt propre du thread d'entrée si actif
810         # Vérification de l'existence et de l'activité du thread
811         if input_thread and input_thread.is_alive():
812             print("[MANUAL] Arrêt du thread de surveillance...")
813             # Attente de la fin du thread avec timeout
814             input_thread.join(timeout=1)
815
816         # Tentatives de fermeture des périphériques matériels
817         # Fermeture de l'ADC
818         try:
819             adc.close()
820         except Exception:
821             pass
822         # Fermeture du multiplexeur
823         try:
824             tmux.close()
825         except Exception:
826             pass
827         # Fermeture du gestionnaire LED
828         try:
829             if led_indicator:
830                 led_indicator.close()
831         except Exception:
832             pass
833         # Désactivation et fermeture du relais
834         try:
835             relay.write(False)
836             relay.close()
837         except Exception:
838             pass
839
840         # Message final de confirmation d'arrêt
841         print("[SEQUENCER] Système arrêté proprement")
842
843 # -----
844 # Point d'entrée principal
845 # -----
846
847 if __name__ == "__main__":
848     main()
849

```

app\mesure_24v_dry_contact.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      mesure_24v_dry_contact.py
4  @brief     Module de test des entrées/sorties 24V pour contacts secs.
5  @details   Ce module fournit des fonctions pour tester et valider les
6             contacts secs via les interfaces I/O 24V. Permet de détecter
7             l'état des contacts (ouvert/fermé) sur deux canaux indépendants
8             avec gestion des GPIO et temporisations appropriées.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 from __future__ import annotations
18
19 # Fonctions temporelles pour les délais de stabilisation
20 import time
21 # Bibliothèque pour la gestion des GPIO
22 from periphery import GPIO
23 # Importation des constantes matérielles pour les chemins GPIO
24 from pins_cm5 import GPIO_CHIP_PATH
25
26 # -----
27 # Constantes du module
28 # -----
29
30 # Sortie 24V canal 1
31 CM_24V_OUT_1 = 13
32 # Sense 24V canal 1
33 CM_24V_SENSE_1 = 14
34 # Sortie 24V canal 2
35 CM_24V_OUT_2 = 15
36 # Sense 24V canal 2
37 CM_24V_SENSE_2 = 16
38
39 # -----
40 # Variables globales d'état
41 # -----
42
43 # 0=ouvert, 1=fermé
44 io_24v_ch1_ok = 0
45 # 0=ouvert, 1=fermé
46 io_24v_ch2_ok = 0
47
48 # -----
49 # Fonctions utilitaires
50 # -----
51
52 def test_24v_dry_contact():
53     """
54     @brief    Test des contacts secs I/O 24V.
55     @details  Effectue un test complet des deux canaux de contacts secs 24V.
56               Active séquentiellement chaque sortie et lit l'état du capteur
57               correspondant pour déterminer si le contact est fermé ou ouvert.
58               Met à jour les variables globales d'état et affiche les résultats.
59
60     @return   Tuple contenant l'état des deux canaux (ch1_state, ch2_state).
61               Valeurs: 0=ouvert, 1=fermé.
62
63     @note     Utilise des temporisations pour la stabilisation des signaux.
64     """
65     global io_24v_ch1_ok, io_24v_ch2_ok
66
67     # Affichage du début du test
68     print("\n[Test-24V] === Test contacts secs I/O 24V ===")
69
70     try:
71         # Configuration GPIO
72         # Configuration de la sortie 24V canal 1
73         out1 = GPIO(GPIO_CHIP_PATH, CM_24V_OUT_1, "out")
74         # Configuration de la sortie 24V canal 2
75         out2 = GPIO(GPIO_CHIP_PATH, CM_24V_OUT_2, "out")
76         # Configuration de l'entrée sense canal 1
77         sense1 = GPIO(GPIO_CHIP_PATH, CM_24V_SENSE_1, "in")
78         # Configuration de l'entrée sense canal 2
79         sense2 = GPIO(GPIO_CHIP_PATH, CM_24V_SENSE_2, "in")
80
81         # Test Canal 1
82         # Mise au repos
83         out1.write(False)
84         # Temporisation de stabilisation

```



```
85     time.sleep(0.01)
86     # Activation
87     out1.write(True)
88     # Stabilisation
89     time.sleep(0.02)
90     # Lecture de l'état du capteur
91     state1 = sense1.read()
92     # Désactivation
93     out1.write(False)
94
95     # Interprétation du résultat canal 1
96     if state1:
97         print("[TEST-24V] Canal 1: fermé")
98         # Contact fermé détecté
99         io_24v_ch1_ok = 1
100     else:
101         print("[TEST-24V] Canal 1: ouvert")
102         # Contact ouvert détecté
103         io_24v_ch1_ok = 0
104
105     # Test Canal 2
106     # Mise au repos
107     out2.write(False)
108     # Temporisation de stabilisation
109     time.sleep(0.01)
110     # Activation
111     out2.write(True)
112     # Stabilisation
113     time.sleep(0.02)
114     # Lecture de l'état du capteur
115     state2 = sense2.read()
116     # Désactivation
117     out2.write(False)
118
119     # Interprétation du résultat canal 2
120     if state2:
121         print("[TEST-24V] Canal 2: fermé")
122         # Contact fermé détecté
123         io_24v_ch2_ok = 1
124     else:
125         print("[TEST-24V] Canal 2: ouvert")
126         # Contact ouvert détecté
127         io_24v_ch2_ok = 0
128
129     # Nettoyage
130     # Fermeture des GPIO de sortie
131     out1.close()
132     out2.close()
133     # Fermeture des GPIO d'entrée
134     sense1.close()
135     sense2.close()
136
137     # Affichage du résultat final
138     print(f"[TEST-24V] Résultat: CH1={io_24v_ch1_ok}, CH2={io_24v_ch2_ok}")
139
140     # Retour du tuple d'état
141     return (io_24v_ch1_ok, io_24v_ch2_ok)
142
143 except Exception as e:
144     # Gestion des erreurs de test
145     print(f"[TEST-24V] Erreur: {e}")
146     # Remise à zéro des états en cas d'erreur
147     io_24v_ch1_ok = 0
148     io_24v_ch2_ok = 0
149     # Retour d'état d'erreur
150     return (0, 0)
151
152 def get_dry_contact_status():
153     """
154     @brief   Retourne l'état actuel des contacts secs.
155     @details Fonction utilitaire qui retourne les derniers états mesurés
156             des contacts secs sans effectuer de nouveau test. Utilise
157             les variables globales mises à jour par test_24v_dry_contact().
158
159     @return  Dictionnaire contenant l'état des deux canaux.
160             Format: {"ch1": 0|1, "ch2": 0|1}
161             Valeurs: 0=ouvert, 1=fermé.
162     """
163     # Retour du dictionnaire d'état des contacts
164     return {
165         "ch1": io_24v_ch1_ok,
166         "ch2": io_24v_ch2_ok
167     }
```

app\pins_cm5.py

```
1  # -*- coding: utf-8 -*-
2  """
3  @file      pins_cm5.py
4  @brief     Mapping GPIO et SPI fixés pour la plateforme CM5.
5  @details   Ce module définit tous les assignements de broches GPIO et
6             les paramètres SPI fixes pour la carte CM5. Ces constantes
7             ne doivent pas être modifiées via la configuration utilisateur
8             car elles correspondent au routage matériel de la carte.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17
18 # -----
19 # GPIO principal et configuration système
20 # -----
21
22 # Chemin du contrôleur GPIO principal sur le système Linux
23 GPIO_CHIP_PATH = "/dev/gpiochip0"
24
25
26 # -----
27 # Multiplexeur ADG731 - Chip Select
28 # -----
29
30 # Broches de sélection CS pour les 4 multiplexeurs ADG731
31 MUX_CS_4 = 2
32 # Broches de sélection CS pour les 3 multiplexeurs ADG731
33 MUX_CS_3 = 3
34 # Broches de sélection CS pour les 2 multiplexeurs ADG731
35 MUX_CS_2 = 7
36 # Broches de sélection CS pour les 1 multiplexeurs ADG731
37 MUX_CS_1 = 8
38
39
40 # -----
41 # Broches SPI multiplexeur (dummy)
42 # -----
43
44 # Broche MOSI dummy pour le multiplexeur (non utilisée directement)
45 MUX_MOSI_DUMMY = 10
46 # Broche SCLK dummy pour le multiplexeur (non utilisée directement)
47 MUX_SCLK_DUMMY = 11
48
49
50 # -----
51 # Commande et mesure 24V
52 # -----
53
54 # Sortie de commande 24V pour le canal 1
55 CM_24V_OUT_1 = 13
56 # Entrée de mesure (sense) 24V pour le canal 1
57 CM_24V_SENSE_1 = 14
58 # Sortie de commande 24V pour le canal 2
59 CM_24V_OUT_2 = 15
60 # Entrée de mesure (sense) 24V pour le canal 2
61 CM_24V_SENSE_2 = 16
62
63
64 # -----
65 # Relais de commande
66 # -----
67
68 # Broche de commande du relais principal de puissance
69 CMD_RELAY = 17
70
71
72 # -----
73 # ADC ADS124S08 - Configuration SPI
74 # -----
75
76 # Chip Select ADC dummy (géré par spidev1.0)
77 CS_ADC_DUMMY = 18
78 # Broche MISO ADC dummy (gérée par le driver SPI)
79 SPI_MISO_ADC_DMY = 19
80 # Broche MOSI ADC dummy (gérée par le driver SPI)
81 SPI_MOSI_ADC_DMY = 20
82 # Broche SCLK ADC dummy (gérée par le driver SPI)
83 SPI_SCLK_ADC_DMY = 21
84
```

```
85
86 # -----
87 # ADC ADS124S08 - Signaux de contrôle
88 # -----
89
90 # Signal Data Ready de l'ADC (interruption de fin de conversion)
91 ADC_DRDY = 22
92 # Signal START/SYNC de l'ADC (non utilisé, démarrage via SPI)
93 ADC_START_SYNC = 23
94
95
96 # -----
97 # ADC ADS124S08 - Sélection d'entrées
98 # -----
99
100 # Bit A0 de sélection du canal analogique de l'ADC
101 ADC_A0 = 24
102 # Bit A1 de sélection du canal analogique de l'ADC
103 ADC_A1 = 25
104
105
106 # -----
107 # Indicateurs visuels
108 # -----
109
110 # LED de façade pour indication d'état du système
111 FRONT_LED = 27
112
113
114 # -----
115 # Configuration des bus SPI
116 # -----
117
118 # Numéro du bus SPI0 (pour multiplexeurs éventuels)
119 SPI0_BUS = 0
120 # Numéro du bus SPI1 (pour ADC ADS124S08)
121 SPI1_BUS = 1
122
123
124 # -----
125 # Configuration des devices SPI
126 # -----
127
128 # Device 0 sur le bus SPI0
129 SPI0_DEV0 = 0
130 # Device 1 sur le bus SPI0
131 SPI0_DEV1 = 1
132 # Device 0 sur le bus SPI1 (correspond à /dev/spidev1.0 pour l'ADC)
133 SPI1_DEV0 = 0
134
135
136 # -----
137 # Vitesses de communication SPI
138 # -----
139
140 # Fréquence SPI pour la communication avec les multiplexeurs ADG731 [Hz]
141 SPI_MUX_SPEED_HZ = 100000
142 # Fréquence SPI pour la communication avec l'ADC ADS124S08 [Hz]
143 SPI_ADC_SPEED_HZ = 100000
144
```

app\resistance_simulation.py

```
1  # -*- coding: utf-8 -*-
2  """
3  @file      resistance_simulation.py
4  @brief     Module pour commander le MUX de simulation de résistance.
5  @details   Ce module permet d'appliquer une résistance simulée R_sim via
6             le réseau de résistances contrôlé par multiplexeur ADG731.
7             Il gère le calcul des commutateurs à activer, la commande du
8             multiplexeur, et la validation des valeurs appliquées.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 # Import des modules système standard
18 import time
19 # Import des annotations de type pour la documentation
20 from typing import Optional, List, Dict, Tuple
21 # Import du driver multiplexeur ADG731
22 from app.hw_mux_adg731 import Adg731MuxSpi
23 # Import des utilitaires de mapping de résistances
24 from app.resistance_mapping import get_channel_mux_map, find_closest_channel
25
26
27 # -----
28 # Classes d'exception personnalisées
29 # -----
30
31 class ResistanceSimulationError(Exception):
32     """
33     @brief   Exception pour les erreurs de simulation de résistance.
34     @details Exception spécialisée levée lors d'erreurs dans le processus
35             de simulation de résistance via le multiplexeur.
36     """
37     pass
38
39
40 # -----
41 # Classe principale de simulation
42 # -----
43
44 class ResistanceSimulator:
45     """
46     @brief   Contrôleur pour la simulation de résistance via MUX.
47     @details Classe principale qui encapsule toute la logique de simulation
48             de résistance en utilisant le multiplexeur ADG731 et les
49             mappings de résistances calibrées.
50     """
51
52     def __init__(self, speed_hz: int = 100000):
53         """
54         @brief   Initialise le simulateur de résistance.
55         @details Constructeur qui configure le multiplexeur ADG731 avec
56                 la vitesse SPI spécifiée et initialise les variables
57                 d'état du simulateur.
58
59         @param speed_hz  Vitesse de communication SPI en Hz (défaut: 100kHz).
60
61         @exception ResistanceSimulationError si l'initialisation du MUX échoue.
62         """
63         # Référence vers le multiplexeur ADG731
64         self.mux = None
65         # Vitesse de communication SPI configurée
66         self.speed_hz = speed_hz
67         # Résistance actuellement appliquée (None si aucune)
68         self.current_resistance = None
69         # Liste des canaux MUX actuellement actifs
70         self.active_channels = []
71
72         try:
73             # Initialisation du multiplexeur ADG731 avec la vitesse SPI
74             self.mux = Adg731MuxSpi(speed_hz=speed_hz)
75             # Affichage de confirmation d'initialisation
76             print(f"[SIM_R] Simulateur de résistance initialisé (SPI: {speed_hz} Hz)")
77         except Exception as e:
78             # Affichage d'erreur en cas d'échec d'initialisation
79             print(f"[SIM_R] ERREUR d'initialisation du MUX: {e}")
80             # Levée d'exception avec contexte détaillé
81             raise ResistanceSimulationError(f"Impossible d'initialiser le MUX: {e}")
82
83     def close(self):
84         """
```

```
85     @brief Ferme les connexions du simulateur.
86     @details Méthode de nettoyage qui ferme proprement la connexion
87             SPI avec le multiplexeur et libère les ressources.
88     """
89     # Vérification de l'existence de la connexion MUX
90     if self.mux:
91         try:
92             # Fermeture de la connexion SPI
93             self.mux.close()
94             # Confirmation de fermeture
95             print("[SIM_R] Simulateur fermé")
96         except Exception as e:
97             # Affichage d'erreur en cas de problème de fermeture
98             print(f"[SIM_R] Erreur lors de la fermeture: {e}")
99
100 def calculate_parallel_resistance(self, resistances: List[float]) -> float:
101     """
102     @brief Calcule la résistance équivalente de résistances en parallèle.
103     @details Applique la formule  $1/R_{eq} = 1/R_1 + 1/R_2 + \dots + 1/R_n$ 
104             pour calculer la résistance équivalente d'un ensemble
105             de résistances connectées en parallèle.
106
107     @param resistances Liste des valeurs de résistance en parallèle [Ohms].
108
109     @return Résistance équivalente calculée [Ohms].
110             Retourne l'infini si la liste est vide.
111     """
112     # Cas d'une liste vide: résistance infinie
113     if not resistances:
114         return float('inf')
115
116     # Cas d'une seule résistance: pas de calcul parallèle
117     if len(resistances) == 1:
118         return resistances[0]
119
120     # Calcul de la somme des inverses ( $1/R_1 + 1/R_2 + \dots$ )
121     reciprocal_sum = sum(1.0 / r for r in resistances)
122     # Application de la formule:  $R_{eq} = 1 / (\text{somme des } 1/R_i)$ 
123     return 1.0 / reciprocal_sum
124
125 def apply_resistance_simulation(self, target_resistance: float, measurement_channel: int, sensor_type: str = "Ni1000 TK5000") -> bool:
126     """
127     @brief Applique une résistance simulée via le MUX sur le canal de mesure.
128     @details Fonction principale qui recherche le canal MUX optimal pour
129             simuler la résistance cible, configure le multiplexeur et
130             mémorise l'état de la simulation.
131
132     @param target_resistance Résistance à simuler [Ohms].
133     @param measurement_channel Canal sur lequel la mesure a été faite (info).
134     @param sensor_type Type de sonde pour sélectionner la carte MUX.
135
136     @return True si succès, False en cas d'erreur.
137     """
138     # Affichage de l'en-tête de la procédure de simulation
139     print(f"\n[SIM_R] === APPLICATION RÉSISTANCE SIMULÉE ===")
140     # Affichage de la résistance cible avec précision
141     print(f"[SIM_R] Résistance cible: {target_resistance:.2f} Ω")
142     # Affichage du canal de mesure pour traçabilité
143     print(f"[SIM_R] Canal de mesure: {measurement_channel}")
144     # Affichage du type de sonde sélectionné
145     print(f"[SIM_R] Type de sonde: {sensor_type}")
146
147     # Vérification de l'initialisation du multiplexeur
148     if self.mux is None:
149         # Erreur critique: MUX non initialisé
150         print(f"[SIM_R] ERREUR MUX non initialisé")
151         return False
152
153     # Recherche du canal MUX optimal en utilisant le mapping dynamique
154     try:
155         # Appel de la fonction de recherche du canal le plus proche
156         channel, actual_resistance, error_percent = find_closest_channel(target_resistance, sensor_type)
157
158         # Vérification que la recherche a trouvé un canal valide
159         if channel is None:
160             # Aucun canal MUX disponible pour cette résistance
161             print(f"[SIM_R] ERREUR Aucun canal MUX trouvé pour {target_resistance:.1f}Ω")
162             return False
163
164         # Affichage du canal MUX sélectionné
165         print(f"[SIM_R] Canal MUX trouvé: {channel}")
166         # Affichage de la résistance réelle vs cible
167         print(f"[SIM_R] Résistance MUX: {actual_resistance}Ω (cible: {target_resistance:.1f}Ω)")
168         # Affichage de l'erreur relative en pourcentage
169         print(f"[SIM_R] Erreur: {error_percent:.1f}%")
170
171     except KeyError as e:
```

```
172         # Erreur: type de sonde non supporté
173         print(f"[SIM_R] ERREUR {e}")
174         return False
175
176     try:
177         # Configuration du multiplexeur (toujours carte 0 par défaut)
178         board_index = 0
179         # Affichage de l'action de configuration MUX
180         print(f"[SIM_R] Application MUX canal {channel} (carte {board_index})")
181         # Activation du canal sélectionné sur la carte MUX
182         self.mux.set_channel(board_index, channel)
183
184         # Mémorisation de l'état actuel de simulation
185         self.active_channels = [channel]
186         # Sauvegarde de la résistance réellement appliquée
187         self.current_resistance = actual_resistance
188
189         # Confirmation de succès avec résistance appliquée
190         print(f"[SIM_R] SUCCES Résistance appliquée: {actual_resistance}Ω sur canal MUX {channel}")
191         return True
192
193     except Exception as e:
194         # Erreur lors de la communication avec le MUX
195         print(f"[SIM_R] ERREUR lors de l'application: {e}")
196         return False
197
198     def get_current_simulation(self) -> Dict[str, any]:
199         """
200         @brief Retourne l'état actuel de la simulation.
201         @details Fonction d'interrogation qui fournit un dictionnaire
202                 contenant toutes les informations sur l'état actuel
203                 de la simulation de résistance.
204
205         @return Dictionnaire contenant les informations actuelles:
206                 - resistance: valeur appliquée [Ohms] ou None
207                 - active_channels: liste des canaux MUX actifs
208                 - mux_available: disponibilité du multiplexeur
209         """
210         # Construction du dictionnaire d'état avec résistance actuelle
211         return {
212             "resistance": self.current_resistance,
213             # Copie de la liste des canaux actifs (protection contre modification)
214             "active_channels": self.active_channels.copy() if self.active_channels else [],
215             # Indication de disponibilité du MUX (toujours True si initialisé)
216             "mux_available": True
217         }
218
219     def validate_resistance_applied(self, expected_resistance: float, tolerance: float = 0.10) -> bool:
220         """
221         @brief Valide que la résistance appliquée correspond à l'attendu.
222         @details Fonction de validation qui compare la résistance actuellement
223                 appliquée avec la valeur attendue en tenant compte d'une
224                 tolérance relative spécifiée.
225
226         @param expected_resistance Résistance attendue pour la validation [Ohms].
227         @param tolerance Tolérance relative acceptée (défaut: 10%).
228
229         @return True si la résistance est dans la tolérance.
230         """
231         # Vérification qu'une résistance est actuellement appliquée
232         if self.current_resistance is None:
233             # Erreur: aucune résistance n'est actuellement simulée
234             print(f"[SIM_R] ERREUR Aucune résistance actuellement appliquée")
235             return False
236
237         # Calcul de l'erreur relative entre résistance appliquée et attendue
238         error = abs(self.current_resistance - expected_resistance) / expected_resistance
239
240         # Comparaison de l'erreur avec la tolérance spécifiée
241         if error <= tolerance:
242             # Validation réussie: affichage des détails
243             print(f"[SIM_R] VALIDATION OK: {self.current_resistance:.2f}Ω (attendu: {expected_resistance:.2f}Ω, erreur: {error*100:.1f}%)")
244             return True
245         else:
246             # Validation échouée: affichage des détails d'erreur
247             print(f"[SIM_R] VALIDATION KO: {self.current_resistance:.2f}Ω (attendu: {expected_resistance:.2f}Ω, erreur: {error*100:.1f}%)")
248             return False
249
250     def reset_simulation(self):
251         """
252         @brief Remet la simulation à zéro (désactive tous les canaux).
253         @details Procédure de nettoyage qui désactive tous les canaux de
254                 tous les multiplexeurs et remet l'état de simulation
255                 à sa configuration initiale.
256         """
257         # Affichage du début de la procédure de remise à zéro
258         print("[SIM_R] Remise à zéro de la simulation")
```

```
259
260 # Vérification de l'initialisation du multiplexeur
261 if self.mux is None:
262     return
263
264 try:
265     # Désactivation systématique de tous les canaux
266     for board in range(4):
267         # Parcours de tous les canaux de chaque carte
268         for channel in range(32):
269             try:
270                 # Tentative de désactivation du canal
271                 self.mux.set_channel(board, channel)
272             except Exception:
273                 # Ignore les erreurs individuelles de canal
274                 pass
275
276     # Remise à zéro des variables d'état
277     self.active_channels = []
278     # Effacement de la résistance mémorisée
279     self.current_resistance = None
280     # Confirmation de succès de la remise à zéro
281     print("[SIM_R] SUCCES Simulation remise à zéro")
282
283 except Exception as e:
284     # Erreur lors de la procédure de remise à zéro
285     print(f"[SIM_R] ERREUR lors de la remise à zéro: {e}")
286
287
288 # -----
289 # Fonctions utilitaires globales
290 # -----
291
292 def apply_resistance_simulation(resistance_ohm: float) -> bool:
293     """
294     @brief   Fonction utilitaire pour appliquer une résistance simulée.
295     @details Fonction de commodité qui encapsule la création d'un simulateur,
296             l'application d'une résistance et le nettoyage automatique.
297             Utilise le pattern RAII pour la gestion des ressources.
298
299     @param resistance_ohm Résistance à appliquer [Ohms].
300
301     @return      True si l'application a réussi, False sinon.
302     """
303     # Initialisation du simulateur à None pour le nettoyage
304     simulator = None
305     try:
306         # Création d'une instance du simulateur
307         simulator = ResistanceSimulator()
308         # Application de la résistance avec paramètres par défaut
309         return simulator.apply_resistance_simulation(resistance_ohm)
310     except Exception as e:
311         # Gestion d'erreur avec affichage du contexte
312         print(f"[SIM_R] ERREUR dans apply_resistance_simulation: {e}")
313         return False
314     finally:
315         # Nettoyage automatique du simulateur si créé
316         if simulator:
317             simulator.close()
318
319
320 # -----
321 # Section de test et démonstration
322 # -----
323 if __name__ == "__main__":
324     """
325     @brief   Point d'entrée pour les tests de développement.
326     @details Section de test qui démontre l'utilisation du simulateur
327             avec différents cas d'usage et validation des résultats.
328     """
329     # Affichage de l'en-tête des tests de développement
330     print("=== Test du simulateur de résistance ===")
331
332     # Création d'une instance de simulateur pour les tests
333     sim = ResistanceSimulator()
334
335     # Test 1: Application d'une résistance simple de 1000 Ohms
336     print("\nTest 1: Application de 1000Ω")
337     # Appel de la fonction de simulation avec résistance cible
338     success = sim.apply_resistance_simulation(1000.0)
339     # Affichage du résultat du test avec évaluation booléenne
340     print(f"Résultat: {'Succès' if success else 'Échec'}")
341
342     # Test 2: Application d'une résistance complexe (parallèle calculé)
343     print("\nTest 2: Application de 667Ω (parallèle de 1k et 2k)")
344     # Test avec une valeur nécessitant une approximation par le MUX
345     success = sim.apply_resistance_simulation(667.0)
```

```
346 # Évaluation et affichage du résultat
347 print(f"Résultat: {'Succès' if success else 'Échec'}")
348
349 # Test 3: Validation de la résistance appliquée avec tolérance
350 print("\nTest 3: Validation de la résistance appliquée")
351 # Vérification qu'une résistance est effectivement appliquée
352 if sim.current_resistance:
353     # Validation avec tolérance élargie de 15%
354     valid = sim.validate_resistance_applied(667.0, tolerance=0.15)
355     # Affichage du résultat de validation
356     print(f"Validation: {'OK' if valid else 'KO'}")
357
358 # Procédure de nettoyage et fermeture
359 sim.reset_simulation()
360 # Fermeture propre du simulateur
361 sim.close()
```


app\sensor_profiles.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      sensor_profiles.py
4  @brief     Profils capteurs extraits de 'Calcul ADC.xlsx' (Feuill).
5  @details   Ce module contient les profils de configuration pour différents
6             types de capteurs de température avec leurs paramètres ADC
7             optimisés (Rref, IDAC, PGA) et leurs tables de conversion
8             température/résistance. Les données proviennent des calculs
9             d'optimisation du fichier Excel de référence.
10 @author    Léo Mendes
11 @project   2510_RegulationsChauffage
12 @Mandant   Oblo_solution
13 @date      2025-09-18
14 @version   1.0.0
15 @copyright Copyright (c) 2025
16 """
17
18
19 # -----
20 # Profils de configuration ADC par capteur
21 # -----
22
23 # Dictionnaire principal contenant les profils de configuration pour chaque capteur
24 # Structure: paramètres optimisés pour l'acquisition ADC
25 # - rref_ohm: Résistance de référence sélectionnée via TMUX1204 (A1 A0: 00=2.2k, 01=2.7k, 10=10k, 11=100k)
26 # - idac_uA: Courant IDAC en microampères pour l'excitation du capteur
27 # - pga_gain: Gain PGA programmable (1,2,4,8,16,32,64,128)
28 # Sortie du système: résistance [Ohm] uniquement
29 SENSOR_PROFILES = {
30     # Profil pour le capteur De Dietrich AF60 (sonde industrielle)
31     "De Dietrich AF60": {
32         # Résistance de référence optimale pour ce type de capteur [Ohms]
33         "rref_ohm": 2200,
34         # Courant d'excitation IDAC configuré [microampères]
35         "idac_uA": 250,
36         # Gain d'amplification programmable sélectionné
37         "pga_gain": 1
38     },
39     # Profil pour le capteur Siemens QAC32 (sonde qualité d'air)
40     "Siemens QAC32": {
41         # Résistance de référence optimale pour ce type de capteur [Ohms]
42         "rref_ohm": 2700,
43         # Courant d'excitation IDAC configuré [microampères]
44         "idac_uA": 250,
45         # Gain d'amplification programmable sélectionné
46         "pga_gain": 4
47     },
48     # Profil pour le capteur PT1000 (sonde platine de précision)
49     "PT1000": {
50         # Résistance de référence optimale pour ce type de capteur [Ohms]
51         "rref_ohm": 10000,
52         # Courant d'excitation IDAC configuré [microampères]
53         "idac_uA": 100,
54         # Gain d'amplification programmable sélectionné
55         "pga_gain": 8
56     },
57     # Profil pour le capteur Ni1000 TK5000 (sonde nickel coefficient standard)
58     "Ni1000 TK5000": {
59         # Résistance de référence optimale pour ce type de capteur [Ohms]
60         "rref_ohm": 10000,
61         # Courant d'excitation IDAC configuré [microampères]
62         "idac_uA": 100,
63         # Gain d'amplification programmable sélectionné
64         "pga_gain": 8
65     },
66     # Profil pour le capteur Ni1000 TK6180 (sonde nickel coefficient élevé)
67     "Ni1000 TK6180": {
68         # Résistance de référence optimale pour ce type de capteur [Ohms]
69         "rref_ohm": 10000,
70         # Courant d'excitation IDAC configuré [microampères]
71         "idac_uA": 50,
72         # Gain d'amplification programmable sélectionné
73         "pga_gain": 8
74     },
75     # Profil pour le capteur NTC 1k 3528 (thermistance 1kOhm)
76     "NTC 1k 3528": {
77         # Résistance de référence optimale pour ce type de capteur [Ohms]
78         "rref_ohm": 10000,
79         # Courant d'excitation IDAC configuré [microampères]
80         "idac_uA": 100,
81         # Gain d'amplification programmable sélectionné
82         "pga_gain": 1
83     },
84     # Profil pour le capteur KTY81-210 (capteur silicium)

```

```

85     "KTY81-210": {
86         # Résistance de référence optimale pour ce type de capteur [Ohms]
87         "rref_ohm": 10000,
88         # Courant d'excitation IDAC configuré [microampères]
89         "idac_uA": 100,
90         # Gain d'amplification programmable sélectionné
91         "pga_gain": 4
92     },
93     # Profil pour le capteur NTC 2k 3390 (thermistance 2k0hm haute précision)
94     "NTC 2k 3390": {
95         # Résistance de référence optimale pour ce type de capteur [Ohms]
96         "rref_ohm": 100000,
97         # Courant d'excitation IDAC configuré [microampères]
98         "idac_uA": 10,
99         # Gain d'amplification programmable sélectionné
100        "pga_gain": 8
101    },
102    # Profil pour le capteur NTC 2.2k 3528 (thermistance 2.2k0hm)
103    "NTC 2.2k 3528": {
104        # Résistance de référence optimale pour ce type de capteur [Ohms]
105        "rref_ohm": 100000,
106        # Courant d'excitation IDAC configuré [microampères]
107        "idac_uA": 10,
108        # Gain d'amplification programmable sélectionné
109        "pga_gain": 4
110    },
111    # Profil pour le capteur NTC 10k 3977 (thermistance 10k0hm haute résolution)
112    "NTC 10k 3977": {
113        # Résistance de référence optimale pour ce type de capteur [Ohms]
114        "rref_ohm": 100000,
115        # Courant d'excitation IDAC configuré [microampères]
116        "idac_uA": 10,
117        # Gain d'amplification programmable sélectionné
118        "pga_gain": 1
119    }
120 }
121
122
123 # -----
124 # Tables de conversion température/résistance
125 # -----
126
127 # Dictionnaire contenant les tables de correspondance température [°C] / résistance [Ohms]
128 # Structure: liste de tuples (température, résistance) pour interpolation linéaire
129 # Données calibrées issues des fiches techniques des capteurs
130 SENSOR_TABLES = {
131     # Table de conversion température/résistance pour capteur De Dietrich AF60
132     # Points de calibration (température_°C, résistance_Ohms) - sonde industrielle
133     "De Dietrich AF60": [
134         (-15.0, 2010), (-13.5, 1900), (-12.0, 1800), (-10.5, 1700), (-9.0, 1609),
135         (-7.5, 1518), (-6.0, 1436), (-4.5, 1354), (-3.0, 1279), (-1.5, 1211),
136         (0.0, 1143), (1.5, 1081), (3.0, 1019), (4.5, 963), (6.0, 912),
137         (7.5, 861), (9.0, 814), (10.5, 767), (12.0, 724), (13.5, 685),
138         (15.0, 646), (16.5, 610), (18.0, 577), (19.5, 544), (21.0, 514),
139         (22.5, 484), (24.0, 457), (25.5, 430), (27.0, 406), (28.5, 382),
140         (30.0, 360), (31.5, 340), (32.0, 334), (32.5, 328), (33.0, 322),
141         (33.5, 316), (34.0, 311), (34.5, 305), (35.0, 300), (35.5, 295),
142         (36.0, 290), (36.5, 284), (37.0, 279), (37.5, 275), (38.0, 270),
143         (38.5, 265), (39.0, 260), (39.5, 256), (40.0, 251.37)
144     ],
145     # Table de conversion température/résistance pour capteur Ni1000 TK6180
146     # Points de calibration (température_°C, résistance_Ohms) - sonde nickel haute sensibilité
147     "Ni1000 TK6180": [
148         (-15.0, 919), (-13.5, 927), (-12.0, 935), (-10.5, 943), (-9.0, 951),
149         (-7.5, 959), (-6.0, 968), (-4.5, 976), (-3.0, 984), (-1.5, 992),
150         (0.0, 1000), (1.5, 1009), (3.0, 1017), (4.5, 1025), (6.0, 1033),
151         (7.5, 1041), (9.0, 1050), (10.5, 1059), (12.0, 1067), (13.5, 1075),
152         (15.0, 1084), (16.5, 1092), (18.0, 1101), (19.5, 1110), (21.0, 1118),
153         (22.5, 1127), (24.0, 1135), (25.5, 1144), (27.0, 1153), (28.5, 1162),
154         (30.0, 1171), (32.0, 1182), (33.0, 1188), (34.0, 1194.21), (35.0, 1200),
155         (36.0, 1206), (37.0, 1212), (38.0, 1218), (39.0, 1224), (40.0, 1230.7)
156     ],
157     # Table de conversion température/résistance pour capteur Ni1000 TK5000
158     # Points de calibration (température_°C, résistance_Ohms) - sonde nickel standard
159     "Ni1000 TK5000": [
160         (-15.0, 935), (-13.5, 941), (-12.0, 947), (-10.5, 954), (-9.0, 960),
161         (-7.5, 967), (-6.0, 973), (-4.5, 980), (-3.0, 987), (-1.5, 993),
162         (0.0, 1000), (1.5, 1007), (3.0, 1014), (4.5, 1020), (6.0, 1027),
163         (7.5, 1033), (9.0, 1040), (10.5, 1047), (12.0, 1054), (13.5, 1061),
164         (15.0, 1067), (16.5, 1074), (18.0, 1082), (19.5, 1089), (21.0, 1095),
165         (22.5, 1102), (24.0, 1110), (25.5, 1116), (27.0, 1123), (28.5, 1131),
166         (30.0, 1138), (31.5, 1145), (32.0, 1147), (33.0, 1152), (34.0, 1157),
167         (35.0, 1162), (36.0, 1166), (37.0, 1171), (38.0, 1176), (39.0, 1181),
168         (40.0, 1185.71)
169     ],
170     # Table de conversion température/résistance pour capteur PT1000
171     # Points de calibration (température_°C, résistance_Ohms) - sonde platine de précision

```

```
172 "PT1000": [  
173     (-15.0, 942), (-13.5, 948), (-12.0, 953), (-10.5, 959), (-9.0, 965),  
174     (-7.5, 971), (-6.0, 977), (-4.5, 982), (-3.0, 989), (-1.5, 994),  
175     (0.0, 1000), (1.5, 1006), (3.0, 1012), (4.5, 1018), (6.0, 1023),  
176     (7.5, 1029), (9.0, 1035), (10.5, 1041), (12.0, 1047), (13.5, 1053),  
177     (15.0, 1059), (16.5, 1064), (18.0, 1070), (19.5, 1076), (21.0, 1082),  
178     (22.5, 1088), (24.0, 1094), (25.5, 1099), (27.0, 1105), (28.5, 1111),  
179     (30.0, 1117), (31.5, 1123), (32.0, 1124), (32.5, 1126), (33.0, 1128),  
180     (33.5, 1130), (34.0, 1132), (34.5, 1134), (35.0, 1136), (35.5, 1138),  
181     (36.0, 1140), (36.5, 1142), (37.0, 1144), (37.5, 1146), (38.0, 1148),  
182     (38.5, 1150), (39.0, 1152), (39.5, 1153), (40.0, 1155.5)  
183 ],  
184 # Table de conversion température/résistance pour capteur Siemens QAC32  
185 # Points de calibration (température_°C, résistance_Ohms) - sonde qualité d'air  
186 "Siemens QAC32": [  
187     (-15.0, 653), (-13.5, 650), (-12.0, 647), (-10.5, 643), (-9.0, 640),  
188     (-7.5, 637), (-6.0, 633), (-4.5, 630), (-3.0, 627), (-1.5, 623),  
189     (0.0, 620), (1.5, 617), (3.0, 614), (4.5, 611), (6.0, 607),  
190     (7.5, 604), (9.0, 601), (10.5, 597), (12.0, 594), (13.5, 591),  
191     (15.0, 588), (16.5, 584), (18.0, 581), (19.5, 578), (21.0, 575),  
192     (22.5, 571), (24.0, 568), (25.5, 565), (27.0, 561), (28.5, 558),  
193     (30.0, 555), (31.5, 552), (40.0, 525)  
194 ],  
195 # Table de conversion température/résistance pour capteur NTC 1k 3528  
196 # Points de calibration (température_°C, résistance_Ohms) - thermistance 1kΩ  
197 "NTC 1k 3528": [  
198     (-15.0, 5855), (-13.5, 5431), (-12.0, 5041), (-10.5, 4683), (-9.0, 4353),  
199     (-7.5, 4050), (-6.0, 3771), (-4.5, 3513), (-3.0, 3276), (-1.5, 3057),  
200     (0.0, 2854), (1.5, 2667), (3.0, 2493), (4.5, 2333), (6.0, 2184),  
201     (7.5, 2046), (9.0, 1918), (10.5, 1799), (12.0, 1689), (13.5, 1586),  
202     (15.0, 1491), (16.5, 1402), (18.0, 1319), (19.5, 1242), (21.0, 1170),  
203     (22.5, 1102), (24.0, 1040), (25.5, 981), (27.0, 926), (28.5, 875),  
204     (30.0, 827), (31.5, 782), (32.0, 767), (32.5, 753), (33.0, 739),  
205     (33.5, 726), (34.0, 713), (34.5, 700), (35.0, 687), (35.5, 675),  
206     (36.0, 663), (36.5, 651), (37.0, 639), (37.5, 628), (38.0, 617),  
207     (38.5, 606), (39.0, 595), (39.5, 585), (40.0, 574.6)  
208 ],  
209 # Table de conversion température/résistance pour capteur KTY81-210  
210 # Points de calibration (température_°C, résistance_Ohms) - capteur silicium  
211 "KTY81-210": [  
212     (-15.0, 1423), (-13.5, 1445), (-12.0, 1466), (-10.5, 1488), (-9.0, 1509),  
213     (-7.5, 1531), (-6.0, 1552), (-4.5, 1573), (-3.0, 1595), (-1.5, 1616),  
214     (0.0, 1638), (1.5, 1659), (3.0, 1681), (4.5, 1702), (6.0, 1723),  
215     (7.5, 1745), (9.0, 1766), (10.5, 1788), (12.0, 1809), (13.5, 1831),  
216     (15.0, 1852), (16.5, 1873), (18.0, 1895), (19.5, 1916), (21.0, 1938),  
217     (22.5, 1959), (24.0, 1981), (25.5, 2002), (27.0, 2023), (28.5, 2045),  
218     (30.0, 2066), (31.5, 2088), (40.0, 2245)  
219 ],  
220 # Table de conversion température/résistance pour capteur NTC 2k 3390  
221 # Points de calibration (température_°C, résistance_Ohms) - thermistance 2kΩ haute précision  
222 "NTC 2k 3390": [  
223     (-15.0, 11124), (-13.5, 10340), (-12.0, 9618), (-10.5, 8954), (-9.0, 8341),  
224     (-7.5, 7777), (-6.0, 7255), (-4.5, 6774), (-3.0, 6329), (-1.5, 5917),  
225     (0.0, 5536), (1.5, 5183), (3.0, 4855), (4.5, 4551), (6.0, 4269),  
226     (7.5, 4007), (9.0, 3764), (10.5, 3537), (12.0, 3326), (13.5, 3130),  
227     (15.0, 2947), (16.5, 2776), (18.0, 2616), (19.5, 2468), (21.0, 2328),  
228     (22.5, 2198), (24.0, 2077), (25.5, 1963), (27.0, 1856), (28.5, 1756),  
229     (30.0, 1663), (31.5, 1575), (32.0, 1546), (32.5, 1519), (33.0, 1492),  
230     (33.5, 1466), (34.0, 1440), (34.5, 1415), (35.0, 1390), (35.5, 1365),  
231     (36.0, 1342), (36.5, 1318), (37.0, 1296), (37.5, 1273), (38.0, 1251),  
232     (38.5, 1230), (39.0, 1209), (39.5, 1188), (40.0, 1168.05)  
233 ],  
234 # Table de conversion température/résistance pour capteur NTC 2.2k 3528  
235 # Points de calibration (température_°C, résistance_Ohms) - thermistance 2.2kΩ  
236 "NTC 2.2k 3528": [  
237     (-15.0, 12881), (-13.5, 11947), (-12.0, 11089), (-10.5, 10301), (-9.0, 9576),  
238     (-7.5, 8909), (-6.0, 8295), (-4.5, 7729), (-3.0, 7206), (-1.5, 6724),  
239     (0.0, 6278), (1.5, 5866), (3.0, 5485), (4.5, 5132), (6.0, 4804),  
240     (7.5, 4501), (9.0, 4220), (10.5, 3958), (12.0, 3715), (13.5, 3489),  
241     (15.0, 3279), (16.5, 3084), (18.0, 2901), (19.5, 2732), (21.0, 2573),  
242     (22.5, 2425), (24.0, 2287), (25.5, 2158), (27.0, 2037), (28.5, 1924),  
243     (30.0, 1819), (31.5, 1720), (32.0, 1600), (32.5, 1569), (33.0, 1538),  
244     (33.5, 1508), (34.0, 1479), (34.5, 1450), (35.0, 1422), (35.5, 1395),  
245     (36.0, 1368), (36.5, 1342), (37.0, 1316), (37.5, 1291), (38.0, 1266),  
246     (38.5, 1242), (39.0, 1219), (39.5, 1196), (40.0, 1173)  
247 ],  
248 # Table de conversion température/résistance pour capteur NTC 10k 3977  
249 # Points de calibration (température_°C, résistance_Ohms) - thermistance 10kΩ haute résolution  
250 "NTC 10k 3977": [  
251     (-15.0, 72502), (-13.5, 66690), (-12.0, 61394), (-10.5, 56563), (-9.0, 52153),  
252     (-7.5, 48123), (-6.0, 44439), (-4.5, 41068), (-3.0, 37980), (-1.5, 35151),  
253     (0.0, 32555), (1.5, 30173), (3.0, 27985), (4.5, 25973), (6.0, 24122),  
254     (7.5, 22419), (9.0, 20850), (10.5, 19403), (12.0, 18068), (13.5, 16836),  
255     (15.0, 15698), (16.5, 14646), (18.0, 13673), (19.5, 12773), (21.0, 11939),  
256     (22.5, 11166), (24.0, 10449), (25.5, 9784), (27.0, 9166), (28.5, 8593),  
257     (30.0, 8059), (31.5, 7563), (32.0, 7330), (32.5, 7183), (33.0, 7037),  
258     (33.5, 6895), (34.0, 6754), (34.5, 6620), (35.0, 6487), (35.5, 6358),
```

```
259         (36.0, 6231), (36.5, 6108), (37.0, 5987), (37.5, 5869), (38.0, 5754),
260         (38.5, 5642), (39.0, 5532), (39.5, 5424), (40.0, 5320)
261     ],
262 }
263
264
265 # -----
266 # Fonctions d'accès aux profils
267 # -----
268
269 def get_profile(sensor_name):
270     """
271     @brief   Récupère le profil de configuration ADC pour un capteur donné.
272     @details Fonction d'accès qui retourne le dictionnaire de configuration
273             contenant les paramètres optimisés (Rref, IDAC, PGA) pour
274             un type de capteur spécifique.
275
276     @param sensor_name  Nom du type de capteur recherché.
277
278     @return             Dictionnaire contenant les paramètres de configuration:
279                         - rref_ohm: résistance de référence [Ohms]
280                         - idac_uA: courant d'excitation [microampères]
281                         - pga_gain: gain programmable [1-128]
282
283     @exception          KeyError si le type de capteur n'est pas reconnu.
284     """
285     # Vérification de l'existence du capteur dans les profils
286     if sensor_name in SENSOR_PROFILES:
287         # Retour du profil de configuration pour ce capteur
288         return SENSOR_PROFILES[sensor_name]
289     # Levée d'exception avec message explicite pour capteur inconnu
290     raise KeyError("Type de sonde inconnu: " + str(sensor_name))
291
```

config_module\sensors.json

```
1 {
2   "_comment_file": "sensors.json",
3   "_comment_brief": "Configuration des capteurs et paramètres de mesure CM5.",
4   "_comment_details": "Ce fichier JSON contient la configuration complète du système de mesure de température. Il définit les paramètres globaux de
communication, les timeouts, les intervalles de mesure et la configuration individuelle de chaque canal de capteur.",
5   "_comment_author": "Léo Mendes",
6   "_comment_project": "2510_RegulationsChauffage",
7   "_comment_mandant": "Oblo_solution",
8   "_comment_date": "2025-09-18",
9   "_comment_version": "1.0.0",
10  "_comment_copyright": "Copyright (c) 2025",
11
12  "_comment_section_device": "--- Configuration du dispositif ---",
13  "mac_address": "0030DEABCDEF",
14  "_comment_mac_address": "Identifiant unique du Raspberry Pi CM5 pour l'API. Format: 6 octets en hexadécimal.",
15
16  "_comment_section_timeouts": "--- Paramètres de temporisation ---",
17  "timeout_s": 180,
18  "_comment_timeout_s": "Temps maximum d'attente pour une mesure ADC avant abandon [secondes]. Valeur recommandée: 180s.",
19
20  "inter_measure_sleep_s": 0.1,
21  "_comment_inter_measure_sleep_s": "Pause entre chaque échantillonnage ADC pour stabilisation [secondes]. Valeur recommandée: 0.1s.",
22
23  "loop_sleep_s": 1.0,
24  "_comment_loop_sleep_s": "Délai de pause dans la boucle principale d'acquisition [secondes]. Évite la surcharge CPU.",
25
26  "_comment_section_automation": "--- Configuration de l'automatisation ---",
27  "sequence_interval_minutes": 5,
28  "_comment_sequence_interval_minutes": "Fréquence d'exécution automatique des mesures complètes [minutes]. 0 = désactivé. Valeur recommandée: 5
min.",
29
30  "auto_sequence": true,
31  "_comment_auto_sequence": "Active/désactive l'exécution automatique des séquences de mesure selon l'intervalle défini. true = actif, false =
manuel.",
32
33  "_comment_section_channels": "--- Configuration des canaux de mesure ---",
34  "channels": [
35    {
36      "_comment_channel_1": "Canal 1 - Configuration du premier capteur",
37      "channel": 1,
38      "_comment_channel_num": "Numéro de canal physique sur le multiplexeur ADC",
39      "sensor": "Ni1000 TK5000",
40      "_comment_sensor_type": "Type de capteur de température. Définit la courbe de conversion résistance-température.",
41      "enabled": true,
42      "_comment_enabled": "État d'activation du canal. true = mesures actives, false = canal ignoré"
43    },
44    {
45      "_comment_channel_2": "Canal 2 - Configuration du deuxième capteur ",
46      "channel": 2,
47      "sensor": "Ni1000 TK5000",
48      "enabled": false
49    },
50    {
51      "_comment_channel_3": "Canal 3 - Configuration du troisième capteur",
52      "channel": 3,
53      "sensor": "Ni1000 TK5000",
54      "enabled": false
55    },
56    {
57      "_comment_channel_4": "Canal 4 - Configuration du quatrième capteur",
58      "channel": 4,
59      "sensor": "Ni1000 TK5000",
60      "enabled": false
61    }
62  ],
63  "_comment_channels_detail": "Liste des 4 canaux de mesure disponibles. Chaque canal peut être configuré individuellement avec son type de capteur
et son état d'activation."
64 }
65
```

app\temperature_conversion.py

```
1  # -*- coding: utf-8 -*-
2  """
3  @file      temperature_conversion.py
4  @brief     Module de conversion résistance/température via interpolation linéaire.
5  @details   Ce module fournit les fonctions de conversion bidirectionnelle
6             entre résistance mesurée et température pour différents types
7             de capteurs. Utilise l'interpolation linéaire sur les tables
8             de calibration et supporte la conversion inverse pour la
9             simulation de résistance.
10 @author    Léo Mendes
11 @project   2510_RegulationsChauffage
12 @Mandant   Oblo_solution
13 @date      2025-09-18
14 @version   1.0.0
15 @copyright Copyright (c) 2025
16 """
17
18 # Import des tables de capteurs depuis le module de profils
19 from app.sensor_profiles import SENSOR_TABLES
20
21
22 # -----
23 # Fonctions de conversion de base
24 # -----
25
26 def resistance_to_temperature(resistance, table):
27     """
28     @brief   Convertit une résistance mesurée en température via interpolation linéaire.
29     @details Algorithme d'interpolation linéaire qui recherche l'intervalle
30             contenant la résistance mesurée et calcule la température
31             correspondante. Gère les cas limites hors plage de mesure.
32
33     @param resistance Résistance mesurée [Ohms].
34     @param table      Liste de tuples (résistance, température) pour interpolation.
35
36     @return           Température interpolée [°C], ou valeurs limites si hors plage.
37     """
38     # Parcours des paires de points consécutifs dans la table de conversion
39     for i in range(len(table) - 1):
40         # Extraction des coordonnées du premier point de l'intervalle
41         r1, t1 = table[i]
42         # Extraction des coordonnées du second point de l'intervalle
43         r2, t2 = table[i + 1]
44         # Vérification si la résistance mesurée est dans l'intervalle courant
45         if r1 <= resistance <= r2:
46             # Application de la formule d'interpolation linéaire
47             temperature = t1 + (resistance - r1) / (r2 - r1) * (t2 - t1)
48             # Retour de la température interpolée
49             return temperature
50
51     # Gestion des cas hors limites de la table de conversion
52     r_min, t_min = table[0]
53     # Extraction des valeurs maximales de la table
54     r_max, t_max = table[-1]
55     # Retour de la température minimale si résistance trop faible
56     if resistance < r_min:
57         return t_min
58     # Retour de la température maximale si résistance trop élevée
59     if resistance > r_max:
60         return t_max
61     # Retour de None si aucune condition n'est satisfaite (cas d'erreur)
62     return None
63
64 def resistance_to_temperature_dynamic(resistance, sensor):
65     """
66     @brief   Convertit une résistance mesurée en température pour un capteur donné.
67     @details Fonction dynamique qui sélectionne automatiquement la table de
68             conversion appropriée selon le type de capteur spécifié.
69             Effectue la validation du capteur avant conversion.
70
71     @param resistance Résistance mesurée [Ohms].
72     @param sensor      Nom du type de capteur à utiliser.
73
74     @return           Température interpolée [°C] ou None si hors limites.
75
76     @exception        ValueError si le capteur n'est pas reconnu.
77     """
78     # Vérification de l'existence du capteur dans les tables disponibles
79     if sensor not in SENSOR_TABLES:
80         # Levée d'exception avec message explicite si capteur introuvable
81         raise ValueError(f"Capteur {sensor} non trouvé dans les données.")
82
83     # Récupération de la table de conversion spécifique au capteur
84     table = SENSOR_TABLES[sensor]
```

```
85 # Parcours des paires de points consécutifs dans la table
86 for i in range(len(table) - 1):
87     # Extraction température et résistance du premier point (format inversé)
88     t1, r1 = table[i]
89     # Extraction température et résistance du second point
90     t2, r2 = table[i + 1]
91     # Vérification si la résistance mesurée est dans l'intervalle
92     if r1 <= resistance <= r2:
93         # Application de l'interpolation linéaire pour calculer la température
94         temperature = t1 + (resistance - r1) / (r2 - r1) * (t2 - t1)
95         # Retour de la température calculée
96         return temperature
97
98 # Gestion des valeurs minimales et maximales de la table
99 t_min, r_min = table[0]
100 # Extraction des valeurs maximales
101 t_max, r_max = table[-1]
102 # Retour de la température minimale si résistance trop faible
103 if resistance < r_min:
104     return t_min
105 # Retour de la température maximale si résistance trop élevée
106 if resistance > r_max:
107     return t_max
108 # Retour de None si aucune condition n'est remplie
109 return None
110
111 def temperature_to_resistance_dynamic(temperature, sensor):
112     """
113     @brief Convertit une température en résistance pour un capteur donné (conversion inverse).
114     @details Fonction de conversion inverse utilisée pour la simulation de résistance.
115             Effectue l'interpolation linéaire sur l'axe température pour obtenir
116             la résistance correspondante selon le type de capteur.
117
118     @param temperature Température d'entrée [°C].
119     @param sensor       Nom du type de capteur à utiliser.
120
121     @return             Résistance interpolée [Ohms] ou None si hors limites.
122
123     @exception         ValueError si le capteur n'est pas reconnu.
124     """
125     # Vérification de l'existence du capteur dans les tables de conversion
126     if sensor not in SENSOR_TABLES:
127         # Levée d'exception avec message d'erreur explicite
128         raise ValueError(f"Capteur {sensor} non trouvé dans les données.")
129
130     # Récupération de la table de conversion pour le capteur spécifié
131     table = SENSOR_TABLES[sensor]
132
133     # Parcours des paires de points consécutifs pour la recherche d'intervalle
134     for i in range(len(table) - 1):
135         # Extraction des coordonnées du premier point (température, résistance)
136         t1, r1 = table[i]
137         # Extraction des coordonnées du second point
138         t2, r2 = table[i + 1]
139
140         # Vérification si la température est contenue dans l'intervalle courant
141         if t1 <= temperature <= t2:
142             # Application de l'interpolation linéaire inverse (température -> résistance)
143             resistance = r1 + (temperature - t1) / (t2 - t1) * (r2 - r1)
144             # Retour de la résistance calculée
145             return resistance
146
147     # Gestion des cas hors limites de la table de conversion
148     t_min, r_min = table[0]
149     # Extraction des valeurs maximales de température et résistance
150     t_max, r_max = table[-1]
151
152     # Retour de la résistance minimale si température trop basse
153     if temperature < t_min:
154         return r_min
155     # Retour de la résistance maximale si température trop élevée
156     if temperature > t_max:
157         return r_max
158
159     # Retour de None si aucune condition n'est satisfaite
160     return None
161
162 def convert_temperature_to_resistance(temperature, probe_type):
163     """
164     @brief Fonction de conversion principale pour la simulation de résistance.
165     @details Fonction de haut niveau qui supporte différents types de sondes
166             avec leurs caractéristiques spécifiques. Inclut la normalisation
167             des noms de sondes et la gestion des variantes de nomenclature.
168
169     @param temperature Température simulée [°C].
170     @param probe_type   Type de sonde (formats multiples supportés).
```

```

172     @return          Résistance simulée [Ohms] ou None si échec.
173     """
174     # Affichage d'information sur la conversion en cours
175     print(f"[CONV] Conversion T->R: {temperature:.2f}°C -> ? ohms (sonde: {probe_type})")
176
177     # Dictionnaire de normalisation des noms de sondes pour compatibilité
178     probe_mapping = {
179         "PT1000": "PT1000",
180         "Ni1000_TK5000": "Ni1000 TK5000",
181         "Ni1000_TK5000": "Ni1000 TK5000",
182         "NTC_10k": "NTC 10k 3977",
183         "NTC_10k_3977": "NTC 10k 3977",
184         "NTC 10k": "NTC 10k 3977"
185     }
186
187     # Recherche du nom normalisé de la sonde dans le mapping
188     normalized_probe = probe_mapping.get(probe_type, probe_type)
189
190     try:
191         # Tentative de conversion avec le nom normalisé
192         resistance = temperature_to_resistance_dynamic(temperature, normalized_probe)
193
194         # Vérification de la validité du résultat de conversion
195         if resistance is None:
196             # Affichage d'information si température hors plage supportée
197             print(f"[CONV] Température {temperature:.2f}°C hors plage pour {normalized_probe}")
198             return None
199
200         # Affichage de confirmation de conversion réussie
201         print(f"[CONV] Conversion réussie: {temperature:.2f}°C -> {resistance:.2f} ohms")
202         # Retour de la résistance calculée
203         return resistance
204
205     except ValueError as e:
206         # Affichage d'erreur lors de la conversion principale
207         print(f"[CONV] Erreur de conversion: {e}")
208
209         # Liste des noms alternatifs pour tentatives de récupération
210         alternative_names = [
211             "PT1000",
212             "Ni1000 TK5000",
213             "Ni1000 TK6180",
214             "NTC 10k 3977",
215             "NTC 1k 3528",
216             "De Dietrich AF60",
217             "Siemens QAC32"
218         ]
219
220         # Tentative de correspondance avec les noms alternatifs
221         for alt_name in alternative_names:
222             # Comparaison de noms avec normalisation (minuscules, espaces)
223             if probe_type.lower().replace("_", " ") in alt_name.lower():
224                 try:
225                     # Tentative de conversion avec le nom alternatif
226                     resistance = temperature_to_resistance_dynamic(temperature, alt_name)
227                     # Vérification de la validité du résultat
228                     if resistance is not None:
229                         # Affichage de succès avec nom alternatif utilisé
230                         print(f"[CONV] Conversion avec nom alternatif '{alt_name}': {temperature:.2f}°C -> {resistance:.2f} ohms")
231                         return resistance
232                 except ValueError:
233                     # Continuation de la boucle si échec avec ce nom alternatif
234                     continue
235
236         # Affichage d'erreur si aucune sonde compatible trouvée
237         print(f"[CONV] Aucune sonde compatible trouvée pour '{probe_type}'")
238         # Affichage de la liste des sondes disponibles pour assistance
239         print(f"[CONV] Sondes disponibles: {list(SENSOR_TABLES.keys())}")
240         return None
241
242 # -----
243 # Fonctions utilitaires et validation
244 # -----
245
246 def get_supported_probe_types():
247     """
248     @brief  Retourne la liste des types de sondes supportés.
249     @details Fonction utilitaire qui extrait la liste complète des noms
250             de capteurs disponibles dans les tables de conversion.
251
252     @return Liste des noms de capteurs disponibles pour conversion.
253     """
254     # Retour direct des clés du dictionnaire des tables de capteurs
255     return list(SENSOR_TABLES.keys())
256
257 def validate_temperature_range(temperature, probe_type):
258     """

```



```
259 @brief Valide si une température est dans la plage supportée par une sonde.
260 @details Fonction de validation qui vérifie si une température donnée
261 est compatible avec la plage de fonctionnement d'un type de
262 capteur spécifique selon sa table de calibration.
263
264 @param temperature Température à valider [°C].
265 @param probe_type Type de sonde à vérifier.
266
267 @return True si la température est dans la plage supportée.
268 """
269 try:
270     # Dictionnaire de normalisation des noms pour la validation
271     probe_mapping = {
272         "PT1000": "PT1000",
273         "Ni1000_TK5000": "Ni1000 TK5000",
274         "NTC_10k": "NTC 10k 3977"
275     }
276     # Application de la normalisation du nom de sonde
277     normalized_probe = probe_mapping.get(probe_type, probe_type)
278
279     # Vérification de l'existence de la sonde dans les tables
280     if normalized_probe not in SENSOR_TABLES:
281         return False
282
283     # Récupération de la table de conversion pour la sonde
284     table = SENSOR_TABLES[normalized_probe]
285     # Extraction de la température minimale de la table
286     t_min, _ = table[0]
287     # Extraction de la température maximale de la table
288     t_max, _ = table[-1]
289
290     # Vérification si la température est dans la plage [t_min, t_max]
291     return t_min <= temperature <= t_max
292
293 except (KeyError, IndexError):
294     # Retour de False en cas d'erreur d'accès aux données
295     return False
```

app\temperature_simulation.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      temperature_simulation.py
4  @brief     Module de calcul de température simulée selon formule prédictive.
5  @details   Ce module implémente la formule de simulation de température:
6              $T_{sim} = T_{mes} + ((T_{prev} - T_{mes}) * n * \exp(k_m))$ 
7             Inclut la validation complète des paramètres, la gestion
8             des erreurs numériques et l'exécution de simulations complètes.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 # Import des modules système standard
18 import math
19 # Import des annotations de type pour la documentation
20 from typing import Optional
21
22
23 # -----
24 # Fonctions de calcul principal
25 # -----
26
27 def calculate_simulated_temperature(t_mes: float, t_prev: float, n: float, k_m: float) -> Optional[float]:
28     """
29     @brief   Calcule la température simulée selon la formule prédictive.
30     @details Implémente la formule  $T_{sim} = T_{mes} + ((T_{prev} - T_{mes}) * n * \exp(k_m))$ 
31             avec validation complète des paramètres et gestion des erreurs
32             numériques (overflow, underflow, valeurs infinies).
33
34     @param t_mes    Température mesurée [°C].
35     @param t_prev    Température prévue [°C].
36     @param n         Paramètre de pondération [-∞, +∞].
37     @param k_m       Paramètre exposant pour fonction exponentielle [-∞, +∞].
38
39     @return         Température simulée [°C] ou None en cas d'erreur.
40
41     @exception      OverflowError si exp(k_m) dépasse les limites numériques.
42     @warning         Limitation automatique des paramètres extrêmes.
43     """
44     try:
45         # Validation du type des paramètres d'entrée
46         if not all(isinstance(param, (int, float)) for param in [t_mes, t_prev, n, k_m]):
47             # Affichage d'erreur si types incorrects
48             print("[CALC] Erreur: tous les paramètres doivent être des nombres")
49             return None
50
51         # Vérification des plages de températures raisonnables pour mesure
52         if not (-100.0 <= t_mes <= 200.0):
53             # Avertissement pour température mesurée hors plage normale
54             print(f"[CALC] Attention: température mesurée hors plage normale: {t_mes}°C")
55
56         # Vérification des plages de températures raisonnables pour prévision
57         if not (-100.0 <= t_prev <= 200.0):
58             # Avertissement pour température prévue hors plage normale
59             print(f"[CALC] Attention: température prévue hors plage normale: {t_prev}°C")
60
61         # Limitation du paramètre k_m pour éviter l'overflow exponentiel
62         if abs(k_m) > 50:
63             # Erreur critique si k_m trop élevé
64             print(f"[CALC] Erreur: paramètre k_m trop élevé: {k_m}")
65             return None
66
67         # Calcul de l'exponentielle avec gestion des cas d'overflow
68         try:
69             # Application de la fonction exponentielle
70             exp_km = math.exp(k_m)
71         except OverflowError:
72             # Gestion de l'overflow lors du calcul exponentiel
73             print(f"[CALC] Erreur: overflow lors du calcul de exp({k_m})")
74             return None
75
76         # Vérification que l'exponentielle reste dans une plage numérique raisonnable
77         if exp_km > 1e10:
78             # Limitation de la valeur exponentielle pour éviter instabilité
79             print(f"[CALC] Erreur: exp(k_m) = {exp_km} trop élevé")
80             return None
81
82         # Calcul de la différence entre température prévue et mesurée
83         temp_diff = t_prev - t_mes
84

```

```

85     # Application de la formule complète de simulation
86     t_sim = t_mes + (temp_diff * n * exp_km)
87
88     # Vérification que le résultat est un nombre fini valide
89     if not math.isfinite(t_sim):
90         # Erreur si résultat infini ou NaN
91         print(f"[CALC] Erreur: résultat non fini: {t_sim}")
92         return None
93
94     # Limitation du résultat dans une plage physiquement raisonnable
95     if not (-200.0 <= t_sim <= 500.0):
96         # Avertissement pour température simulée hors plage physique
97         print(f"[CALC] Attention: température simulée hors plage raisonnable: {t_sim}°C")
98
99     # Affichage de confirmation avec détail du calcul effectué
100    print(f"[CALC] Calcul réussi: T_sim = {t_mes} + (({t_prev} - {t_mes}) * {n} * exp({k_m})) = {t_sim:.2f}°C")
101    # Retour de la température simulée calculée
102    return t_sim
103
104    except Exception as e:
105        # Gestion d'exception générale pour erreurs inattendues
106        print(f"[CALC] Erreur inattendue lors du calcul: {e}")
107        return None
108
109    def validate_simulation_parameters(probe_type: str, n: float, k_m: float, t_mes: float, t_prev: float) -> bool:
110        """
111        @brief Valide tous les paramètres de simulation avant exécution.
112        @details Fonction de validation complète qui vérifie la cohérence
113                et les plages de tous les paramètres d'entrée. Inclut
114                la vérification interactive pour les écarts importants.
115
116        @param probe_type Type de sonde à utiliser pour la simulation.
117        @param n Paramètre de pondération de la simulation.
118        @param k_m Paramètre exposant de la fonction exponentielle.
119        @param t_mes Température mesurée à valider [°C].
120        @param t_prev Température prévue à valider [°C].
121
122        @return True si tous les paramètres sont valides.
123
124        @warning Demande confirmation utilisateur pour écarts importants.
125        """
126        # Affichage du début de la procédure de validation
127        print("[VALIDATION] Vérification des paramètres de simulation...")
128
129        # Liste des types de sondes supportés par le système
130        valid_probe_types = ["PT1000", "Ni1000_TK5000", "NTC_10k", "Ni1000 TK5000", "NTC 10k 3977"]
131        # Vérification que le type de sonde est dans la liste supportée
132        if probe_type not in valid_probe_types:
133            # Affichage d'erreur avec liste des types supportés
134            print(f"[VALIDATION] Type de sonde non supporté: {probe_type}")
135            print(f"[VALIDATION] Types supportés: {valid_probe_types}")
136            return False
137        else:
138            # Confirmation que le type de sonde est valide
139            print(f"[VALIDATION] Type de sonde valide: {probe_type}")
140
141        # Vérification du paramètre n dans sa plage de fonctionnement recommandée
142        if not (-2.0 <= n <= 2.0):
143            # Erreur si paramètre n hors plage acceptable
144            print(f"[VALIDATION] Paramètre n hors plage [-2.0, 2.0]: {n}")
145            return False
146        else:
147            # Confirmation de validité du paramètre n
148            print(f"[VALIDATION] Paramètre n valide: {n}")
149
150        # Vérification du paramètre k_m dans sa plage de sécurité numérique
151        if not (-10.0 <= k_m <= 10.0):
152            # Erreur si k_m risque de causer overflow exponentiel
153            print(f"[VALIDATION] Paramètre k_m hors plage [-10.0, 10.0]: {k_m}")
154            return False
155        else:
156            # Confirmation de validité du paramètre k_m
157            print(f"[VALIDATION] Paramètre k_m valide: {k_m}")
158
159        # Vérification de la température mesurée dans plage physique normale
160        if not (-50.0 <= t_mes <= 150.0):
161            # Erreur si température mesurée hors plage d'application
162            print(f"[VALIDATION] Température mesurée hors plage [-50°C, 150°C]: {t_mes}°C")
163            return False
164        else:
165            # Confirmation de validité de la température mesurée
166            print(f"[VALIDATION] Température mesurée valide: {t_mes}°C")
167
168        # Vérification de la température prévue dans plage physique normale
169        if not (-50.0 <= t_prev <= 150.0):
170            # Erreur si température prévue hors plage d'application
171            print(f"[VALIDATION] Température prévue hors plage [-50°C, 150°C]: {t_prev}°C")

```

```

172         return False
173     else:
174         # Confirmation de validité de la température prévue
175         print(f"[VALIDATION] Température prévue valide: {t_prev}°C")
176
177     # Calcul de l'écart absolu entre températures pour vérification cohérence
178     temp_diff = abs(t_prev - t_mes)
179     # Vérification si l'écart entre températures est anormalement important
180     if temp_diff > 50.0:
181         # Avertissement pour écart important entre températures
182         print(f"[VALIDATION] Écart important entre températures: {temp_diff:.1f}°C")
183         print(f"[VALIDATION] T_mes={t_mes}°C, T_prev={t_prev}°C")
184
185         # Procédure de demande de confirmation utilisateur pour écart important
186         try:
187             # Demande de confirmation interactive
188             confirm = input("[VALIDATION] Continuer malgré l'écart important? (o/N): ").strip().lower()
189             # Vérification de la réponse utilisateur
190             if confirm != 'o' and confirm != 'oui':
191                 # Annulation si utilisateur refuse de continuer
192                 print("[VALIDATION] Validation annulée par l'utilisateur")
193                 return False
194         except KeyboardInterrupt:
195             # Gestion de l'interruption par Ctrl+C
196             print("\n[VALIDATION] Validation interrompue")
197             return False
198
199     # Confirmation finale que tous les paramètres sont validés
200     print("[VALIDATION] Tous les paramètres sont valides")
201     return True
202
203 def run_temperature_simulation(simulation_data: dict) -> Optional[float]:
204     """
205     @brief Exécute une simulation complète de température avec dictionnaire.
206     @details Fonction principale d'orchestration qui extrait les paramètres
207             d'un dictionnaire, valide tous les paramètres et exécute la
208             simulation complète avec gestion d'erreur robuste.
209
210     @param simulation_data Dictionnaire contenant tous les paramètres requis:
211             - probe_type: Type de sonde (str)
212             - n: Paramètre de pondération (float)
213             - k_m: Paramètre exposant (float)
214             - temperature: Température mesurée (float)
215             - forecast_temperature: Température prévue (float)
216
217     @return Température simulée [°C] ou None si erreur.
218
219     @warning Retourne None pour toute erreur de validation ou calcul.
220     """
221     # Affichage d'en-tête de la simulation avec séparateur visuel
222     print("\n=== SIMULATION DE TEMPÉRATURE ===")
223
224     # Bloc de traitement avec gestion complète des exceptions
225     try:
226         # Extraction sécurisée de tous les paramètres du dictionnaire
227         probe_type = simulation_data["probe_type"]
228         n = float(simulation_data["n"])
229         k_m = float(simulation_data["k_m"])
230         t_mes = float(simulation_data["temperature"])
231         t_prev = float(simulation_data["forecast_temperature"])
232
233         # Affichage formaté de tous les paramètres extraits
234         print(f"[SIM] Paramètres d'entrée:")
235         print(f"[SIM] Type de sonde: {probe_type}")
236         print(f"[SIM] Paramètre n: {n}")
237         print(f"[SIM] Paramètre k_m: {k_m}")
238         print(f"[SIM] Température mesurée: {t_mes}°C")
239         print(f"[SIM] Température prévue: {t_prev}°C")
240
241         # Étape de validation complète des paramètres extraits
242         if not validate_simulation_parameters(probe_type, n, k_m, t_mes, t_prev):
243             # Arrêt immédiat si validation échoue
244             print("[SIM] Échec de la validation des paramètres")
245             return None
246
247         # Calcul de la température simulée avec paramètres validés
248         t_sim = calculate_simulated_temperature(t_mes, t_prev, n, k_m)
249
250         # Vérification que le calcul a réussi
251         if t_sim is None:
252             # Gestion du cas d'échec de calcul
253             print("[SIM] Échec du calcul de la température simulée")
254             return None
255
256         # Section d'affichage des résultats avec formatage soigné
257         print(f"\n[SIM] RÉSULTAT:")
258         print(f"[SIM] Température simulée: {t_sim:.2f}°C")

```

```
259
260     # Analyse comparative automatique du résultat obtenu
261     ecart_mesure = abs(t_sim - t_mes)
262     ecart_prevision = abs(t_sim - t_prev)
263
264     # Classification intelligente du résultat selon les écarts
265     if ecart_mesure < 0.1:
266         # Résultat très proche de la mesure actuelle
267         print(f"[SIM] Simulation proche de la mesure (écart: {ecart_mesure:.3f}°C)")
268     elif ecart_prevision < ecart_mesure:
269         # Résultat tend vers la prévision plutôt que la mesure
270         print(f"[SIM] Simulation tend vers la prévision")
271     else:
272         # Résultat tend vers la mesure plutôt que la prévision
273         print(f"[SIM] Simulation tend vers la mesure")
274
275     # Retour du résultat calculé et validé
276     return t_sim
277
278 except KeyError as e:
279     # Gestion des paramètres manquants dans le dictionnaire
280     print(f"[SIM] Paramètre manquant: {e}")
281     return None
282
283 except (ValueError, TypeError) as e:
284     # Gestion des erreurs de conversion de type
285     print(f"[SIM] Erreur de conversion de type: {e}")
286     return None
287
288 except Exception as e:
289     # Capture de toute autre exception non prévue
290     print(f"[SIM] Erreur inattendue: {e}")
291     return None
```

ADS124S0x Low-Power, Low-Noise, Highly Integrated, 6- and 12-Channel, 4-kSPS, 24-Bit, Delta-Sigma ADC with PGA and Voltage Reference

1 Features

- Low Power Consumption: As Low as 280 μ A
- Low-Noise PGA: 19 nV_{RMS} at Gain = 128
- Programmable Gain: 1 to 128
- Programmable Data Rates: 2.5 SPS to 4 kSPS
- Simultaneous 50-Hz and 60-Hz Rejection at ≤ 20 SPS with Low-Latency Digital Filter
- Analog Multiplexer with 12 (ADS124S08) or 6 (ADS124S06) Independently Selectable Inputs
- Dual-Matched Programmable Current Sources for Sensor Excitation: 10 μ A to 2000 μ A
- Internal Reference: 2.5 V, 10 ppm/ $^{\circ}$ C (max) Drift
- Internal Oscillator: 4.096 MHz, 1.5% Accuracy
- Internal Temperature Sensor
- Extended Fault Detection Circuits
- Self Offset and System Calibration
- Four General-Purpose I/Os
- SPI-Compatible Interface with Optional CRC
- Analog Supply: Unipolar (2.7 V to 5.25 V) or Bipolar (± 2.5 V)
- Digital Supply: 2.7 V to 3.6 V
- Operating Temperature: -50° C to $+125^{\circ}$ C

2 Applications

- Sensor Transducers and Transmitters: Temperature, Pressure, Strain, Flow
- PLC and DCS Analog Input Modules
- Temperature Controllers
- Climate Chambers, Industrial Ovens

3 Description

The ADS124S06 and ADS124S08 are precision, 24-bit, delta-sigma ($\Delta\Sigma$), analog-to-digital converters (ADCs) that offer low power consumption and many integrated features to reduce system cost and component count in applications measuring small-signal sensors.

These ADCs feature configurable digital filters that offer low-latency conversion results and 50-Hz or 60-Hz rejection for noisy industrial environments. A low-noise, programmable gain amplifier (PGA) provides gains ranging from 1 to 128 to amplify low-level signals for resistive bridge or thermocouple applications. Additionally, these devices integrate a low-drift, 2.5-V reference that reduces printed circuit board (PCB) area. Finally, two programmable excitation current sources (IDACs) allow for easy and accurate RTD biasing.

An input multiplexer supports 12 inputs for the ADS124S08 and six inputs for the ADS124S06 that can be connected to the ADC in any combination for design flexibility. In addition, these devices include features such as sensor burn-out detection, voltage bias for thermocouples, system monitoring, and four general-purpose I/Os.

The devices are offered in a leadless VQFN-32 or a TQFP-32 package.

Device Information

ORDER NUMBER	PACKAGE (PIN)	BODY SIZE
ADS124S0x	TQFP (32)	5.0 mm $\times$ 5.0 mm
	VQFN (32)	5.0 mm $\times$ 5.0 mm

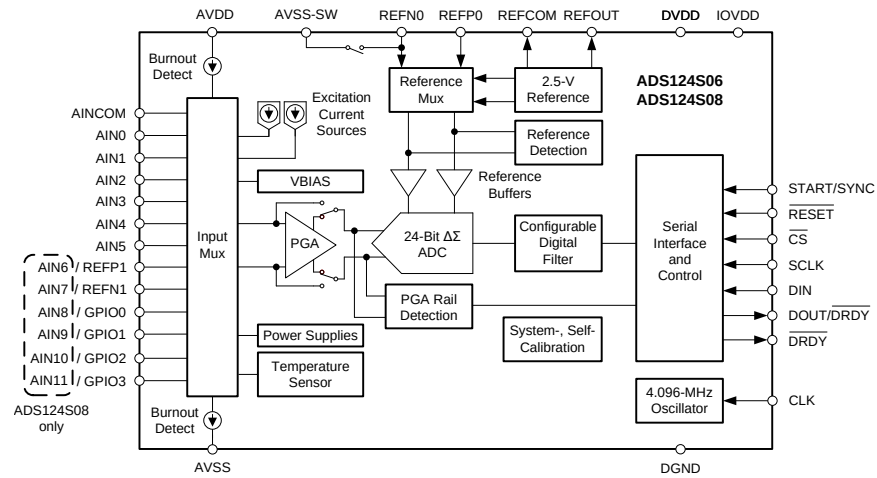


ADS124S06, ADS124S08

SBAS660C –AUGUST 2016–REVISED JUNE 2017

www.ti.com

Functional Block Diagram



Copyright © 2016, Texas Instruments Incorporated

Table of Contents

1 Features	1	10 Application and Implementation	86
2 Applications	1	10.1 Application Information.....	86
3 Description	1	10.2 Typical Application	91
4 Revision History	3	10.3 Do's and Don'ts	96
5 Device Family Comparison Table	5	11 Power Supply Recommendations	98
6 Pin Configuration and Functions	5	11.1 Power Supplies	98
7 Specifications	7	11.2 Power-Supply Sequencing.....	98
7.1 Absolute Maximum Ratings	7	11.3 Power-On Reset.....	98
7.2 ESD Ratings.....	7	11.4 Power-Supply Decoupling.....	98
7.3 Recommended Operating Conditions.....	8	12 Layout	99
7.4 Thermal Information	8	12.1 Layout Guidelines	99
7.5 Electrical Characteristics.....	9	12.2 Layout Example	100
7.6 Timing Characteristics.....	14	13 Device and Documentation Support	101
7.7 Switching Characteristics	14	13.1 Device Support.....	101
7.8 Typical Characteristics	17	13.2 Documentation Support	101
8 Parameter Measurement Information	24	13.3 Related Links	101
8.1 Noise Performance	24	13.4 Receiving Notification of Documentation Updates.....	101
9 Detailed Description	29	13.5 Community Resources.....	101
9.1 Overview	29	13.6 Trademarks	101
9.2 Functional Block Diagram	30	13.7 Electrostatic Discharge Caution.....	101
9.3 Feature Description.....	31	13.8 Glossary	102
9.4 Device Functional Modes.....	58	14 Mechanical, Packaging, and Orderable Information	102
9.5 Programming.....	62		
9.6 Register Map.....	72		

4 Revision History

NOTE: Page numbers for previous revisions may differ from page numbers in the current version.

Changes from Revision B (November 2016) to Revision C	Page
• Added note to <i>Pin Configuration and Functions</i> section	5
• Added TQFP package to test conditions of first row and added second row to Internal Voltage Reference, <i>Accuracy</i> parameter	11
• Changed typical specification of Reference noise parameter from 17.5 μV_{PP} to 9 μV_{PP}	11
• Changed $\overline{\text{DRDY}}$ trace of <i>START Command Timing Requirements</i> figure.....	16
• Changed <i>Typical Characteristics</i> section	17
• Changed values in Table 1.....	24
• Changed values in Table 3.....	25
• Added footnote 1 to Table 5	26
• Changed values in Table 5.....	26
• Added footnote 1 to Table 5 and Table 6	26
• Changed values in Table 7.....	27
• Added last paragraph to <i>PGA Input-Voltage Requirements</i> section	34
• Added <i>when the internal reference voltage is enabled</i> to fifth sentence of <i>Internal Reference</i> section	36
• Added last paragraph to <i>Low-Latency Filter Frequency Response</i> section	38
• Changed footnotes 1 and 3 of Table 13.....	42
• Added link to ADS1x4S0x design calculator to <i>Sinc³ Filter Frequency Response</i> section.....	43
• Changed footnotes 1 and 3 of Table 15	45
• Changed footnotes 1 and 3 of Table 18.....	48
• Changed footnotes 1 and 3 of Table 19.....	49

Revision History (continued)

• Changed IDAC routing description in <i>IDAC Block Diagram</i> figure	50
• Added last sentence to first paragraph of <i>PGA Output Voltage Rail Monitors</i> section	53
• Changed third sentence in last paragraph of <i>WREG</i> section for clarity	68
• Added footnote 1 to <i>GPIO Configuration Register</i> section.....	85
• Changed cross-reference in last paragraph of <i>External Reference and Ratiometric Measurements</i> section to point to the <i>Typical Application</i> section	88
• Added last sentence to second paragraph of <i>Unused Inputs and Outputs</i> section	89
• Added <i>Send the RDATA command</i> ; to <i>Pseudo Code Example</i> section	90
• Changed <i>Internal Reference</i> block text to <i>2.5-V Reference</i> and deleted connection dot from AVSS-SW pin in <i>3-Wire RTD Application</i> figure.....	91
• Changed <i>3-Wire RTD Measurement with Lead-Wire Compensation</i> section title to <i>Register Settings</i>	94
• Added <i>on all other analog inputs</i> to first sentence of sixth bullet in <i>Do's and Don'ts</i> section	96
• Changed second sentence of <i>Power-Supply Decoupling</i> section for clarity	98
• Added <i>Device Support</i> section	101

Changes from Revision A (August 2016) to Revision B

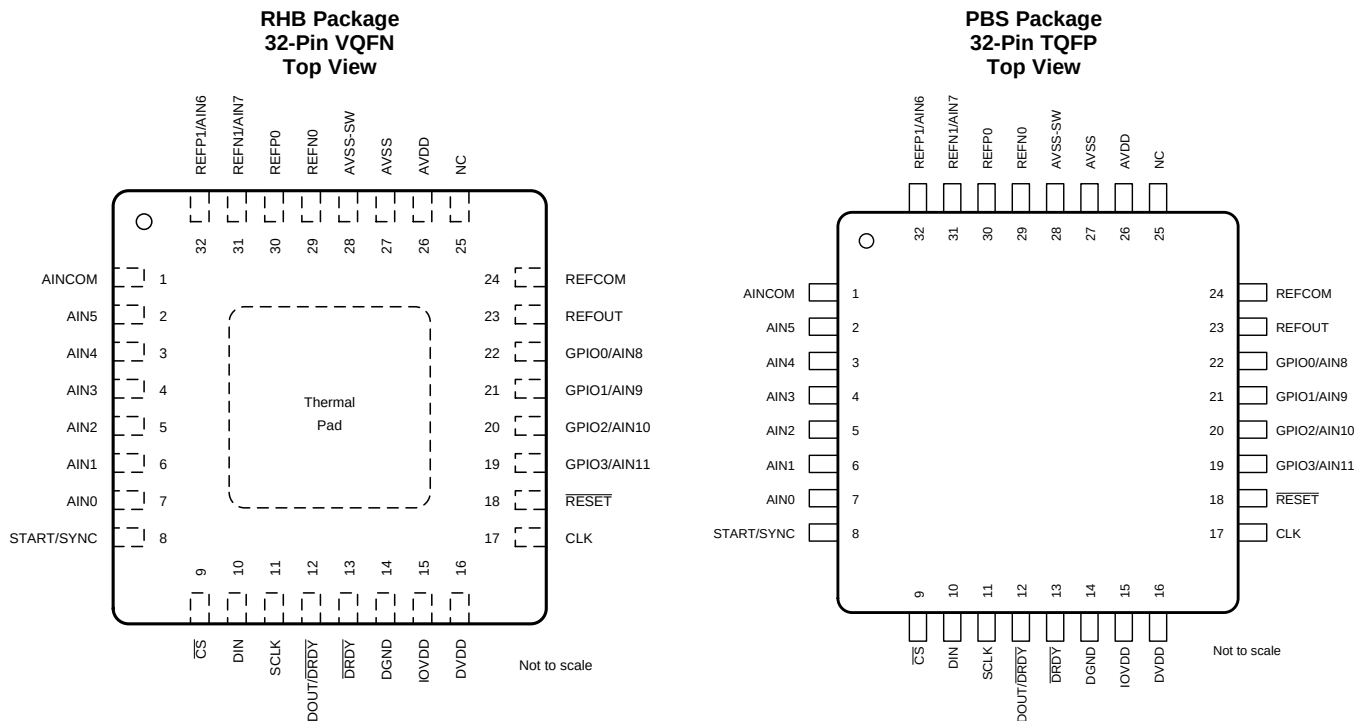
Page

• Released to production	1
--------------------------------	---

5 Device Family Comparison Table

PRODUCT	RESOLUTION (Bits)	NUMBER OF INPUTS
ADS124S08	24	12 analog inputs
ADS124S06	24	6 analog inputs
ADS114S08	16	12 analog inputs
ADS114S06	16	6 analog inputs

6 Pin Configuration and Functions



NOTE: The analog input functions (AIN6–AIN11) are not available on pins 19 to 22, 31, and 32 for the ADS124S06.

ADS124S06, ADS124S08

SBAS660C –AUGUST 2016–REVISED JUNE 2017

www.ti.com
Pin Functions

PIN		FUNCTION	DESCRIPTION ⁽¹⁾
NO.	NAME		
1	AINCOM	Analog input	Common analog input for single-ended measurements
2	AIN5	Analog input	Analog input 5
3	AIN4	Analog input	Analog input 4
4	AIN3	Analog input	Analog input 3
5	AIN2	Analog input	Analog input 2
6	AIN1	Analog input	Analog input 1
7	AIN0	Analog input	Analog input 0
8	START/SYNC	Digital input	Start conversion
9	$\overline{\text{CS}}$	Digital input	Chip select; active low
10	DIN	Digital input	Serial data input
11	SCLK	Digital input	Serial clock input
12	DOUT/ $\overline{\text{DRDY}}$	Digital output	Serial data output combined with data ready; active low
13	$\overline{\text{DRDY}}$	Digital output	Data ready; active low
14	DGND	Digital ground	Digital ground
15	IOVDD	Digital supply	Digital I/O power supply. In case IOVDD is not tied to DVDD, connect a 100-nF (or larger) capacitor to DGND.
16	DVDD	Digital supply	Digital core power supply. Connect a 100-nF (or larger) capacitor to DGND.
17	CLK	Digital input	External clock input. Connect to DGND to use the internal oscillator.
18	$\overline{\text{RESET}}$	Digital input	Reset; active low
19	GPIO3/AIN11	Analog input/output	General-purpose I/O ⁽²⁾ ; analog input 11 (ADS124S08 only)
20	GPIO2/AIN10	Analog input/output	General-purpose I/O ⁽²⁾ ; analog input 10 (ADS124S08 only)
21	GPIO1/AIN9	Analog input/output	General-purpose I/O ⁽²⁾ ; analog input 9 (ADS124S08 only)
22	GPIO0/AIN8	Analog input/output	General-purpose I/O ⁽²⁾ ; analog input 8 (ADS124S08 only)
23	REFOUT	Analog output	Positive voltage reference output. Connect a 1- μ F to 47- μ F capacitor to REFCOM if the internal voltage reference is enabled.
24	REFCOM	Analog output	Negative voltage reference output. Connect to AVSS.
25	NC	—	Leave unconnected or connect to AVSS
26	AVDD	Analog supply	Positive analog power supply. Connect a 330-nF (or larger) capacitor to AVSS.
27	AVSS	Analog supply	Negative analog power supply
28	AVSS-SW	Analog supply	Negative analog power supply; low-side switch. Connect to AVSS.
29	REFN0	Analog input	Negative external reference input 0
30	REFP0	Analog input	Positive external reference input 0
31	REFN1/AIN7	Analog input	Negative external reference input 1; analog input 7 (ADS124S08 only)
32	REFP1/AIN6	Analog input	Positive external reference input 1; analog input 6 (ADS124S08 only)
Pad	Thermal pad	—	RHB package only. Thermal power pad. Connect to AVSS.

(1) See the [Unused Inputs and Outputs](#) section for details on how to connect unused pins.

(2) General-purpose inputs and outputs use logic levels based on the analog supply.

7 Specifications

7.1 Absolute Maximum Ratings⁽¹⁾

		MIN	MAX	UNIT
Power-supply voltage	AVDD to AVSS	−0.3	5.5	V
	AVSS to DGND	−2.8	0.3	
	DVDD to DGND	−0.3	3.9	
	IOVDD to DGND	−0.3	5.5	
Analog input voltage	AINx, GPIOx, REFPx, REFNx, REFCOM	AVSS − 0.3	AVDD + 0.3	V
Digital input voltage	$\overline{\text{CS}}$, SCLK, DIN, DOUT/DRDY, $\overline{\text{DRDY}}$, START, $\overline{\text{RESET}}$, CLK	DGND − 0.3	IOVDD + 0.3	V
Input current	Continuous, AVSS-SW, REFN0, REFOUT	−100	100	mA
	Continuous, all other pins except power-supply pins	−10	10	
Temperature	Junction, T _J		150	°C
	Storage, T _{stg}	−60	150	

- (1) Stresses beyond those listed under *Absolute Maximum Ratings* may cause permanent damage to the device. These are stress ratings only, which do not imply functional operation of the device at these or any other conditions beyond those indicated under *Recommended Operating Conditions*. Exposure to absolute-maximum-rated conditions for extended periods may affect device reliability.

7.2 ESD Ratings

		VALUE	UNIT
V _(ESD) Electrostatic discharge	Human-body model (HBM), per ANSI/ESDA/JEDEC JS-001 ⁽¹⁾	±2500	V
	Charged-device model (CDM), per JEDEC specification JESD22-C101 ⁽²⁾	±1000	

- (1) JEDEC document JEP155 states that 500-V HBM allows safe manufacturing with a standard ESD control process.

- (2) JEDEC document JEP157 states that 250-V CDM allows safe manufacturing with a standard ESD control process.

7.3 Recommended Operating Conditions

over operating ambient temperature range (unless otherwise noted)

		MIN	NOM	MAX	UNIT
POWER SUPPLY					
Analog power supply	AVDD to AVSS	2.7		5.25	V
	AVSS to DGND	–2.625	0	0.05	
	AVDD to DGND	1.5		5.25	
Digital core power supply	DVDD to DGND	2.7		3.6	V
Digital IO power supply	IOVDD to DGND	DVDD		5.25	V
ANALOG INPUTS⁽¹⁾					
$V_{(AINx)}$ Absolute input voltage ⁽²⁾	PGA bypassed	AVSS – 0.05		AVDD + 0.05	V
	PGA enabled, gain = 1 to 16	AVSS + 0.15 + $ V_{INMAX} \cdot (\text{Gain} - 1) / 2$		AVDD – 0.15 – $ V_{INMAX} \cdot (\text{Gain} - 1) / 2$	
	PGA enabled, gain = 32 to 128	AVSS + 0.15 + $15.5 \cdot V_{INMAX} $		AVDD – 0.15 – $15.5 \cdot V_{INMAX} $	
V_{IN} Differential input voltage	$V_{IN} = V_{AINP} - V_{AINN}$	– V_{REF} / Gain		V_{REF} / Gain	V
VOLTAGE REFERENCE INPUTS⁽³⁾					
V_{REF} Differential reference input voltage	$V_{REF} = V_{(REFPx)} - V_{(REFNx)}$	0.5		AVDD – AVSS	V
$V_{(REFNx)}$ Absolute negative reference voltage	Negative reference buffer disabled	AVSS – 0.05		$V_{(REFPx)} - 0.5$	V
	Negative reference buffer enabled	AVSS		$V_{(REFPx)} - 0.5$	V
$V_{(REFPx)}$ Absolute positive reference voltage	Positive reference buffer disabled	$V_{(REFNx)} + 0.5$		AVDD + 0.05	V
	Positive reference buffer enabled	$V_{(REFNx)} + 0.5$		AVDD	V
EXTERNAL CLOCK SOURCE⁽⁴⁾					
f_{CLK} External clock frequency		2	4.096	4.5	MHz
Duty cycle		40%	50%	60%	
GENERAL-PURPOSE INPUTS (GPIOs)					
Input voltage		AVSS – 0.05		AVDD + 0.05	V
DIGITAL INPUTS (Other than GPIOs)					
Input voltage		DGND		IOVDD	V
TEMPERATURE RANGE					
T_A Operating ambient temperature		–50		125	°C

- (1) A_{INP} and A_{INN} denote the positive and negative inputs of the PGA. Any of the available analog inputs (A_{INx}) can be selected as either A_{INP} or A_{INN} by the input multiplexer.
- (2) V_{INMAX} denotes the maximum differential input voltage, V_{IN} , that is expected in the application. $|V_{INMAX}|$ can be smaller than V_{REF} / Gain .
- (3) REFPx and REFNx denote one of the two available external differential reference input pairs.
- (4) An external clock is not required when the internal oscillator is used.

7.4 Thermal Information

THERMAL METRIC ⁽¹⁾		ADS124S06, ADS124S08		UNIT
		VQFN (RHB)	TQFP (PBS)	
		32 PINS	32 PINS	
$R_{\theta JA}$	Junction-to-ambient thermal resistance	45.2	75.5	°C/W
$R_{\theta JC(top)}$	Junction-to-case (top) thermal resistance	28.3	17.1	°C/W
$R_{\theta JB}$	Junction-to-board thermal resistance	15.8	28.5	°C/W
Ψ_{JT}	Junction-to-top characterization parameter	0.4	0.4	°C/W
Ψ_{JB}	Junction-to-board characterization parameter	15.7	28.3	°C/W
$R_{\theta JC(bot)}$	Junction-to-case (bottom) thermal resistance	2.3	n/a	°C/W

- (1) For more information about traditional and new thermal metrics, see the [Semiconductor and IC Package Thermal Metrics](#) application report.

7.5 Electrical Characteristics

minimum and maximum specifications apply from $T_A = -50^{\circ}\text{C}$ to $+125^{\circ}\text{C}$; Typical specifications are at $T_A = 25^{\circ}\text{C}$; all specifications are at $AVDD = 2.7\text{ V}$ to 5.25 V , $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, all gains, internal reference, internal oscillator, all data rates, and global chop disabled (unless otherwise noted)

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
ANALOG INPUTS						
Absolute input current	PGA bypassed, AVSS + 0.1 V ≤ V _(AINx) ≤ AVDD − 0.1 V		0.5			nA
	PGA enabled, all gains, V _{(AINx)MIN} ≤ V _(AINx) ≤ V _{(AINx)MAX}	−2	0.1 2			
Absolute input current drift	PGA bypassed, AVSS + 0.1 V ≤ V _(AINx) ≤ AVDD − 0.1 V		2			pA/°C
	PGA enabled, all gains, V _{(AINx)MIN} ≤ V _(AINx) ≤ V _{(AINx)MAX}		2			
Differential input current	PGA bypassed, V _{CM} = AVDD / 2, −V _{REF} ≤ V _{IN} ≤ V _{REF}		1			nA/V
	PGA enabled, all gains, V _{CM} = AVDD / 2, −V _{REF} / Gain ≤ V _{IN} ≤ V _{REF} / Gain	−1	0.02	1		nA
Differential input current drift	PGA bypassed, V _{CM} = AVDD / 2, −V _{REF} ≤ V _{IN} ≤ V _{REF}		3			pA/°C
	PGA enabled, all gains, V _{CM} = AVDD / 2, −V _{REF} / Gain ≤ V _{IN} ≤ V _{REF} / Gain		1			
PGA						
Gain settings			1, 2, 4, 8, 16, 32, 64, 128			
Startup time		Enabling the PGA in conversion mode	190			μs
SYSTEM PERFORMANCE						
Resolution (no missing codes)			24			Bits
DR	Data rate		2.5, 5, 10, 16.6, 20, 50, 60, 100, 200, 400, 800, 1000, 2000, 4000			SPS
INL	Integral nonlinearity (best fit)	PGA bypassed, V _{CM} = AVDD / 2	1 10			ppm _{FSR}
		PGA enabled, gain = 1 to 8, V _{CM} = AVDD / 2	2 15			
		PGA enabled, gain = 16 to 128, V _{CM} = AVDD / 2, T _A = −40°C to +85°C	3 15			
V _{IO}	Input offset voltage	T _A = 25°C, PGA bypassed	−120	20	120	μV
		T _A = 25°C, PGA enabled, gain = 1 to 8	−120 / Gain	20 / Gain	120 / Gain	
		T _A = 25°C, PGA enabled, gain = 16 to 128	−15	2	15	
		T _A = 25°C, PGA bypassed, after internal offset calibration	On the order of noise _{pp} at the set DR and gain			
		T _A = 25°C, PGA enabled, gain = 1 to 128, after internal offset calibration	On the order of noise _{pp} at the set DR and gain			
		T _A = 25°C, PGA bypassed, global chop enabled	−2	0.2	2	
		T _A = 25°C, PGA enabled, gain = 1 to 128, global chop enabled	−2	0.2	2	
Offset drift	T _A = −40°C to +85°C, PGA bypassed	−75	10	75	nV/°C	
	T _A = −40°C to +85°C, PGA enabled, gain = 1 to 128	−100	15	100		
	PGA bypassed	−75	10	75		
	PGA enabled, gain = 1 to 8	−200	15	200		
	PGA enabled, gain = 16 to 128	−150	15	150		
	PGA bypassed, global chop enabled	−10	2	10		
	PGA enabled, gain = 1 to 128, global chop enabled	−10	2	10		

Electrical Characteristics (continued)

minimum and maximum specifications apply from $T_A = -50^{\circ}\text{C}$ to $+125^{\circ}\text{C}$; Typical specifications are at $T_A = 25^{\circ}\text{C}$; all specifications are at $AVDD = 2.7\text{ V}$ to 5.25 V , $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, all gains, internal reference, internal oscillator, all data rates, and global chop disabled (unless otherwise noted)

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
SYSTEM PERFORMANCE (continued)						
Gain error ⁽¹⁾		$T_A = 25^{\circ}\text{C}$, PGA bypassed		40	120	ppm
		$T_A = 25^{\circ}\text{C}$, PGA enabled, gain = 1 to 32		40	120	
		$T_A = 25^{\circ}\text{C}$, PGA enabled, gain = 64 and 128		40	200	
Gain drift ⁽¹⁾		$T_A = -40^{\circ}\text{C}$ to $+85^{\circ}\text{C}$, PGA bypassed		0.5	1	ppm/ $^{\circ}\text{C}$
		$T_A = -40^{\circ}\text{C}$ to $+85^{\circ}\text{C}$, PGA enabled, gain = 1 to 128		0.5	2	
		PGA bypassed		0.5	1	
		PGA enabled, gain = 1 to 128		1	4	
Noise (input-referred) ⁽²⁾		PGA enabled, gain = 128, DR = 2.5 SPS, sinc ³ filter		19		nV _{RMS}
NMRR Normal-mode rejection ratio ⁽³⁾		$f_{IN} = 50\text{ Hz}$ or 60 Hz ($\pm 1\text{ Hz}$), DR = 10 SPS, sinc ³ filter	88			dB
		$f_{IN} = 50\text{ Hz}$ or 60 Hz ($\pm 1\text{ Hz}$), DR = 10 SPS, sinc ³ filter, external $f_{CLK} = 4.096\text{ MHz}$	102			
		$f_{IN} = 50\text{ Hz}$ or 60 Hz ($\pm 1\text{ Hz}$), DR = 20 SPS, low-latency filter	79			
		$f_{IN} = 50\text{ Hz}$ or 60 Hz ($\pm 1\text{ Hz}$), DR = 20 SPS, low-latency filter, external $f_{CLK} = 4.096\text{ MHz}$	95			
		$f_{IN} = 50\text{ Hz}$ ($\pm 1\text{ Hz}$), DR = 50 SPS, sinc ³ filter	87			
		$f_{IN} = 50\text{ Hz}$ ($\pm 1\text{ Hz}$), DR = 50 SPS, sinc ³ filter, external $f_{CLK} = 4.096\text{ MHz}$	101			
		$f_{IN} = 60\text{ Hz}$ ($\pm 1\text{ Hz}$), DR = 60 SPS, sinc ³ filter	89			
		$f_{IN} = 60\text{ Hz}$ ($\pm 1\text{ Hz}$), DR = 60 SPS, sinc ³ filter, external $f_{CLK} = 4.096\text{ MHz}$	105			
CMRR Common-mode rejection ratio		At dc	110	120		dB
		$f_{CM} = 50\text{ Hz}$ or 60 Hz ($\pm 1\text{ Hz}$), DR = 2.5 SPS to 10 SPS, sinc ³ filter	120	130		
		$f_{CM} = 50\text{ Hz}$ or 60 Hz ($\pm 1\text{ Hz}$), DR = 2.5 SPS, 5 SPS, 10 SPS, 20 SPS, low-latency filter	115	125		
PSRR Power-supply rejection ratio		AVDD at dc	90	105		dB
		AVDD at 50 Hz or 60 Hz	100	115		
		DVDD at dc	100	115		

(1) Excluding error of voltage reference.

(2) See the [Noise Performance](#) section for more information.

(3) See the [50-Hz and 60-Hz Line Cycle Rejection](#) section for more information.

Electrical Characteristics (continued)

minimum and maximum specifications apply from $T_A = -50^{\circ}\text{C}$ to $+125^{\circ}\text{C}$; Typical specifications are at $T_A = 25^{\circ}\text{C}$; all specifications are at $AVDD = 2.7\text{ V}$ to 5.25 V , $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, all gains, internal reference, internal oscillator, all data rates, and global chop disabled (unless otherwise noted)

PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
VOLTAGE REFERENCE INPUTS					
Absolute input current	Reference buffers disabled, external $V_{REF} = 2.5\text{ V}$, REF1P/REF1N inputs	-6	4	6	$\mu\text{A/V}$
	Reference buffers enabled, external $V_{REF} = 2.5\text{ V}$, REF1P/REF1N inputs	-15	5	15	nA
INTERNAL VOLTAGE REFERENCE					
V_{REF} Output voltage			2.5		V
Accuracy	$T_A = 25^{\circ}\text{C}$, TQFP package	-0.05%	$\pm 0.01\%$	0.05%	
	$T_A = 25^{\circ}\text{C}$, VQFN package	-0.1%	$\pm 0.01\%$	0.1%	
Temperature drift	$T_A = -40^{\circ}\text{C}$ to $+85^{\circ}\text{C}$		2.5	8	ppm/ $^{\circ}\text{C}$
	$T_A = -50^{\circ}\text{C}$ to $+125^{\circ}\text{C}$		3	10	
Output current	$AVDD = 2.7\text{ V}$ to 3.3 V , sink and source	-5		5	mA
	$AVDD = 3.3\text{ V}$ to 5.25 V , sink and source	-10		10	
Short-circuit current limit	Sink and source		70	100	mA
PSRR Power-supply rejection ratio	$AVDD$ at dc		85		dB
Load regulation	$AVDD = 2.7\text{ V}$ to 3.3 V , load current = -5 mA to 5 mA		8		$\mu\text{V/mA}$
	$AVDD = 3.3\text{ V}$ to 5.25 V , load current = -10 mA to 10 mA		8		
Startup time	1- μF capacitor on REFOUT, 0.001% settling		5.9		ms
Capacitive load stability	Capacitor on REFOUT	1		47	μF
Reference noise	$f = 0.1\text{ Hz}$ to 10 Hz , 1- μF capacitor on REFOUT		9		μV_{PP}
INTERNAL OSCILLATOR					
f_{CLK} Frequency			4.096		MHz
Accuracy		-1.5%		1.5%	
EXCITATION CURRENT SOURCES (IDACS)					
Current settings			10, 50, 100, 250, 500, 750, 1000, 1500, 2000		μA
Compliance voltage ⁽⁴⁾	10 μA to 750 μA , 0.1% deviation	$AVSS$		$AVDD - 0.4$	V
	1 mA to 2 mA, 0.1% deviation	$AVSS$		$AVDD - 0.6$	
Accuracy (each IDAC)	$T_A = 25^{\circ}\text{C}$, 10 μA to 100 μA	-5%	$\pm 0.7\%$	5%	
	$T_A = 25^{\circ}\text{C}$, 250 μA to 2 mA	-3%	$\pm 0.5\%$	3%	
Current mismatch between IDACs	$T_A = 25^{\circ}\text{C}$, 10 μA to 100 μA		0.15%	0.8%	
	$T_A = 25^{\circ}\text{C}$, 250 μA to 750 μA		0.10%	0.6%	
	$T_A = 25^{\circ}\text{C}$, 1 mA to 2 mA		0.07%	0.4%	
Temperature drift (each IDAC)	10 μA to 750 μA		20	120	ppm/ $^{\circ}\text{C}$
	1 mA to 2 mA		10	80	
Temperature drift matching between IDACs	10 μA to 100 μA		3	25	ppm/ $^{\circ}\text{C}$
	250 μA to 2 mA		2	15	
Startup time	With internal reference already settled. From end of WREG command to current flowing out of pin.		22		μs

(4) The IDAC current does not change by more than 0.1% from the nominal value when staying within the specified compliance voltage.

Electrical Characteristics (continued)

minimum and maximum specifications apply from $T_A = -50^{\circ}\text{C}$ to $+125^{\circ}\text{C}$; Typical specifications are at $T_A = 25^{\circ}\text{C}$; all specifications are at $\text{AVDD} = 2.7\text{ V}$ to 5.25 V , $\text{AVSS} = 0\text{ V}$, $\text{DVDD} = \text{IOVDD} = 3.3\text{ V}$, all gains, internal reference, internal oscillator, all data rates, and global chop disabled (unless otherwise noted)

PARAMETER		TEST CONDITIONS	MIN	TYP	MAX	UNIT
BIAS VOLTAGE						
V _{BIAS}	Output voltage settings		(AVDD + AVSS) / 2, (AVDD + AVSS) / 12			V
	Output impedance		350			Ω
	Startup time	Combined capacitive load on all selected analog inputs C _{LOAD} = 1 μF, 0.1% settling	2.8			ms
BURNOUT CURRENT SOURCES (BOCS)						
	Current settings		0.2, 1, 10			μA
Accuracy		0.2 μA, sinking or sourcing	±8%			
		1 μA, sinking or sourcing	±4%			
		10 μA, sinking or sourcing	±2%			
PGA RAIL DETECTION						
	Positive rail threshold	Referred to the output of the PGA	AVDD – 0.15			V
	Negative rail threshold	Referred to the output of the PGA	AVSS + 0.15			V
REFERENCE DETECTION						
	Threshold 1		0.3			V
	Threshold 2		1/3·(AVDD – AVSS)			V
	Threshold 2 accuracy		–3%	±1%	3%	
	Pull-together resistance		10			MΩ
SUPPLY VOLTAGE MONITORS						
Accuracy		(AVDD – AVSS) / 4 monitor	±1%			
		DVDD / 4 monitor	±1%			
TEMPERATURE SENSOR						
	Output voltage	T _A = 25°C	129			mV
	Temperature coefficient		403			μV/°C
LOW-SIDE POWER SWITCH						
R _{ON}	On-resistance		1			3 Ω
	Current through switch		75			mA
GENERAL-PURPOSE INPUT/OUTPUTS (GPIOs)						
V _{IL}	Logic input level, low		AVSS – 0.05		0.3 AVDD	V
V _{IH}	Logic input level, high		0.7 AVDD		AVDD + 0.05	V
V _{OL}	Logic output level, low	I _{OL} = 1 mA	AVSS		0.2 AVDD	V
V _{OH}	Logic output level, high	I _{OH} = 1 mA	0.8 AVDD		AVDD	V
DIGITAL INPUT/OUTPUTS						
V _{IL}	Logic input level, low		DGND		0.3 IOVDD	V
V _{IH}	Logic input level, high		0.7 IOVDD		IOVDD	V
V _{OL}	Logic output level, low	I _{OL} = 1 mA	DGND		0.2 IOVDD	V
V _{OH}	Logic output level, high	I _{OH} = 1 mA	0.8 IOVDD		IOVDD	V
	Input current	DGND ≤ V _{Digital Input} ≤ IOVDD	–1		1	μA

Electrical Characteristics (continued)

minimum and maximum specifications apply from $T_A = -50^{\circ}\text{C}$ to $+125^{\circ}\text{C}$; Typical specifications are at $T_A = 25^{\circ}\text{C}$; all specifications are at $AVDD = 2.7\text{ V}$ to 5.25 V , $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, all gains, internal reference, internal oscillator, all data rates, and global chop disabled (unless otherwise noted)

PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT
ANALOG SUPPLY CURRENT (AVDD = 3.3 V, External Reference, Internal Reference Disabled, Reference Buffers Disabled, IDACs Disabled, VBIAS Disabled, Flags Disabled, Internal Oscillator, All Data Rates, VIN = 0 V)					
IAVDD Analog supply current	Power-down mode		0.1	1.5	μA
	Standby mode, PGA bypassed		70		
	Conversion mode, PGA bypassed		85		
	Conversion mode, PGA enabled, gain = 1, 2		120	135	
	Conversion mode, PGA enabled, gain = 4, 8		140	155	
	Conversion mode, PGA enabled, gain = 16, 32		165	180	
	Conversion mode, PGA enabled, gain = 64		200		
	Conversion mode, PGA enabled, gain = 128		250		
ADDITIONAL ANALOG SUPPLY CURRENTS PER FUNCTION (AVDD = 3.3 V)					
IAVDD Analog supply current	Internal 2.5-V reference, no external load		185	280	μA
	Positive reference buffer		35	60	
	Negative reference buffer		25	40	
	VBIAS buffer, no external load		10		
	IDAC overhead, 10 μA to 250 μA		20	35	
	IDAC overhead, 500 μA to 750 μA		30		
	IDAC overhead, 1 mA		40		
	IDAC overhead, 1.5 mA		50		
	IDAC overhead, 2 mA		65		
	PGA rail detection and reference detection circuit		10		
DIGITAL SUPPLY CURRENT (DVDD = IOVDD = 3.3 V, All Data Rates, SPI Not Active)					
IDVDD + IOVDD Digital supply current	Power-down mode, internal oscillator		0.1		μA
	Standby mode, internal oscillator		185		
	Conversion mode, internal oscillator		225	300	
	Conversion mode, external fCLK = 4.096 MHz		195		
POWER DISSIPATION (AVDD = DVDD = IOVDD = 3.3 V, Internal Reference Enabled, Reference Buffers Disabled, IDACs Disabled, VBIAS Disabled, Flags Disabled, Internal Oscillator, All Data Rates, VIN = 0 V, SPI Not Active)					
PD Power dissipation	Conversion mode, PGA enabled, gain = 1		1.75		mW

7.6 Timing Characteristics

over operating ambient temperature range, DVDD = 2.7 V to 3.6 V, IOVDD = DVDD to 5.25 V, and DOUT/DRDY load = 20 pF || 100 kΩ to DGND (unless otherwise noted)

	MIN	MAX	UNIT ⁽¹⁾
SERIAL INTERFACE			
t _{d(CSSC)} Delay time, first SCLK rising edge after $\overline{\text{CS}}$ falling edge	20		ns
t _{d(SCCS)} Delay time, $\overline{\text{CS}}$ rising edge after final SCLK falling edge	20		ns
t _{w(CSH)} Pulse duration, $\overline{\text{CS}}$ high	30		ns
t _{c(SC)} SCLK period	100		ns
t _{w(SCH)} Pulse duration, SCLK high	40		ns
t _{w(SCL)} Pulse duration, SCLK low	40		ns
t _{su(DI)} Setup time, DIN valid before SCLK falling edge	15		ns
t _{h(DI)} Hold time, DIN valid after SCLK falling edge	20		ns
t _{d(CMD)} Delay time, between bytes or commands	0		ns
RESET PIN			
t _{w(RSL)} Pulse duration, $\overline{\text{RESET}}$ low	4		t _{CLK}
t _{d(RSSC)} Delay time, first SCLK rising edge after $\overline{\text{RESET}}$ rising edge (or 7th SCLK falling edge of RESET command)	4096		t _{CLK}
START/SYNC PIN			
t _{w(STH)} Pulse duration, START/SYNC high	4		t _{CLK}
t _{w(STL)} Pulse duration, START/SYNC low	4		t _{CLK}
t _{su(STDR)} Setup time, START/SYNC falling edge (or 7th SCLK falling edge of STOP command) before $\overline{\text{DRDY}}$ falling edge to stop further conversions (continuous conversion mode)	32		t _{CLK}
READING CONVERSION DATA WITHOUT RDATA COMMAND			
t _{h(SCDR)} Hold time, SCLK low before $\overline{\text{DRDY}}$ falling edge ⁽²⁾	28		t _{CLK}
t _{d(DRSC)} Delay time, SCLK rising edge after $\overline{\text{DRDY}}$ falling edge ⁽²⁾	4		t _{CLK}

(1) t_{CLK} = 1 / f_{CLK}.

(2) Only applicable when reading data without the RDATA command. All commands can be send without any SCLK to $\overline{\text{DRDY}}$ signal timing restrictions.

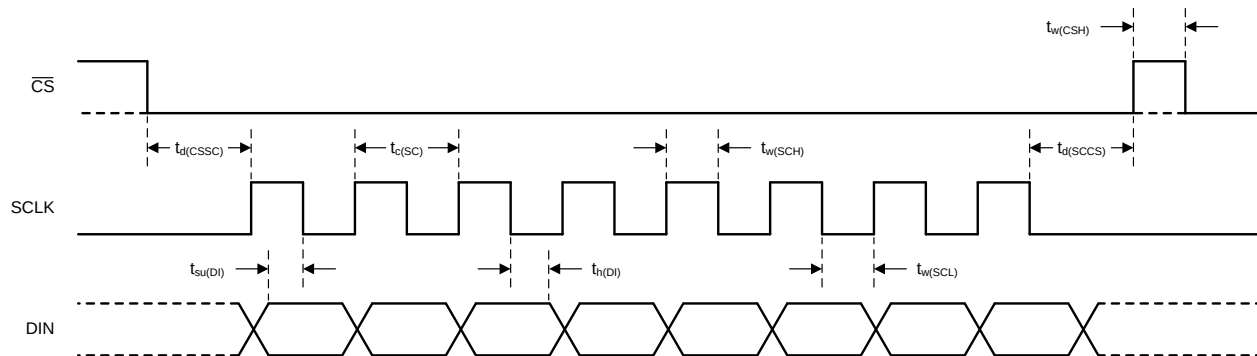
7.7 Switching Characteristics

over operating ambient temperature range, DVDD = 2.7 V to 3.6 V, IOVDD = DVDD to 5.25 V, and DOUT/DRDY load = 20 pF || 100 kΩ to DGND (unless otherwise noted)

PARAMETER	TEST CONDITIONS	MIN	TYP	MAX	UNIT ⁽¹⁾
t _{p(CSDO)} Propagation delay time, $\overline{\text{CS}}$ falling edge to DOUT driven		0		25	ns
t _{p(SCDO)} Propagation delay time, SCLK rising edge to valid new DOUT		3		30	ns
t _{p(CSDOZ)} Propagation delay time, $\overline{\text{CS}}$ rising edge to DOUT high impedance		0		25	ns
t _{p(STDR)} Propagation delay time, START/SYNC rising edge (or first SCLK rising edge of any command or data read) to $\overline{\text{DRDY}}$ rising edge				2	t _{CLK}
t _{w(DRH)} Pulse duration, $\overline{\text{DRDY}}$ high		24			t _{CLK}
t _{p(GPIO)} Propagation delay time, last SCLK falling edge of WREG command to GPIOx output valid		3		100	ns
SPI timeout per 8 bit ⁽²⁾		2 ¹⁵			t _{CLK}

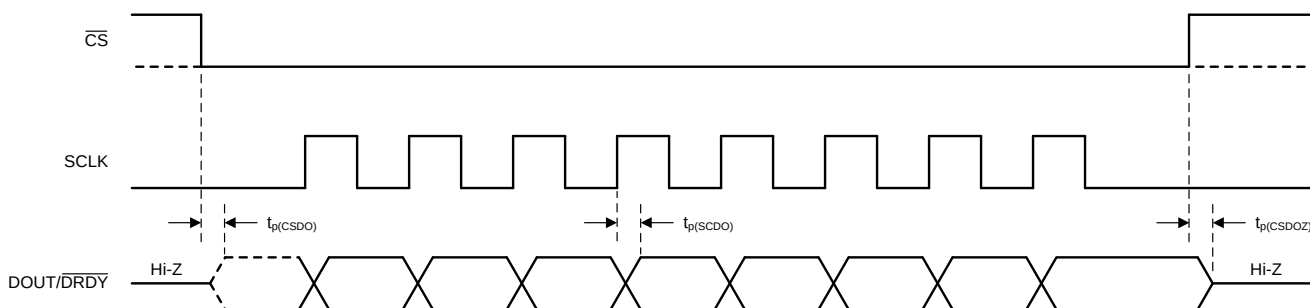
(1) t_{CLK} = 1 / f_{CLK}.

(2) The SPI interface resets when an entire byte is not sent within the specified timeout time.



NOTE: Single-byte communication is shown. Actual communication can be multiple bytes.

Figure 1. Serial Interface Timing Requirements



NOTE: Single-byte communication is shown. Actual communication can be multiple bytes.

Figure 2. Serial Interface Switching Characteristics

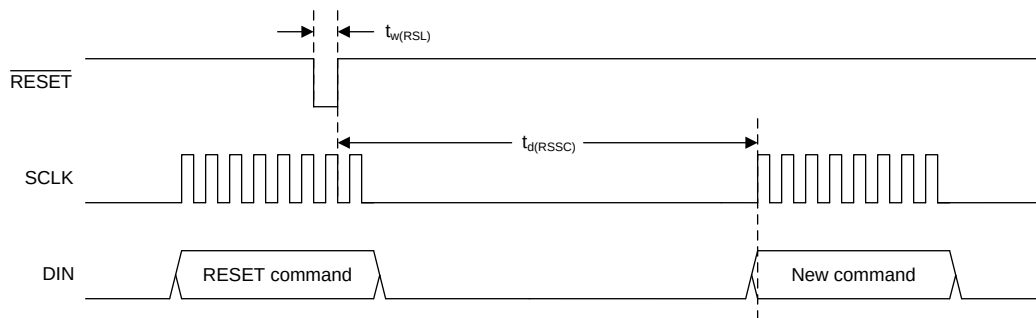


Figure 3. RESET Pin and RESET Command Timing Requirements

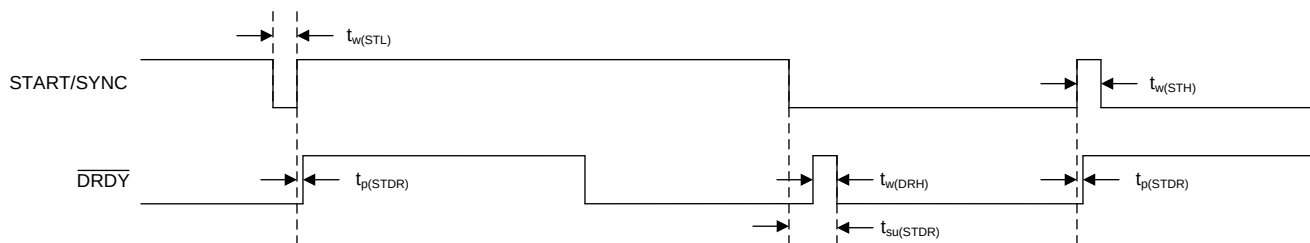
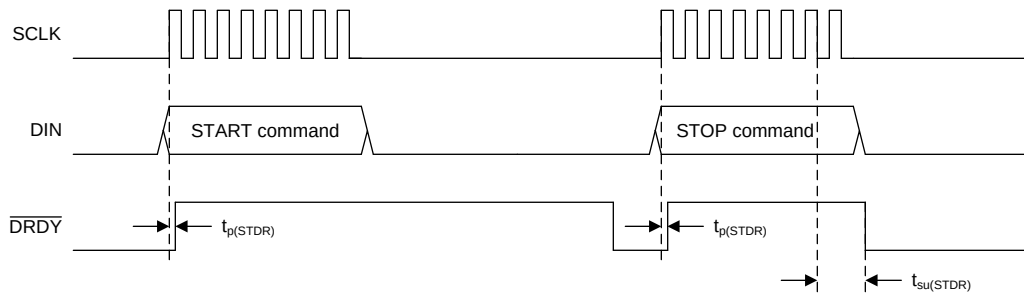
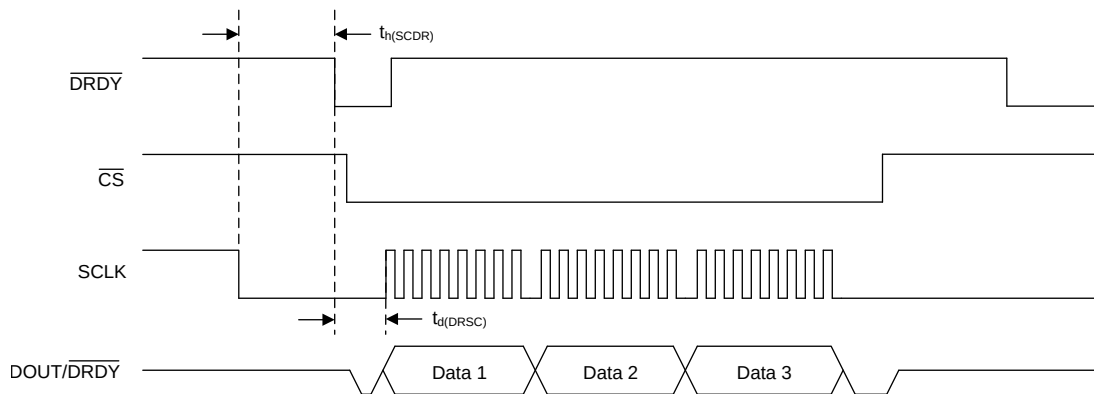
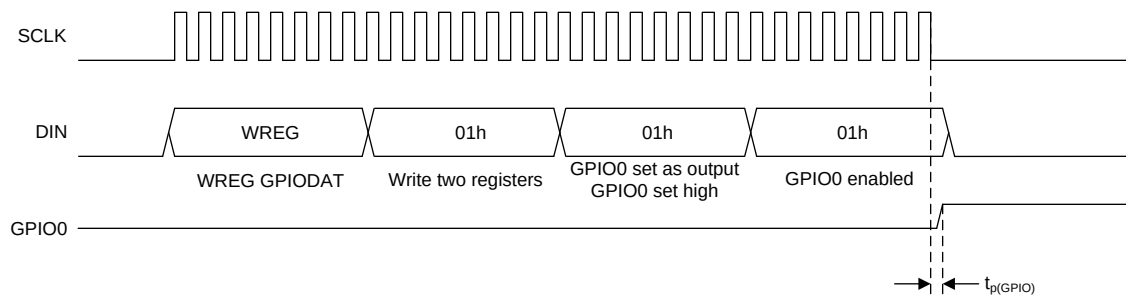


Figure 4. START/SYNC Pin Timing Requirements


Figure 5. START Command Timing Requirements

Figure 6. Read Data Direct (Without an RDATA Command) Timing Requirements

Figure 7. GPIO Switching Characteristics

7.8 Typical Characteristics

at $T_A = 25^\circ\text{C}$, $AVDD = 3.3\text{ V}$, $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, using internal $V_{REF} = 2.5\text{ V}$, internal 4.096-MHz oscillator, and PGA enabled (unless otherwise noted)

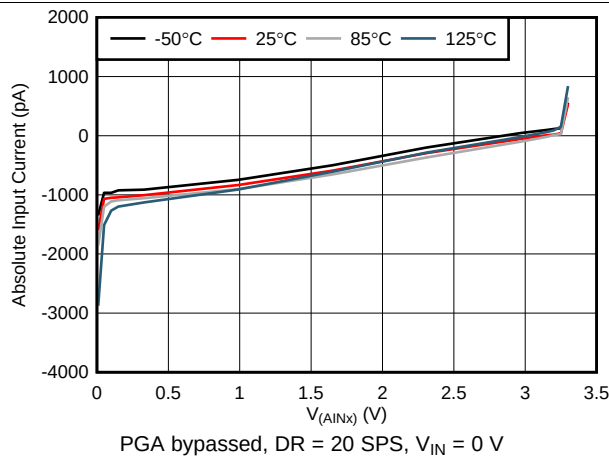


Figure 8. Absolute Input Current vs Absolute Input Voltage

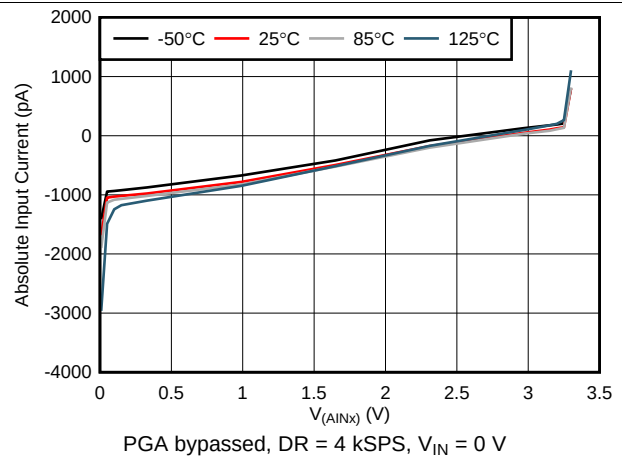


Figure 9. Absolute Input Current vs Absolute Input Voltage

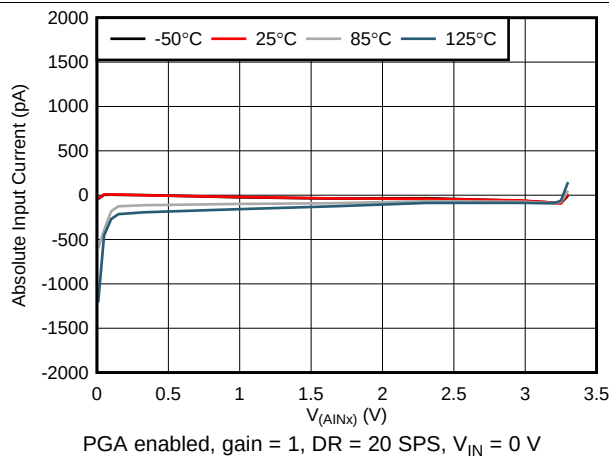


Figure 10. Absolute Input Current vs Absolute Input Voltage

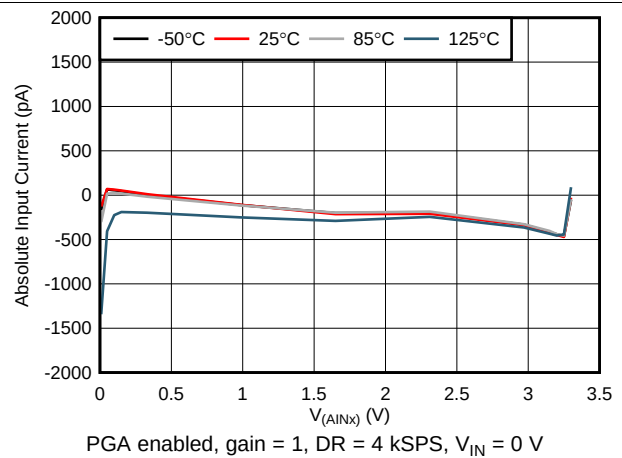


Figure 11. Absolute Input Current vs Absolute Input Voltage

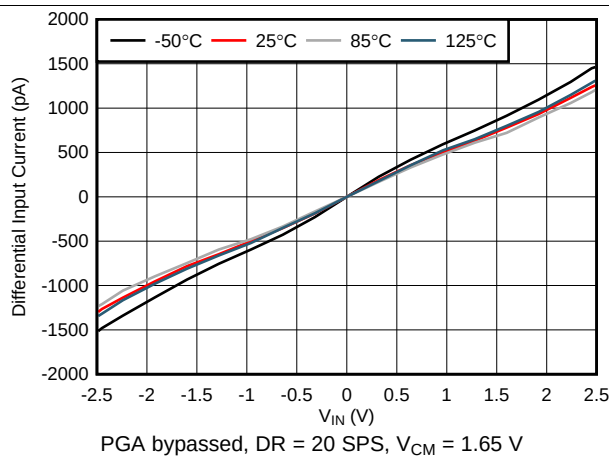


Figure 12. Differential Input Current vs Differential Input Voltage

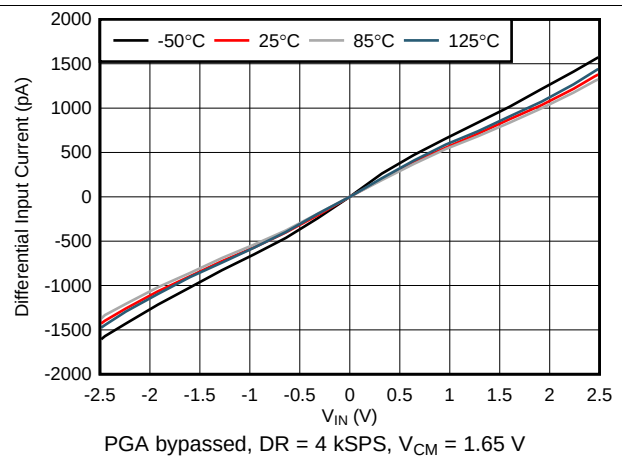


Figure 13. Differential Input Current vs Differential Input Voltage

Typical Characteristics (continued)

at $T_A = 25^\circ\text{C}$, $AVDD = 3.3\text{ V}$, $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, using internal $V_{REF} = 2.5\text{ V}$, internal 4.096-MHz oscillator, and PGA enabled (unless otherwise noted)

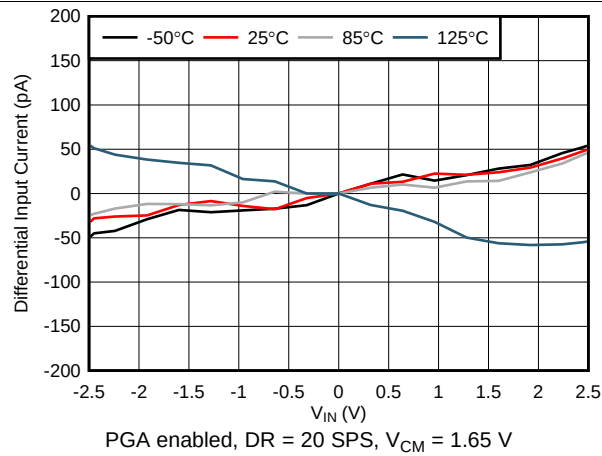


Figure 14. Differential Input Current vs Differential Input Voltage

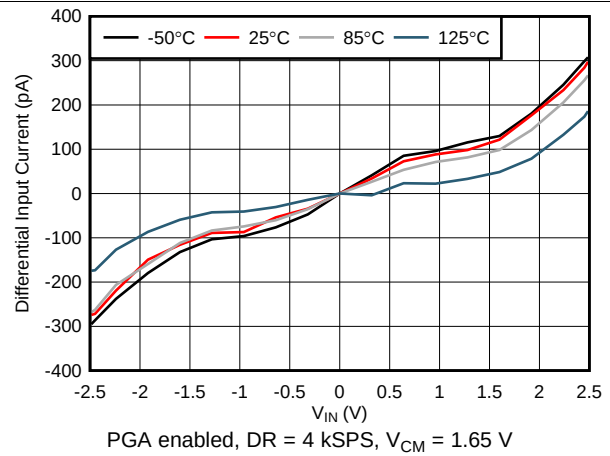


Figure 15. Differential Input Current vs Differential Input Voltage

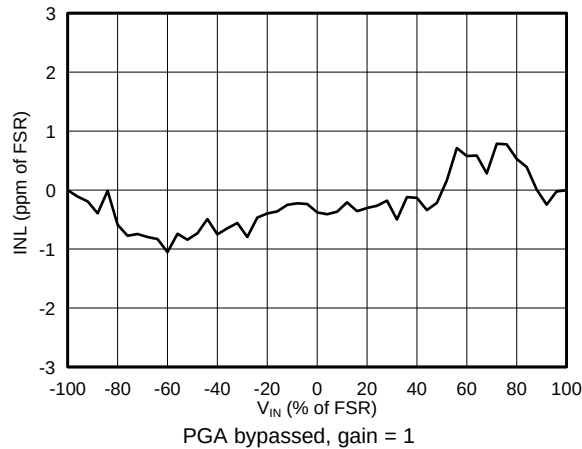


Figure 16. INL vs Differential Input Voltage

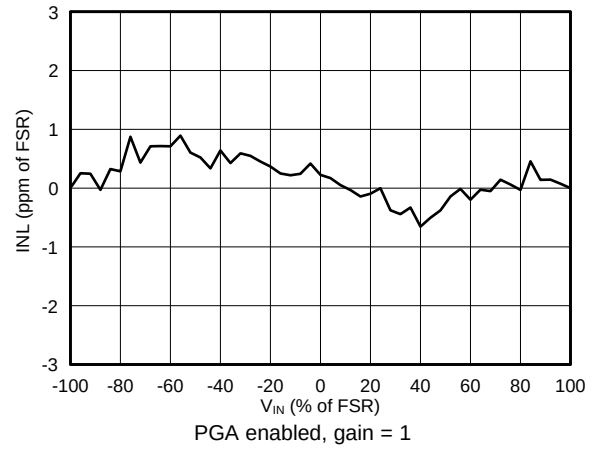


Figure 17. INL vs Differential Input Voltage

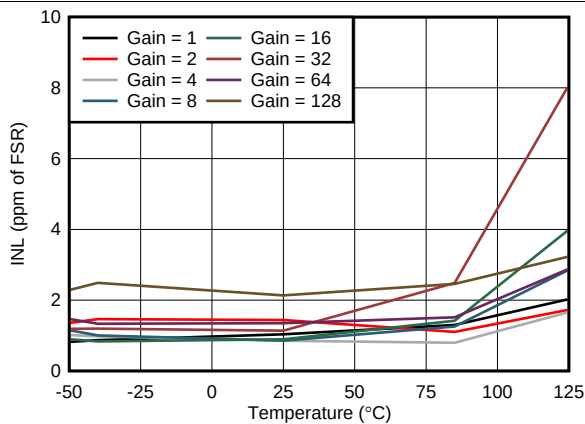


Figure 18. INL vs Temperature

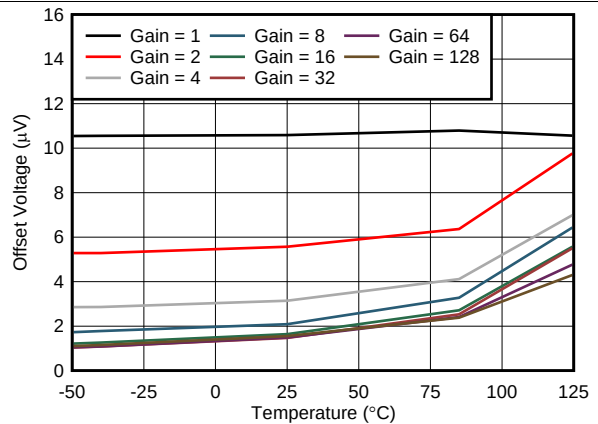


Figure 19. Offset Voltage vs Temperature

Typical Characteristics (continued)

at $T_A = 25^\circ\text{C}$, $AVDD = 3.3\text{ V}$, $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, using internal $V_{REF} = 2.5\text{ V}$, internal 4.096-MHz oscillator, and PGA enabled (unless otherwise noted)

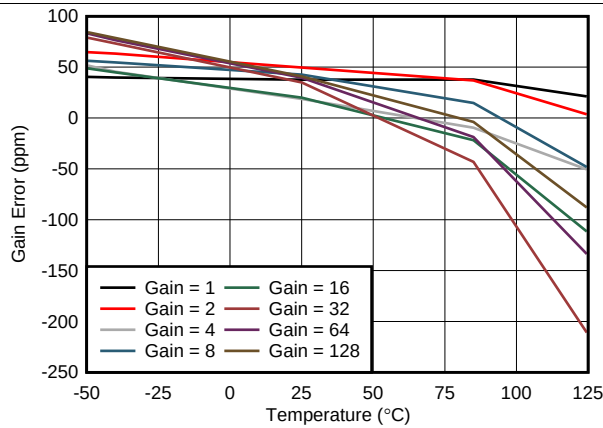


Figure 20. Gain Error vs Temperature

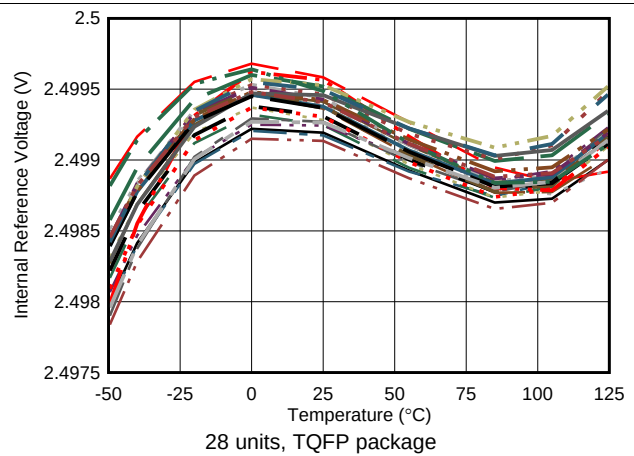


Figure 21. Internal Reference Voltage vs Temperature

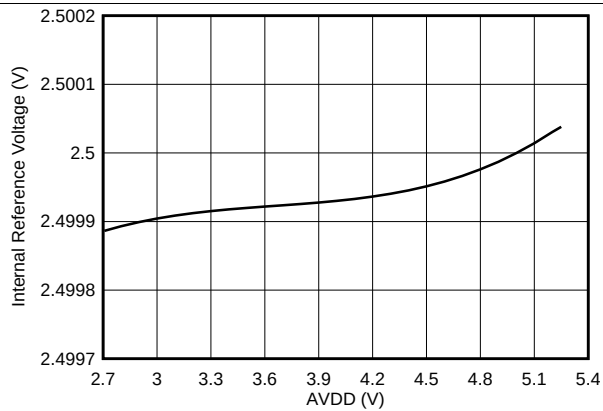


Figure 22. Internal Reference Voltage vs AVDD

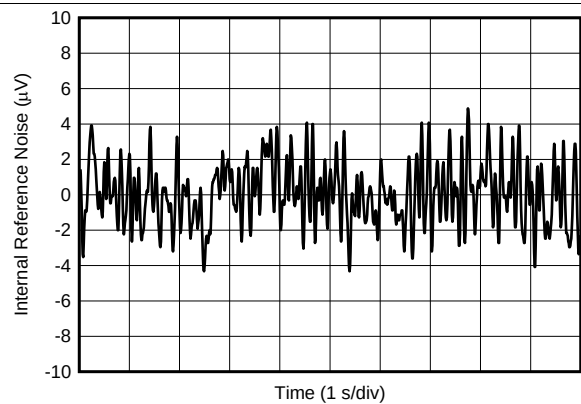


Figure 23. Internal Reference Voltage Noise

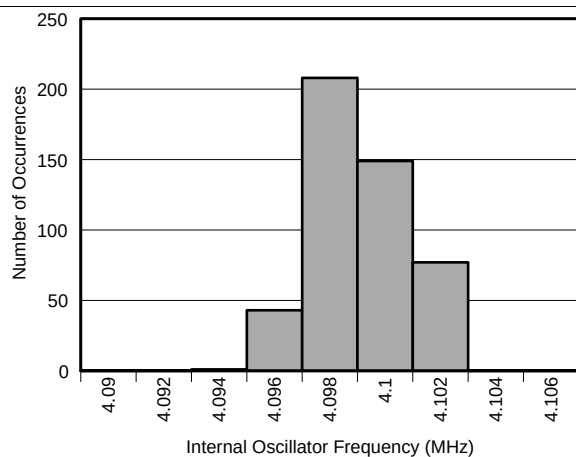


Figure 24. Internal Oscillator Frequency Histogram

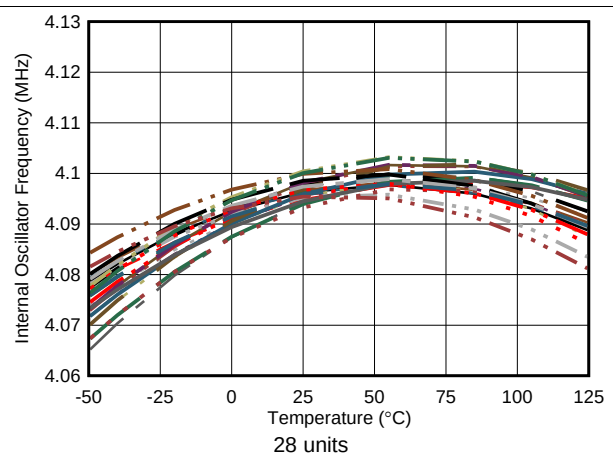


Figure 25. Internal Oscillator Frequency vs Temperature

Typical Characteristics (continued)

at $T_A = 25^\circ\text{C}$, $AVDD = 3.3\text{ V}$, $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, using internal $V_{REF} = 2.5\text{ V}$, internal 4.096-MHz oscillator, and PGA enabled (unless otherwise noted)

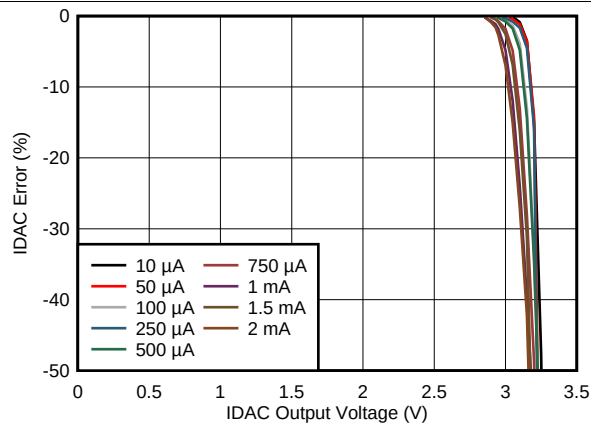


Figure 26. IDAC Accuracy vs Compliance Voltage

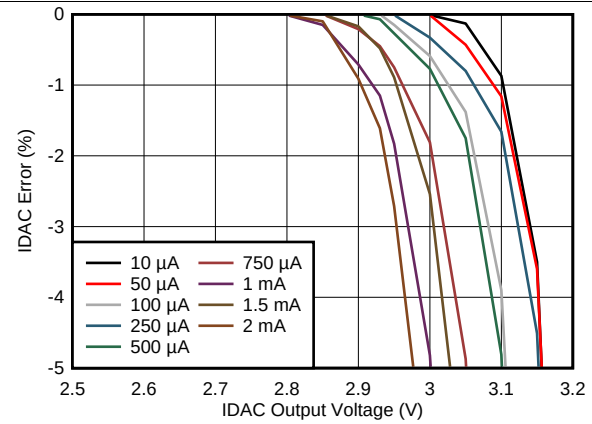


Figure 27. IDAC Accuracy vs Compliance Voltage

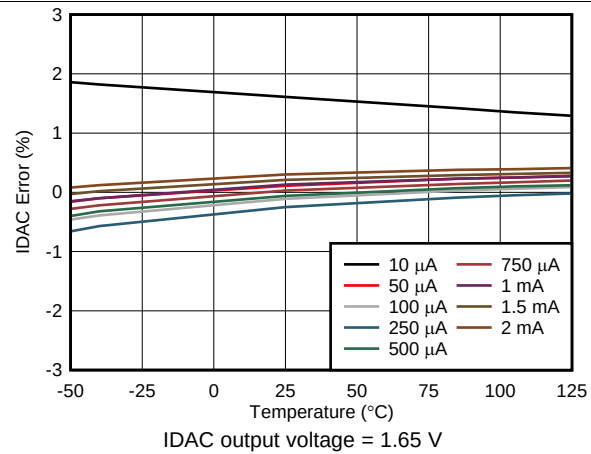


Figure 28. IDAC Accuracy vs Temperature

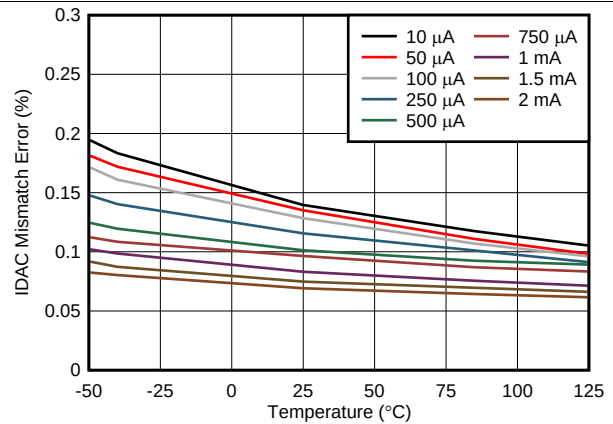


Figure 29. IDAC Matching vs Temperature

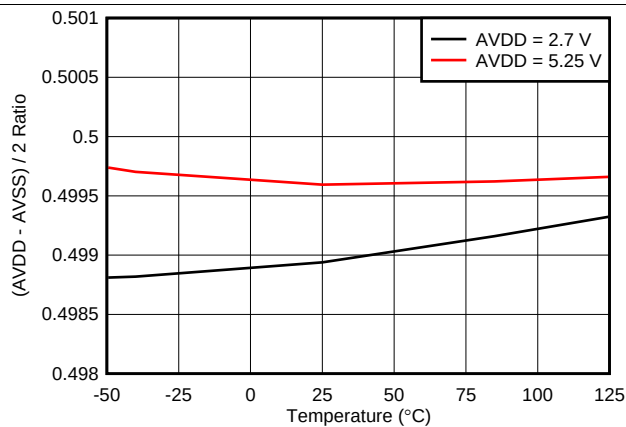


Figure 30. VBIAS Voltage $[(AVDD - AVSS) / 2]$ vs Temperature

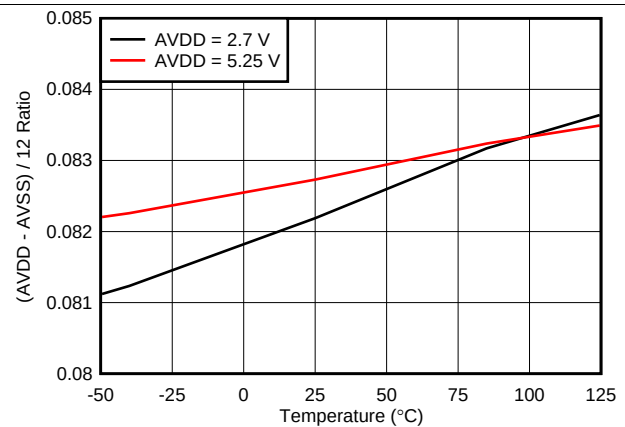


Figure 31. VBIAS Voltage $[(AVDD - AVSS) / 12]$ vs Temperature

Typical Characteristics (continued)

at $T_A = 25^\circ\text{C}$, $AVDD = 3.3\text{ V}$, $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, using internal $V_{REF} = 2.5\text{ V}$, internal 4.096-MHz oscillator, and PGA enabled (unless otherwise noted)

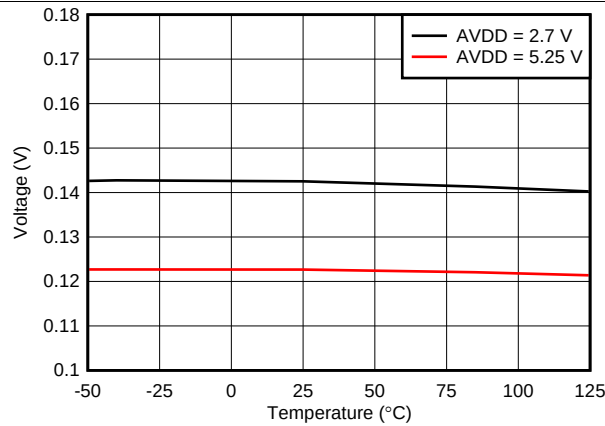


Figure 32. PGA Rail Detection, PGAN_RAILP, PGAP_RAILP Threshold From AVDD

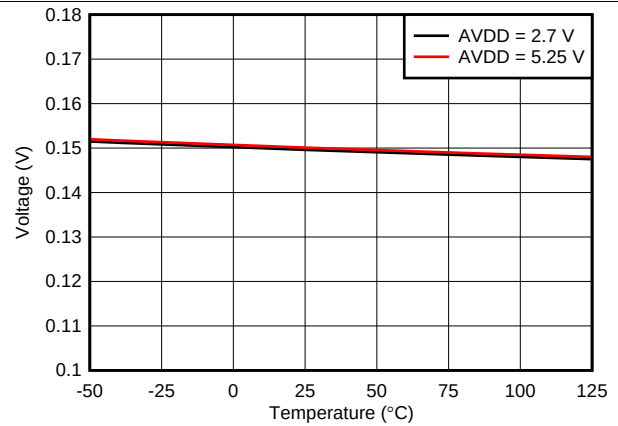


Figure 33. PGA Rail Detection, PGAN_RAILN, PGAP_RAILN Threshold From AVSS

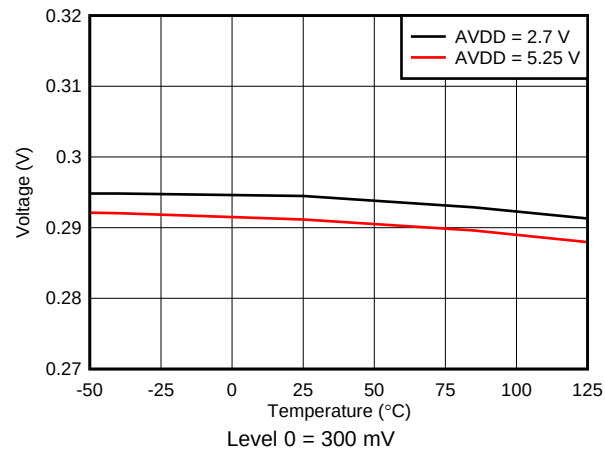


Figure 34. Reference Threshold Voltage, Level 0

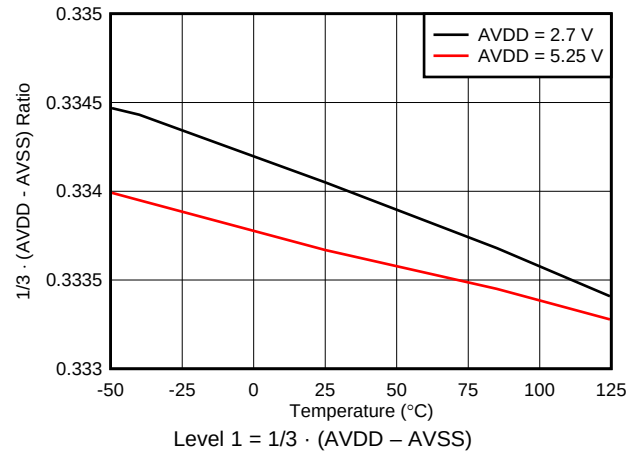


Figure 35. Reference Threshold Voltage, Level 1

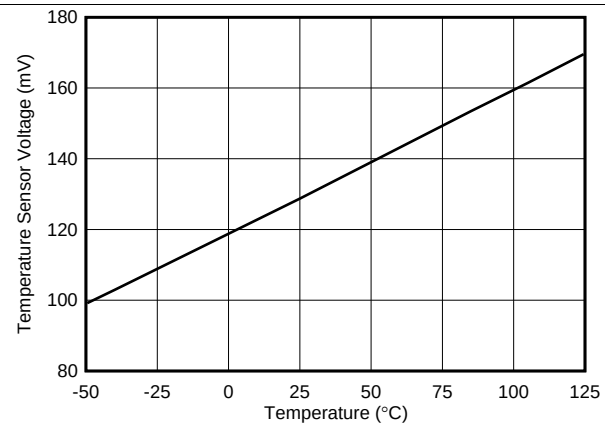


Figure 36. Temperature Sensor Voltage vs Temperature

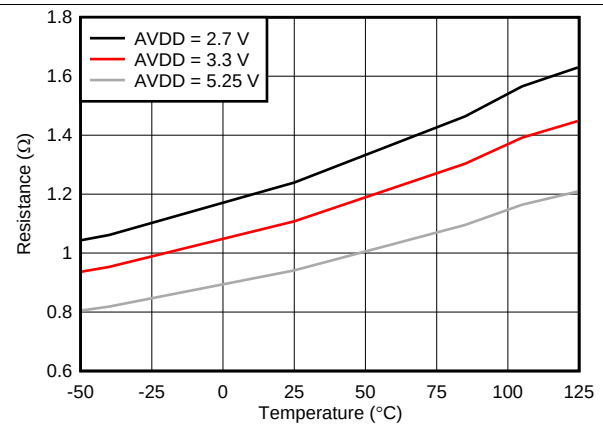
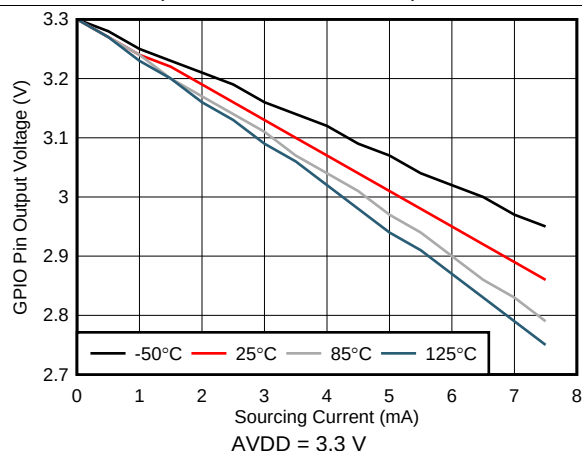
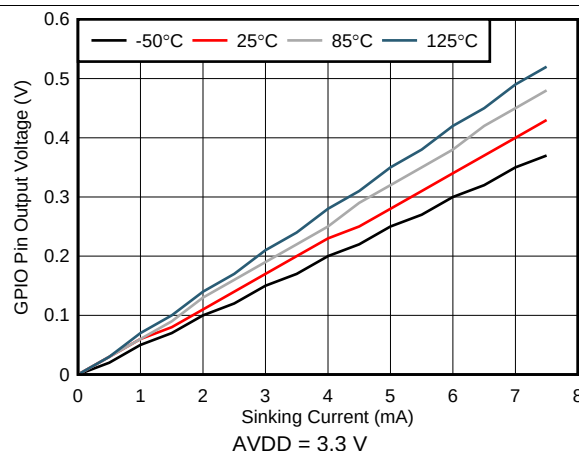
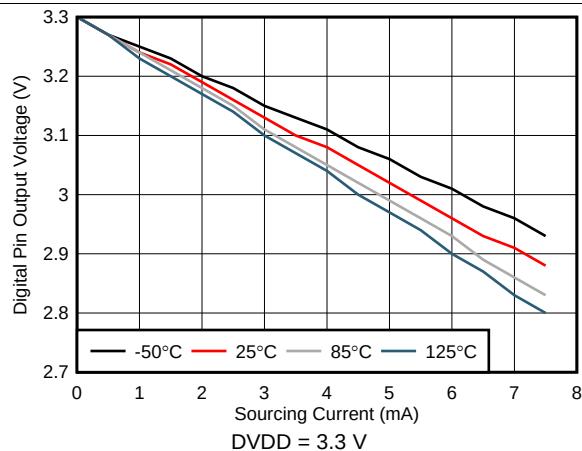
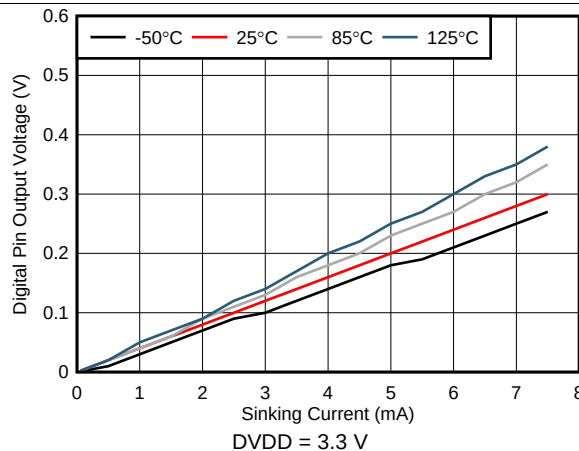
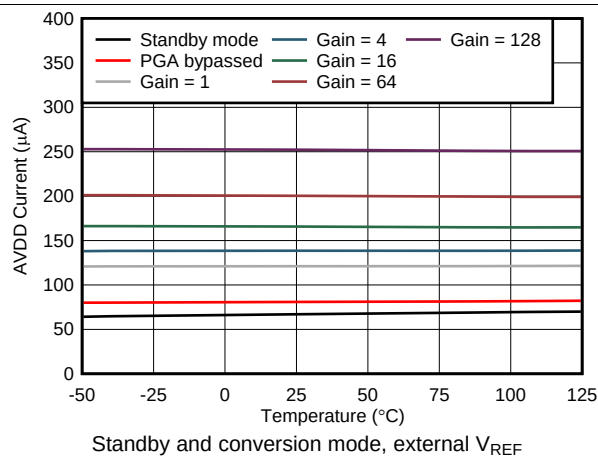
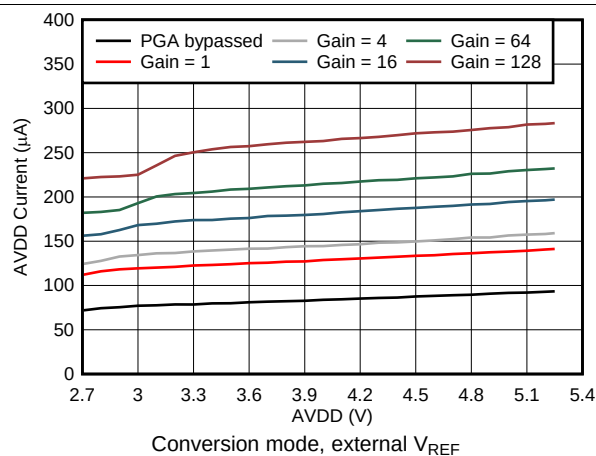


Figure 37. Low-Side Switch R_{ON} vs Temperature

Typical Characteristics (continued)

at $T_A = 25^\circ\text{C}$, $AVDD = 3.3\text{ V}$, $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, using internal $V_{REF} = 2.5\text{ V}$, internal 4.096-MHz oscillator, and PGA enabled (unless otherwise noted)


Figure 38. GPIO Pin Output Voltage vs Sourcing Current

Figure 39. GPIO Pin Output Voltage vs Sinking Current

Figure 40. Digital Pin Output Voltage vs Sourcing Current

Figure 41. Digital Pin Output Voltage vs Sinking Current

Figure 42. Analog Supply Current vs Temperature

Figure 43. Analog Supply Current vs AVDD

Typical Characteristics (continued)

at $T_A = 25^\circ\text{C}$, $AVDD = 3.3\text{ V}$, $AVSS = 0\text{ V}$, $DVDD = IOVDD = 3.3\text{ V}$, using internal $V_{REF} = 2.5\text{ V}$, internal 4.096-MHz oscillator, and PGA enabled (unless otherwise noted)

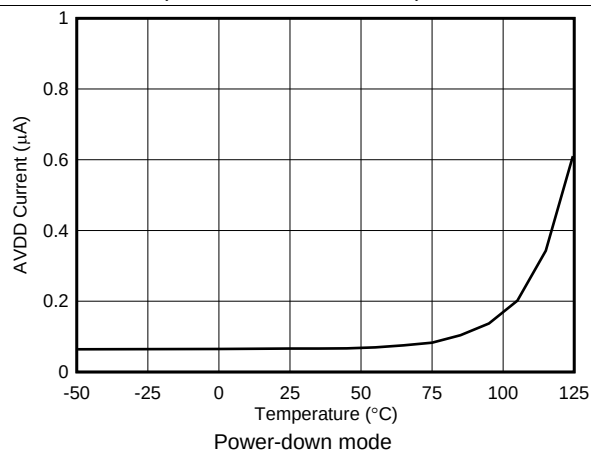


Figure 44. Analog Supply Current vs Temperature

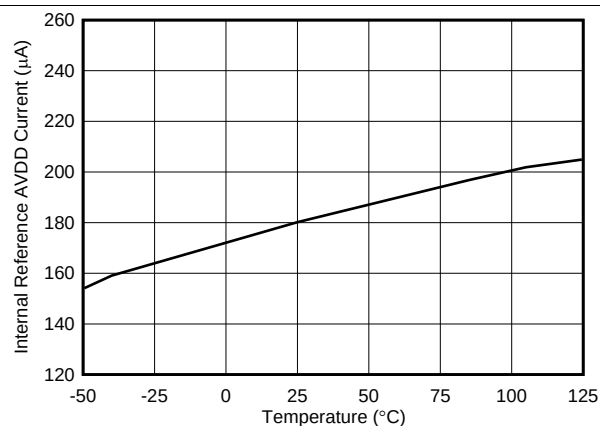


Figure 45. Internal Reference AVDD Current vs Temperature

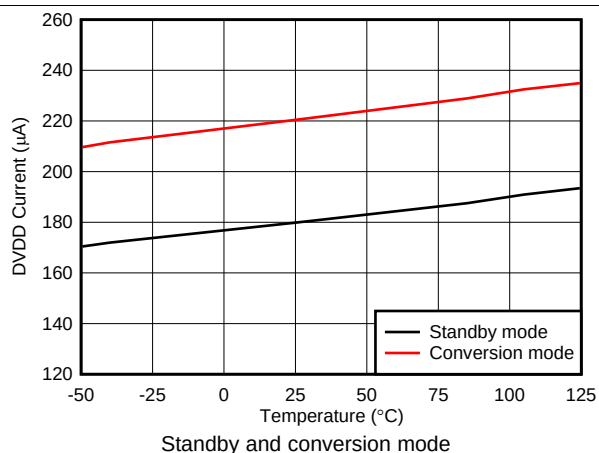


Figure 46. Digital Supply Current vs Temperature

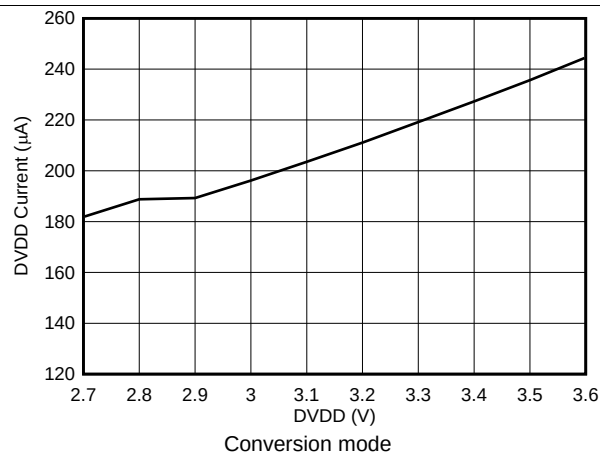


Figure 47. Digital Supply Current vs DVDD

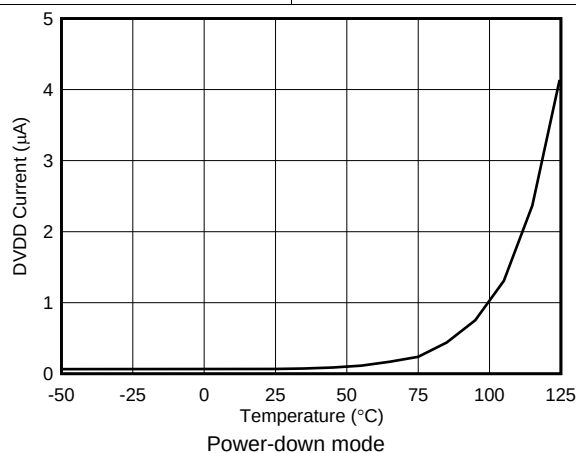


Figure 48. Digital Supply Current vs Temperature

8 Parameter Measurement Information

8.1 Noise Performance

Delta-sigma ($\Delta\Sigma$) analog-to-digital converters (ADCs) are based on the principle of oversampling. The input signal of a $\Delta\Sigma$ ADC is sampled at a high frequency (modulator frequency) and subsequently filtered and decimated in the digital domain to yield a conversion result at the respective output data rate. The ratio between modulator frequency and output data rate is called the *oversampling ratio* (OSR). By increasing the OSR, and thus reducing the output data rate, the noise performance of the ADC can be optimized. In other words, the input-referred noise drops when reducing the output data rate because more samples of the internal modulator are averaged to yield one conversion result. Increasing the gain also reduces the input-referred noise, which is particularly useful when measuring low-level signals.

Table 1 to Table 4 summarize the device noise performance. Table 1 and Table 2 list the ADC measurement noise using the sinc³ digital filter at different data rates and different PGA settings, and Table 3 and Table 4 list the ADC measurement noise using the low-latency digital filter. Data are representative of typical noise performance at T_A = 25°C using the internal 2.5-V reference. Data shown are based on 512 consecutive samples from a single device with inputs internally shorted. Table 1 and Table 3 list the input-referred root mean square noise in units of μV_{RMS} for the conditions shown. Note that peak-to-peak (μV_{PP}) values are shown in parentheses. Table 2 and Table 4 list the corresponding data in effective resolution calculated from μV_{RMS} values using Equation 1. Noise-free resolution is calculated from μV_{PP} values using Equation 2.

The input-referred noise (Table 1 and Table 3) only changes marginally when using an external low-noise reference, such as the REF5025. To calculate effective resolution and noise-free resolution when using a reference voltage other than 2.5 V, use Equation 1 and Equation 2:

$$\text{Effective Resolution} = \ln[(2 \cdot V_{\text{REF}} / \text{Gain}) / V_{\text{RMS-Noise}}] / \ln(2) \quad (1)$$

$$\text{Noise-Free Resolution} = \ln[(2 \cdot V_{\text{REF}} / \text{Gain}) / V_{\text{PP-Noise}}] / \ln(2) \quad (2)$$

Table 5 to Table 8 repeat the measurements of Table 1 to Table 4 but use the global chop feature of the device. The global chop feature averages two measurement of the ADC with the inputs swapped. This feature significantly reduces the input offset of the device, and reduces noise in the measurement.

Noise performance with the PGA bypassed are identical to the noise performance of the device with gain = 1 in Table 1 to Table 8.

**Table 1. Noise in μV_{RMS} (μV_{PP}) with Sinc³ Filter,
at AVDD = 3.3 V, AVSS = 0 V, PGA Enabled, Global Chop Disabled, and Internal 2.5-V Reference**

DATA RATE (SPS)	GAIN							
	1	2	4	8	16	32	64	128
2.5	0.32 (1.8)	0.16 (0.89)	0.085 (0.45)	0.049 (0.26)	0.036 (0.20)	0.025 (0.14)	0.020 (0.13)	0.019 (0.11)
5	0.40 (2.4)	0.21 (1.0)	0.11 (0.60)	0.066 (0.37)	0.040 (0.30)	0.033 (0.20)	0.029 (0.18)	0.027 (0.16)
10	0.53 (3.0)	0.29 (1.6)	0.16 (0.89)	0.088 (0.52)	0.061 (0.39)	0.046 (0.25)	0.040 (0.24)	0.036 (0.23)
16.6	0.76 (4.2)	0.36 (2.2)	0.20 (1.2)	0.11 (0.67)	0.077 (0.47)	0.060 (0.35)	0.052 (0.32)	0.046 (0.30)
20	0.81 (4.8)	0.41 (2.4)	0.22 (1.4)	0.12 (0.71)	0.082 (0.48)	0.064 (0.44)	0.056 (0.37)	0.048 (0.30)
50	1.3 (7.2)	0.62 (3.7)	0.33 (1.9)	0.18 (1.2)	0.13 (0.76)	0.11 (0.69)	0.091 (0.54)	0.080 (0.53)
60	1.4 (8.0)	0.70 (4.5)	0.37 (2.4)	0.21 (1.3)	0.12 (0.71)	0.11 (0.87)	0.10 (0.67)	0.089 (0.58)
100	1.8 (9.2)	0.91 (5.8)	0.49 (2.6)	0.27 (1.8)	0.17 (1.1)	0.14 (0.88)	0.12 (0.72)	0.11 (0.52)
200	2.4 (13)	1.2 (7.7)	0.64 (4.0)	0.39 (2.3)	0.26 (1.6)	0.21 (1.3)	0.19 (1.1)	0.15 (0.95)
400	3.6 (19)	1.8 (10)	0.87 (5.0)	0.54 (3.4)	0.34 (2.1)	0.28 (1.7)	0.25 (1.5)	0.22 (1.3)
800	5.0 (29)	2.6 (16)	1.3 (7.8)	0.76 (5.2)	0.49 (2.7)	0.41 (2.5)	0.37 (2.2)	0.34 (2.0)
1000	6.0 (32)	2.9 (17)	1.5 (10)	0.85 (5.2)	0.56 (3.2)	0.42 (2.5)	0.41 (2.3)	0.37 (2.2)
2000	7.8 (45)	4.2 (26)	2.1 (14)	1.2 (6.9)	0.78 (4.5)	0.64 (4.0)	0.57 (3.5)	0.54 (2.8)
4000	15. (95)	7.3 (45)	3.9 (25)	2.0 (11)	1.2 (7.1)	0.87 (5.2)	0.83 (5.1)	0.80 (5.0)

Table 2. Effective Resolution from RMS Noise (Noise-Free Resolution from Peak-to-Peak Noise) with Sinc³ Filter at AVDD = 3.3 V, AVSS = 0 V, PGA Enabled, Global Chop Disabled, and Internal 2.5-V Reference

DATA RATE (SPS)	GAIN							
	1	2	4	8	16	32	64	128
2.5	23.9 (21.4)	23.9 (21.4)	23.8 (21.4)	23.6 (21.2)	23.0 (20.5)	22.6 (20.2)	21.9 (19.1)	21.0 (18.4)
5	23.6 (21.0)	23.5 (21.2)	23.4 (21.0)	23.2 (20.7)	22.9 (20.0)	22.2 (19.5)	21.4 (18.9)	20.5 (17.6)
10	23.2 (20.7)	23.0 (20.5)	22.9 (20.4)	22.8 (20.2)	22.3 (19.6)	21.7 (19.2)	20.9 (18.4)	20.1 (17.3)
16.6	22.6 (20.2)	22.7 (20.1)	22.6 (20.0)	22.4 (19.8)	22.0 (19.4)	21.3 (18.8)	20.5 (17.9)	19.7 (17.0)
20	22.6 (20.0)	22.5 (20.0)	22.4 (19.8)	22.3 (19.8)	21.9 (19.3)	21.2 (18.4)	20.4 (17.7)	19.6 (17.0)
50	21.9 (19.4)	21.9 (19.4)	21.9 (19.3)	21.7 (19.0)	21.2 (18.6)	20.5 (17.8)	19.7 (17.2)	18.9 (16.2)
60	21.8 (19.2)	21.8 (19.1)	21.7 (19.0)	21.5 (18.8)	21.3 (18.8)	20.4 (17.5)	19.5 (16.8)	18.7 (16.0)
100	21.4 (19.0)	21.4 (18.7)	21.3 (18.9)	21.1 (18.4)	20.8 (18.1)	20.1 (17.4)	19.3 (16.7)	18.4 (16.0)
200	21.0 (18.6)	20.9 (18.3)	20.9 (18.2)	20.6 (18.1)	20.2 (17.6)	19.5 (16.9)	18.6 (16.1)	18.0 (15.3)
400	20.4 (18.0)	20.4 (17.9)	20.5 (17.9)	20.2 (17.5)	19.8 (17.2)	19.1 (16.5)	18.2 (15.7)	17.4 (14.9)
800	19.9 (17.4)	19.9 (17.2)	19.9 (17.3)	19.7 (16.9)	19.3 (16.8)	18.6 (16.0)	17.7 (15.1)	16.8 (14.3)
1000	19.7 (17.2)	19.7 (17.2)	19.6 (16.9)	19.5 (16.9)	19.1 (16.6)	18.5 (16.1)	17.6 (14.9)	16.7 (14.1)
2000	19.3 (16.8)	19.2 (16.6)	19.2 (16.5)	18.9 (16.5)	18.6 (16.1)	17.9 (15.3)	17.1 (14.4)	16.2 (13.8)
4000	18.4 (15.7)	18.4 (15.7)	18.3 (15.6)	18.3 (15.7)	18.0 (15.4)	17.5 (14.9)	16.5 (13.9)	15.6 (12.9)

Table 3. Noise in μV_{RMS} (μV_{PP}) with Low-Latency Filter, at AVDD = 3.3 V, AVSS = 0 V, PGA Enabled, Global Chop Disabled, and Internal 2.5-V Reference

DATA RATE (SPS)	GAIN							
	1	2	4	8	16	32	64	128
2.5	0.46 (2.7)	0.24 (1.3)	0.12 (0.82)	0.072 (0.41)	0.051 (0.30)	0.041 (0.27)	0.037 (0.20)	0.034 (0.19)
5	0.62 (3.9)	0.33 (1.6)	0.18 (1.1)	0.10 (0.52)	0.064 (0.37)	0.053 (0.31)	0.050 (0.33)	0.046 (0.27)
10	0.94 (6.6)	0.47 (3.0)	0.24 (1.4)	0.14 (0.78)	0.09 (0.52)	0.074 (0.44)	0.065 (0.37)	0.057 (0.35)
16.6	1.3 (8.0)	0.66 (3.9)	0.32 (2.0)	0.18 (1.2)	0.12 (0.69)	0.10 (0.56)	0.086 (0.55)	0.079 (0.52)
20	1.2 (8.0)	0.65 (3.9)	0.34 (1.9)	0.20 (1.2)	0.13 (0.67)	0.10 (0.64)	0.093 (0.54)	0.082 (0.51)
50	2.1 (13)	1.0 (6.6)	0.54 (3.1)	0.30 (1.9)	0.22 (1.3)	0.16 (1.2)	0.15 (0.90)	0.13 (0.84)
60	2.3 (15)	1.2 (7.5)	0.62 (3.7)	0.35 (2.2)	0.22 (1.4)	0.19 (1.1)	0.16 (0.99)	0.15 (0.90)
100	2.8 (18)	1.6 (10)	0.83 (5.1)	0.45 (2.9)	0.30 (1.7)	0.23 (1.3)	0.38 (2.6)	0.20 (1.3)
200	4.6 (28)	2.2 (13)	1.1 (6.9)	0.67 (4.1)	0.42 (2.6)	0.34 (2.0)	0.29 (1.9)	0.28 (1.7)
400	6.1 (31)	3.0 (19)	1.6 (11)	0.92 (5.1)	0.46 (2.6)	0.48 (2.8)	0.42 (2.9)	0.39 (2.3)
800	8.6 (36)	4.3 (27)	2.3 (15)	1.3 (8.0)	0.81 (4.6)	0.66 (4.0)	0.58 (3.8)	0.55 (3.2)
1000	10 (50)	5.0 (29)	2.6 (15)	1.4 (8.9)	0.90 (5.4)	0.79 (5.1)	0.66 (4.3)	0.59 (3.8)
2000	22 (83)	11 (80)	5.5 (32)	2.9 (17)	1.6 (11)	1.1 (6.7)	1.1 (6.7)	1.0 (6.6)
4000	103 (629)	48 (404)	24 (160)	12 (70)	6.4 (39)	3.3 (21)	3.6 (20)	3.1 (19)

Table 4. Effective Resolution from RMS Noise (Noise-Free Resolution from Peak-to-Peak Noise) with Low-Latency Filter, at AVDD = 3.3 V, AVSS = 0 V, PGA Enabled, Global Chop Disabled, and Internal 2.5-V Reference

DATA RATE (SPS)	GAIN							
	1	2	4	8	16	32	64	128
2.5	23.4 (20.8)	23.3 (20.8)	23.3 (20.5)	23.0 (20.5)	22.6 (20.0)	21.9 (19.1)	21.1 (18.6)	20.0 (17.6)
5	22.9 (20.3)	22.8 (20.5)	22.7 (20.1)	22.6 (20.2)	22.2 (19.7)	21.5 (19.0)	20.6 (17.9)	19.7 (17.1)
10	22.3 (19.5)	22.3 (19.7)	22.3 (19.8)	22.1 (19.6)	21.7 (19.2)	21.0 (18.4)	20.2 (17.8)	19.4 (16.7)
16.6	21.9 (19.2)	21.9 (19.3)	21.9 (19.2)	21.7 (19.0)	21.3 (18.8)	20.5 (18.1)	19.8 (17.1)	18.9 (16.2)
20	22.0 (19.2)	21.9 (19.3)	21.8 (19.3)	21.6 (19.0)	21.3 (18.8)	20.5 (17.9)	19.7 (17.2)	18.9 (16.2)
50	21.2 (18.5)	21.2 (18.5)	21.1 (18.6)	21.0 (18.3)	20.5 (17.9)	19.9 (17.0)	19.0 (16.4)	18.2 (15.5)
60	21.0 (18.3)	21.0 (18.4)	21.0 (18.4)	20.8 (18.1)	20.4 (17.7)	19.7 (17.1)	18.9 (16.3)	18.0 (15.4)
100	20.8 (18.1)	20.6 (17.9)	20.5 (17.9)	20.4 (17.7)	20.0 (17.5)	19.4 (16.9)	17.6 (14.9)	17.6 (14.9)
200	20.1 (17.4)	20.1 (17.6)	20.1 (17.5)	19.8 (17.2)	19.5 (16.9)	18.8 (16.2)	18.0 (15.3)	17.1 (14.5)
400	19.6 (17.3)	19.7 (17.0)	19.6 (16.8)	19.4 (16.9)	19.4 (16.9)	18.3 (15.8)	17.5 (14.7)	16.6 (14.0)
800	19.2 (17.1)	19.1 (16.5)	19.0 (16.4)	18.9 (16.3)	18.6 (16.0)	17.9 (15.2)	17.0 (14.3)	16.1 (13.6)
1000	19.0 (16.6)	18.9 (16.4)	18.9 (16.3)	18.7 (16.1)	18.4 (15.8)	17.6 (14.9)	16.8 (14.2)	16.0 (13.3)
2000	17.8 (15.9)	17.8 (14.9)	17.8 (15.3)	17.7 (15.2)	17.5 (14.8)	17.1 (14.5)	16.1 (13.5)	15.2 (12.5)
4000	15.6 (13.0)	15.7 (12.6)	15.7 (12.9)	15.7 (13.1)	15.6 (13.0)	15.5 (12.9)	14.4 (11.9)	13.6 (11.0)

Table 5. Noise in μV_{RMS} (μV_{PP}) with Sinc³ Filter, at AVDD = 3.3 V, AVSS = 0 V, PGA Enabled, Global Chop Enabled, and Internal 2.5-V Reference

DATA RATE (SPS) ⁽¹⁾	GAIN							
	1	2	4	8	16	32	64	128
2.5	0.22 (1.2)	0.11 (0.60)	0.063 (0.30)	0.031 (0.19)	0.022 (0.15)	0.017 (0.09)	0.014 (0.08)	0.013 (0.07)
5	0.31 (1.8)	0.15 (0.70)	0.080 (0.52)	0.049 (0.26)	0.032 (0.19)	0.024 (0.15)	0.020 (0.12)	0.019 (0.11)
10	0.41 (2.4)	0.21 (1.2)	0.12 (0.60)	0.061 (0.37)	0.042 (0.24)	0.031 (0.20)	0.029 (0.17)	0.027 (0.16)
16.6	0.54 (3.3)	0.28 (1.5)	0.14 (0.80)	0.080 (0.48)	0.052 (0.32)	0.043 (0.26)	0.039 (0.23)	0.034 (0.20)
20	0.60 (3.3)	0.31 (1.8)	0.16 (0.80)	0.090 (0.56)	0.059 (0.34)	0.046 (0.25)	0.043 (0.27)	0.037 (0.19)
50	0.91 (4.8)	0.49 (2.5)	0.23 (1.3)	0.14 (0.90)	0.090 (0.45)	0.070 (0.41)	0.069 (0.36)	0.058 (0.33)
60	1.0 (5.7)	0.51 (3.1)	0.26 (1.5)	0.16 (0.90)	0.10 (0.54)	0.090 (0.45)	0.070 (0.41)	0.067 (0.40)
100	1.2 (6.9)	0.63 (3.7)	0.33 (1.7)	0.19 (1.0)	0.13 (0.70)	0.11 (0.59)	0.089 (0.52)	0.084 (0.49)
200	1.8 (10)	0.90 (4.9)	0.48 (2.8)	0.27 (1.4)	0.19 (1.1)	0.15 (0.80)	0.14 (0.80)	0.12 (0.75)
400	2.5 (14)	1.2 (7.0)	0.60 (3.4)	0.39 (2.4)	0.26 (1.6)	0.20 (1.1)	0.18 (1.0)	0.16 (0.90)
800	3.5 (21)	1.8 (11)	1.0 (5.7)	0.55 (3.2)	0.35 (2.1)	0.29 (1.7)	0.26 (1.4)	0.23 (1.3)
1000	3.9 (23)	2.1 (13)	1.1 (6.0)	0.65 (3.5)	0.39 (2.3)	0.32 (1.9)	0.28 (1.6)	0.26 (1.5)
2000	6.0 (35)	2.8 (16)	1.6 (8.0)	0.90 (5.0)	0.59 (3.5)	0.47 (2.5)	0.39 (2.2)	0.36 (2.1)
4000	10 (63)	5.3 (30)	2.8 (16)	1.4 (9.0)	0.90 (5.0)	0.67 (4.0)	0.58 (3.3)	0.54 (3.2)

(1) The actual data conversion period changes with the sinc³ filter and global chop mode enabled; see [Table 19](#) for details.

Table 6. Effective Resolution from RMS Noise (Noise-Free Resolution from Peak-to-Peak Noise) with Sinc³ Filter at AVDD = 3.3 V, AVSS = 0 V, PGA Enabled, Global Chop Enabled, and Internal 2.5-V Reference

DATA RATE (SPS) ⁽¹⁾	GAIN							
	1	2	4	8	16	32	64	128
2.5	24.4 (22.0)	24.5 (22.0)	24.2 (22.0)	24.3 (21.7)	23.7 (21.0)	23.1 (20.7)	22.4 (19.8)	21.5 (19.1)
5	24.0 (21.4)	24.0 (21.7)	23.9 (21.2)	23.6 (21.2)	23.2 (20.7)	22.6 (20.0)	21.9 (19.4)	20.9 (18.5)
10	23.5 (21.0)	23.5 (21.0)	23.3 (21.0)	23.3 (21.0)	22.8 (20.3)	22.2 (19.6)	21.4 (18.9)	20.5 (17.9)
16.6	23.1 (20.5)	23.1 (20.7)	23.1 (20.5)	22.8 (20.5)	22.5 (19.9)	21.8 (19.2)	20.9 (18.4)	20.1 (17.6)
20	23.0 (20.5)	22.9 (20.4)	22.9 (20.5)	22.7 (20.5)	22.3 (19.8)	21.7 (19.2)	20.8 (18.2)	20.0 (17.6)
50	22.4 (20.0)	22.3 (19.9)	22.4 (19.8)	22.1 (19.8)	21.8 (19.4)	21.1 (18.5)	20.1 (17.7)	19.4 (16.8)
60	22.3 (19.8)	22.2 (19.6)	22.2 (19.7)	21.9 (19.7)	21.6 (19.1)	20.8 (18.4)	20.0 (17.5)	19.2 (16.6)
100	22.0 (19.5)	21.9 (19.4)	21.8 (19.5)	21.6 (19.5)	21.2 (18.7)	20.5 (18.0)	19.7 (17.2)	18.8 (16.3)
200	21.4 (18.9)	21.5 (19.0)	21.3 (18.8)	21.1 (18.8)	20.7 (18.1)	20.0 (17.6)	19.1 (16.6)	18.3 (15.7)
400	21.0 (18.5)	20.9 (18.4)	21.0 (18.5)	20.6 (18.5)	20.2 (17.6)	19.6 (17.1)	18.7 (16.1)	17.9 (15.4)
800	20.4 (17.9)	20.4 (17.8)	20.3 (17.7)	20.1 (17.7)	19.8 (17.2)	19.0 (16.5)	18.2 (15.7)	17.4 (14.9)
1000	20.3 (17.7)	20.2 (17.6)	20.1 (17.7)	19.9 (17.7)	19.6 (17.0)	18.9 (16.3)	18.1 (15.6)	17.2 (14.7)
2000	19.7 (17.1)	19.8 (17.2)	19.6 (17.2)	19.4 (17.2)	19.0 (16.5)	18.3 (15.9)	17.6 (15.2)	16.7 (14.2)
4000	19.0 (16.3)	18.9 (16.4)	18.8 (16.2)	18.7 (16.2)	18.4 (15.9)	17.8 (15.3)	17.0 (14.5)	16.1 (13.6)

(1) The actual data conversion period changes with the sinc³ filter and global chop mode enabled; see [Table 19](#) for details.

Table 7. Noise in μV_{RMS} (μV_{PP}) with Low-Latency Filter, at AVDD = 3.3 V, AVSS = 0 V, PGA Enabled, Global Chop Enabled, and Internal 2.5-V Reference

DATA RATE (SPS)	GAIN							
	1	2	4	8	16	32	64	128
2.5	0.34 (2.1)	0.16 (0.90)	0.09 (0.52)	0.051 (0.26)	0.034 (0.17)	0.027 (0.15)	0.024 (0.14)	0.021 (0.13)
5	0.47 (2.7)	0.22 (1.3)	0.12 (0.60)	0.076 (0.45)	0.038 (0.22)	0.039 (0.23)	0.034 (0.20)	0.029 (0.19)
10	0.63 (3.6)	0.33 (1.8)	0.18 (1.0)	0.10 (0.56)	0.069 (0.37)	0.051 (0.30)	0.045 (0.26)	0.041 (0.25)
16.6	0.90 (4.5)	0.43 (2.5)	0.24 (1.4)	0.14 (0.90)	0.090 (0.50)	0.070 (0.40)	0.065 (0.38)	0.056 (0.35)
20	0.90 (5.1)	0.42 (2.5)	0.23 (1.4)	0.14 (0.90)	0.090 (0.63)	0.070 (0.41)	0.070 (0.39)	0.059 (0.36)
50	1.3 (8.0)	0.80 (4.6)	0.39 (2.5)	0.23 (1.2)	0.14 (0.90)	0.12 (0.70)	0.11 (0.61)	0.090 (0.50)
60	1.7 (10)	0.80 (4.3)	0.42 (2.8)	0.23 (1.4)	0.18 (1.0)	0.13 (0.70)	0.12 (0.69)	0.11 (0.67)
100	2.0 (12)	1.1 (6.0)	0.55 (3.2)	0.34 (1.9)	0.21 (1.3)	0.18 (1.1)	0.16 (0.90)	0.14 (0.80)
200	3.0 (17)	1.5 (8.0)	0.70 (4.1)	0.48 (2.9)	0.28 (1.7)	0.26 (1.4)	0.23 (1.3)	0.19 (1.0)
400	4.3 (22)	2.3 (13)	1.1 (6.0)	0.64 (3.7)	0.44 (2.4)	0.33 (1.9)	0.29 (1.8)	0.28 (1.7)
800	6.2 (25)	3.1 (19)	1.5 (9.0)	0.90 (5.0)	0.62 (3.4)	0.49 (2.8)	0.45 (2.5)	0.40 (2.3)
1000	6.0 (38)	3.3 (19)	1.8 (11)	1.1 (6.3)	0.66 (3.7)	0.52 (2.8)	0.45 (2.7)	0.45 (2.5)
2000	15 (51)	8.0 (50)	4.0 (23)	2.2 (13)	1.2 (7.0)	0.90 (5.2)	0.80 (5.6)	0.70 (3.9)
4000	65 (275)	35 (211)	18 (116)	9.0 (51)	4.4 (24)	2.3 (15)	2.4 (13)	2.2 (12)

Table 8. Effective Resolution from RMS Noise (Noise-Free Resolution from Peak-to-Peak Noise) with Low-Latency Filter, at AVDD = 3.3 V, AVSS = 0 V, PGA Enabled, Global Chop Enabled, and Internal 2.5-V Reference

DATA RATE (SPS)	GAIN							
	1	2	4	8	16	32	64	128
2.5	23.8 (21.2)	23.9 (21.4)	23.7 (21.2)	23.5 (21.2)	23.1 (20.8)	22.5 (20.0)	21.7 (19.1)	20.8 (18.2)
5	23.3 (20.8)	23.4 (20.8)	23.3 (21.0)	23.0 (20.4)	23.0 (20.4)	21.9 (19.4)	21.1 (18.6)	20.4 (17.7)
10	22.9 (20.4)	22.9 (20.4)	22.7 (20.3)	22.6 (20.1)	22.1 (19.7)	21.6 (19.9)	20.7 (18.2)	19.9 (17.2)
16.6	22.4 (20.1)	22.5 (19.9)	22.3 (19.8)	22.1 (19.4)	21.8 (19.2)	21.1 (18.6)	20.2 (17.6)	19.4 (16.8)
20	22.4 (19.9)	22.5 (19.9)	22.4 (19.8)	22.1 (19.4)	21.8 (18.9)	21.0 (18.6)	20.1 (17.8)	19.3 (16.7)
50	21.8 (19.3)	21.7 (19.0)	21.6 (19.0)	21.4 (19.0)	21.0 (18.4)	20.4 (17.8)	19.4 (17.0)	18.7 (16.3)
60	21.5 (19.0)	21.6 (19.1)	21.5 (18.8)	21.3 (18.8)	20.8 (18.3)	20.2 (17.7)	19.3 (16.8)	18.5 (15.8)
100	21.2 (18.6)	21.1 (18.6)	21.1 (18.6)	20.8 (18.3)	20.5 (17.9)	19.7 (17.1)	18.9 (16.4)	18.1 (15.6)
200	20.7 (18.2)	20.7 (18.2)	20.7 (18.2)	20.3 (17.7)	20.1 (17.5)	19.2 (16.8)	18.4 (15.9)	17.6 (15.2)
400	20.1 (17.8)	20.1 (17.6)	20.1 (17.6)	19.9 (17.4)	19.4 (17.0)	18.8 (16.4)	18.0 (15.4)	17.1 (14.5)
800	19.6 (17.6)	19.6 (17.0)	19.7 (17.1)	19.4 (16.8)	18.9 (16.5)	18.3 (15.8)	17.4 (14.9)	16.6 (14.0)
1000	19.6 (17.0)	19.5 (17.0)	19.4 (16.8)	19.1 (16.6)	18.9 (16.4)	18.2 (15.8)	17.4 (14.8)	16.4 (13.9)
2000	18.4 (16.6)	18.2 (15.6)	18.2 (15.7)	18.1 (15.6)	18.0 (15.4)	17.4 (14.9)	16.6 (13.8)	15.9 (13.3)
4000	16.2 (14.1)	16.1 (13.5)	16.1 (13.4)	16.1 (13.6)	16.1 (13.7)	16.1 (13.4)	15.0 (12.7)	14.1 (11.5)

9 Detailed Description

9.1 Overview

The ADS124S06 and ADS124S08 are precision 24-bit, delta-sigma ($\Delta\Sigma$) ADCs with an integrated analog front end (AFE) to simplify precision sensor connections. The ADC provides output data rates from 2.5 SPS to 4000 SPS for flexibility in resolution and data rates over a wide range of applications. The low-noise and low-drift architecture make these devices suitable for precise measurement of low-voltage sensors, such as load cells and temperature sensors.

The ADS124S0x incorporate several features that simplify precision sensor measurements. Key integrated features include:

- Low-noise, CMOS PGA with integrated signal fault detection
- Low-drift, 2.5-V voltage reference
- Two sets of buffered external reference inputs with reference voltage level detection
- Dual, matched, sensor-excitation current sources (IDACs)
- Internal 4.096-MHz oscillator
- Temperature sensor
- Four general-purpose input/output pins (GPIOs)
- A low-resistance switch (when connected to AVSS) can be used to disconnect bridge sensors to reduce current consumption

As described in the [Functional Block Diagram](#) section, these devices provide 13 (ADS124S08) or 7 (ADS124S06) analog inputs that are configurable as either single-ended inputs, differential inputs, or any combination of the two. Many of the analog inputs have additional features as programmed by the user. The analog inputs can be programmed to enable the following extended features:

- Two sensor excitation current sources: all analog input pins (and REFP1 and REFN1 on the ADS124S06)
- Sensor biasing voltage (VBIAS): pins AIN0, AIN1, AIN2, AIN3, AIN4, AIN5, AINCOM
- Four GPIO pins: AIN8, AIN9, AIN10, AIN11 (ADS124S08 only, the ADS124S06 has dedicated GPIOs)
- Sensor burn-out current sources: analog input pins selected for ADC input

Following the input multiplexer (MUX), the ADC features a high input-impedance, low-noise, programmable gain amplifier (PGA), eliminating the need for an external amplifier. The PGA gain is programmable from 1 to 128 in binary steps. The PGA can be bypassed to allow the input range to extend 50 mV below ground or above supply. The PGA has output voltage monitors to verify the integrity of the conversion result.

An inherently stable delta-sigma modulator measures the ratio of the input voltage to the reference voltage to provide the ADC result. The ADC operates with the internal 2.5-V reference, or with up to two external reference inputs. The external reference inputs can be continuously monitored for low (or missing) voltage. The REFOUT pin provides the buffered 2.5-V internal voltage reference output that can be used to bias external circuitry.

The digital filter provides two filter modes, sinc^3 and low-latency, allowing optimization of settling time and line-cycle rejection. The third-order sinc filter offers simultaneous 50-Hz and 60-Hz line-cycle rejection at data rates of 2.5 SPS, 5 SPS, and 10 SPS, 50-Hz rejection at data rates of 16.6 SPS and 50 SPS, and 60-Hz rejection at data rates of 20 SPS and 60 SPS. The low-latency filter provides settled data with 50-Hz and 60-Hz line-cycle rejection at data rates of 2.5 SPS, 5 SPS, 10 SPS, and 20 SPS, 50-Hz rejection at data rates of 16.6 SPS and 50 SPS, and 60-Hz rejection at a data rate of 60 SPS.

Two programmable excitation current sources provide bias to resistive sensors [such as resistance temperature detectors (RTDs) or thermistors]. The ADC integrates several system monitors for read back, such as temperature sensor and supply monitors. Four GPIO pins are available as either dedicated pins (ADS124S06) or combined with analog input pins (ADS124S08).

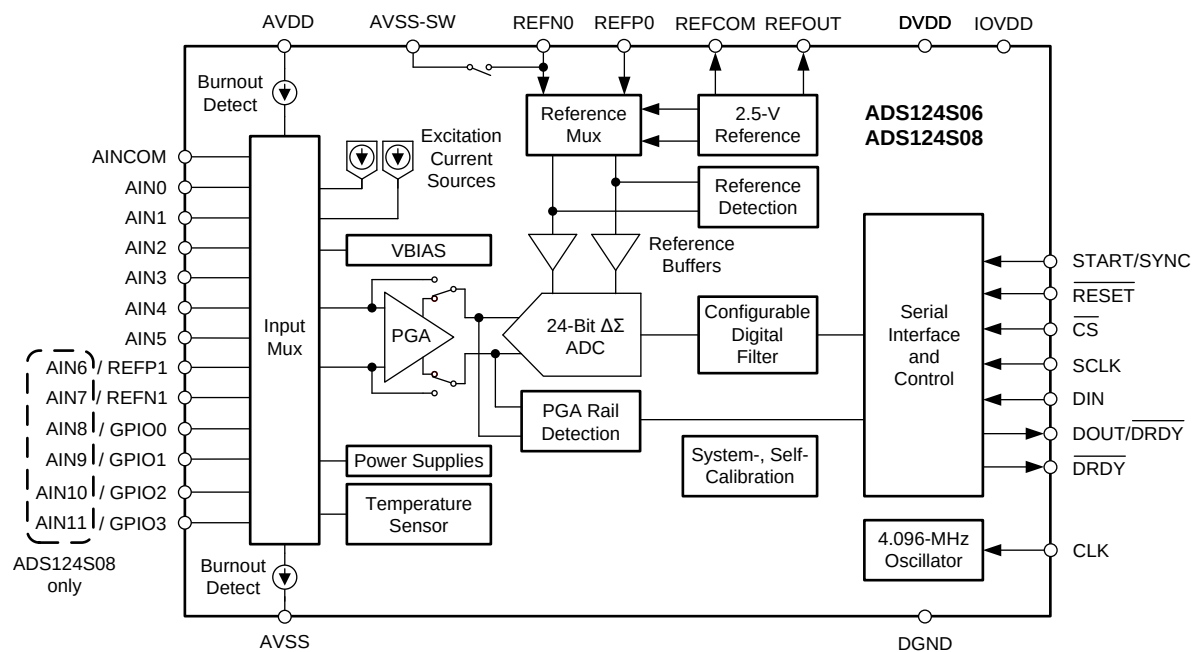
The ADS124S0x system clock is either provided by the internal low-drift, 4.096-MHz oscillator or an external clock source on the CLK input.

Overview (continued)

The SPI-compatible serial interface is used to read the conversion data and also to configure and control the ADC. The serial interface consists of four signals: $\overline{\text{CS}}$, SCLK, DIN, and DOUT/DRDY. The conversion data are provided with an optional CRC code for improved data integrity. The dual function DOUT/DRDY output indicates when conversion data are ready and also provides the data output. The serial interface can be implemented with as little as three connections by tying CS low. Start ADC conversions with either the START/SYNC pin or with commands. The ADC can be programmed for a continuous conversion mode or to perform single-shot conversions.

The AVDD analog supply operates with bipolar supplies from ± 1.5 V to ± 2.625 V or with a unipolar supply from 2.7 V to 5.25 V. For unipolar-supply operation, use the VBIAS voltage to bias isolated (floating) sensors. The digital supplies operate with unipolar supplies only. The DVDD digital power supply operates from 2.7 V to 3.6 V and the IOVDD supply operates from DVDD to 5.25 V.

9.2 Functional Block Diagram



Copyright © 2016, Texas Instruments Incorporated

9.3 Feature Description

9.3.1 Multiplexer

The ADS124S0x contains a flexible input multiplexer; see [Figure 49](#). Select any of the six (ADS124S06) or 12 (ADS124S08) analog inputs as the positive or negative input for the PGA using the MUX_P[3:0] and MUX_N[3:0] bits in the input multiplexer register (02h). In addition, AINCOM can be selected as the positive or negative PGA input. AINCOM is treated as a regular analog input, as is AINx. Use AINCOM in single-ended measurement applications as the common input for the other analog inputs.

The multiplexer also routes the excitation current sources to drive resistive sensors (bridges, RTDs, and thermistors) and can provide bias voltages for unbiased sensors (unbiased thermocouples for example) to analog input pins.

The ADS124S0x also contain a set of system monitor functions measured through the multiplexer. The inputs can be shorted together at mid-supply $[(AVDD + AVSS) / 2]$ to measure and calibrate the input offset of the analog front-end and the ADC. The system monitor also includes a temperature sensor that provides a measurement of the device temperature. The system monitor can also measure the analog and digital supplies, measuring $[(AVDD - AVSS) / 4]$ for the analog supply or $DVDD / 4$ for the digital supply. Finally, the system monitor contains a set of burn-out current sources that pull the inputs to either supply if the sensor has burned out and has a high impedance so that the ADC measures a full-scale reading.

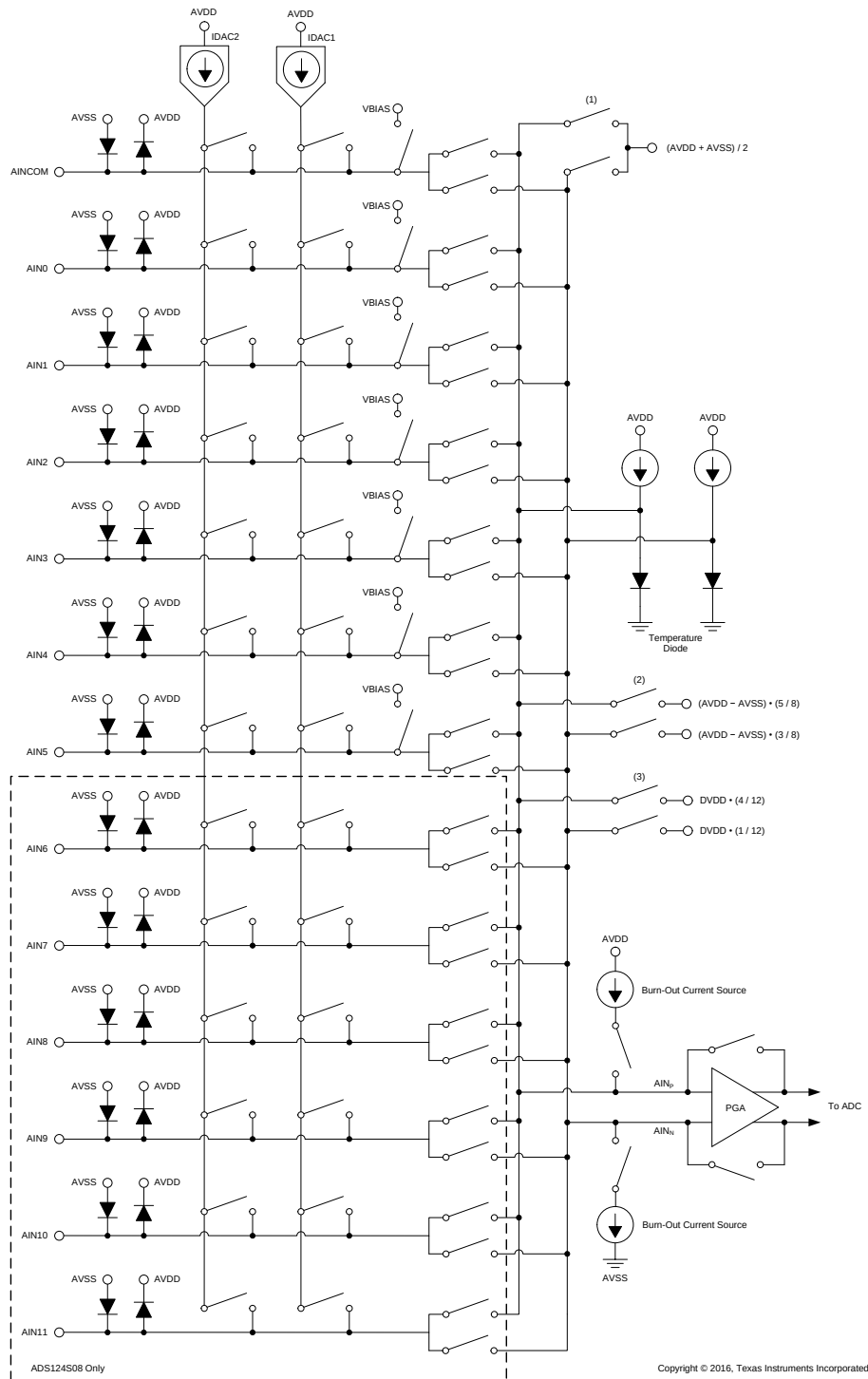
The multiplexer implements a break-before-make circuit. When changing the multiplexer channels using the MUX_P[3:0] and MUX_N[3:0] bits, the device first disconnects the PGA inputs from the analog inputs and connects them to mid-supply for $2 \cdot t_{CLK}$. In the next step, the PGA inputs connect to the selected new analog input channels. This break-before-make behavior ensures the ADC always starts from a known state and that the analog inputs are not momentarily shorted together.

Electrostatic discharge (ESD) diodes to AVDD and AVSS protect the inputs. To prevent the ESD diodes from turning on, the absolute voltage on any input must stay within the range provided by [Equation 3](#):

$$AVSS - 0.3 \text{ V} < V_{(AINx)} < AVDD + 0.3 \text{ V} \quad (3)$$

External Schottky clamp diodes or series resistors may be required to limit the input current to safe values (see the [Absolute Maximum Ratings](#) table). Overdriving an unselected input on the device can affect conversions taking place on other input pins.

Feature Description (continued)



- (1) A_{IN_P} and A_{IN_N} are connected together to $(AVDD + AVSS) / 2$ for offset measurement.
- (2) Measurement for the analog supply equivalent to $(AVDD - AVSS) / 4$.
- (3) Measurement for the analog supply equivalent to $DVDD / 4$.

Figure 49. Analog Input Multiplexer

Feature Description (continued)

9.3.2 Low-Noise Programmable Gain Amplifier

The ADS124S06 and ADS124S08 feature a low-drift, low-noise, high input impedance programmable gain amplifier (PGA). Figure 50 shows a simplified diagram of the PGA. The PGA consists of two chopper-stabilized amplifiers (A1 and A2) and a resistor feedback network that sets the gain of the PGA. The PGA input is equipped with an electromagnetic interference (EMI) filter and an antialiasing filter on the output.

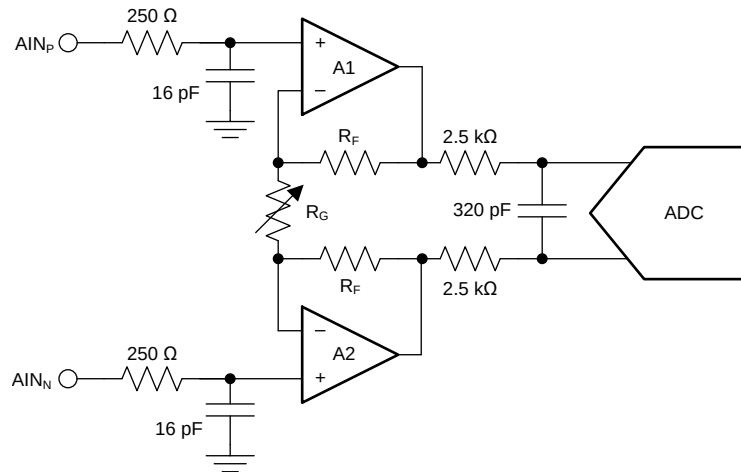


Figure 50. Simplified PGA Diagram

The PGA can be set to gains of 1, 2, 4, 8, 16, 32, 64, or 128 using the GAIN[2:0] bits in the gain setting register (03h). Gain is changed inside the device using a variable resistor, R_G . The differential full-scale input voltage range (FSR) of the PGA is defined by the gain setting and the reference voltage used, as shown in Equation 4:

$$\text{FSR} = \pm V_{\text{REF}} / \text{Gain} \quad (4)$$

Table 9 shows the corresponding full-scale ranges when using the internal 2.5-V reference.

Table 9. PGA Full-Scale Range

GAIN SETTING	FSR
1	± 2.5 V
2	± 1.25 V
4	± 0.625 V
8	± 0.313 V
16	± 0.156 V
32	± 0.078 V
64	± 0.039 V
128	± 0.020 V

The PGA must be enabled with the PGA_EN[1:0] bits of the gain setting register (03h). Setting these bits to 00 powers down and bypasses the PGA. A setting of 01 enables the PGA. The 10 and 11 settings are reserved and must not be written to the device.

With the PGA enabled, gains 64 and 128 are established in the digital domain. When the device is set to 64 or 128, the PGA is set to a gain of 32, and additional gain is established with digital scaling. The input-referred noise does still improve compared to the gain = 32 setting because the PGA is biased with a higher supply current to reduce noise.

9.3.2.1 PGA Input-Voltage Requirements

As with many amplifiers, the PGA has an absolute input voltage range requirement that cannot be exceeded. The maximum and minimum absolute input voltages are limited by the voltage swing capability of the PGA output. The specified minimum and maximum absolute input voltages (V_{AINP} and V_{AINN}) depend on the PGA gain, the maximum differential input voltage (V_{INMAX}), and the tolerance of the analog power-supply voltages ($AVDD$ and $AVSS$). Use the maximum voltage expected in the application for V_{INMAX} . The absolute positive and negative input voltages must be within the specified range, as shown in Equation 5:

$$AVSS + 0.15 \text{ V} + |V_{INMAX}| \cdot (\text{Gain} - 1) / 2 < V_{AINP}, V_{AINN} < AVDD - 0.15 \text{ V} - |V_{INMAX}| \cdot (\text{Gain} - 1) / 2$$

where

- V_{AINP}, V_{AINN} = absolute input voltage
 - $V_{INMAX} = V_{AINP} - V_{AINN}$ = maximum differential input voltage
- (5)

As mentioned in the previous section, PGA gain settings of 64 and 128 are scaled in the digital domain and are not implemented with the amplifier. When using the PGA in gains of 64 and 128, set the gain in Equation 5 to 32 to calculate the absolute input voltage range.

The relationship between the PGA input to the PGA output is shown graphically in Figure 51. The PGA output voltages (V_{OUTP}, V_{OUTN}) depend on the PGA gain and the input voltage magnitudes. For linear operation, the PGA output voltages must not exceed $AVDD - 0.15 \text{ V}$ or $AVSS + 0.15 \text{ V}$. Note that the diagram depicts a positive differential input voltage that results in a positive differential output voltage.

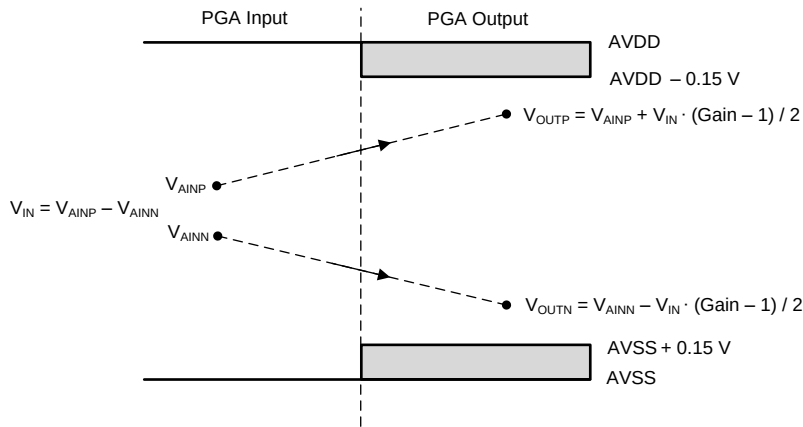


Figure 51. PGA Input/Output Range

Download the [ADS1x4S0x design calculator](http://www.ti.com) from www.ti.com. This calculator can be used to determine the input voltage range of the PGA.

9.3.2.2 PGA Rail Flags

The PGA rail flags (FL_P_RAILP, FL_P_RAILN, FL_N_RAILP, and FL_N_RAILN) in the status register (01h) indicate if the positive or negative output of the PGA is closer to the analog supply rails than 150 mV. Enable the PGA output rail detection circuit using the FL_RAIL_EN bit in the excitation current register 1 (06h). A flag going high indicates that the PGA is operating outside the linear operating or absolute input voltage range. PGA rail flags are discussed in more detail in the [PGA Output Voltage Rail Monitors](#) section.

9.3.2.3 Bypassing the PGA

At a gain of 1, the device can be configured to disable and bypass the low-noise PGA. Disabling the PGA lowers the overall power consumption and also removes the restrictions of Equation 5 for the input voltage range. If the PGA is bypassed, the ADC absolute input voltage range extends beyond the $AVDD$ and $AVSS$ power supplies, allowing input voltages at or below ground. The absolute input voltage range when the PGA is bypassed is shown in Equation 6:

$$AVSS - 0.05 \text{ V} < V_{AINP}, V_{AINN} < AVDD + 0.05 \text{ V}$$
(6)

In order to measure single-ended signals that are referenced to AVSS ($A_{INP} = V_{IN}$, $A_{INN} = AVSS$), the PGA must be bypassed. The PGA is bypassed and powered down by setting the PGA_EN[1:0] bits to 00 in the gain setting register (03h).

For signal sources with high output impedance, external buffering may still be necessary. Note that active buffers introduce noise and also introduce offset and gain errors. Consider all of these factors in high-accuracy applications.

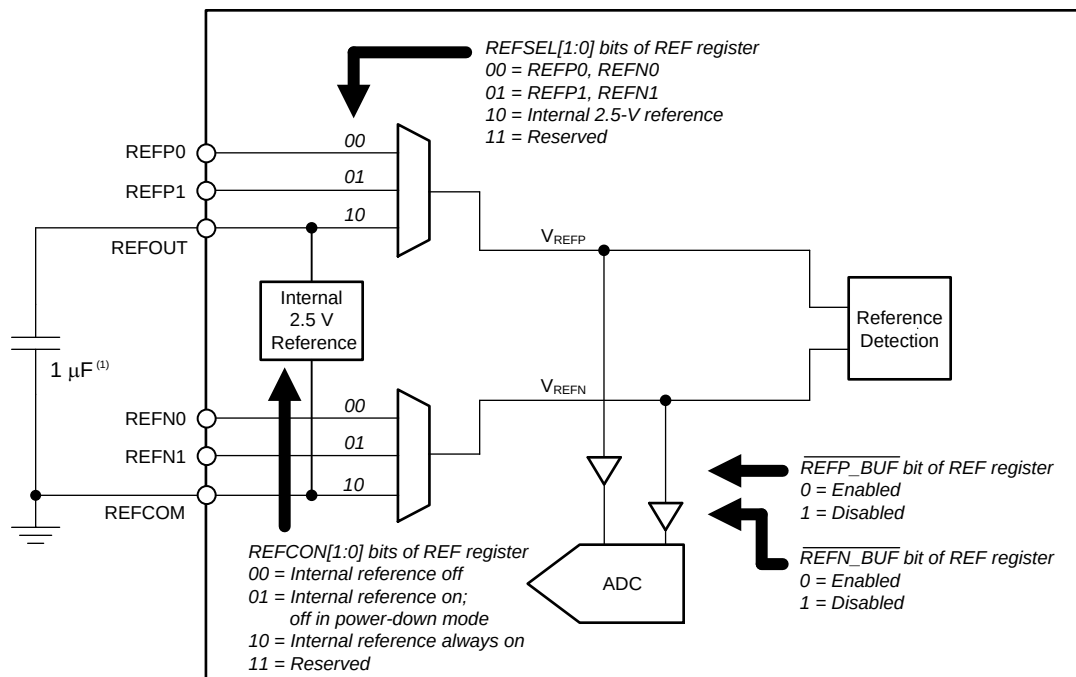
9.3.3 Voltage Reference

The devices require a reference voltage for operation. The ADS124S0x offers an integrated low-drift 2.5-V reference. For applications that require a different reference voltage value or a ratiometric measurement approach, the ADS124S08 offers two differential reference input pairs (REFP0, REFN0 and REFP1, REFN1). The differential reference inputs allow freedom in the reference common-mode voltage. REFP0 and REFN0 are dedicated reference inputs, whereas REFP1 and REFN1 are shared with inputs AIN6 and AIN7 (respectively) on the ADS124S08. The specified external reference voltage range is 0.5 V to AVDD. The reference voltage is shown in Equation 7, where $V_{(REFPx)}$ and $V_{(REFNx)}$ are the absolute positive and absolute negative reference voltages.

$$V_{REF} = V_{(REFPx)} - V_{(REFNx)} \quad (7)$$

The polarity of the reference voltage internal to the ADC must be positive. The magnitude of the reference voltage together with the PGA gain establishes the ADC full-scale differential input range as defined by $FSR = \pm V_{REF} / \text{Gain}$.

Figure 52 shows the block diagram of the reference multiplexer. The ADC reference multiplexer selects between the internal reference and two external references (REF0 and REF1). The reference multiplexer is programmed with the REFSEL[1:0] bits in the reference control register (05h). By default, the external reference pair REFP0, REFN0 is selected.



(1) The internal reference requires a minimum 1-μF capacitor connected from REFP0 to REFCOM.

Figure 52. Reference Multiplexer Block Diagram

The ADC also contains an integrated reference voltage monitor. This monitor provides continuous detection of a low or missing reference during the conversion cycle. The reference monitor flags (FL_REF_L0 and FL_REF_L1) are set in the STATUS byte and described in the [Reference Monitor](#) section.

9.3.3.1 Internal Reference

The ADC integrates a precision, low-drift, 2.5-V reference. The internal reference is enabled by setting REFCON[1:0] to 10 (reference is always on) or 01 (reference is on, but powers down in power-down mode) in the reference control register (05h). By default, the internal voltage reference is powered down. To select the internal reference for use with the ADC, set the REFSEL[1:0] bits to 10. The REFOUT pin provides a buffered reference output voltage when the internal reference voltage is enabled. The negative reference output is the REFCOM pin, as shown in [Figure 52](#). Connect a capacitor in the range of 1 μ F to 47 μ F between REFOUT and REFCOM. Larger capacitor values help filter more noise at the expense of a longer reference start-up time.

The capacitor is not required if the internal reference is not used. However, the internal reference must be powered on if using the IDACs.

The internal reference requires a start-up time that must be accounted for before starting a conversion, as shown in [Table 10](#).

Table 10. Internal Reference Settling Time

REFOUT CAPACITOR	SETTLING ERROR	SETTLING TIME (ms)
1 μ F	0.01%	4.5
	0.001%	5.9
10 μ F	0.01%	4.9
	0.001%	6.3
47 μ F	0.01%	5.5
	0.001%	7.0

9.3.3.2 External Reference

The ADS124S0x provides two external reference inputs selectable through the reference multiplexer. The reference inputs are differential with independent positive and negative inputs. REFP0 and REFN0 or REFP1 and REFN1 can be selected as the ADC reference. REFP1 and REFN1 are shared inputs with analog pins AIN6 and AIN7 in the ADS124S08.

Without buffering, the reference input impedance is approximately 250 k Ω . The reference input current can lead to possible errors from either high reference source impedance or through reference input filtering. To reduce the input current, use either internal or external reference buffers. In most applications external reference buffering is not necessary.

Connect a 100-nF bypass capacitor across the external reference input pins. Follow the specified absolute and differential reference voltage requirements.

9.3.3.3 Reference Buffers

The device has two individually selectable reference input buffers to lower the reference input current. Use the REFP_BUF and REFN_BUF bits in the reference control register (05h) to enable or disable the positive and negative reference buffers respectively. Note that these bits are active low. Writing a 1 to REFP_BUF or REFN_BUF disables the reference buffers.

The reference buffers are recommended to be disabled when the internal reference is selected for measurements. When the external reference input is at the supply voltage (REFPx at AVDD or REFNx at AVSS), the reference buffer is recommended to be disabled.

9.3.4 Clock Source

The ADS124S0x system clock is either provided by the internal low-drift 4.096-MHz oscillator or an external clock source on the CLK input. Use the CLK bit within the data rate register (04h) to select the internal 4.096-MHz oscillator or an external clock source.

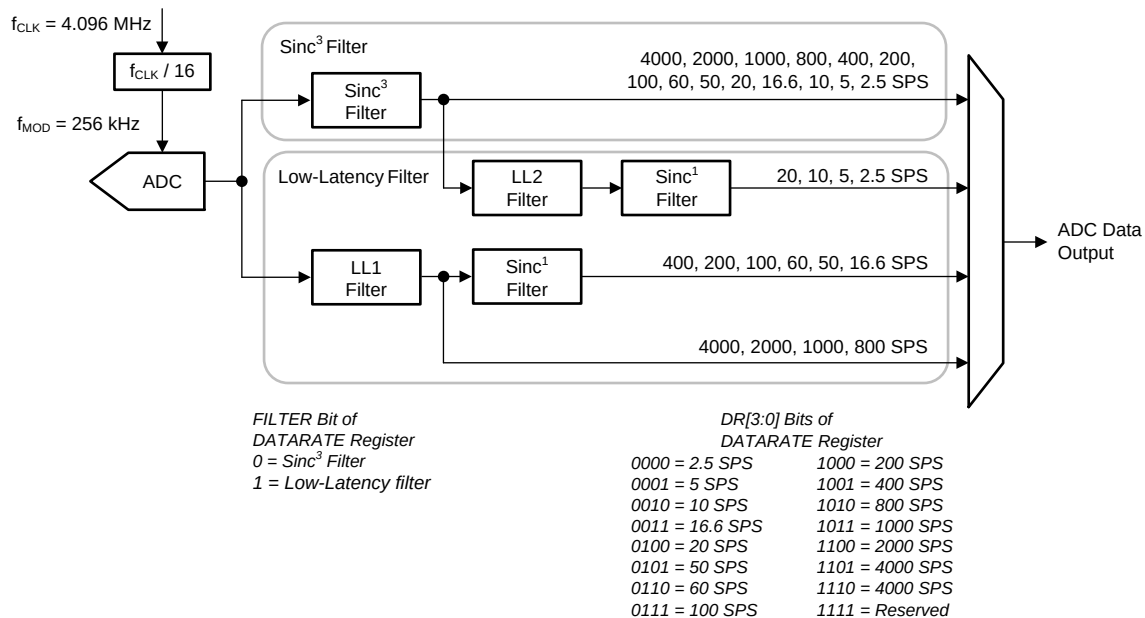
The device defaults to using the internal oscillator. If the device is reset (from either the $\overline{\text{RESET}}$ pin, or the RESET command), then the clock source returns to using the internal oscillator even if an external clock is selected.

9.3.5 Delta-Sigma Modulator

A delta-sigma ($\Delta\Sigma$) modulator is used in the devices to convert the analog input voltage into a pulse code modulated (PCM) data stream. The modulator runs at a modulator clock frequency of $f_{\text{MOD}} = f_{\text{CLK}} / 16$, where f_{CLK} is either provided by the internal 4.096-MHz oscillator or the external clock source.

9.3.6 Digital Filter

The devices offer digital filter options for both filtering and decimation of the digital data stream coming from the delta-sigma modulator. The implementation of the digital filter is determined by the data rate and filter mode setting. Figure 53 shows the digital filter implementation. Choose between a third-order sinc filter (sinc^3) and a low-latency filter (low-latency filter with multiple components) using the FILTER bit in the data rate register (04h).



NOTE: LL filter = low-latency filter.

Figure 53. Digital Filter Architecture

Regardless of the FILTER type setting, the oversampling ratio is the same for each given data rate, meaning that the device requires a set number of modulator clocks to output a single ADC conversion data. The output data rate is selected using the DR[3:0] bits in the data rate register and is shown in [Table 11](#).

Table 11. ADC Data Rates and Digital Filter Oversampling Ratios

NOMINAL DATA RATE (SPS) ⁽¹⁾	DATA RATE REGISTER DR[3:0]	OVERSAMPLING RATIO ⁽²⁾
2.5	0000	102400
5	0001	51200
10	0010	25600
16.6	0011	15360
20	0100	12800
50	0101	5120
60	0110	4264
100	0111	2560
200	1000	1280
400	1001	640
800	1010	320
1000	1011	256
2000	1100	128
4000	1101	64

(1) Valid for the internal oscillator or an external 4.096-MHz clock.

(2) The oversampling ratio is f_{MOD} divided by the data rate; $f_{MOD} = f_{CLK} / 16$.

9.3.6.1 Low-Latency Filter

The low-latency filter is selected when the FILTER bit is set to 0 in the data rate register (04h). The filter is a finite impulse response (FIR) filter that provides settled data, given that the analog input signal has settled to the final value before the conversion is started. The low-latency filter is especially useful when multiple channels must be scanned in minimal time.

9.3.6.1.1 Low-Latency Filter Frequency Response

The low-latency filter provides many data rate options for rejecting 50-Hz and 60-Hz line cycle noise. At data rates of 2.5 SPS, 5 SPS, 10 SPS, and 20 SPS, the filter rejects both 50-Hz and 60-Hz line frequencies. At data rates of 16.6 SPS and 50 SPS, the filter has a notch at 50 Hz. At a 60-SPS data rate, the filter has a notch at 60 Hz.

For detailed frequency response plots showing line cycle noise rejection, download the [ADS1x4S0x design calculator](#) from www.ti.com.

Figure 54 to Figure 68 show the frequency response of the low-latency filter for different data rates. Table 12 gives the bandwidth of the low-latency filter for each data rate.

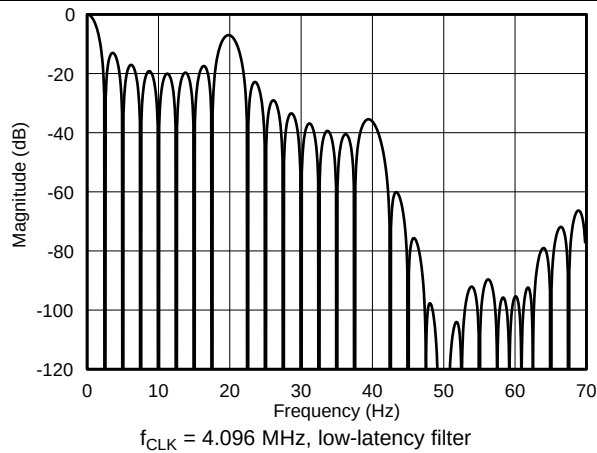


Figure 54. Low-Latency Filter Frequency Response, Data Rate = 2.5 SPS

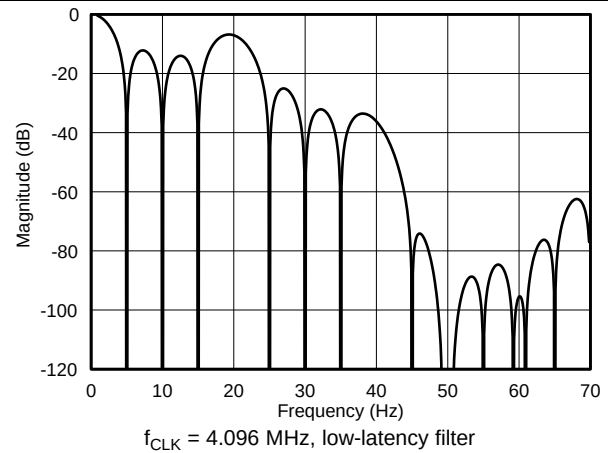


Figure 55. Low-Latency Filter Frequency Response, Data Rate = 5 SPS

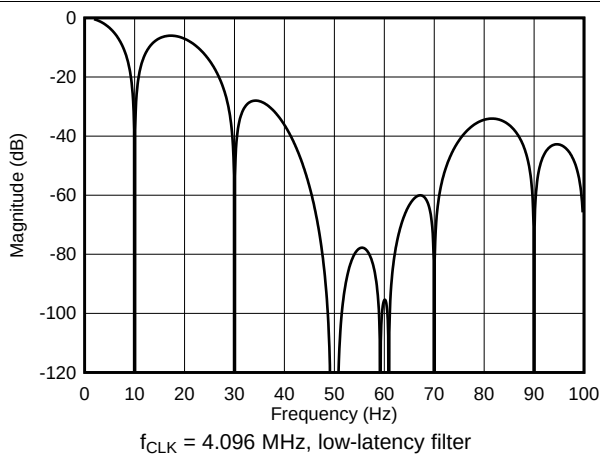


Figure 56. Low-Latency Filter Frequency Response, Data Rate = 10 SPS

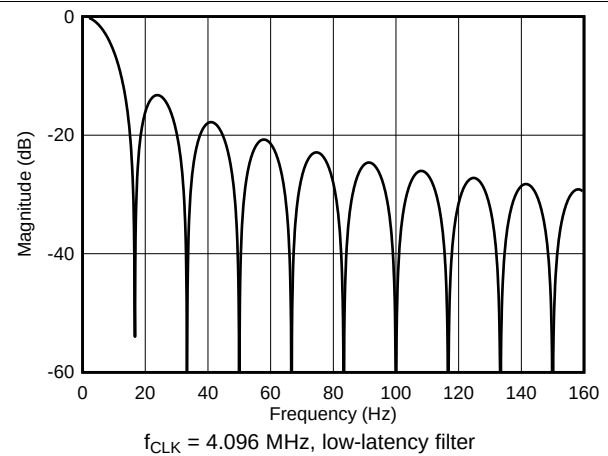


Figure 57. Low-Latency Filter Frequency Response, Data Rate = 16.6 SPS

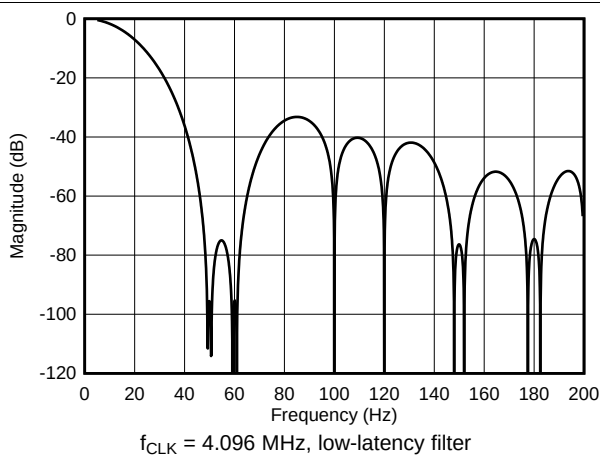


Figure 58. Low-Latency Filter Frequency Response, Data Rate = 20 SPS

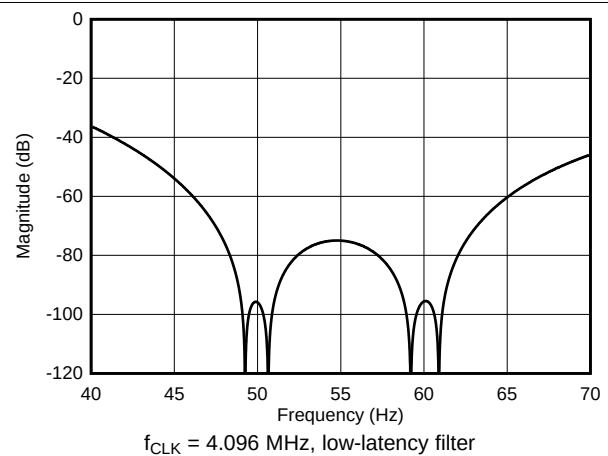
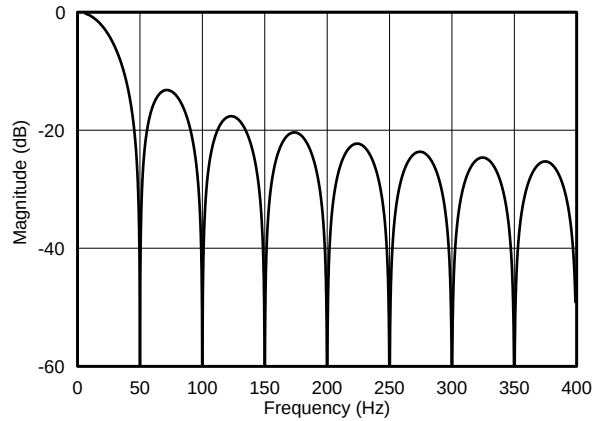


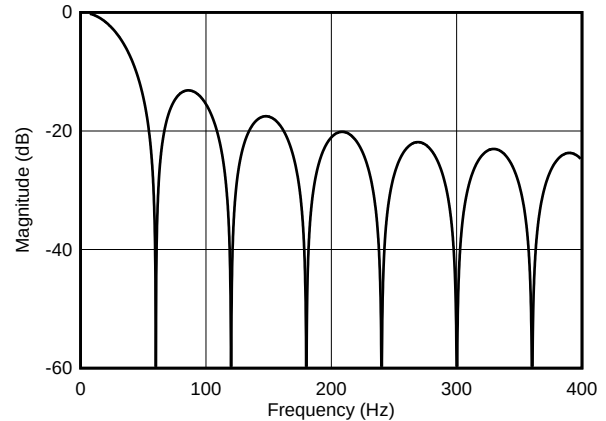
Figure 59. Low-Latency Filter Frequency Response, Data Rate = 20 SPS, Zoomed to 50 Hz and 60 Hz

ADS124S06, ADS124S08

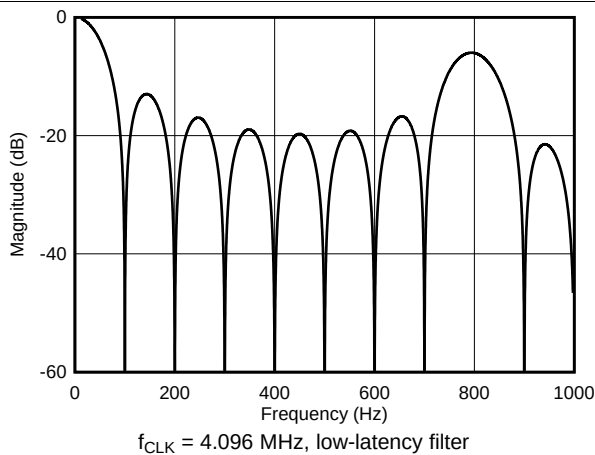
SBAS660C –AUGUST 2016–REVISED JUNE 2017

www.ti.com


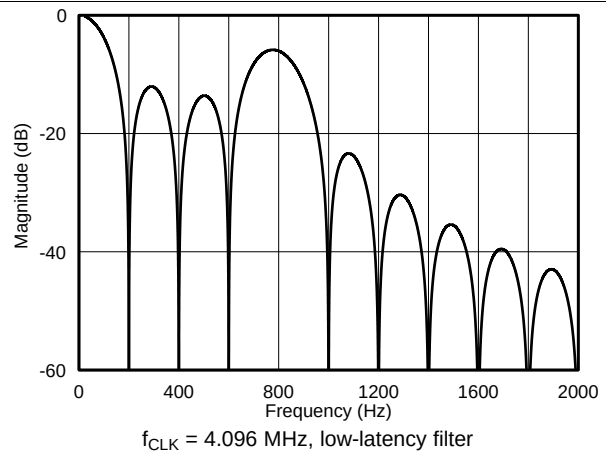
**Figure 60. Low-Latency Filter Frequency Response,
Data Rate = 50 SPS**



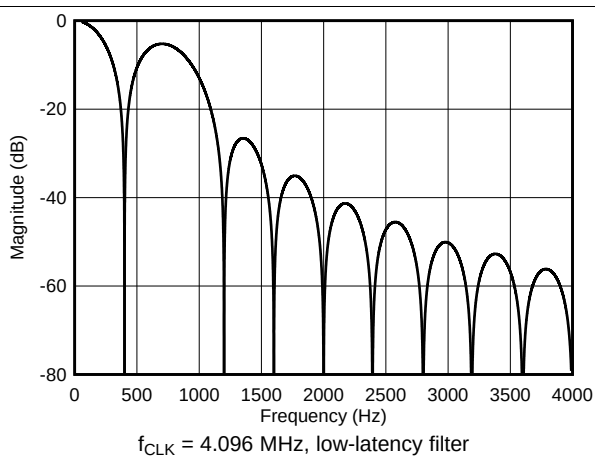
**Figure 61. Low-Latency Filter Frequency Response,
Data Rate = 60 SPS**



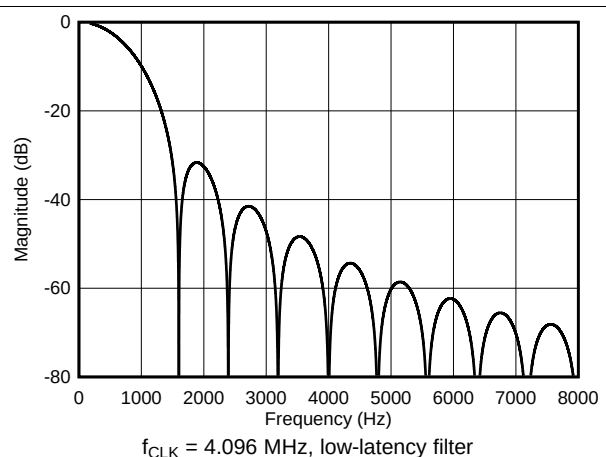
**Figure 62. Low-Latency Filter Frequency Response,
Data Rate = 100 SPS**



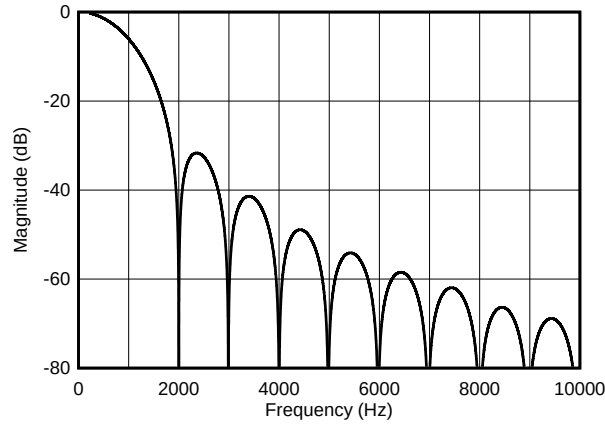
**Figure 63. Low-Latency Filter Frequency Response,
Data Rate = 200 SPS**



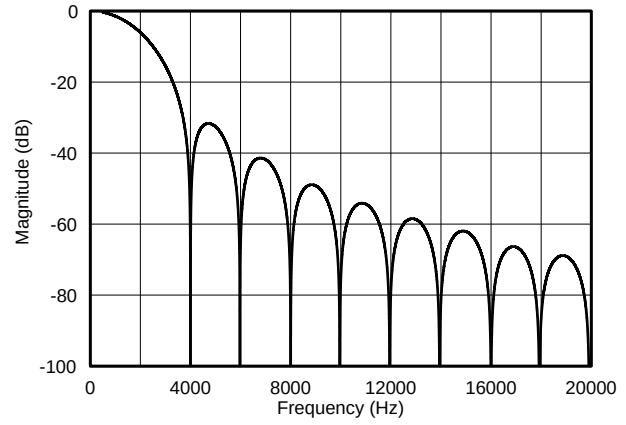
**Figure 64. Low-Latency Filter Frequency Response,
Data Rate = 400 SPS**



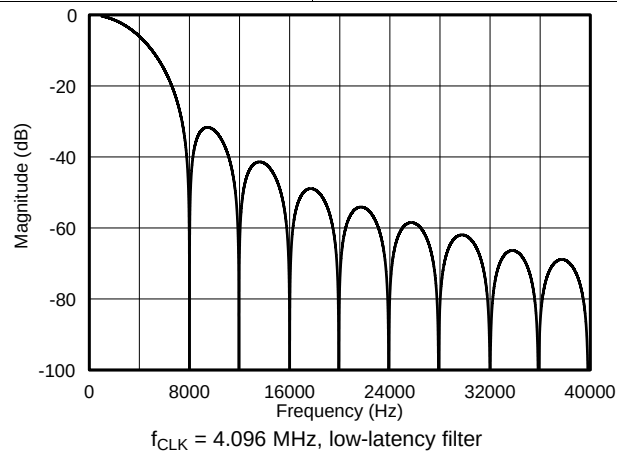
**Figure 65. Low-Latency Filter Frequency Response,
Data Rate = 800 SPS**



**Figure 66. Low-Latency Filter Frequency Response,
Data Rate = 1 kSPS**



**Figure 67. Low-Latency Filter Frequency Response,
Data Rate = 2 kSPS**



**Figure 68. Low-Latency Filter Frequency Response,
Data Rate = 4 kSPS**

Table 12. Low-Latency Filter Bandwidth

NOMINAL DATA RATE (SPS) ⁽¹⁾	-3-dB BANDWIDTH (Hz) ⁽¹⁾
2.5	1.1
5	2.2
10	4.7
16.6	7.4
20	13.2
50	22.1
60	26.6
100	44.4
200	89.9
400	190
800	574
1000	718
2000	718
4000	718

(1) Valid for the internal oscillator or an external 4.096-MHz clock. Scales proportional with f_{CLK} .

The low-latency filter notches and output data rate scale proportionally with the clock frequency. For example, a notch that appears at 20 Hz when using a 4.096-MHz clock appears at 10 Hz if a 2.048-MHz clock is used. Note that the internal oscillator can vary over temperature as specified in the [Electrical Characteristics](#) table. The data rate, conversion time, and filter notches consequently vary by the same percentage. Consider using an external precision clock source if a digital filter notch at a specific frequency with a tighter tolerance is required.

9.3.6.1.2 Data Conversion Time for the Low-Latency Filter

The amount of time required to receive data from the ADC depends on more than just the nominal data rate of the device. The data period also depends on the mode of operation and other configurations of the device. When the low-latency filter is enabled, the data settles in one data period. However, a small amount of latency exists to set up the device, calculate the conversion data from the modulator samples, and other overhead that adds time to the conversion. For this reason, the first conversion data takes longer than subsequent data conversions.

[Table 13](#) shows the conversion times for the low-latency filter for each ADC data rate and various conversion modes.

Table 13. Data Conversion Time for the Low-Latency Filter

NOMINAL DATA RATE ⁽¹⁾ (SPS)	FIRST DATA FOR CONTINUOUS CONVERSION MODE OR SINGLE-SHOT CONVERSION MODE ⁽²⁾		SECOND AND SUBSEQUENT CONVERSIONS FOR CONTINUOUS CONVERSION MODE	
	ms ⁽³⁾	NUMBER OF t _{MOD} PERIODS ⁽³⁾	ms ⁽⁴⁾	NUMBER OF t _{MOD} PERIODS ⁽⁴⁾
2.5	406.504	104065	400	102400
5	206.504	52865	200	51200
10	106.504	27265	100	25600
16.6	60.254	15425	60	15360
20	56.504	14465	50	12800
50	20.156	5160	20	5120
60	16.910	4329	16.66	4264
100	10.156	2600	10	2560
200	5.156	1320	5	1280
400	2.656	680	2.5	640
800	1.406	360	1.25	320
1000	1.156	296	1	256
2000	0.656	168	0.5	128
4000	0.406	104	0.25	64

(1) Valid for the internal oscillator or an external 4.096-MHz clock. Scales proportional with f_{CLK} .

(2) Conversions start at the rising edge of the START/SYNC pin or on the seventh SCLK falling edge for a START command.

(3) Time does not include the programmable delay set by the DELAY[2:0] bits in the gain setting register. The default setting is an additional $14 \cdot t_{MOD}$, where $t_{MOD} = t_{CLK} \cdot 16$.

(4) Subsequent readings in continuous conversion mode do not have the programmable delay time.

9.3.6.2 Sinc³ Filter

The sinc³ digital filter is selected when the FILTER bit is set to 0 in the data rate register (04h). Compared to the low-latency filter, the sinc³ filter has improved noise performance but has a three-cycle latency in the data output.

9.3.6.2.1 Sinc³ Filter Frequency Response

The low-pass nature of the sinc³ filter establishes the overall frequency response. The frequency response is given by [Equation 8](#):

$$|H(f)| = |H_{\text{Sinc}^3}(f)| = \left| \frac{\sin\left(\frac{16\pi f \cdot \text{OSR}}{f_{\text{CLK}}}\right)}{\text{OSR} \times \sin\left(\frac{16\pi f}{f_{\text{CLK}}}\right)} \right|^3$$

where

- f = signal frequency
- f_{CLK} = ADC clock frequency
- OSR = oversampling ratio

(8)

The sinc³ filter offers simultaneous 50-Hz and 60-Hz line cycle rejection at data rates of 2.5 SPS, 5 SPS, and 10 SPS. The sinc³ filter offers only 50-Hz rejection at data rates of 16.6 SPS and 50 SPS, and only 60-Hz rejection at data rates of 20 SPS and 60 SPS. The sinc³ digital filter response scales with the data rate and has notches at multiples of the data rate. [Figure 69](#) shows the sinc³ digital filter frequency response normalized to the data rate. As an example, [Figure 70](#) shows the frequency response when the data rate is set to 10 SPS, and [Figure 71](#) illustrates a close-up of the filter rejection of 50-Hz and 60-Hz line frequencies. For more detailed frequency response plots, download the [ADS1x4S0x design calculator](#) from www.ti.com.

[Table 14](#) gives the bandwidth of the sinc³ filter for each data rate.

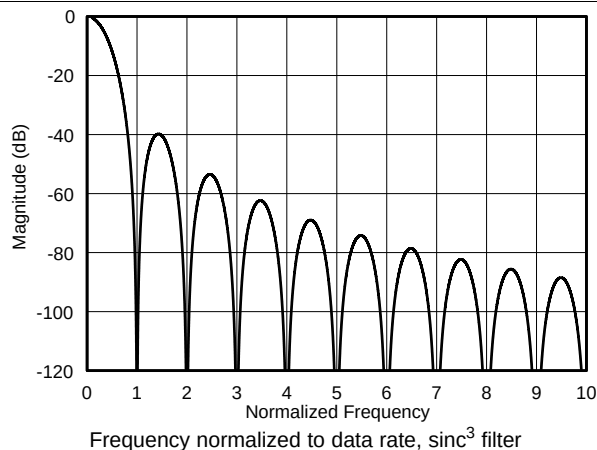


Figure 69. Sinc³ Filter Frequency Response, Normalized to Data Rate

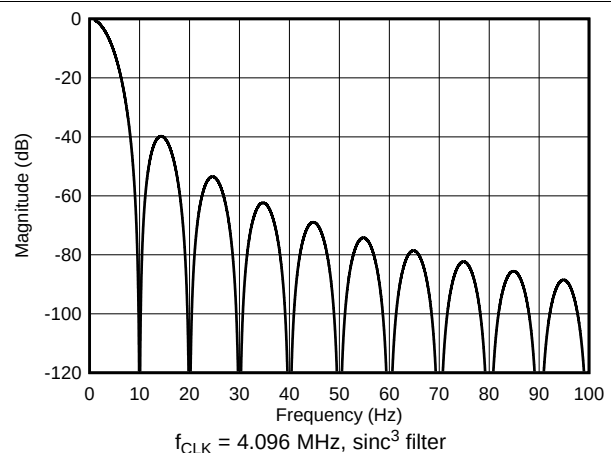
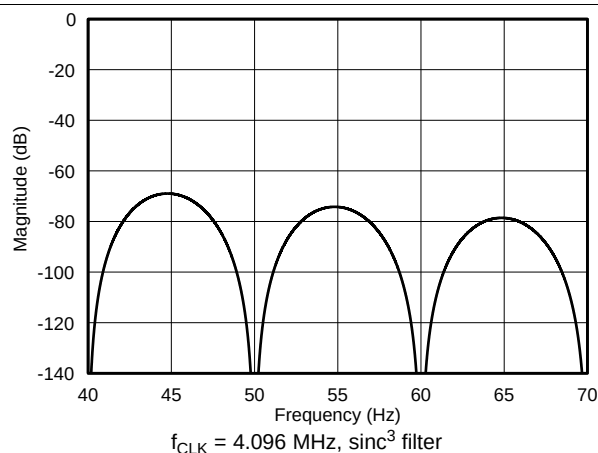


Figure 70. Sinc³ Filter Frequency Response, Data Rate = 10 SPS



**Figure 71. Sinc³ Filter Frequency Response,
Data Rate = 10 SPS, Zoomed to 50 Hz and 60 Hz**

Table 14. Sinc³ Filter –3-dB Bandwidth

NOMINAL DATA RATE (SPS) ⁽¹⁾	–3-dB BANDWIDTH (Hz) ⁽¹⁾
2.5	0.65
5	1.3
10	2.6
16.6	4.4
20	5.2
50	13.1
60	15.7
100	26.2
200	52.3
400	105
800	209
1000	262
2000	523
4000	1046

(1) Valid for the internal oscillator or an external 4.096-MHz clock. Scales proportional with f_{CLK} .

As mentioned in the previous section, filter notches and output data rate scale proportionally with the clock frequency and the internal oscillator can change frequency with temperature.

9.3.6.2.2 Data Conversion Time for the Sinc³ Filter

Similar to the low-latency filter, the sinc³ filter requires different amounts of time to complete a conversion. By nature, the sinc³ filter normally takes three conversion to settle. In both single-shot conversion mode and continuous conversion mode, the first two conversions are suppressed so that only settled data are output by the ADC.

Table 15 shows the conversion times for the sinc³ filter for each ADC data rate and various conversion modes.

Table 15. Data Conversion Time for the Sinc³ Filter

NOMINAL DATA RATE ⁽¹⁾ (SPS)	FIRST DATA FOR CONTINUOUS CONVERSION MODE OR SINGLE-SHOT CONVERSION MODE ⁽²⁾		SECOND AND SUBSEQUENT CONVERSIONS FOR CONTINUOUS CONVERSION MODE	
	ms ⁽³⁾	NUMBER OF t _{MOD} PERIODS ⁽³⁾	ms ⁽⁴⁾	NUMBER OF t _{MOD} PERIODS ⁽⁴⁾
2.5	1200.254	307265	400	102400
5	600.254	153665	200	51200
10	300.254	76865	100	25600
16.6	180.254	46145	60	15360
20	150.254	38465	50	12800
50	60.254	15425	20	5120
60	50.223	12857	16.66	4264
100	30.254	7745	10	2560
200	15.254	3905	5	1280
400	7.754	1985	2.5	640
800	4.004	1025	1.25	320
1000	3.156	808	1	256
2000	1.656	424	0.5	128
4000	0.906	232	0.25	64

(1) Valid for the internal oscillator or an external 4.096-MHz clock. Scales proportional with f_{CLK} .

(2) Conversions start at the rising edge of the START/SYNC pin or on the seventh SCLK falling edge for a START command.

(3) Time does not include the programmable delay set by the DELAY[2:0] bits in the gain setting register. The default setting is an additional $14 \cdot t_{MOD}$, where $t_{MOD} = t_{CLK} \cdot 16$.

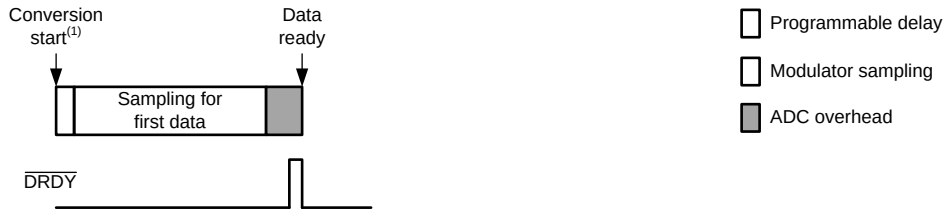
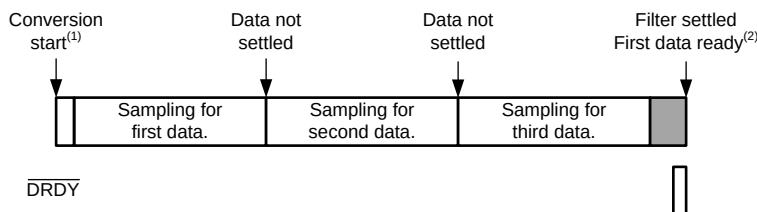
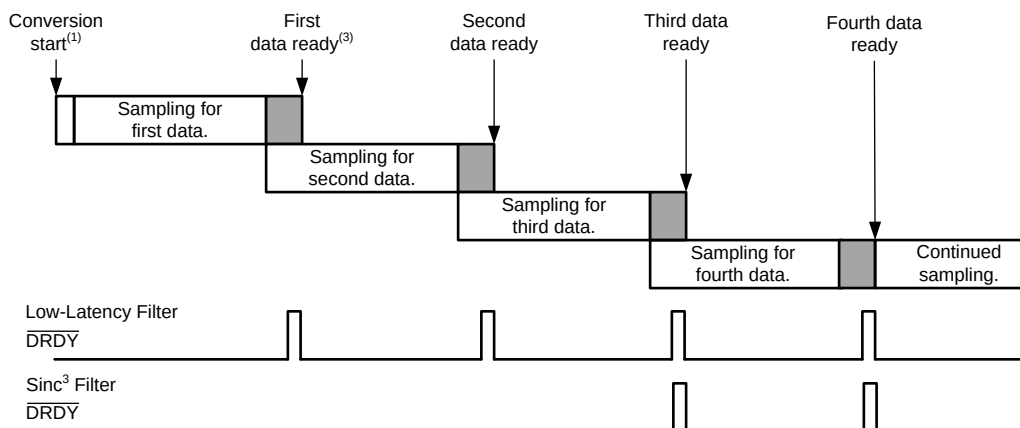
(4) Subsequent readings in continuous conversion mode do not have the programmable delay time.

9.3.6.3 Note on Conversion Time

Each data period consists of time required for the modulator to sample the analog inputs. However, there is additional time required before the samples become an ADC conversion result. First, there is a programmable conversion delay (described in the [Programmable Conversion Delay](#) section) that is added before the conversion starts. This delay allows for additional settling time for input filtering on the analog inputs and for the antialiasing filter after the PGA. The default programmable conversion delay is $14 \cdot t_{MOD}$. Also, overhead time is needed to convert the modulator samples into an ADC conversion result. This overhead time includes any necessary offset or gain compensation after the digital filter accumulates a data result.

The first conversion when the device is in continuous conversion mode (just as in single-shot conversion mode) includes the programmable conversion delay, the modulator sampling time, and the overhead time. The second and subsequent conversions are the normal data period (period as given by the inverse of the data rate).

Figure 72 shows the time sequence for the ADC in both continuous conversion and single-shot conversion modes. The sequence is the same regardless of the filter setting. However, when the low-latency filter settles for each data, the sinc³ filter does not settle until the third data.

Single-shot conversion mode: Low-Latency filter

Single-shot conversion mode: Sinc³ Filter

Continuous conversion mode: Low-Latency or Sinc³ Filter


(1) Conversions start at the rising edge of the START/SYNC pin or on the seventh SCLK falling edge for a START command.

(2) In sinc³ filter mode, the first two data outputs are suppressed to allow for the measurement data to settle.

(3) In sinc³ filter mode, there is no overhead time for the first two data, which are not available to be read.

Figure 72. Single-Shot Conversion Mode and Continuous Conversion Mode Sequences

9.3.6.4 50-Hz and 60-Hz Line Cycle Rejection

If the ADC connection leads are in close proximity to industrial motors and conductors, coupling of 50-Hz and 60-Hz power line frequencies can occur. The coupled noise interferes with the signal voltage, and can lead to inaccurate or unstable conversions. The digital filter provides enhanced rejection of power-line-coupled noise for data rates of 60 SPS and less. Program the filter to tradeoff data rate and conversion latency versus the desired level of line cycle rejection. [Table 16](#) and [Table 17](#) summarize the ADC 50-Hz and 60-Hz line-cycle rejection based on ± 1 -Hz and ± 2 -Hz tolerance of power-line to ADC clock frequency. The best possible power-line rejection is provided by using an accurate ADC clock.

Table 16. Low-Latency Filter, 50-Hz and 60-Hz Line Cycle Rejection

DATA RATE (SPS) ⁽¹⁾	LOW-LATENCY DIGITAL FILTER LINE CYCLE REJECTION (dB)			
	50 Hz ± 1 Hz	60 Hz ± 1 Hz	50 Hz ± 2 Hz	60 Hz ± 2 Hz
2.5	–113.7	–95.4	–97.7	–92.4
5	–111.9	–95.4	–87.6	–81.8
10	–111.5	–95.4	–85.7	–81.0
16.6	–33.8	–20.9	–27.8	–20.8
20	–95.4	–95.4	–75.5	–80.5
50	–33.8	–15.5	–27.6	–15.1
60	–13.4	–35.0	–12.6	–29.0

(1) $f_{CLK} = 4.096$ MHz.

Table 17. Sinc³ Filter, 50-Hz and 60-Hz Line Cycle Rejection

DATA RATE (SPS) ⁽¹⁾	SINC ³ DIGITAL FILTER LINE CYCLE REJECTION (dB)			
	50 Hz ± 1 Hz	60 Hz ± 1 Hz	50 Hz ± 2 Hz	60 Hz ± 2 Hz
2.5	–108.7	–113.4	–107.2	–112.1
5	–103.2	–107.8	–90.1	–95.0
10	–101.8	–106.4	–84.6	–89.4
16.6	–101.6	–63.0	–83.4	–62.4
20	–53.5	–106.1	–53.5	–88.0
50	–101.4	–46.7	–82.9	–45.3
60	–40.3	–105.1	–37.8	–87.2

(1) $f_{CLK} = 4.096$ MHz.

9.3.6.5 Global Chop Mode

The device uses a very low-drift PGA and modulator in order to provide very low input voltage offset drift. However, a small amount of offset voltage drift sometimes remains in normal measurement. The ADC incorporates a global chop option to reduce the offset voltage and offset voltage drift to very low levels. When the global chop is enabled, the ADC performs two internal conversions to cancel the input offset voltage. The first conversion is taken with normal input polarity. The ADC reverses the internal input polarity for a second conversion. The average of the two conversions yields the final corrected result, removing the offset voltage. The global chop mode is enabled using the G_CHOP bit in the data rate register (04h). [Figure 73](#) shows a block diagram of the global chop implementation. The combined PGA and ADC internal offset voltage is modeled as V_{OFS} .

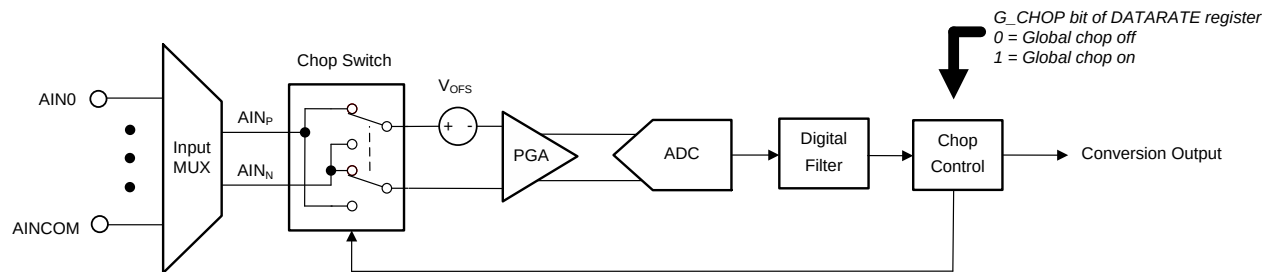


Figure 73. ADC Global Chop Block Diagram

The first conversion result is available after the ADC takes two separate conversions with settled data. When using the low-latency filter, data settles in a single conversion. When the global chop mode is enabled, the first conversion result appears after a time period of approximately two conversions. When using the sinc³ filter, data settles in three conversions. If the global chop mode is enabled, the first conversion result appears after a time period of approximately six conversions.

In continuous conversion mode with the global chop mode enabled, subsequent conversions complete in half the time as the first conversion completed. Data for alternating inputs are pipelined so that averaging appears on each ADC data cycle. Conversion times using the global chop mode are given in [Table 18](#) and [Table 19](#).

Table 18. Data Conversion Time for Global Chop Mode Using the Low-Latency Filter

NOMINAL DATA RATE ⁽¹⁾ (SPS)	FIRST DATA CONVERSION PERIOD FOR GLOBAL CHOP MODE ⁽²⁾		SECOND AND SUBSEQUENT CONVERSION PERIODS FOR GLOBAL CHOP MODE	
	ms ⁽³⁾	NUMBER OF t_{MOD} PERIODS ⁽³⁾	ms ⁽³⁾	NUMBER OF t_{MOD} PERIODS ⁽³⁾
2.5	813.008	208130	406.504	104065
5	413.008	105730	206.504	52865
10	213.008	54530	106.504	27265
16.66	120.508	30850	60.254	15425
20	113.008	28930	56.504	14465
50	40.313	10320	20.156	5160
60	33.820	8658	16.910	4329
100	20.313	5200	10.156	2600
200	10.313	2640	5.156	1320
400	5.313	1360	2.656	680
800	2.813	720	1.406	360
1000	2.313	592	1.156	296
2000	1.313	336	0.656	168
4000	0.813	208	0.406	104

(1) Valid for the internal oscillator or an external 4.096-MHz clock. Scales proportional with f_{CLK} .

(2) Conversions start at the rising edge of the START/SYNC pin or on the seventh SCLK falling edge for a START command.

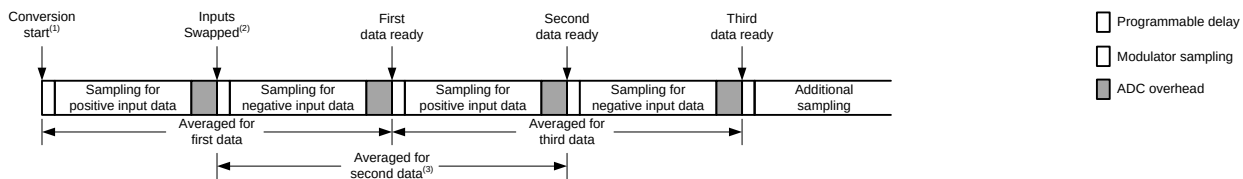
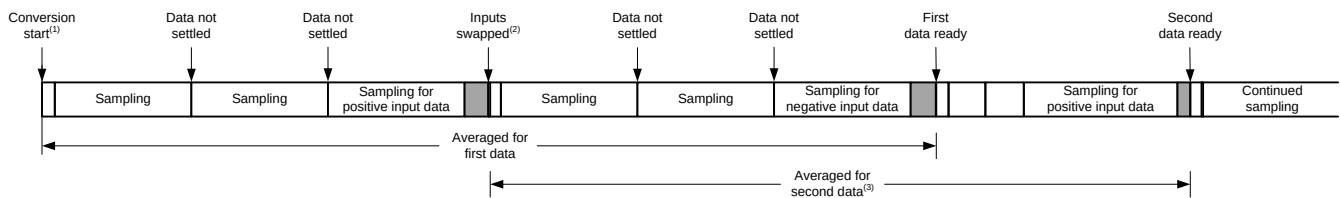
(3) Time does not include the programmable delay set by the DELAY[2:0] bits in the gain setting register. Global chop mode requires two conversions, doubling the additional time. The default setting adds an extra $28 \cdot t_{MOD}$ (where $t_{MOD} = t_{CLK} \cdot 16$) to this column.

Table 19. Data Conversion Time for Global Chop Mode Using the Sinc³ Filter

NOMINAL DATA RATE ⁽¹⁾ (SPS)	FIRST DATA CONVERSION PERIOD FOR GLOBAL CHOP MODE ⁽²⁾		SECOND AND SUBSEQUENT CONVERSION PERIODS FOR GLOBAL CHOP MODE	
	ms ⁽³⁾	NUMBER OF t_{MOD} PERIODS ⁽³⁾	ms ⁽³⁾	NUMBER OF t_{MOD} PERIODS ⁽³⁾
2.5	2400.508	614530	1200.254	307265
5	1200.508	307330	600.254	153665
10	600.508	153730	300.254	76865
16.66	360.508	92290	180.254	46145
20	300.508	76930	150.254	38465
50	120.508	30850	60.254	15425
60	100.445	25714	50.223	12857
100	60.508	15490	30.254	7745
200	30.508	7810	15.254	3905
400	15.508	3970	7.754	1985
800	8.008	2050	4.004	1025
1000	6.313	1616	3.156	808
2000	3.313	848	1.656	424
4000	1.813	464	0.906	232

- (1) Valid for the internal oscillator or an external 4.096-MHz clock. Scales proportional with f_{CLK} .
- (2) Conversions start at the rising edge of the START/SYNC pin or on the seventh SCLK falling edge for a START command.
- (3) Time does not include the programmable delay set by the DELAY[2:0] bits in the gain setting register. Global chop mode requires two conversions, doubling the additional time. The default setting adds an extra $28 \cdot t_{MOD}$ (where $t_{MOD} = t_{CLK} \cdot 16$) to this column.

In global chop mode, sequences are similar to taking consecutive single-shot conversions and swapping the input on each conversion. Output data are averaged using the last two data read operations by the ADC with the inputs swapped. [Figure 74](#) shows the time sequence for the ADC using global chop mode.

Global chop enabled, continuous conversion mode: Low-Latency filter

Global chop enabled, continuous conversion mode: Sinc³ Filter


- (1) Conversions start at the rising edge of the START/SYNC pin or on the seventh SCLK falling edge for a START command.
- (2) When the first data are collected, the inputs are swapped.
- (3) Measurements are averaged after the inputs are swapped for each conversion.

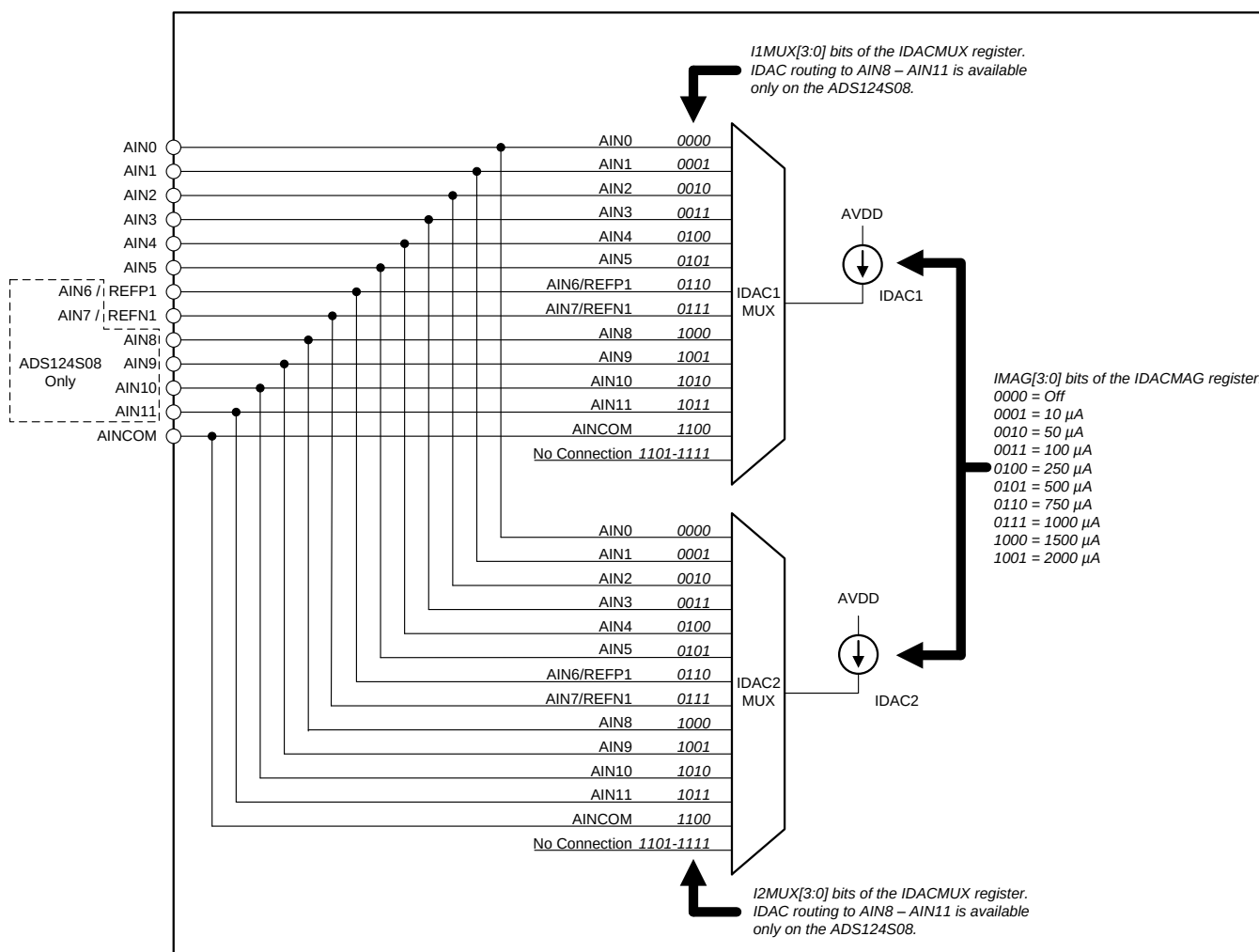
Figure 74. Global Chop Enabled Conversion Mode Sequences

Because the digital filter must settle after reversing the inputs, the global chop mode data rate is less than the nominal data rate, depending on the digital filter and programmed settling delay. However, if the data rate in use has 50-Hz and 60-Hz frequency response notches, the null frequencies remain unchanged.

The global chop mode also reduces the ADC noise by a factor of $\sqrt{2}$ because two conversions are averaged. In some cases, the programmable conversion delay must be increased, DELAY[2:0] in the gain setting register (03h), to allow for settling of external components.

9.3.7 Excitation Current Sources (IDACs)

The ADS124S0x incorporates two integrated, matched current sources (IDAC1, IDAC2). The current sources provide excitation current to resistive temperature devices (RTDs), thermistors, diodes, and other resistive sensors that require constant current biasing. The current sources are programmable to output values between 10 μA to 2000 μA using the IMAG[3:0] bits in the excitation current register 1 (06h). Each current source can be connected to any of the analog inputs AINx as well as the REFP1 and REFN1 inputs for the ADS124S06. Both current sources can also be connected to the same pin. The routing of the IDACs is configured by the I1MUX[3:0] and I2MUX[3:0] bits in the excitation current register 2 (07h). In three-wire RTD applications, the matched current sources can be used to cancel errors caused by sensor lead resistance (see the [Typical Application](#) section for more details). [Figure 75](#) details the IDAC connection through the input multiplexer.



Copyright © 2017, Texas Instruments Incorporated

Figure 75. IDAC Block Diagram

The internal reference must be enabled for IDAC operation. As a current source, the IDAC requires voltage headroom to the positive supply to operate. This voltage headroom is the compliance voltage. When driving resistive sensors and biasing resistors, take care not to exceed the compliance voltage of the IDACs, otherwise the specified accuracy of the IDAC current may not be met. For IDAC compliance voltage specifications, see the [Electrical Characteristics](#) table.

9.3.8 Bias Voltage Generation

The ADS124S0x provides an internal bias voltage generator, VBIAS, that can be set to two different levels, $(AVDD + AVSS) / 2$ and $(AVDD + AVSS) / 12$ by using the VB_LEVEL bit in the sensor biasing register (08h). The bias voltage is internally buffered and can be established on the analog inputs AIN0 to AIN5 and AINCOM using the VB_AINx bits in the sensor biasing register (08h). A typical use case for VBIAS is biasing unbiased thermocouples to within the common-mode voltage range of the PGA. A block diagram of the VBIAS voltage generator and connection diagram is shown in Figure 76.

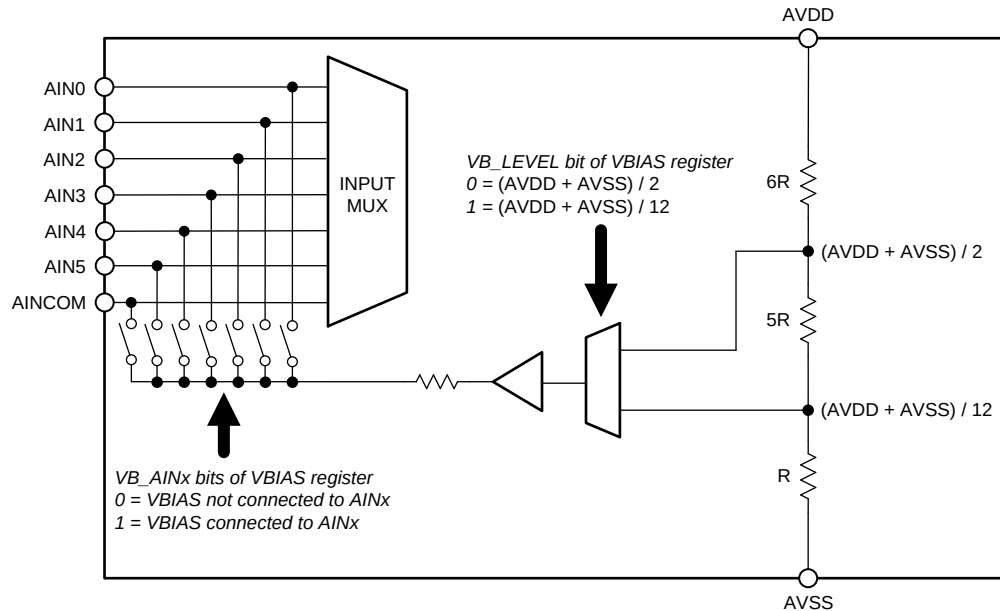


Figure 76. VBIAS Block Diagram

The start-up time of the VBIAS voltage depends on the pin load capacitance. The total capacitance includes any capacitance connected from VBIAS to AVDD, AVSS, and ground. Table 20 lists the VBIAS voltage settling times for various external load capacitances. Ensure the VBIAS voltage is fully settled before starting a conversion.

Table 20. VBIAS Settling Time

LOAD CAPACITANCE	SETTLING TIME
0.1 μ F	280 μ s
1 μ F	2.8 ms
10 μ F	28 ms

9.3.9 System Monitor

The ADS124S0x provides a set of system monitor functions. These functions measure the device temperature, analog power supply, digital power supply, or use current sources to detect sensor malfunction. System monitor functions are enabled through the SYS_MON[2:0] bits of the system control register (09h).

9.3.9.1 Internal Temperature Sensor

On-chip diodes provide temperature-sensing capability. Enable the internal temperature sensor by setting SYS_MON[2:0] = 010 in the system control register (09h). The temperature sensor outputs a voltage proportional to the device temperature as specified in the [Electrical Characteristics](#) table.

When measuring the internal temperature sensor, the analog inputs are disconnected from the ADC and the output voltage of the temperature sensor is routed to the ADC for measurement using the selected PGA gain, data rate, and voltage reference. If enabled, PGA gain must be limited to 4 for the temperature sensor measurement to remain within the allowed absolute input voltage range of the PGA. As a result of the low device junction-to-PCB thermal resistance ($R_{\theta JB}$), the internal device temperature closely tracks the printed circuit board (PCB) temperature.

9.3.9.2 Power Supply Monitors

The ADS124S0x provides a means for monitoring both the analog and digital power supply (AVDD and DVDD). The power-supply voltages are divided by a resistor network to reduce the voltages to within the ADC input range. The reduced power-supply voltage is routed to the ADC input multiplexer. The analog (V_{ANLMON}) and digital (V_{DIGMON}) power-supply readings are scaled by [Equation 9](#) and [Equation 10](#), respectively:

$$V_{ANLMON} = (AVDD - AVSS) / 4 \quad (9)$$

$$V_{DIGMON} = (DVDD - DGND) / 4 \quad (10)$$

Enable the supply voltage monitors using the SYS_MON[2:0] bits in the system control register (09h). Setting SYS_MON[2:0] to 011 measures V_{ANLMON} , and setting SYS_MON[2:0] to 100 measures V_{DIGMON} .

When the supply voltage monitor is enabled, the analog inputs are disconnected from the ADC and the PGA gain is set to 1, regardless of the GAIN[2:0] bit values in the gain setting register (03h). Supply voltage monitor measurements can be done with either the PGA enabled or PGA disabled via the PGA_EN[1:0] register. To obtain valid power supply monitor readings, the reference voltage must be larger than the power-supply measurements shown in [Equation 9](#) and [Equation 10](#).

9.3.9.3 Burn-Out Current Sources

To help detect a possible sensor malfunction, the ADS124S0x provides selectable current sources to function as burn-out current sources (BOCS) using the SYS_MON[2:0] bits in the system control register (09h). Current sources are set to values of 0.2 μA , 1 μA , and 10 μA with SYS_MON[2:0] settings of 101, 110, and 111, respectively.

When enabled, one BOCS sources current to the selected positive analog input (AIN_P) and the other BOCS sinks current from the selected negative analog input (AIN_N). With an open-circuit in a burned out sensor, these BOCSs pull the positive input towards AVDD and the negative input towards AVSS, resulting in a full-scale reading. A full-scale reading can also indicate that the sensor is overloaded or that the reference voltage is absent. A near-zero reading can indicate a shorted sensor. Distinguishing a shorted sensor condition from a normal reading can be difficult, especially if an RC filter is used at the inputs. The voltage drop across the external filter resistance and the residual resistance of the multiplexer can cause the output to read a value higher than zero.

The ADC readings of a functional sensor can be corrupted when the burn-out current sources are enabled. The burn-out current sources are recommended to be disabled when performing the precision measurement, and only enabling them to test for sensor fault conditions. If the global chop mode is enabled, disable this mode before making a measurement with the burn-out current sources.

9.3.10 Status Register

The ADS124S0x has a one-byte status register that contains flags to indicate if a fault condition has occurred. This byte can be read out from the status register (01h), or can be prepended to each data read as the first byte when reading data from the ADC. To prepend the STATUS byte to each conversion result, set the SENDSTAT bit to 1 in the system control register (09h).

The STATUS byte data field and field description are found in [Figure 94](#) and [Table 27](#). The following sections describe various flagged fault conditions that are indicated in the STATUS byte.

Flags for the PGA output voltage rail monitors and reference monitor are set after each conversion. Reading the STATUS byte reads the flags latched during the last conversion cycle.

9.3.10.1 POR Flag

After the power supplies are turned on, the ADC remains in reset until DVDD, IOVDD, and the analog power supply (AVDD – AVSS) voltage exceed the respective power-on reset (POR) voltage thresholds. If a POR event has occurred, the FL_POR flag (bit 7 of the STATUS byte) is set. This flag indicates that a POR event has occurred and has not been cleared. This flag is cleared with a user register write to set the bit to 0. The power-on reset is described further in the [Power-On Reset](#) section.

9.3.10.2 $\overline{\text{RDY}}$ Flag

The $\overline{\text{RDY}}$ flag indicates that the device has started up and is ready to receive a configuration change. During a reset or POR event, the device is resetting the register map and may not be available. The $\overline{\text{RDY}}$ flag is shown with bit 6 of the STATUS byte.

9.3.10.3 PGA Output Voltage Rail Monitors

The PGA contains an integrated output-voltage monitor. If the level of the PGA output voltage exceeds $\text{AVDD} - 0.15 \text{ V}$ or drops below $\text{AVSS} + 0.15 \text{ V}$, a flag is set to indicate that the output has gone beyond the output range of the PGA. Each PGA output V_{OUTN} and V_{OUTP} can trigger an overvoltage or undervoltage flag, giving a total of four flags. The PGA output voltage rail monitors are enabled with the FL_REF_EN bit of excitation current register 1. The PGA output voltage rail monitor block diagram is shown in [Figure 77](#). If the PGA is bypassed, then the rail monitor is still operational and is sensing the connection at the input of the ADC.

The PGA output voltage rail monitors are:

- FL_P_RAILP (bit 5 of the STATUS byte): V_{OUTP} has exceeded $\text{AVDD} - 0.15 \text{ V}$
- FL_P_RAILN (bit 4 of the STATUS byte): V_{OUTP} dropped below $\text{AVSS} + 0.15 \text{ V}$
- FL_N_RAILP (bit 3 of the STATUS byte): V_{OUTN} has exceeded $\text{AVDD} - 0.15 \text{ V}$
- FL_N_RAILN (bit 2 of the STATUS byte): V_{OUTN} dropped below $\text{AVSS} + 0.15 \text{ V}$

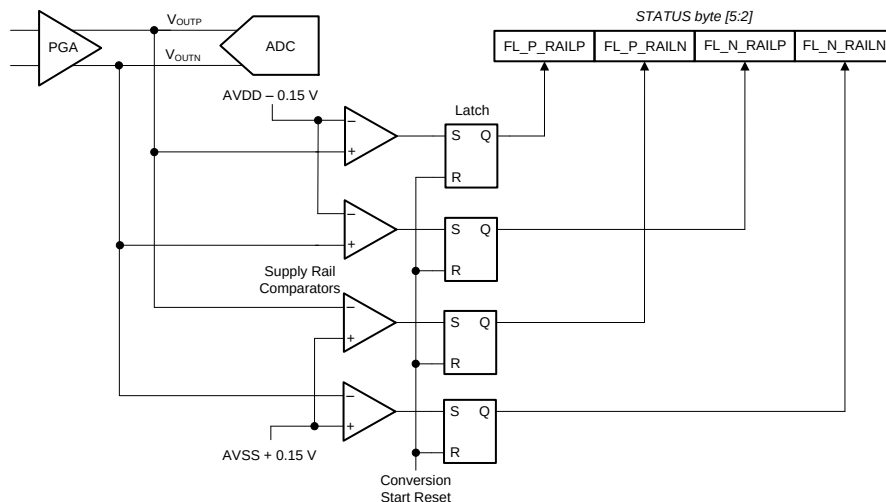


Figure 77. PGA Output Voltage Rail Monitors

Figure 78 shows an example of a PGA output voltage rail monitor overrange event and the respective behavior of the flags. A fault is latched during a conversion cycle. The flags are updated (set or cleared) only at the end of a conversion cycle.

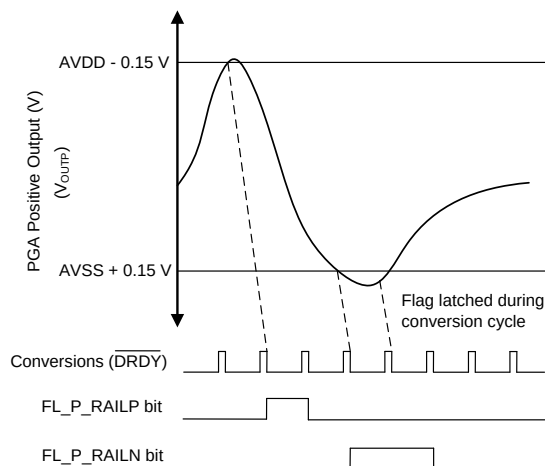


Figure 78. PGA Output Voltage Rail Monitor Timing

9.3.10.4 Reference Monitor

The user can select to continuously monitor the ADC reference inputs for a shorted or missing reference voltage. The reference detection circuit offers two thresholds, the first threshold is 300 mV and the second threshold is $\frac{1}{3} \cdot (AVDD - AVSS)$. The reference detection circuit measures the differential reference voltage and sets a flag latched after each conversion in the STATUS byte if the voltage is below the threshold. A reference voltage less than 300 mV can indicate a potential short on the reference inputs or, in case of a ratiometric RTD measurement, a broken wire between the RTD and the reference resistor. A reference voltage between 300 mV and $\frac{1}{3} \cdot (AVDD - AVSS)$ can indicate a broken sensor excitation wire in a 3-wire RTD setup.

Additionally, a resistor of 10 M Ω can be connected between the selected REFPx and REFNx inputs. The resistor can be used to detect a floating reference input. With a floating input, the resistor pulls both reference inputs to the same potential so that the reference detection circuit can detect this condition. The pull-together reference resistor is not recommended to be continuously connected to active reference inputs. This resistor lowers the input impedance of the reference inputs and can contribute gain error to the measurement.

The reference detection circuits must be enabled with the FL_REF_EN[1:0] bits of the reference control register (05h). The FL_REF_L0 flag (bit 0 of the STATUS byte) indicates if the reference voltage is lower than 0.3 V. The FL_REF_L1 flag (bit 1 of the STATUS byte) indicates if the reference voltage is lower than $\frac{1}{3} \cdot (AVDD - AVSS)$. A diagram of the reference detection circuit is shown in Figure 79. A reference monitor fault is latched at each conversion cycle and the flags in the status register are updated at the falling edge of DRDY.

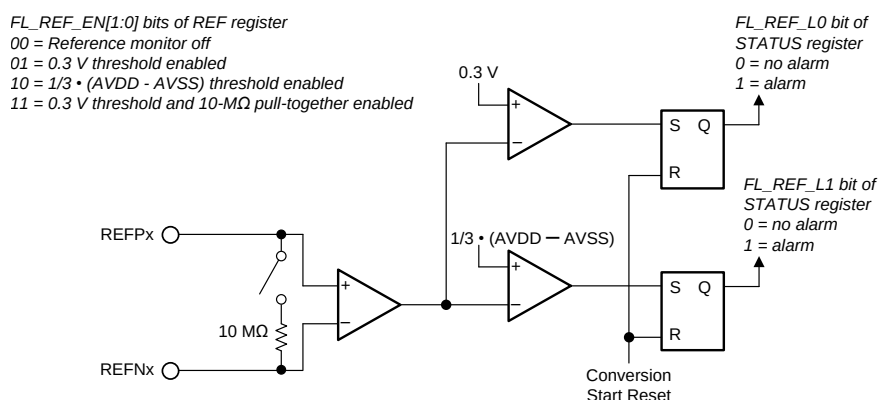


Figure 79. Reference Monitor Block Diagram

9.3.11 General-Purpose Inputs and Outputs (GPIOs)

The ADS124S06 offers four dedicated general-purpose input and output (GPIO) pins, and the ADS124S08 offers four pins (AIN8 to AIN11) that serve a dual purpose as either analog inputs or GPIOs.

Two registers control the function of the GPIO pins. Use the CON[3:0] bits of the GPIO configuration register (11h) to configure a pin as a GPIO pin. The upper four bits (DIR[3:0]) of the GPIO data register (10h) configure the GPIO pin as either an input or an output. The lower four bits (DAT[3:0]) of the GPIO data register contain the input or output GPIO data. If a GPIO pin is configured as an input, the respective DAT[x] bit reads the status of the pin; if a GPIO pin is configured as an output, write the output status to the respective DAT[x] bit. For more information about the use of GPIO pins, see the [Configuration Registers](#) section.

Figure 80 shows a diagram of how these functions are combined onto a single pin. Note that when the pin is configured as a GPIO, the corresponding logic is powered from AVDD and AVSS. When the devices are operated with bipolar analog supplies, the GPIO outputs bipolar voltages. Care must be taken to not load the GPIO pins when used as outputs because large currents can cause droop or noise on the analog supplies. GPIO pins use Schmitt triggered inputs, with hysteresis to make the input more resistance to noise; see the [Electrical Characteristics](#) table for GPIO thresholds.

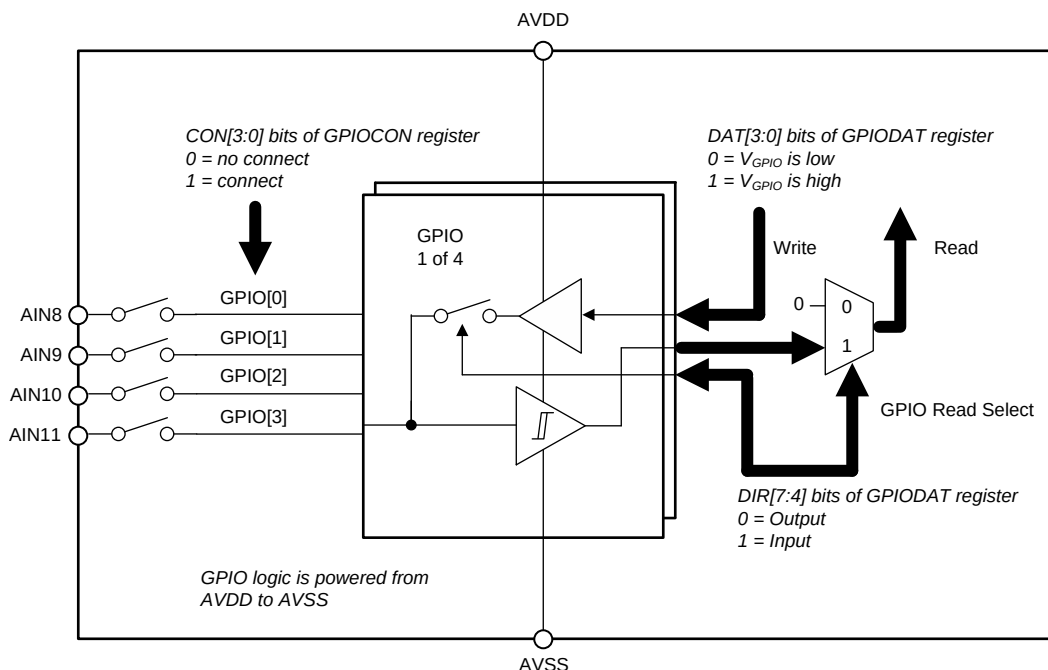


Figure 80. GPIO Block Diagram

For connections of unused GPIO pins, see the [Unused Inputs and Outputs](#) section.

9.3.12 Low-Side Power Switch

A low-side power switch with low on-resistance connected between REFNO and AVSS-SW is integrated in the devices. This power switch can be used to reduce system power consumption in resistive bridge sensor applications by powering down the bridge circuit between conversions. When the PSW bit in the excitation current register 1 (06h) is set to 1, the switch closes. The switch automatically opens when the POWERDOWN command is issued. The switch is opened by setting the PSW bit to 0. By default, the switch is open. Connect AVSS-SW to AVSS.

9.3.13 Cyclic Redundancy Check (CRC)

A cyclic redundancy check (CRC) is enabled by setting the CRC bit to 1 in the system control register (10h). When CRC mode is enabled, the 8-bit CRC is appended to the conversion result. The CRC is calculated for the 24-bit conversion result and the STATUS byte when enabled.

In CRC mode, the checksum byte is the 8-bit remainder of the bitwise exclusive-OR (XOR) of the data bytes by a CRC polynomial. For conversion data, use three data bytes. The CRC is based on the CRC-8-ATM (HEC) polynomial: $X^8 + X^2 + X + 1$.

The nine binary coefficients of the polynomial are: 100000111. To calculate the CRC, divide (XOR operation) the data bytes (excluding the CRC) with the polynomial and compare the calculated CRC values to the ADC CRC value. If the values do not match, a data transmission error has occurred. In the event of a data transmission error, read the data again.

The following list shows a general procedure to compute the CRC value:

1. Left-shift the initial 24-bit data value (32-bit data when the STATUS byte is enabled) by 8 bits, with zeros padded to the right, creating a new 32-bit data value (the starting data value).
2. Align the MSB of the CRC polynomial (100000111) to the left-most, logic-one value of the data.
3. Perform an XOR operation on the data value with the aligned CRC polynomial. The XOR operation creates a new, shorter-length value. The bits of the data values that are not in alignment with the CRC polynomial drop down and append to the right of the new XOR result.
4. When the XOR result is less than 100000000, the procedure ends, yielding the 8-bit CRC value. Otherwise, continue with the XOR operation shown in step 2, using the current data value. The number of loop iterations depends on the value of the initial data.

9.3.14 Calibration

The ADC incorporates offset and gain calibration commands, as well as user-offset and full-scale (gain) calibration registers to calibrate the ADC. The ADC calibration registers are 24 bits wide. Use calibration to correct internal ADC errors or overall system errors. Calibrate by sending calibration commands to the ADC, or by direct user calibration. In user calibration, the user calculates and writes the correction values to the calibration registers. The ADC performs self or system-offset calibration, or a system gain calibration. Perform offset calibration before system gain calibration. After power-on, wait for the power supplies and reference voltage to fully settle before calibrating.

As shown in Figure 81, the value of the offset calibration register is subtracted from the filter output and then multiplied by the full-scale register value divided by 400000h. The data are then clipped to a 24-bit value to provide the final output.

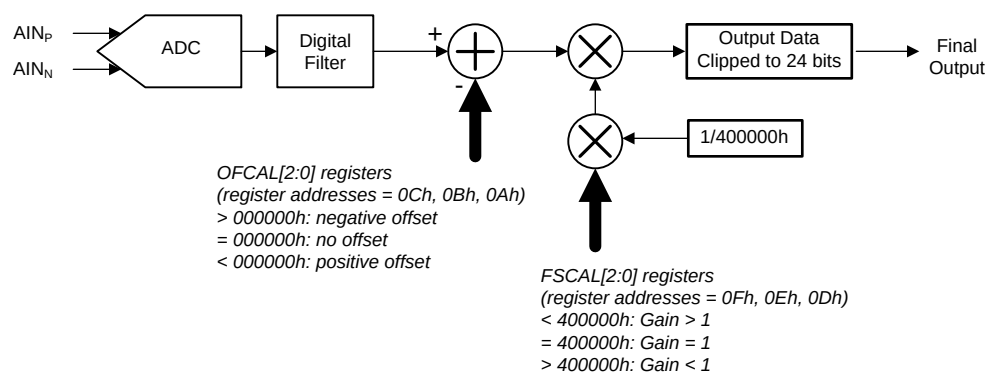


Figure 81. ADC Calibration Block Diagram

Calibration commands cannot be used when the device is in standby mode (when the START/SYNC pin is low, or when the STOP command is issued).

9.3.14.1 Offset Calibration

The offset calibration word is 24 bits, consisting of three 8-bit registers, as shown in the three registers starting with [offset calibration register 1](#). The offset value is twos complement format with a maximum positive value equal to 7FFFFFFh, and a maximum negative value equal to 800000h. This value is subtracted from each output reading as an offset correction. A register value equal to 000000h has no offset correction. If global chop mode is enabled, the offset calibration register is disabled. [Table 21](#) shows example settings of the offset register.

Table 21. Offset Calibration Register Values

OFC REGISTER VALUE	OFFSET CALIBRATED OUTPUT CODE ⁽¹⁾
000001h	FFFFFFh
000000h	000000h
FFFFFFh	000001h

(1) Ideal output code with shorted input, excluding ADC noise and offset voltage error.

The user can select how many samples (1, 4, 8, or 16) to average for self or system offset calibration using the CAL_SAMP[1:0] bits in the system control register (09h). Fewer readings shorten the calibration time but also provide less accuracy. Averaging more readings takes longer but yields a more accurate calibration result by reducing the noise level.

Two commands can be used to perform offset calibration. SFOCAL is a self offset calibration that internally sets the input to mid-scale using the SYS_MON[2:0] = 001 setting and takes a measurement of the offset. SYOCAL is a system offset calibration where the user must input a null voltage to calibrate the system offset. After either command is issued, the OFC register is updated.

After an offset calibration is performed, the device starts a new conversion and $\overline{\text{DRDY}}$ falls to indicate a new conversion has completed.

9.3.14.2 Gain Calibration

The full-scale (gain) calibration word is 24 bits consisting of three 8-bit registers, as shown in the three registers starting with [gain calibration register 1](#). The gain calibration value is straight binary, normalized to a unity-gain correction factor at a register value equal to 400000h. [Table 22](#) shows register values for selected gain factors. Do not exceed the PGA input range limits during gain calibration.

Table 22. Gain Calibration Register Values

FSC REGISTER VALUE	GAIN FACTOR
433333h	1.05
400000h	1.00
3CCCCCh	0.95

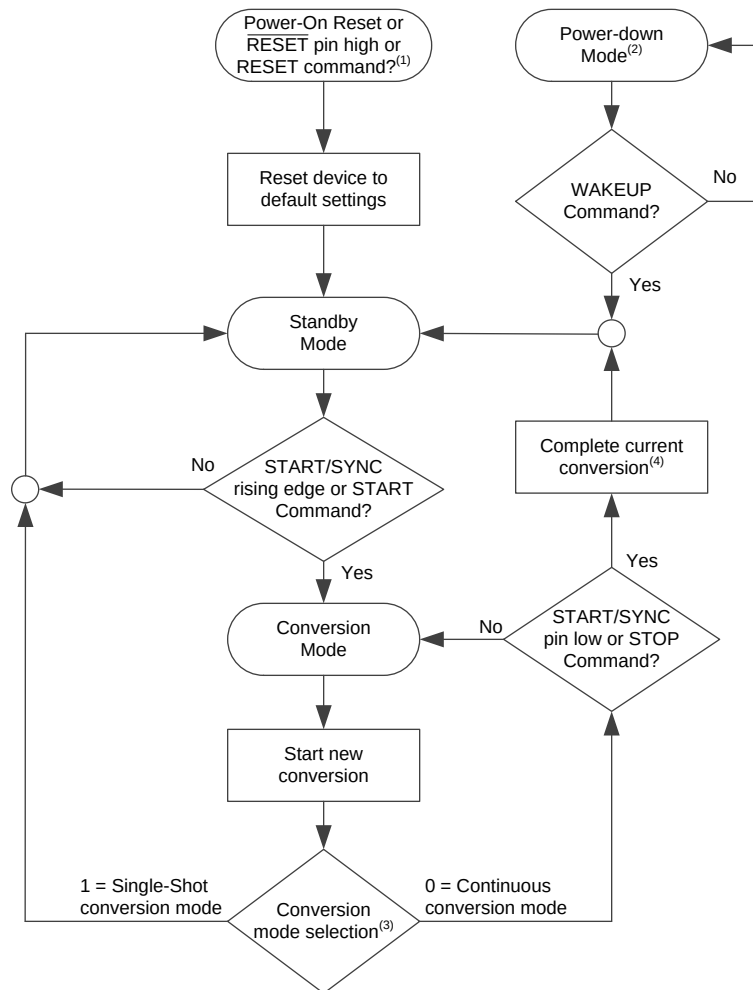
All gains of the ADS124S0x are factory trimmed to meet the gain error specified in the [Electrical Characteristics](#) table at $T_A = 25^\circ\text{C}$. When the gain drift of the devices over temperature is very low, there is typically no need for self gain calibration.

The SYGCAL command initiates a system gain calibration, where the user sets the input to full-scale to remove gain error. After the SYGCAL is issued, the FSC register is updated. As with the offset calibration, the CAL_SAMP[1:0] bits determine the number of samples used for a gain calibration.

As with an offset calibration, the device starts a new conversion after a gain calibration and $\overline{\text{DRDY}}$ falls to indicate a new conversion has completed.

9.4 Device Functional Modes

The device operates in three different modes: power-down mode, standby mode, and conversion mode. [Figure 82](#) shows a flow chart of the different operating modes and how the device transitions from one mode to another.



- (1) Any reset (power-on, command, or pin), immediately resets the device.
- (2) A POWERDOWN command aborts an ongoing conversion and immediately puts the device into power-down mode.
- (3) The conversion mode is selected with the MODE bit in the data rate register.
- (4) The rising edge of the START/SYNC pin or the START command starts a new conversion without completing the current conversion.

Figure 82. Operating Flow Chart

9.4.1 Reset

The ADS124S0x is reset in one of three ways:

- Power-on reset
- $\overline{\text{RESET}}$ pin
- RESET command

When a reset occurs, the configuration registers reset to default values and the device enters standby mode. The device then waits for the rising edge of the START/SYNC pin or a START command to enter conversion mode. Note that if the device had been using an external clock, the reset sets the device to use the internal oscillator as a default configuration. See the [Timing Characteristics](#) section for reset timing information.

Device Functional Modes (continued)

9.4.1.1 Power-On Reset

The ADS124S0x incorporates a power-on reset circuit that holds the device in reset until all supplies reach approximately 1.65 V. The power-on reset also ensures that the device starts operating in a known state in case a brown-out event occurs, when the supplies have dipped below the minimum operating voltages. When the device completes a POR sequence, the FL_POR flag in the status register is set high to indicate that a POR has occurred.

Begin communications with the device 2.2 ms after the power supplies reach minimum operating voltages. The only exception is polling the status register for the RDY bit. If the user polls the RDY bit, then use an SCLK rate of half the maximum-specified SCLK rate to get a proper reading when the device is making internal configurations. This 2.2-ms POR time is required for the internal oscillator to start up and the device to properly set internal configurations. After the internal configurations are set, the device sets the RDY bit in the device status register (01h). When this bit is set to 0, user configurations can be programmed into the device. Figure 83 shows the power-on reset timing sequence for the device.

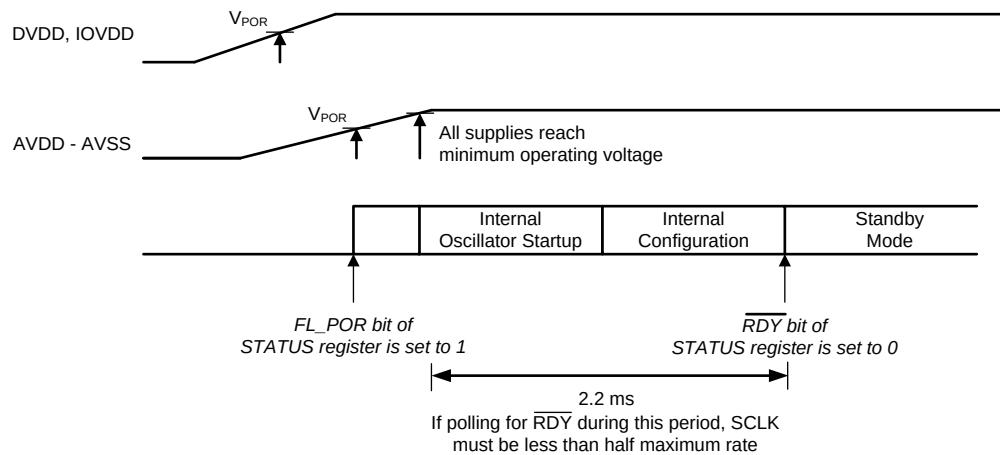


Figure 83. Power-On Reset Timing Sequence

9.4.1.2 RESET Pin

Reset the ADC by taking the RESET pin low for a minimum of $4 \cdot t_{CLK}$ cycles, and then returning the pin high. After the rising edge of the RESET pin, a delay time of $t_{d(RSSC)}$ is required before sending the first serial interface command or starting a conversion. See the [Timing Characteristics](#) section for reset timing information.

9.4.1.3 Reset by Command

Reset the ADC by using the RESET command (06h or 07h). The command is decoded on the seventh SCLK falling edge. After sending the RESET command, a delay time of $t_{d(RSSC)}$ is required before sending the first serial interface command or starting a conversion. See the [Timing Characteristics](#) section for reset timing information.

9.4.2 Power-Down Mode

Power-down mode is entered by sending the POWERDOWN command. In this mode, all analog and digital circuitry is powered down for lowest power consumption regardless of the register settings. Only the internal voltage reference can be configured to stay on during power-down mode in case a faster start-up time is required. All register values retain the current settings during power-down mode. The configuration registers can be read and written in power-down mode. A WAKEUP command must be issued in order to exit power-down mode and to enter standby mode.

When the POWERDOWN command is issued, the device enters power-down mode $2 \cdot t_{CLK}$ after the seventh SCLK falling edge of the command. For lowest power consumption (on DVDD and IOVDD), stop the external clock when in power-down mode. The device does not gate the external clock when in power-down mode. Selecting the internal oscillator before sending the POWERDOWN command is recommended.

Device Functional Modes (continued)

To release the device from POWERDOWN, issue the WAKEUP command to enter standby mode. The device then waits for the rising edge of the START/SYNC pin or a START command to go into conversion mode.

When in power-down mode, the device responds to the RREG, RDATA, and WAKEUP commands. The WREG and RESET commands can also be sent, but are ignored until a WAKEUP command is sent and the internal oscillator resumes operation.

9.4.3 Standby Mode

The device powers up in standby mode and automatically enters this mode whenever there is no ongoing conversion. When the STOP command is sent (or the START/SYNC pin is taken low) in continuous conversion mode, or when a conversion completes in single-shot conversion mode, the device enters standby mode.

Standby mode offers several different options and features to lower the power consumption:

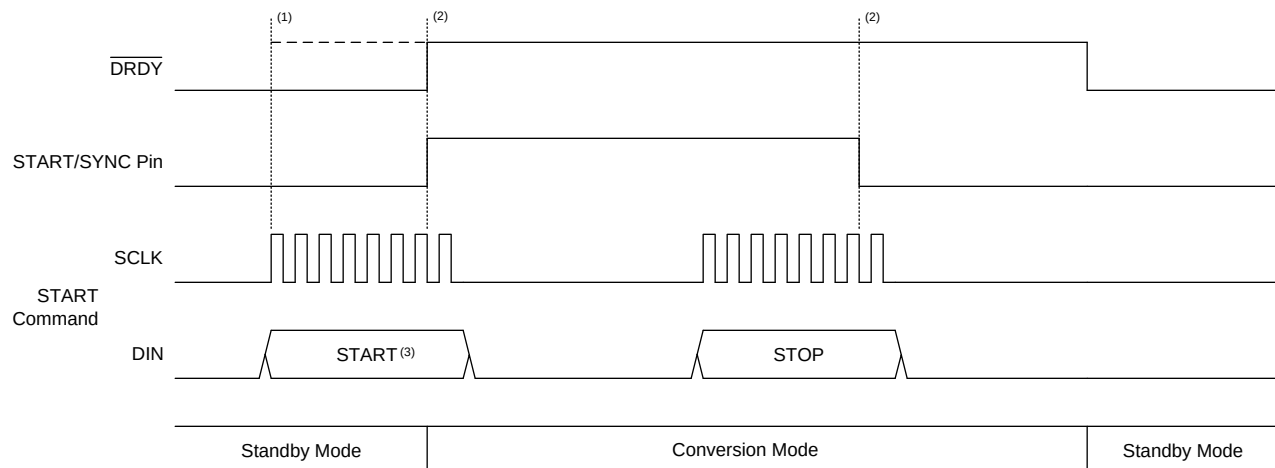
- The PGA can be powered down by setting PGA_EN[1:0] to 00 in the gain setting register (03h).
- The internal voltage reference can be powered down by setting REFCON[1:0] to 00 in the reference control register (05h). This setting also turns off the IDACs.
- The digital filter is held in reset state.
- The clock to the modulator and digital core is gated to decrease dynamic switching losses.

If powered down in standby mode, the PGA and internal reference can require extra time to power up. Extra delay may be required between power up of the PGA or the internal reference, and the start of conversions. In particular, the reference power up time is dependent on the capacitance between REFOUT and REFCOM.

Calibration commands are not decoded when the device is in standby mode.

9.4.4 Conversion Modes

The ADS124S0x offers two conversion modes: continuous conversion and single-shot conversion mode. Continuous-conversion mode converts indefinitely until stopped by the user. Single-shot conversion mode performs one conversion after the START/SYNC pin is taken high or after the START command is sent. Use the MODE bit in the data rate register (04h) to program the conversion mode. [Figure 84](#) shows how the START/SYNC pin and the START command are used to control ADC conversions.



- (1) $\overline{\text{DRDY}}$ rises at the first SCLK rising edge or the rising edge of the START/SYNC pin.
- (2) START and STOP commands take effect $2 \cdot t_{\text{CLK}}$ after the seventh SCLK falling edge. The conversion starts $2 \cdot t_{\text{CLK}}$ after the START/SYNC rising edge.
- (3) To synchronize a conversion, the STOP command must be issued prior to the START command. STOP and START commands can be issued without a delay between the commands.

Figure 84. Conversion Start and Stop Timing

Device Functional Modes (continued)

ADC conversions are controlled by the START/SYNC pin or by serial commands. For the device to start converting in continuous conversion or single-shot conversion mode, a START command must be sent or the START/SYNC pin must be taken high. If using commands to control conversions, keep the START/SYNC pin low to avoid possible contentions between the START/SYNC pin and commands.

Conversions can be synchronized to perform a conversion at a particular time. To synchronize the conversion with the START/SYNC pin, take the pin low. The rising edge of the START/SYNC pin starts a new conversion. Similarly, a conversion can be synchronized using the START command. If the device is in standby mode, issue a START command. If the device is in conversion mode, issue a STOP command followed by a START command. The STOP and START commands can be consecutive. A new conversion starts on the seventh SCLK falling edge of the START command.

9.4.4.1 Continuous Conversion Mode

The device is configured for continuous conversion mode by setting the MODE bit to 0 in the data rate register (04h). A START command must be sent or the START/SYNC pin must be taken high for the device to start converting continuously. When controlling the device with commands, hold the START/SYNC pin low. Taking the START/SYNC pin low or sending the STOP command stops the device from converting after the currently ongoing conversion completes, indicated by the falling edge of DRDY. The device enters standby mode thereafter.

For information on the exact timing of single-shot conversion mode data, see [Table 13](#) and [Table 15](#).

9.4.4.2 Single-Shot Conversion Mode

The device is configured for single-shot conversion mode by setting the MODE bit to 1 in the data rate register (04h). A START command must be sent or the START/SYNC pin must be taken high for the device to start a single conversion. After the conversion completes, the device enters standby mode again. To start a new conversion, the START command must be sent again or the START/SYNC pin must be taken low and then high again.

When the device uses the sinc³ filter, ADC data requires three conversion cycles to settle. When the sinc³ filter is enabled, a single-shot conversion suppresses the first two ADC conversions and provides the third conversion as the output data so that the user receives settled data. Because three conversions are required for settled data, the conversion time in single-shot conversion mode is approximately three times the normal data period. When the device uses the low-latency filter, the ADC data settles in a single conversion. In single-shot conversion mode with the low-latency filter, the data period is closer to the normal data period.

For information on the exact timing of single-shot conversion mode data, see [Table 13](#) and [Table 15](#).

9.4.4.3 Programmable Conversion Delay

When a new conversion is started, the ADC provides a delay before the actual start of the conversion. This timed delay is provided to allow for the integrated analog anti-alias filter to settle. In some cases more delay is required to allow for external settling effects. The delay time can be configured to automatically delay the start of a conversion after a START command is sent, the START/SYNC pin is taken high, or a WREG command is sent to change any configuration register from address 03h to 07h is issued (as described in the [WREG](#) section). The programmable conversion delay is intended to accommodate the analog settling time on the inputs (for example, when changing a multiplexer channel). Use the DELAY[2:0] bits in the gain setting register (03h) to program a delay time ranging from $1 \cdot t_{MOD}$ to $4096 \cdot t_{MOD}$ (where $t_{MOD} = 16 \cdot t_{CLK}$). The default programmable conversion delay setting is $14 \cdot t_{MOD}$.

9.5 Programming

9.5.1 Serial Interface

The ADC has an SPI-compatible, bidirectional serial interface that is used to read the conversion data as well as to configure and control the ADC. Only SPI mode 1 ($\overline{\text{CPOL}} = 0$, $\text{CPHA} = 1$) is supported. The serial interface consists of five control lines: $\overline{\text{CS}}$, SCLK, DIN, DOUT/ $\overline{\text{DRDY}}$, and $\overline{\text{DRDY}}$ but can be used with only four or even three control signals. If the ADS124S08 or ADS124S06 is the only device connected to the SPI bus, then the $\overline{\text{CS}}$ input can be tied low so that only SCLK, DIN, and DOUT/ $\overline{\text{DRDY}}$ are required to communicate with the device.

9.5.1.1 Chip Select ($\overline{\text{CS}}$)

The $\overline{\text{CS}}$ pin is an active low input that enables the ADC serial interface for communication and is useful when multiple devices share the same serial bus. $\overline{\text{CS}}$ must be low during the entire data transaction. When $\overline{\text{CS}}$ is high, the serial interface is reset, SCLK input activity is ignored (blocking input commands), and the DOUT/ $\overline{\text{DRDY}}$ output enters a high-impedance state. ADC conversions are not affected by the state of $\overline{\text{CS}}$. In situations where multiple devices are present on the bus, the dedicated $\overline{\text{DRDY}}$ pin can provide an uninterrupted monitor of the conversion status and is not affected by $\overline{\text{CS}}$. If the serial bus is not shared with another peripheral, $\overline{\text{CS}}$ can be tied to DGND to permanently enable the ADC interface and DOUT/ $\overline{\text{DRDY}}$ can be used to indicate conversion status. These changes reduce the serial interface from five I/Os to three I/Os.

9.5.1.2 Serial Clock (SCLK)

The serial interface clock is a noise-filtered, Schmidt-triggered input used to clock data into and out of the ADC. Input data to the ADC are latched on the falling SCLK edge and output data from the ADC are updated on the rising SCLK edge. Return SCLK low after the data sequence is complete. Even though the SCLK input has hysteresis, keep SCLK as clean as possible to prevent unintentional SCLK transitions. Avoid ringing and voltage overshoot on the SCLK input. Place a series termination resistor at the SCLK drive pin to help reduce ringing.

9.5.1.3 Serial Data Input (DIN)

The serial data input pin (DIN) is used with SCLK to send data (commands and register data) to the device. The device latches data on DIN on the SCLK falling edge. The device never drives the DIN pin. During data readback, when no command is intended, keep DIN low.

9.5.1.4 Serial Data Output and Data Ready (DOUT/ $\overline{\text{DRDY}}$)

The DOUT/ $\overline{\text{DRDY}}$ pin is a dual-function output. The pin functions as the digital data output and the ADC data-ready indication.

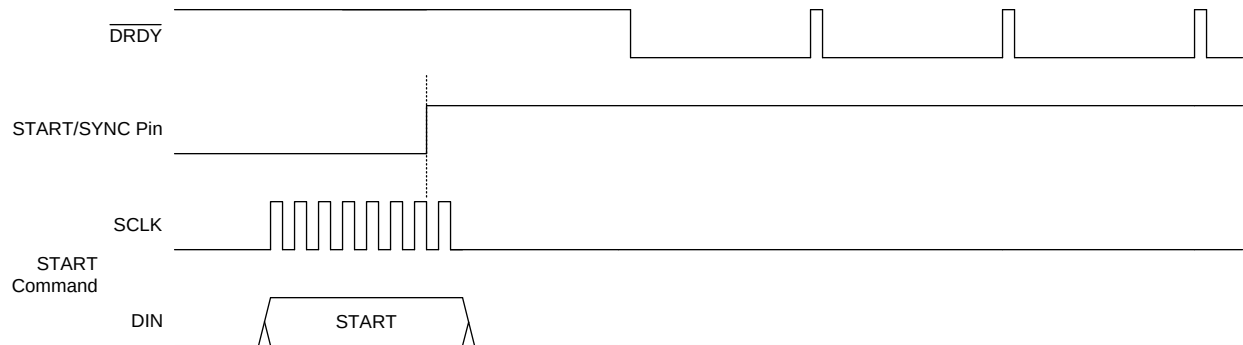
First, this pin is used with SCLK to read conversion and register data from the device. Conversion or register data are shifted out on DOUT/ $\overline{\text{DRDY}}$ on the SCLK rising edge. DOUT/ $\overline{\text{DRDY}}$ goes to a high-impedance state when $\overline{\text{CS}}$ is high.

Second, the DOUT/ $\overline{\text{DRDY}}$ pin indicates availability of new conversion data. DOUT/ $\overline{\text{DRDY}}$ transitions low at the same time that the $\overline{\text{DRDY}}$ pin goes low to indicate new conversion data are available. Both signals can be used to detect if new data are ready. However, because DOUT/ $\overline{\text{DRDY}}$ is disabled when $\overline{\text{CS}}$ is high, use the dedicated $\overline{\text{DRDY}}$ pin when monitoring conversions on multiple devices on the SPI bus.

Programming (continued)

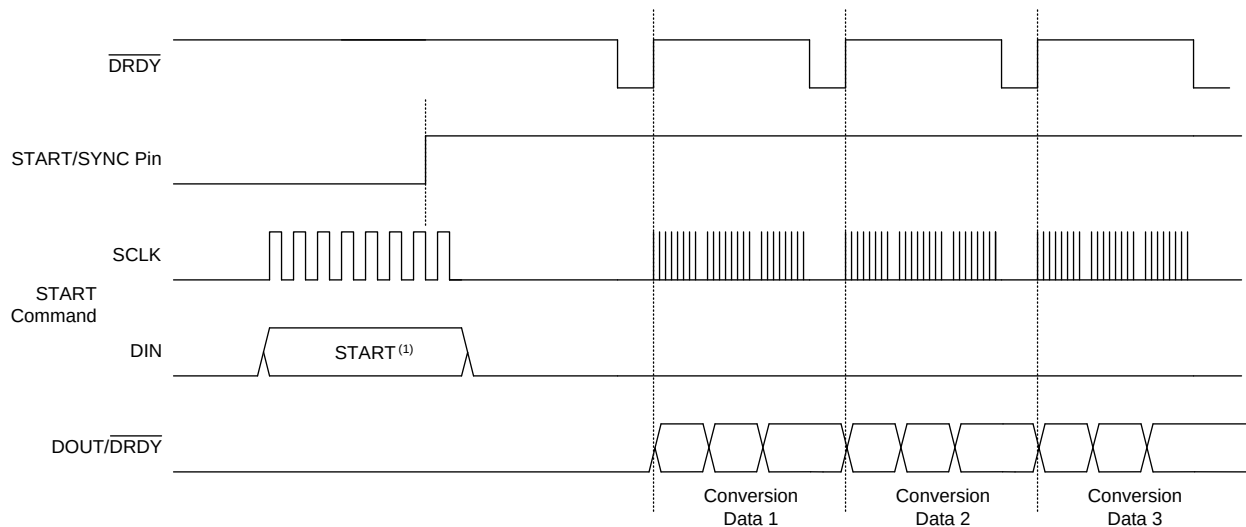
9.5.1.5 Data Ready ($\overline{\text{DRDY}}$)

The $\overline{\text{DRDY}}$ pin is an output that transitions low to indicate when conversion data are ready for retrieval. Initially, $\overline{\text{DRDY}}$ is high at power-on. When converting, the state of $\overline{\text{DRDY}}$ depends on whether the conversion data are retrieved or not. In continuous conversion mode after $\overline{\text{DRDY}}$ goes low, $\overline{\text{DRDY}}$ is driven high on the first SCLK rising edge. If data are not read, $\overline{\text{DRDY}}$ remains low and then pulses high $24 \cdot t_{\text{CLK}}$ before the next $\overline{\text{DRDY}}$ falling edge. The data must be retrieved before the next $\overline{\text{DRDY}}$ update, otherwise the data are overwritten by new data and any previous data are lost. Figure 85 shows the $\overline{\text{DRDY}}$ operation without data retrieval. Figure 86 shows the $\overline{\text{DRDY}}$ operation with data retrieval after each conversion completes.



- (1) $\overline{\text{DRDY}}$ returns high with the rising edge of the first SCLK after a data ready indication.

Figure 85. $\overline{\text{DRDY}}$ Operation Without Data Retrieval



- (1) $\overline{\text{DRDY}}$ returns high with the rising edge of the first SCLK after a data ready indication.

Figure 86. $\overline{\text{DRDY}}$ Operation With Data Retrieval

9.5.1.6 Timeout

The ADS124S0x offers a serial interface timeout feature that is used to recover communication when a serial interface transmission is interrupted. This feature is especially useful in applications where CS is permanently tied low and is not used to frame a communication sequence. The SPI interface resets when no valid 8 bits are received within $2^{15} \cdot t_{\text{CLK}}$. The timeout feature is enabled by setting the TIMEOUT bit to 1 in the system control register (09h).

Programming (continued)

9.5.2 Data Format

The devices provide 24 bits of data in binary twos complement format. The size of one code (LSB) is calculated using [Equation 11](#).

$$1 \text{ LSB} = (2 \cdot V_{\text{REF}} / \text{Gain}) / 2^{24} = +\text{FS} / 2^{23} \quad (11)$$

A positive full-scale input [$V_{\text{IN}} \geq (+\text{FS} - 1 \text{ LSB}) = (V_{\text{REF}} / \text{Gain} - 1 \text{ LSB})$] produces an output code of 7FFFFFFh and a negative full-scale input ($V_{\text{IN}} \leq -\text{FS} = -V_{\text{REF}} / \text{Gain}$) produces an output code of 800000h. The output clips at these codes for signals that exceed full-scale.

[Table 23](#) summarizes the ideal output codes for different input signals.

Table 23. Ideal Output Code vs Input Signal

INPUT SIGNAL, $V_{\text{IN}} = V_{\text{AINP}} - V_{\text{AINN}}$	IDEAL OUTPUT CODE ⁽¹⁾
$\geq \text{FS} (2^{23} - 1) / 2^{23}$	7FFFFFFh
$\text{FS} / 2^{23}$	000001h
0	000000h
$-\text{FS} / 2^{23}$	FFFFFFh
$\leq -\text{FS}$	800000h

(1) Excludes the effects of noise, INL, offset, and gain errors.

Mapping of the analog input signal to the output codes is shown in [Figure 87](#).

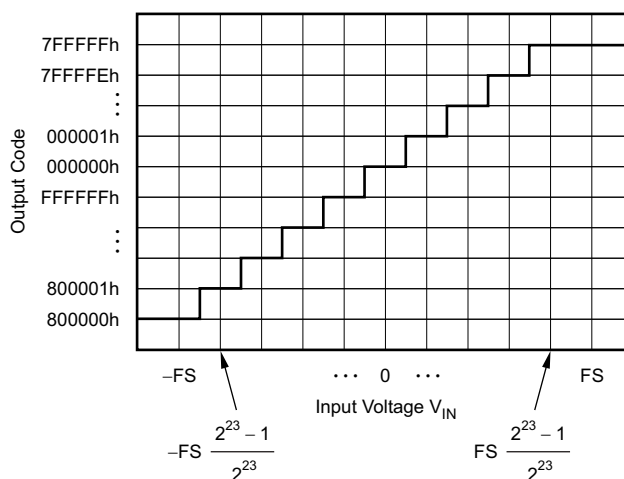


Figure 87. Code Transition Diagram

9.5.3 Commands

Commands are used to control the ADC, access the configuration registers, and retrieve data. Many of the commands are stand-alone (that is, single-byte). The register write and register read commands, however, are multibyte, consisting of two command bytes plus the register data byte or bytes. The commands are listed in [Table 24](#).

Table 24. Command Definitions

COMMAND	DESCRIPTION	FIRST COMMAND BYTE	SECOND COMMAND BYTE
Control Commands			
NOP	No operation	0000 0000 (00h)	—
WAKEUP	Wake-up from power-down mode	0000 001x (02h, 03h) ⁽¹⁾	—
POWERDOWN	Enter power-down mode	0000 010x (04h, 05h) ⁽¹⁾	—
RESET	Reset the device	0000 011x (06h, 07h) ⁽¹⁾	—
START	Start conversions	0000 100x (08h, 09h) ⁽¹⁾	—
STOP	Stop conversions	0000 101x (0Ah, 0Bh) ⁽¹⁾	—
Calibration Commands			
SYOCAL	System offset calibration	0001 0110 (16h)	—
SYGCAL	System gain calibration	0001 0111 (17h)	—
SFOCAL	Self offset calibration	0001 1001 (19h)	—
Data Read Command			
RDATA	Read data by command	0001 001x (12h / 13h) ⁽¹⁾	—
Register Read and Write Commands			
RREG	Read <i>nnnnn</i> registers starting at address <i>rrrr</i>	001r <i>rrrr</i> ⁽²⁾	000n <i>nnnn</i> ⁽³⁾
WREG	Write <i>nnnnn</i> registers starting at address <i>rrrr</i>	010r <i>rrrr</i> ⁽²⁾	000n <i>nnnn</i> ⁽³⁾

(1) x = don't care.

(2) r *rrrr* = starting register address.

(3) n *nnnn* = number of registers to read or write – 1.

Commands can be sent at any time, either during a conversion or when conversions are stopped. However, if register read or write commands are in progress when conversion data are ready, the ADC blocks loading of conversion data to the output shift register. The $\overline{\text{CS}}$ input pin can be taken high between commands; or held low between consecutive commands. $\overline{\text{CS}}$ must stay low for the entire command sequence. Complete the command, or terminate the command before completion by taking $\overline{\text{CS}}$ high. Only send the commands that are listed in [Table 24](#).

9.5.3.1 NOP

NOP is a no-operation command. The NOP command is used to clock out data without clocking in a command.

9.5.3.2 WAKEUP

Issue the WAKEUP command to exit power-down mode and to place the device into standby mode.

When running off the external clock, the external clock must be running before sending the WAKEUP command, otherwise the command is not decoded.

9.5.3.3 POWERDOWN

Sending the POWERDOWN command aborts a currently ongoing conversion and puts the device into power-down mode. The device goes into power-down mode $2 \cdot t_{CLK}$ after the seventh SCLK falling edge of the command.

For lowest power consumption on DVDD and IOVDD, stop the external clock when in power-down mode. The device does not gate the external clock. When running off the external clock, provide at a minimum two additional t_{CLK} s after the POWERDOWN command is issued, otherwise the device does not enter power-down mode. Because an external clock can be gated for lower power consumption, selecting the internal oscillator before sending the POWERDOWN command is recommended.

During power-down mode, the only commands that are available are RREG, RDATA, and WAKEUP.

9.5.3.4 RESET

The RESET command resets the digital filter and sets all configuration register values to default settings. A RESET command also puts the device into standby mode. When in standby mode, the device waits for a rising edge on the START/SYNC pin or a START command to resume conversions. After sending the RESET command, a delay time of $t_{d(RSSC)}$ is required before sending the first serial interface command or starting a conversion. See the [Timing Characteristics](#) section for reset timing information.

Note that if the device had been using an external clock, the reset sets the device to use the internal oscillator as a default configuration.

9.5.3.5 START

When the device is configured for continuous conversion mode, issue the START command for the device to start converting. Every time a conversion completes, the device automatically starts a new conversion until the STOP command is sent.

In single-shot conversion mode, the START command is used to start a single conversion. After the conversion completes, the device enters standby mode.

Tie the START/SYNC pin low when the device is controlled through the START and STOP commands. The START command is not decoded if the START/SYNC pin is high. If the device is already in conversion mode, the command has no effect.

9.5.3.6 STOP

The STOP command is used in continuous conversion mode to stop the device from converting. The current conversion is allowed to complete. After DRDY transitions low, the device enters standby mode. The command has no effect in single-shot conversion mode.

Hold the START/SYNC pin low when the device is controlled through START and STOP commands.

9.5.3.7 SYOCAL

The SYOCAL command initiates a system offset calibration. For a system offset calibration, the inputs must be externally shorted to a voltage within the input range, ideally near the mid-supply voltage of $(AVDD + AVSS) / 2$. The OFC registers are updated when the command completes. Calibration commands must be issued in conversion mode.

9.5.3.8 SYGCAL

The SYGCAL command initiates the system gain calibration. For a system gain calibration, the input must be externally set to full-scale. The FSC registers are updated after this operation. Calibration commands must be issued in conversion mode.

9.5.3.9 SFOCAL

The SFOCAL command initiates a self offset calibration. The device internally shorts the inputs to mid-supply and performs the calibration. The OFC registers are updated after this operation. Calibration commands must be issued in conversion mode.

9.5.3.10 RDATA

The RDATA command is used to read conversion data from the device at any time without concern of data corruption when the DRDY or DOUT/DRDY signal cannot be monitored. The conversion result is read from a buffer so that a new data conversion does not corrupt the conversion read.

9.5.3.11 RREG

Use the RREG command to read the device register data. Read the register data one register at a time, or read a block of register data. The starting register address can be any register in the register map. The RREG command consists of two bytes. The first byte specifies the starting register address: 001r rrrr, where *r rrrr* is the starting register address. The second command byte is the number of registers to read (minus 1): 000n nnnn, where *n nnnn* is the number of registers to read minus 1.

After the read command is sent, the ADC responds with one or more register data bytes, most significant bit first. If the byte count exceeds the last register address, the ADC begins to output zero data. During the register read operation, any conversion data that becomes available is not loaded to the output shift register to avoid data contention. However, the conversion data can be retrieved later by the RDATA command. After the register read command has started, further commands are blocked until one of the following conditions are met:

- The read operation is completed
- The read operation is terminated by taking $\overline{\text{CS}}$ high
- The read operation is terminated by a serial interface timeout
- The ADC is reset by toggling the $\overline{\text{RESET}}$ pin

Figure 88 depicts a two-register read operation example. As shown, the commands required to read data from two registers starting at register REF (address = 05h) are: command byte 1 = 25h and command byte 2 = 01h. Keep DIN low after the two command bytes are sent.

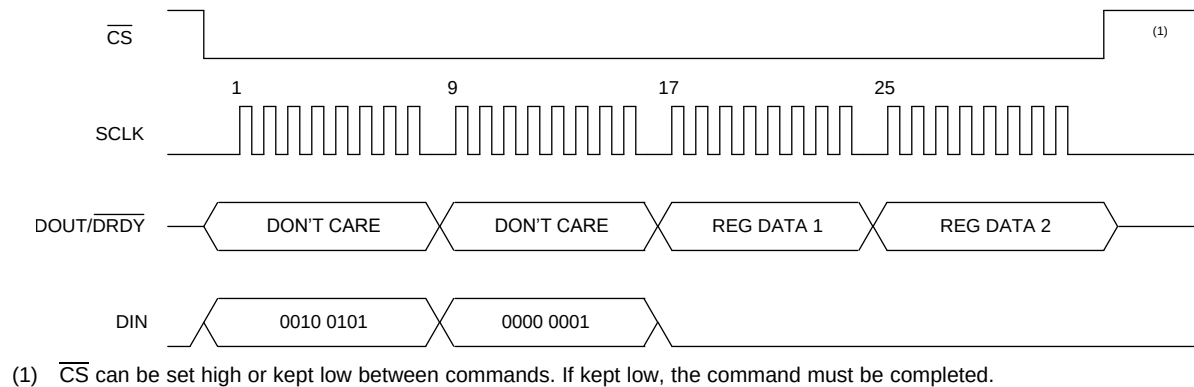


Figure 88. Read Register Sequence

9.5.3.12 WREG

Use the WREG command to write the device register data. The register data are written one register at a time or as a block of register data. The starting register address is any register in the register map.

The WREG command consists of two bytes. The first byte specifies the starting register address: 010r rrrr, where r rrrr is the starting register address. The second command byte is the number of registers to write (minus 1): 000n nnnn, where n nnnn is the number of registers to write minus 1. The following byte (or bytes) is the register data, most significant bit first. If the byte count exceeds the last register address, the ADC ignores the data. After the register write command has started, further commands are blocked until one of the following conditions are met:

- The write operation is completed
- The write operation is terminated by taking $\overline{CS}$ high
- The write operation is terminated by a serial interface timeout
- The ADC is reset by toggling the $\overline{RESET}$ pin

Figure 89 depicts a two-register write operation example. As shown, the required commands to write data to two registers starting at register REF (address = 05h) are: command byte 1 = 45h and command byte 2 = 01h.

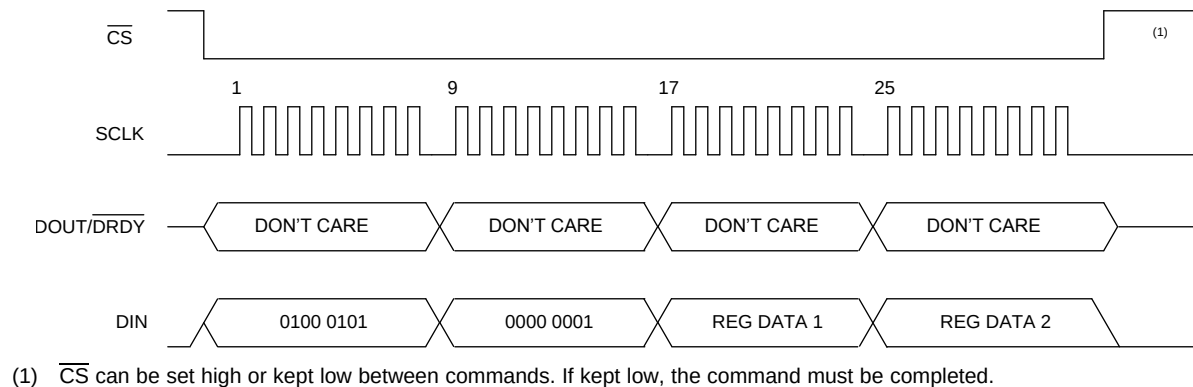


Figure 89. Write Register Sequence

Writing new data to certain configuration registers resets the digital filter and starts a new conversion if a conversion is in progress. Writing to the following registers triggers a new conversion:

- Channel configuration register (02h)
- Gain setting register (03h)
- Data rate register (04h)
- Reference control register (05h), bits [5:0]
- Excitation current register 1 (06h), bits [3:0]
- Excitation current register 2 (07h)
- System control register (09h), bits [7:5]

When the device is configured with WREG, the first data ready indication occurs after the new conversion completes with the new configuration settings. The previous conversion data are cleared at restart; therefore read the previous data before the register write operation. A WREG to the previously mentioned registers only starts a new conversion if the register data are new (differs from the previous register data) and if a conversion is in progress. If the device is in standby mode, the device sets the configuration according to the WREG data, but does not start a conversion until the START/SYNC pin is taken high or a START command is issued.

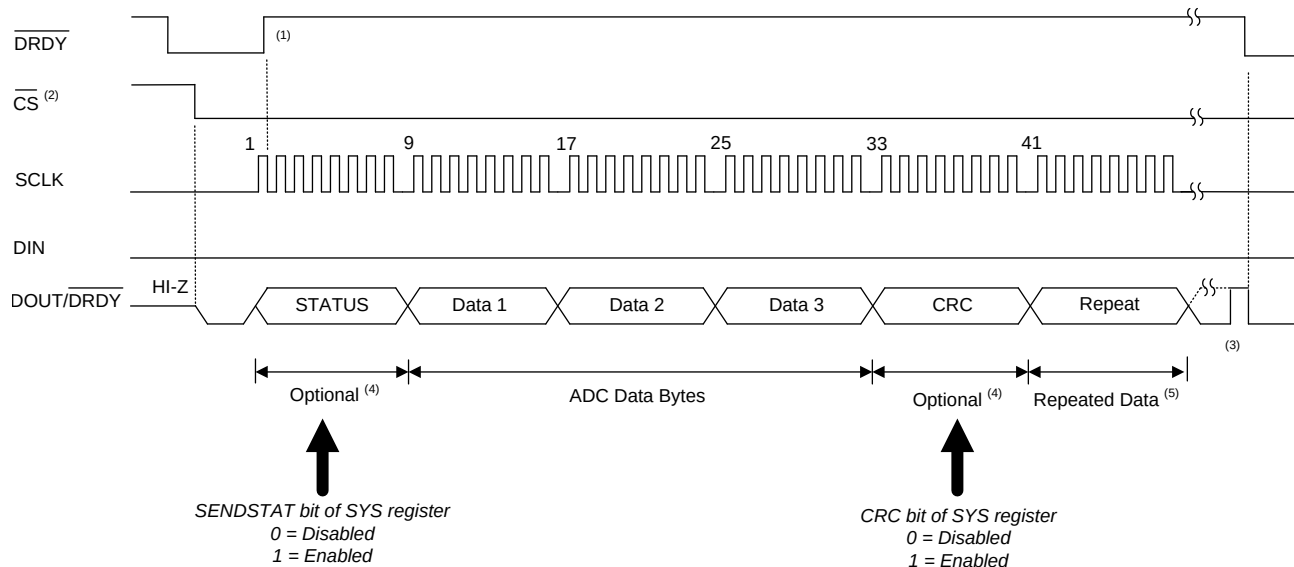
9.5.4 Reading Data

ADC data are read by two methods: read data direct or read data by command. The ADC writes new conversion data to the output shift register and the internal data-holding register. Data are read either from the output shift register (in direct mode) or read from the data-holding register (in command mode). Reading data from the data-holding register (command mode) does not require synchronizing the start of data readback to $\overline{\text{DRDY}}$.

9.5.4.1 Read Data Direct

In this method of data retrieval, ADC conversion data are shifted out directly from the output shift register. No command is necessary. Read data direct requires that no serial activity occur from the falling edge of $\overline{\text{DRDY}}$ to the readback, or the data are invalid. The serial interface is full duplex in the read data direct mode; meaning that commands are decoded during the data readback. If no command is intended, keep DIN low during readback. If an input command is sent during readback, the ADC executes the command, and data corruption can result. Synchronize the data readback to $\overline{\text{DRDY}}$ or to $\text{DOUT}/\overline{\text{DRDY}}$ to make sure the data are read before the next $\overline{\text{DRDY}}$ update, or the old data are overwritten with new data.

As shown in Figure 90, the ADC data field is 3, 4, or 5 bytes long. The data field consists of an optional STATUS byte, three bytes of conversion data, and an optional CRC byte. After all bytes are read, the data-byte sequence (including the STATUS byte and CRC byte, if selected) is repeated when continued SCLKs are sent. The byte sequence repeats starting with the first byte. In order to help verify error-free communication, read the same data multiple times in each conversion interval or use the optional CRC byte.



- (1) $\overline{\text{DRDY}}$ returns high on the first SCLK falling edge.
- (2) $\overline{\text{CS}}$ can be tied low. If $\overline{\text{CS}}$ is low, $\text{DOUT}/\overline{\text{DRDY}}$ asserts low at the same time as $\overline{\text{DRDY}}$.
- (3) Complete data retrieval before new data are ready ($28 \cdot t_{\text{CLK}}$ before the next falling edge of $\text{DOUT}/\overline{\text{DRDY}}$ and $\overline{\text{DRDY}}$).
- (4) The STATUS and CRC bytes are optional.
- (5) The byte sequence, including selected optional bytes, repeats by continuing SCLK.

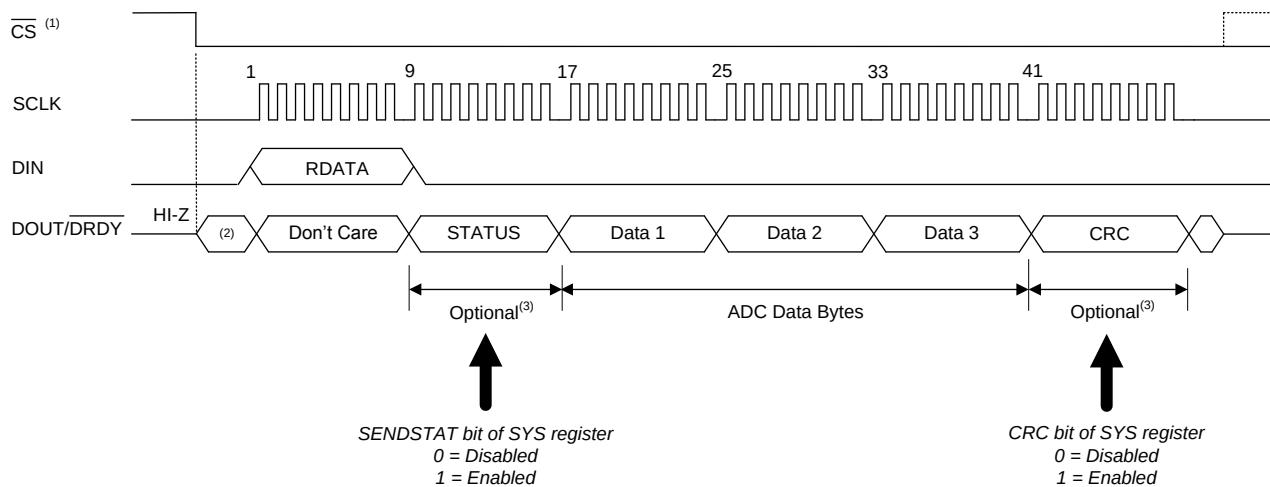
Figure 90. Read Data Direct

9.5.4.2 Read Data by RDATA Command

When the RDATA command is sent, the data are retrieved from the ADC data-holding register. Read data at any time without the risk of data corruption because the command method does not require synchronizing to DRDY. Polling of DRDY to determine when ADC data are ready can still be used.

Figure 91 shows the read data by command sequence. The output data MSB begins on the first SCLK rising edge after the command. The output data field can be 3, 4, or 5 bytes long. The data field consists of an optional STATUS byte, three bytes of conversion data, and an optional CRC byte. An RDATA command must be sent for each read operation. The ADC does not respond to commands until the read operation is complete, or terminated by taking CS high.

After all bytes are read, the data-byte sequence (including the STATUS byte and CRC byte, if selected) is repeated by continuing SCLK.



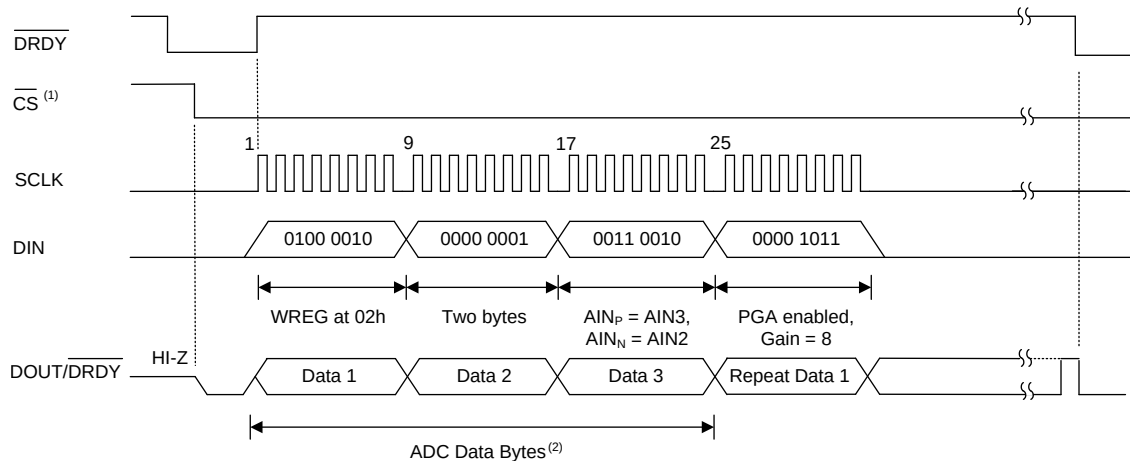
- (1) $\overline{CS}$ can be tied low. If $\overline{CS}$ is low, DOUT/DRDY asserts low with $\overline{DRDY}$.
- (2) DOUT/DRDY is driven low with DRDY. If a read operation occurs after the DRDY falling edge, then DOUT/DRDY can be high or low.
- (3) The STATUS and CRC bytes are optional.

Figure 91. Read Data by Command

9.5.4.3 Sending Commands When Reading Data

The device serial interface is capable of full-duplex operation when reading conversion data and not using the RDATA command. In full-duplex operation, commands are decoded at the same time that conversion data are read. Commands can be sent on any 8-bit data boundary during a data read operation. When a RREG or RDATA command is recognized, the current data read operation is aborted and the conversion data are corrupted, unless the command is sent when the last byte of the conversion result is retrieved. The device starts to output the requested data on DOUT/DRDY at the first SCLK rising edge after the command byte. To read data without interruption, keep DIN low when clocking out data.

A WREG command can be sent without corrupting an ongoing read operation. Sending a WREG command when reading data minimizes the time between reading the data and setting the device configuration for the next conversion. [Figure 92](#) shows an example for sending a WREG command to write two configuration registers when reading conversion data by using read data direct mode. After the command is clocked in, the device resets the digital filter and starts converting with the new register settings as long as the device is in continuous conversion mode. The digital filter is reset and conversions are restarted after each data byte is received. In this example, the digital filter is reset when the first byte is received, decoding the input multiplexer and again when the PGA is set. The WREG command can be sent on any of the 8-bit boundaries. The example in [Figure 92](#) has the STATUS and CRC bytes disabled.



(1) $\overline{CS}$ can be tied low. If $\overline{CS}$ is low, DOUT/DRDY asserts low at the same time as $\overline{DRDY}$.

(2) The output data buffer is cyclical and the original data byte is re-issued when the fourth DIN byte is clocked in.

Figure 92. Issuing a WREG Command When Reading Back ADC Data

9.5.5 Interfacing with Multiple Devices

When connecting multiple devices to a single SPI bus, SCLK, DIN, and DOUT/DRDY can be safely shared by using a dedicated chip-select ($\overline{CS}$) line for each SPI-enabled device. When $\overline{CS}$ transitions high for the respective device, DOUT/DRDY enters a tri-state mode. Therefore, DOUT/DRDY cannot be used to indicate when new data are available if $\overline{CS}$ is high. Only the dedicated $\overline{DRDY}$ pin indicates that new data are available because the $\overline{DRDY}$ pin is actively driven even when $\overline{CS}$ is high.

In some cases, the $\overline{DRDY}$ pin cannot be interfaced to the microcontroller. This scenario can occur if there are insufficient GPIO channels available on the microcontroller or if the serial interface must be galvanically isolated and thus the amount of channels must be limited. In order to evaluate when a new conversion of one of the devices is ready, the microcontroller can periodically drop $\overline{CS}$ to the respective device and poll the state of the DOUT/DRDY pin.

When $\overline{CS}$ goes low, the DOUT/DRDY pin immediately drives either high or low. If the DOUT/DRDY line drives low, new data are available. If the DOUT/DRDY line drives high, no new data are available. This procedure requires that DOUT/DRDY is forced high after reading each conversion result and before taking $\overline{CS}$ high. To make sure DOUT/DRDY is taken high, send a RREG command to read a register where the least significant bit is 1.

ADS124S06, ADS124S08

SBAS660C –AUGUST 2016–REVISED JUNE 2017

www.ti.com

Retrieving data using direct read mode requires knowledge of the $\overline{\text{DRDY}}$ falling edge timing to avoid data corruption. Use the RDATA command so that valid data can be retrieved from the device at any time without concern of data corruption by a new data ready.

9.6 Register Map

9.6.1 Configuration Registers

The ADS124S0x register map consists of 18, 8-bit registers. These registers are used to configure and control the device to the desired mode of operation. Access the registers through the serial interface by using the RREG and WREG register commands. After power-on or reset, the registers default to the initial settings, as shown in the *Default* column of [Table 25](#).

Data can be written as a block to multiple registers using a single WREG command. If data are written as a block, the data of certain registers take effect immediately when data are shifted in. Writing new data to certain registers results in a restart of conversions that are in progress. The registers that result in a conversion restart are discussed in the [WREG](#) section.

Table 25. Configuration Register Map

ADDR	REGISTER	DEFAULT	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
00h	ID	xxh	RESERVED					DEV_ID[2:0]		
01h	STATUS	80h	FL_POR	$\overline{\text{RDY}}$	FL_P_RAILP	FL_P_RAILN	FL_N_RAILP	FL_N_RAILN	FL_REF_L1	FL_REF_L0
02h	INPMUX	01h	MUXP[3:0]				MUXN[3:0]			
03h	PGA	00h	DELAY[2:0]			PGA_EN[1:0]		GAIN[2:0]		
04h	DATARATE	14h	G_CHOP	CLK	MODE	FILTER	DR[3:0]			
05h	REF	10h	FL_REF_EN[1:0]		$\overline{\text{REFP_BUF}}$	$\overline{\text{REFN_BUF}}$	REFSEL[1:0]		REFCON[1:0]	
06h	IDACMAG	00h	FL_RAIL_EN	PSW	0	0	IMAG[3:0]			
07h	IDACMUX	FFh	I2MUX[3:0]				I1MUX[3:0]			
08h	VBIAS	00h	VB_LEVEL	VB_AINC	VB_AIN5	VB_AIN4	VB_AIN3	VB_AIN2	VB_AIN1	VB_AIN0
09h	SYS	10h	SYS_MON[2:0]			CAL_SAMP[1:0]		TIMEOUT	CRC	SENDSTAT
0Ah	OFCAL0	00h	OFC[7:0]							
0Bh	OFCAL1	00h	OFC[15:8]							
0Ch	OFCAL2	00h	OFC[23:16]							
0Dh	FSCAL0	00h	FSC[7:0]							
0Eh	FSCAL1	00h	FSC[15:8]							
0Fh	FSCAL2	40h	FSC[23:16]							
10h	GPIODAT	00h	DIR[3:0]				DAT[3:0]			
11h	GPIOCON	00h	0	0	0	0	CON[3:0]			

9.6.1.1 Device ID Register (address = 00h) [reset = xxh]

Figure 93. Device ID (ID) Register

7	6	5	4	3	2	1	0
RESERVED					DEV_ID[2:0]		
R-xxh					R-xh		

LEGEND: R = Read only; -n = value after reset; -x = variable

Table 26. Device ID (ID) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:3	RESERVED	R	xxh	Reserved Values are subject to change without notice
2:0	DEV_ID[2:0]	R	xh	Device identifier Identifies the model of the device. 000 : ADS124S08 (12 channels, 24 bits) 001 : ADS124S06 (6 channels, 24 bits) 010 : Reserved 011 : Reserved 100 : Reserved 101 : Reserved 110 : Reserved 111 : Reserved

9.6.1.2 Device Status Register (address = 01h) [reset = 80h]

Figure 94. Device Status (STATUS) Register

7	6	5	4	3	2	1	0
FL_POR	$\overline{\text{RDY}}$	FL_P_RAILP	FL_P_RAILN	FL_N_RAILP	FL_N_RAILN	FL_REF_L1	FL_REF_L0
R/W-1h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 27. Device Status (STATUS) Register Field Descriptions

Bit	Field	Type	Reset	Description
7	FL_POR	R/W	1h	POR flag Indicates a power-on reset (POR) event has occurred. 0 : Register has been cleared and no POR event has occurred. 1 : POR event occurred and has not been cleared. Flag must be cleared by user register write (default).
6	$\overline{\text{RDY}}$	R	0h	Device ready flag Indicates the device has started up and is ready for communication. 0 : ADC ready for communication (default) 1 : ADC not ready
5	FL_P_RAILP	R	0h	Positive PGA output at positive rail flag⁽¹⁾ Indicates the positive PGA output is within 150 mV of AVDD. 0 : No error (default) 1 : PGA positive output within 150 mV of AVDD
4	FL_P_RAILN	R	0h	Positive PGA output at negative rail flag⁽¹⁾ Indicates the positive PGA output is within 150 mV of AVSS. 0 : No error (default) 1 : PGA positive output within 150 mV of AVSS
3	FL_N_RAILP	R	0h	Negative PGA output at positive rail flag⁽¹⁾ Indicates the negative PGA output is within 150 mV of AVDD. 0 : No error (default) 1 : PGA negative output within 150 mV of AVDD
2	FL_N_RAILN	R	0h	Negative PGA output at negative rail flag⁽¹⁾ Indicates the negative PGA output is within 150 mV of AVSS. 0 : No error (default) 1 : PGA negative output within 150 mV of AVSS
1	FL_REF_L1	R	0h	Reference voltage monitor flag, level 1⁽²⁾ Indicates the external reference voltage is lower than 1/3 of the analog supply voltage. Can be used to detect an open-excitation lead in a 3-wire RTD application. 0 : Differential reference voltage $\geq 1/3 \cdot (\text{AVDD} - \text{AVSS})$ (default) 1 : Differential reference voltage $< 1/3 \cdot (\text{AVDD} - \text{AVSS})$
0	FL_REF_L0	R	0h	Reference voltage monitor flag, level 0⁽²⁾ Indicates the external reference voltage is lower than 0.3 V. Can be used to indicate a missing or floating external reference voltage. 0 : Differential reference voltage $\geq 0.3 \text{ V}$ (default) 1 : Differential reference voltage $< 0.3 \text{ V}$

(1) The PGA rail monitors are enabled with the FL_RAIL_EN bit in excitation current register 1 (06h).

(2) The reference monitors are enabled with the FL_REF_EN[1:0] bits of the reference control register (05h).

9.6.1.3 Input Multiplexer Register (address = 02h) [reset = 01h]

Figure 95. Input Multiplexer (INPMUX) Register

7	6	5	4	3	2	1	0
MUXP[3:0]				MUXN[3:0]			
R/W-0h				R/W-1h			

LEGEND: R/W = Read/Write; -n = value after reset

Table 28. Input Multiplexer (INPMUX) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:4	MUXP[3:0]	R/W	0h	Positive ADC input selection Selects the ADC positive input channel. 0000 : AIN0 (default) 0001 : AIN1 0010 : AIN2 0011 : AIN3 0100 : AIN4 0101 : AIN5 0110 : AIN6 (ADS124S08 only) 0111 : AIN7 (ADS124S08 only) 1000 : AIN8 (ADS124S08 only) 1001 : AIN9 (ADS124S08 only) 1010 : AIN10 (ADS124S08 only) 1011 : AIN11 (ADS124S08 only) 1100 : AINCOM 1101 : Reserved 1110 : Reserved 1111 : Reserved
3:0	MUXN[3:0]	R/W	1h	Negative ADC input selection Selects the ADC negative input channel. 0000 : AIN0 0001 : AIN1 (default) 0010 : AIN2 0011 : AIN3 0100 : AIN4 0101 : AIN5 0110 : AIN6 (ADS124S08 only) 0111 : AIN7 (ADS124S08 only) 1000 : AIN8 (ADS124S08 only) 1001 : AIN9 (ADS124S08 only) 1010 : AIN10 (ADS124S08 only) 1011 : AIN11 (ADS124S08 only) 1100 : AINCOM 1101 : Reserved 1110 : Reserved 1111 : Reserved

9.6.1.4 Gain Setting Register (address = 03h) [reset = 00h]

Figure 96. Gain Setting (PGA) Register

7	6	5	4	3	2	1	0
DELAY[2:0]			PGA_EN[1:0]		GAIN[2:0]		
R/W-0h			R/W-0h		R/W-0h		

LEGEND: R/W = Read/Write; -n = value after reset

Table 29. Gain Setting (PGA) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:5	DELAY[2:0]	R/W	0h	Programmable conversion delay selection Sets the programmable conversion delay time for the first conversion after a WREG when a configuration change resets of the digital filter and triggers a new conversion ⁽¹⁾ . 000 : $14 \cdot t_{MOD}$ (default) 001 : $25 \cdot t_{MOD}$ 010 : $64 \cdot t_{MOD}$ 011 : $256 \cdot t_{MOD}$ 100 : $1024 \cdot t_{MOD}$ 101 : $2048 \cdot t_{MOD}$ 110 : $4096 \cdot t_{MOD}$ 111 : $1 \cdot t_{MOD}$
4:3	PGA_EN[1:0]	R/W	0h	PGA enable Enables or bypasses the PGA. 00 : PGA is powered down and bypassed. Enables single-ended measurements with unipolar supply (Set gain = 1 ⁽²⁾) (default) 01 : PGA enabled (gain = 1 to 128) 10 : Reserved 11 : Reserved
2:0	GAIN[2:0]	R/W	0h	PGA gain selection Configures the PGA gain. 000 : 1 (default) 001 : 2 010 : 4 011 : 8 100 : 16 101 : 32 110 : 64 111 : 128

(1) For details on which bits and registers trigger a new conversion, see the [WREG](#) section.

(2) When bypassing the PGA, the user must also set GAIN[2:0] to 000.

9.6.1.5 Data Rate Register (address = 04h) [reset = 14h]

Figure 97. Data Rate (DATARATE) Register

7	6	5	4	3	2	1	0
G_CHOP	CLK	MODE	FILTER	DR[3:0]			
R/W-0h	R/W-0h	R/W-0h	R/W-1h	R/W-4h			

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30. Data Rate (DATARATE) Register Field Descriptions

Bit	Field	Type	Reset	Description
7	G_CHOP	R/W	0h	Global chop enable Enables the global chop function. When enabled, the device automatically swaps the inputs and takes the average of two consecutive readings to cancel the offset voltage. 0 : Disabled (default) 1 : Enabled
6	CLK	R/W	0h	Clock source selection Configures the clock source to use either the internal oscillator or an external clock. 0 : Internal 4.096-MHz oscillator (default) 1 : External clock
5	MODE	R/W	0h	Conversion mode selection Configures the ADC for either continuous conversion or single-shot conversion mode. 0 : Continuous conversion mode (default) 1 : Single-shot conversion mode
4	FILTER	R/W	1h	Digital filter selection Configures the ADC to use either the sinc ³ or the low-latency filter. 0 : Sinc ³ filter 1 : Low-latency filter (default)
3:0	DR[3:0]	R/W	4h	Data rate selection Configures the output data rate ⁽¹⁾ . 0000 : 2.5 SPS 0001 : 5 SPS 0010 : 10 SPS 0011 : 16.6 SPS 0100 : 20 SPS (default) 0101 : 50SPS 0110 : 60 SPS 0111 : 100 SPS 1000 : 200 SPS 1001 : 400 SPS 1010 : 800 SPS 1011 : 1000 SPS 1100 : 2000 SPS 1101 : 4000 SPS 1110 : 4000 SPS 1111 : Reserved

(1) Data rates of 60 Hz or less can offer line-cycle rejection; see the [50-Hz and 60-Hz Line Cycle Rejection](#) section for more information.

9.6.1.6 Reference Control Register (address = 05h) [reset = 10h]

Figure 98. Reference Control (REF) Register

7	6	5	4	3	2	1	0
FL_REF_EN[1:0]		REFP_BUF	REFN_BUF	REFSEL[1:0]		REFCON[1:0]	
R/W-0h		R/W-0h	R/W-1h	R/W-0h		R/W-0h	

LEGEND: R/W = Read/Write; -n = value after reset

Table 31. Reference Control (REF) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:6	FL_REF_EN[1:0]	R/W	0h	Reference monitor configuration Enables and configures the reference monitor. 00 : Disabled (default) 01 : FL_REF_L0 monitor enabled, threshold 0.3 V 10 : FL_REF_L0 and FL_REF_L1 monitors enabled, thresholds 0.3 V and 1/3 · (AVDD – AVSS) 11 : FL_REF_L0 monitor and 10-MΩ pull-together enabled, threshold 0.3 V
5	REFP_BUF	R/W	0h	Positive reference buffer bypass Disables the positive reference buffer. Recommended when $V_{(REFP_X)}$ is close to AVDD. 0 : Enabled (default) 1 : Disabled
4	REFN_BUF	R/W	1h	Negative reference buffer bypass Disables the negative reference buffer. Recommended when $V_{(REFN_X)}$ is close to AVSS. 0 : Enabled 1 : Disabled (default)
3:2	REFSEL[1:0]	R/W	0h	Reference input selection Selects the reference input source for the ADC. 00 : REFP0, REFN0 (default) 01 : REFP1, REFN1 10 : Internal 2.5-V reference ⁽¹⁾ 11 : Reserved
1:0	REFCON[1:0]	R/W	0h	Internal voltage reference configuration⁽²⁾ Configures the behavior of the internal voltage reference. 00 : Internal reference off (default) 01 : Internal reference on, but powers down in power-down mode 10 : Internal reference is always on, even in power-down mode 11 : Reserved

(1) Disable the reference buffers when the internal reference is selected for measurements.

(2) The internal voltage reference must be turned on to use the IDACs.

9.6.1.7 Excitation Current Register 1 (address = 06h) [reset = 00h]

Figure 99. Excitation Current Register 1 (IDACMAG)

7	6	5	4	3	2	1	0
FL_RAIL_EN	PSW	0	0	IMAG[3:0]			
R/W-0h	R/W-0h	R-0h	R-0h	R/W-0h			

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 32. Excitation Current Register 1 (IDACMAG) Register Field Descriptions

Bit	Field	Type	Reset	Description
7	FL_RAIL_EN	R/W	0h	PGA output rail flag enable Enables the PGA output voltage rail monitor circuit. 0 : Disabled (default) 1 : Enabled
6	PSW	R/W	0h	Low-side power switch Controls the low-side power switch. The low-side power switch opens automatically in power-down mode. 0 : Open (default) 1 : Closed
5:4	RESERVED	R	0h	Reserved Always write 0h
3:0	IMAG[3:0]	R/W	0h	IDAC magnitude selection Selects the value of the excitation current sources. Sets IDAC1 and IDAC2 to the same value. 0000 : Off (default) 0001 : 10 μ A 0010 : 50 μ A 0011 : 100 μ A 0100 : 250 μ A 0101 : 500 μ A 0110 : 750 μ A 0111 : 1000 μ A 1000 : 1500 μ A 1001 : 2000 μ A 1010 - 1111 : Off

9.6.1.8 Excitation Current Register 2 (address = 07h) [reset = FFh]

Figure 100. Excitation Current Register 2 (IDACMUX)

7	6	5	4	3	2	1	0
I2MUX[3:0]				I1MUX[3:0]			
R/W-Fh				R/W-Fh			

LEGEND: R/W = Read/Write; -n = value after reset

Table 33. Excitation Current Register 2 (IDACMUX) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:4	I2MUX[3:0]	R/W	Fh	IDAC2 output channel selection Selects the output channel for IDAC2. 0000 : AIN0 0001 : AIN1 0010 : AIN2 0011 : AIN3 0100 : AIN4 0101 : AIN5 0110 : AIN6 (ADS124S08), REFP1 (ADS124S06) 0111 : AIN7 (ADS124S08), REFN1 (ADS124S06) 1000 : AIN8 (ADS124S08 only) 1001 : AIN9 (ADS124S08 only) 1010 : AIN10 (ADS124S08 only) 1011 : AIN11 (ADS124S08 only) 1100 : AINCOM 1101 - 1111 : Disconnected (default)
3:0	I1MUX[3:0]	R/W	Fh	IDAC1 output channel selection Selects the output channel for IDAC1. 0000 : AIN0 0001 : AIN1 0010 : AIN2 0011 : AIN3 0100 : AIN4 0101 : AIN5 0110 : AIN6 (ADS124S08 only), REFP1 (ADS124S06) 0111 : AIN7 (ADS124S08 only), REFN1 (ADS124S06) 1000 : AIN8 (ADS124S08 only) 1001 : AIN9 (ADS124S08 only) 1010 : AIN10 (ADS124S08 only) 1011 : AIN11 (ADS124S08 only) 1100 : AINCOM 1101 - 1111 : Disconnected (default)

9.6.1.9 Sensor Biasing Register (address = 08h) [reset = 00h]

Figure 101. Sensor Biasing (VBIAS) Register

7	6	5	4	3	2	1	0
VB_LEVEL	VB_AINC	VB_AIN5	VB_AIN4	VB_AIN3	VB_AIN2	VB_AIN1	VB_AIN0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

LEGEND: R/W = Read/Write; -n = value after reset

Table 34. Sensor Biasing (VBIAS) Register Field Descriptions

Bit	Field	Type	Reset	Description
7	VB_LEVEL	R/W	0h	VBIAS level selection Sets the VBIAS output voltage level. VBIAS is disabled when not connected to any input. 0 : (AVDD + AVSS) / 2 (default) 1 : (AVDD + AVSS) / 12
6	VB_AINC	R/W	0h	AINCOM VBIAS selection⁽¹⁾ Enables VBIAS on the AINCOM pin. 0 : VBIAS disconnected from AINCOM (default) 1 : VBIAS connected to AINCOM
5	VB_AIN5	R/W	0h	AIN5 VBIAS selection⁽¹⁾ Enables VBIAS on the AIN5 pin. 0 : VBIAS disconnected from AIN5 (default) 1 : VBIAS connected to AIN5
4	VB_AIN4	R/W	0h	AIN4 VBIAS selection⁽¹⁾ Enables VBIAS on the AIN4 pin. 0 : VBIAS disconnected from AIN4 (default) 1 : VBIAS connected to AIN4
3	VB_AIN3	R/W	0h	AIN3 VBIAS selection⁽¹⁾ Enables VBIAS on the AIN3 pin. 0 : VBIAS disconnected from AIN3 (default) 1 : VBIAS connected to AIN3
2	VB_AIN2	R/W	0h	AIN2 VBIAS selection⁽¹⁾ Enables VBIAS on the AIN2 pin. 0 : VBIAS disconnected from AIN2 (default) 1 : VBIAS connected to AIN2
1	VB_AIN1	R/W	0h	AIN1 VBIAS selection⁽¹⁾ Enables VBIAS on the AIN1 pin. 0 : VBIAS disconnected from AIN1 (default) 1 : VBIAS connected to AIN1
0	VB_AIN0	R/W	0h	AIN0 VBIAS selection⁽¹⁾ Enables VBIAS on the AIN0 pin. 0 : VBIAS disconnected from AIN0 (default) 1 : VBIAS connected to AIN0

(1) The bias voltage can be selected for multiple analog inputs at the same time.

9.6.1.10 System Control Register (address = 09h) [reset = 10h]

Figure 102. System Control (SYS) Register

7	6	5	4	3	2	1	0
SYS_MON[2:0]			CAL_SAMP[1:0]		TIMEOUT	CRC	SENDSTAT
R/W-0h			R/W-2h		R/W-0h	R/W-0h	R/W-0h

LEGEND: R/W = Read/Write; -n = value after reset

Table 35. System Control (SYS) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:5	SYS_MON[2:0]	R/W	0h	System monitor configuration⁽¹⁾ Enables a set of system monitor measurements using the ADC. 000 : Disabled (default) 001 : PGA inputs shorted to (AVDD + AVSS) / 2 and disconnected from AINx and the multiplexer; gain set by user 010 : Internal temperature sensor measurement; PGA must be enabled (PGA_EN[1:0] = 01); gain set by user ⁽²⁾ 011 : (AVDD – AVSS) / 4 measurement; gain set to 1 ⁽³⁾ 100 : DVDD / 4 measurement; gain set to 1 ⁽³⁾ 101 : Burn-out current sources enabled, 0.2-μA setting 110 : Burn-out current sources enabled, 1-μA setting 111 : Burn-out current sources enabled, 10-μA setting
4:3	CAL_SAMP[1:0]	R/W	2h	Calibration sample size selection Configures the number of samples averaged for self and system offset and system gain calibration. 00 : 1 sample 01 : 4 samples 10 : 8 samples (default) 11 : 16 samples
2	TIMEOUT	R/W	0h	SPI timeout enable Enables the SPI timeout function. 0 : Disabled (default) 1 : Enabled
1	CRC	R/W	0h	CRC enable Enables the CRC byte appended to the conversion result. When enabled, CRC is calculated across the 24-bit conversion result (plus the STATUS byte if enabled). 0 : Disabled (default) 1 : Enabled
0	SENDSTAT	R/W	0h	STATUS byte enable Enables the STATUS byte prepended to the conversion result. 0 : Disabled (default) 1 : Enabled

- (1) With system monitor functions enabled, the AINx multiplexer switches are open for the (AVDD + AVSS) / 2 measurement, the temperature sensor, and the supply monitors.
- (2) When using the internal temperature sensor, gain must be 4 or less to keep the measurement within the PGA input voltage range.
- (3) The PGA gain is automatically set to 1 when the supply monitors are enabled, regardless of the setting in GAIN[2:0].

9.6.1.11 Offset Calibration Register 1 (address = 0Ah) [reset = 00h]

Figure 103. Offset Calibration Register 1 (OFCAL0)

7	6	5	4	3	2	1	0
OFC[7:0]							
R/W-00h							

LEGEND: R/W = Read/Write; -n = value after reset

Table 36. Offset Calibration Register 1 (OFCAL0) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:0	OFC[7:0]	R/W	00h	Bits [7:0] of the offset calibration value.

9.6.1.12 Offset Calibration Register 2 (address = 0Bh) [reset = 00h]

Figure 104. Offset Calibration Register 2 (OFCAL1)

7	6	5	4	3	2	1	0
OFC[15:8]							
R/W-00h							

LEGEND: R/W = Read/Write; -n = value after reset

Table 37. Offset Calibration Register 2 (OFCAL1) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:0	OFC[15:8]	R/W	00h	Bits [15:8] of the offset calibration value.

9.6.1.13 Offset Calibration Register 3 (address = 0Ch) [reset = 00h]

Figure 105. Offset Calibration Register 3 (OFCAL2)

7	6	5	4	3	2	1	0
OFC[23:16]							
R/W-00h							

LEGEND: R/W = Read/Write; -n = value after reset

Table 38. Offset Calibration Register 3 (OFCAL2) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:0	OFC[23:16]	R/W	00h	Bits [23:16] of the offset calibration value.

9.6.1.14 Gain Calibration Register 1 (address = 0Dh) [reset = 00h]

Figure 106. Gain Calibration Register 1 (FSCAL0)

7	6	5	4	3	2	1	0
FSC[7:0]							
R/W-00h							

LEGEND: R/W = Read/Write; -n = value after reset

Table 39. Gain Calibration Register 1 (FSCAL0) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:0	FSC[7:0]	R/W	00h	Bits [7:0] of the gain calibration value.

9.6.1.15 Gain Calibration Register 2 (address = 0Eh) [reset = 00h]

Figure 107. Gain Calibration Register 2 (FSCAL1)

7	6	5	4	3	2	1	0
FSC[15:8]							
R/W-00h							

LEGEND: R/W = Read/Write; -n = value after reset

Table 40. Gain Calibration Register 2 (FSCAL1) Field Descriptions

Bit	Field	Type	Reset	Description
7:0	FSC[15:8]	R/W	00h	Bits [15:8] of the gain calibration value.

9.6.1.16 Gain Calibration Register 3 (address = 0Fh) [reset = 40h]

Figure 108. Gain Calibration Register 3 (FSCAL2)

7	6	5	4	3	2	1	0
FSC[23:16]							
R/W-40h							

LEGEND: R/W = Read/Write; -n = value after reset

Table 41. Gain Calibration Register 3 (FSCAL2) Field Descriptions

Bit	Field	Type	Reset	Description
7:0	FSC[23:16]	R/W	40h	Bits [23:16] of the gain calibration value.

9.6.1.17 GPIO Data Register (address = 10h) [reset = 00h]

Figure 109. GPIO Data (GPIODAT) Register

7	6	5	4	3	2	1	0
DIR[3:0]				DAT[3:0]			
R/W-0h				R/W-0h			

LEGEND: R/W = Read/Write; -n = value after reset

Table 42. GPIO Data (GPIODAT) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:4	DIR[3:0]	R/W	0h	GPIO direction Configures the selected GPIO as an input or output. 0 : GPIO[x] configured as output (default) 1 : GPIO[x] configured as input
3:0	DAT[3:0]	R/W	0h	GPIO data Contains the data of the GPIO inputs or outputs. 0 : GPIO[x] is low (default) 1 : GPIO[x] is high

9.6.1.18 GPIO Configuration Register (address = 11h) [reset = 00h]

Figure 110. GPIO Configuration Register

7	6	5	4	3	2	1	0
0	0	0	0	CON[3:0]			
R-0h	R-0h	R-0h	R-0h	R/W-0h			

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 43. GPIO Configuration (GPIOCON) Register Field Descriptions

Bit	Field	Type	Reset	Description
7:4	RESERVED	R	0h	Reserved Always write 0h
3:0	CON[3:0]	R/W	0h	GPIO pin configuration Configures the GPIO[x] pin as an analog input or GPIO. CON[x] corresponds to the GPIO[x] pin. 0 : GPIO[x] configured as analog input (default) ⁽¹⁾ 1 : GPIO[x] configured as GPIO

(1) On the ADS124S06, the GPIO pins default as disabled. Set the CON[3:0] bits to enable the respective GPIO pins.

10 Application and Implementation

NOTE

Information in the following applications sections is not part of the TI component specification, and TI does not warrant its accuracy or completeness. TI's customers are responsible for determining suitability of components for their purposes. Customers should validate and test their design implementation to confirm system functionality.

10.1 Application Information

The ADS124S06 and ADS124S08 are precision, 24-bit, $\Delta\Sigma$ ADCs that offer many integrated features to simplify the measurement of the most common sensor types (including various types of temperature, flow, and bridge sensors). Primary considerations when designing an application with the ADS124S0x include analog input filtering, establishing an appropriate reference, and setting the absolute input voltage for the internal PGA. Connecting and configuring the serial interface appropriately is another concern. These considerations are discussed in the following sections.

10.1.1 Serial Interface Connections

The principle serial interface connections for the ADS124S0x are shown in [Figure 111](#).

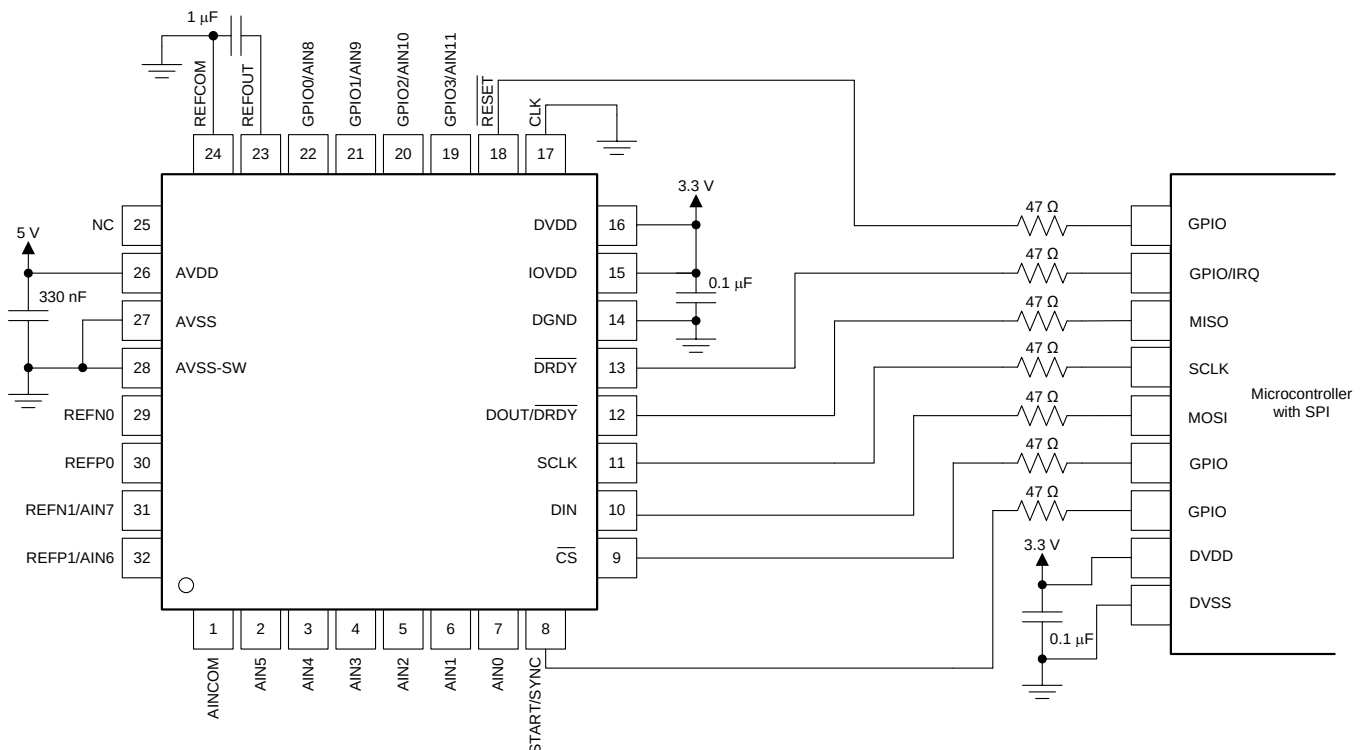


Figure 111. Serial Interface Connections

Most microcontroller SPI peripherals can interface with the ADS124S0x. The interface operates in SPI mode 1 where CPOL = 0 and CPHA = 1. In SPI mode 1, SCLK idles low and data are launched or changed only on SCLK rising edges; data are latched or read by the master and slave on SCLK falling edges. Details of the SPI communication protocol employed by the devices are found in the [Serial Interface](#) section.

Place 47-Ω resistors in series with all digital input and output pins ($\overline{\text{CS}}$, SCLK, DIN, DOUT/ $\overline{\text{DRDY}}$, and $\overline{\text{DRDY}}$). This resistance smooths sharp transitions, suppresses overshoot, and offers some overvoltage protection. Care must be taken to meet all SPI timing requirements because the additional resistors interact with the bus capacitances present on the digital signal lines.

Application Information (continued)

10.1.2 Analog Input Filtering

Analog input filtering serves two purposes: first, to limit the effect of aliasing during the sampling process and second, to reduce external noise from being a part of the measurement.

As with any sampled system, aliasing can occur if proper antialias filtering is not in place. Aliasing occurs when frequency components are present in the input signal that are higher than half the sampling frequency of the ADC (also known as the Nyquist frequency). These frequency components are folded back and show up in the actual frequency band of interest below half the sampling frequency. Note that inside a $\Delta\Sigma$ ADC, the input signal is oversampled at the modulator frequency, f_{MOD} and not at the output data rate. The filter response of the digital filter repeats at multiples of f_{MOD} , as shown in Figure 112. Signals or noise up to a frequency where the filter response repeats are attenuated to a certain amount by the digital filter depending on the filter architecture. Any frequency components present in the input signal around the modulator frequency or multiples thereof are not attenuated and alias back into the band of interest, unless attenuated by an external analog filter.

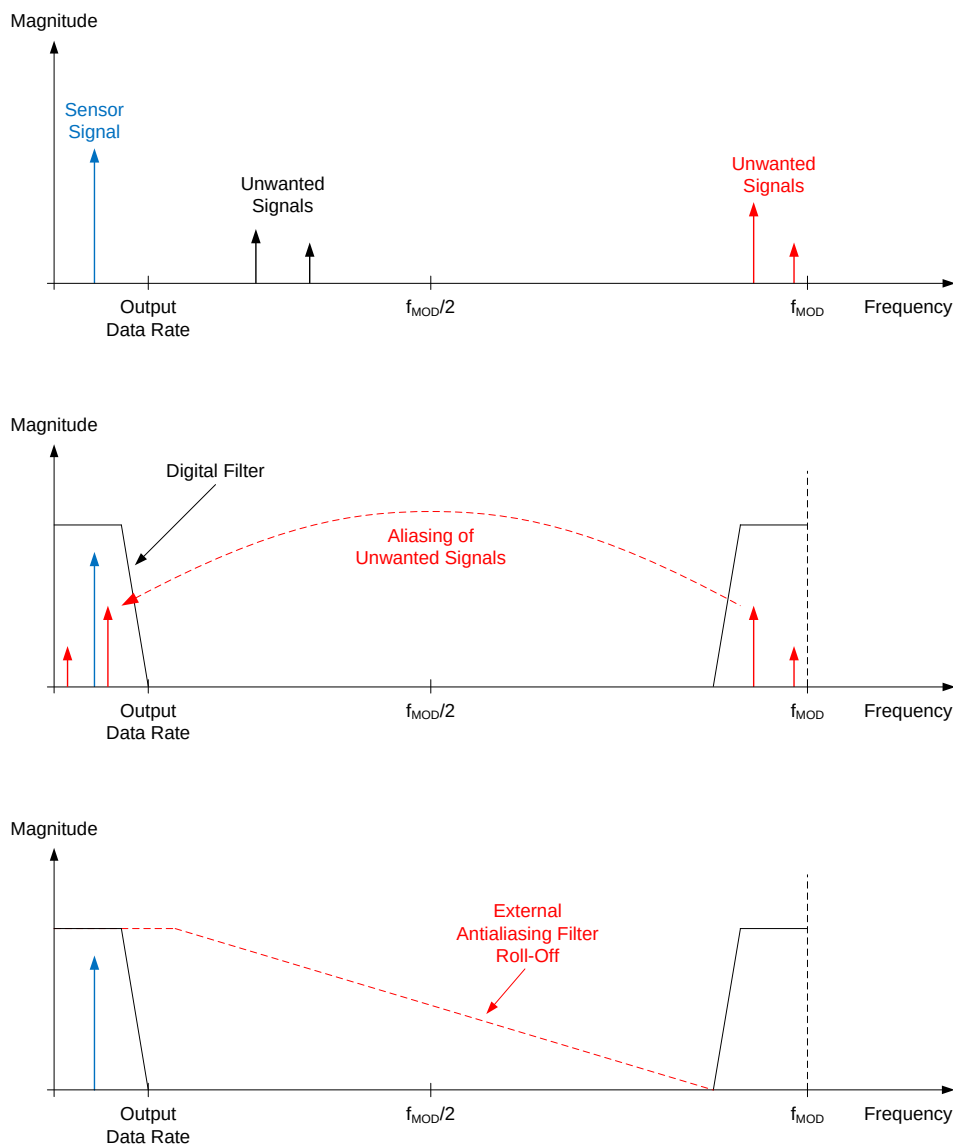


Figure 112. Effect of Aliasing

Application Information (continued)

Many sensor signals are inherently band limited; for example, the output of a thermocouple has a limited rate of change. In this case, the sensor signal does not alias back into the pass band when using a $\Delta\Sigma$ ADC. However, any noise pick-up along the sensor wiring or the application circuitry can potentially alias into the pass band. Power line-cycle frequency and harmonics are one common noise source. External noise can also be generated from electromagnetic interference (EMI) or radio frequency interference (RFI) sources, such as nearby motors and cellular phones. Another noise source typically exists on the printed circuit board (PCB) itself in the form of clocks and other digital signals. Analog input filtering helps remove unwanted signals from affecting the measurement result.

A first-order resistor-capacitor (RC) filter is (in most cases) sufficient to either eliminate aliasing, or to reduce the effect of aliasing to a level below the noise floor of the sensor. Ideally, any signal beyond $f_{\text{MOD}} / 2$ is attenuated to a level below the noise floor of the ADC. The digital filter of the ADS124S0x attenuates signals to a certain degree, as illustrated in the filter response plots in the [Digital Filter](#) section. In addition, noise components are usually smaller in magnitude than the actual sensor signal. Therefore, using a first-order RC filter with a cutoff frequency set at the output data rate or 10 times higher is generally a good starting point for a system design.

Internal to the device, prior to the PGA inputs, is an EMI filter; see [Figure 50](#). The cutoff frequency of this filter is approximately 40 MHz and helps reject high-frequency interference.

10.1.3 External Reference and Ratiometric Measurements

The full-scale range of the ADS124S0x is defined by the reference voltage and the PGA gain ($\text{FSR} = \pm V_{\text{REF}} / \text{Gain}$). An external reference can be used instead of the integrated 2.5-V reference to adapt the FSR to the specific system needs. An external reference must be used if $V_{\text{IN}} > 2.5 \text{ V}$. For example, an external 5-V reference and an $\text{AVDD} = 5 \text{ V}$ are required in order to measure a single-ended signal that can swing between 0 V and 5 V.

The reference inputs of the device also allow the implementation of ratiometric measurements. In a ratiometric measurement, the same excitation source that is used to excite the sensor is also used to establish the reference for the ADC. As an example, a simple form of a ratiometric measurement uses the same current source to excite both the resistive sensor element (such as an RTD) and another resistive reference element that is in series with the element being measured. The voltage that develops across the reference element is used as the reference source for the ADC. Because current noise and drift are common to both the sensor measurement and the reference, these components cancel out in the ADC transfer function. The output code is only a ratio of the sensor element and the value of the reference resistor. The value of the excitation current source itself is not part of the ADC transfer function.

The example in the [Typical Application](#) section describes a system that uses a ratiometric measurement. One excitation current source is used to drive a reference resistor and an RTD. The ADC measurement represents a ratiometric measurement between the RTD value and a known reference resistor value.

10.1.4 Establishing a Proper Input Voltage

The ADS124S0x can be used to measure various types of input signal configurations: single-ended, pseudo-differential, and fully-differential signals (which can be either unipolar or bipolar). However, configuring the device properly for the respective signal type is important.

Signals where the negative analog input is fixed and referenced to analog ground ($V_{\text{AINN}} = 0 \text{ V}$) are commonly called *single-ended signals*. The input voltage of a single-ended signal consequently varies between 0 V and V_{IN} . If the PGA is disabled and bypassed, the input voltage of the ADS124S08 can be as low as 50 mV below AVSS and as large as 50 mV above AVDD . Therefore, set the PGA_EN bits to 10 in the gain setting register (03h) to measure single-ended signals when a unipolar analog supply is used ($\text{AVSS} = 0 \text{ V}$). Only a gain of 1 is possible in this configuration. Measuring a 0-mA to 20-mA or 4-mA to 20-mA signal across a load resistor of 100 Ω referenced to GND is a typical example. The ADS124S0x can directly measure the signal across the load resistor using a unipolar supply, the internal 2.5-V reference, and gain = 1 when the PGA is bypassed.

If gain is needed to measure a single-ended signal, the PGA must be enabled. In this case, a bipolar supply is required for the ADS124S0x to meet the input voltage requirement of the PGA. Signals where the negative analog input (AIN_N) is fixed at a voltage other than 0 V are referred to as *pseudo-differential signals*. The input voltage of a pseudo-differential signal varies between V_{AINN} and $V_{\text{AINN}} + V_{\text{IN}}$.

Application Information (continued)

Fully-differential signals in contrast are defined as signals having a constant common-mode voltage where the positive and negative analog inputs swing 180° out-of-phase but have the same amplitude.

The ADS124S0x can measure pseudo-differential and fully-differential signals both with the PGA enabled or bypassed. However, the PGA must be enabled in order to measure any input with a gain greater than 1. The input voltage must meet the input and output voltage restrictions of the PGA, as explained in the [PGA Input-Voltage Requirements](#) section when the PGA is enabled. Setting the input voltage at or near $(AVSS + AVDD) / 2$ in most cases satisfies the PGA input voltage requirements.

Signals where both the positive and negative inputs are always ≥ 0 V are called *unipolar signals*. These signals can in general be measured with the ADS124S0x using a unipolar analog supply ($AVSS = 0$ V). As mentioned previously, the PGA must be bypassed in order to measure single-ended, unipolar signals when using a unipolar supply.

A signal is called *bipolar* when either the positive or negative input can swing below 0 V. A bipolar analog supply (such as $AVDD = 2.5$ V, $AVSS = -2.5$ V) is required in order to measure bipolar signals with the ADS124S0x. A typical application task is measuring a single-ended, bipolar, ± 10 -V signal where AIN_N is fixed at 0 V and AIN_P swings between -10 V and 10 V. The ADS124S0x cannot directly measure this signal because the 10-V signal exceeds the analog power-supply limits. However, one possible solution is to use a bipolar analog supply ($AVDD = 2.5$ V, $AVSS = -2.5$ V), gain = 1, and a resistor divider in front of the ADS124S0x. The resistor divider must divide the voltage down to $\leq \pm 2.5$ V to be able to measure the voltage using the internal 2.5-V reference.

10.1.5 Unused Inputs and Outputs

To minimize leakage currents on the analog inputs, leave unused analog and reference inputs floating, or connect the inputs to mid-supply or to AVDD. Connecting unused analog or reference inputs to AVSS is possible as well, but can yield higher leakage currents than the previously mentioned options. REFNO is an exception; this pin can be accidentally shorted to AVSS through the internal low-side switch. Leave the REFNO pin floating when not in use or tie the pin to AVSS.

GPIO pins operate on levels based on the analog supply. Do not float GPIO pins that are configured as digital inputs. Tie unused GPIO pins that are configured as digital inputs to the appropriate levels, AVDD or AVSS, including when in power-down mode. Tie unused GPIO output pins to AVSS through a pulldown resistor and set the output to 0 in the GPIO data register. For unused GPIO pins on the ADS124S06, leave the GPIOCON register set to the default register values and connect these GPIO pins in the same manner as for an unused analog input.

Do not float unused digital inputs; excessive power-supply leakage current can result. Tie all unused digital inputs to the appropriate levels, IOVDD or DGND, even when in power-down mode. Connections for unused digital inputs are listed below.

- Tie the $\overline{CS}$ pin to DGND if $\overline{CS}$ is not used
- Tie the CLK pin to DGND if the internal oscillator is used
- Tie the START/SYNC pin to DGND to control conversions by commands
- Tie the $\overline{RESET}$ pin to IOVDD if the $\overline{RESET}$ pin is not used
- If the $\overline{DRDY}$ output is not used, leave the $\overline{DRDY}$ pin unconnected or tie the $\overline{DRDY}$ pin to IOVDD using a weak pullup resistor

Application Information (continued)

10.1.6 Pseudo Code Example

The following list shows a pseudo code sequence with the required steps to set up the device and the microcontroller that interfaces to the ADC in order to take subsequent readings from the ADS124S0x in continuous conversion mode. The dedicated $\overline{\text{DRDY}}$ pin is used to indicate availability of new conversion data.

```

Power-up so that all supplies reach minimum operating levels;
Delay for a minimum of 2.2 ms to allow power supplies to settle and power-up reset to complete;

Configure the SPI interface of the microcontroller to SPI mode 1 (CPOL = 0, CPHA =1);
If the  $\overline{\text{CS}}$  pin is not tied low permanently, configure the microcontroller GPIO connected to  $\overline{\text{CS}}$  as an
output;
Configure the microcontroller GPIO connected to the  $\overline{\text{DRDY}}$  pin as a falling edge triggered interrupt
input;

Set  $\overline{\text{CS}}$  to the device low;
Delay for a minimum of  $t_{d(\text{CSSC})}$ ;
Send the RESET command (06h) to make sure the device is properly reset after power-up; //Optional
Delay for a minimum of  $4096 \cdot t_{\text{CLK}}$ ;
Read the status register using the RREG command to check that the  $\overline{\text{RDY}}$  bit is 0; //Optional
Clear the FL_POR flag by writing 00h to the status register; //Optional
Write the respective register configuration with the WREG command;
For verification, read back all configuration registers with the RREG command;
Send the START command (08h) to start converting in continuous conversion mode;
Delay for a minimum of  $t_{d(\text{SCCS})}$ ;
Clear  $\overline{\text{CS}}$  to high (resets the serial interface);
Loop
{
    Wait for  $\overline{\text{DRDY}}$  to transition low;
    Take  $\overline{\text{CS}}$  low;
    Delay for a minimum of  $t_{d(\text{CSSC})}$ ;
    Send the RDATA command;
    Send 24 SCLK rising edges to read out conversion data on DOUT/ $\overline{\text{DRDY}}$ ;
    Delay for a minimum of  $t_{d(\text{SCCS})}$ ;
    Clear  $\overline{\text{CS}}$  to high;
}
Take  $\overline{\text{CS}}$  low;
Delay for a minimum of  $t_{d(\text{CSSC})}$ ;
Send the STOP command (0Ah) to stop conversions and put the device in standby mode;
Delay for a minimum of  $t_{d(\text{SCCS})}$ ;
Clear  $\overline{\text{CS}}$  to high;

```

10.2 Typical Application

Figure 113 shows a fault-protected, filtered, 3-wire RTD application circuit with hardware-based, lead-wire compensation. Two IDAC current sources provide the lead-wire compensation. One IDAC current source (IDAC1) provides excitation to the RTD element. The ADC reference voltage (pins AIN6 and AIN7) is derived from the voltage across resistor R_{REF} sourcing the same IDAC1 current, providing ratiometric cancellation of current-source drift. The other current source (IDAC2) has the same current setting, providing cancellation of lead-wire resistance by generating a voltage drop across lead-wire resistance R_{LEAD2} equal to the voltage drop of R_{LEAD1} . Because the R_{RTD} voltage is measured differentially at ADC pins AIN1 and AIN2, the voltages across the lead wire resistance cancel. Resistor R_{BIAS} level-shifts the RTD signal to within the ADC specified input range. The current sources are provided by two additional pins (AIN5 and AIN3) that connect to the RTD through blocking diodes. The additional pins are used to route the RTD excitation currents around the input filter resistors, avoiding the voltage drop otherwise caused by the filter resistors R_{F1} and R_{F4} . The diodes protect the ADC inputs in the event of a miswired connection. The input filter resistors limit the input fault currents flowing into the ADC.

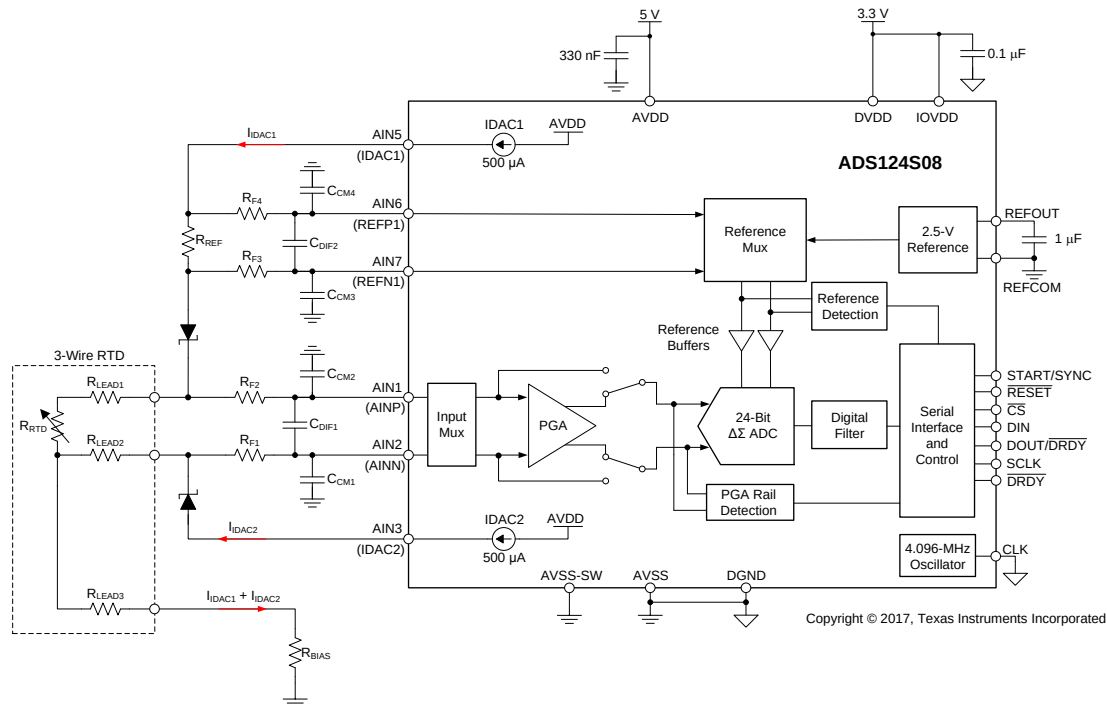


Figure 113. 3-Wire RTD Application

10.2.1 Design Requirements

Table 44 shows the design requirements of the 3-wire RTD application.

Table 44. Design Requirements

DESIGN PARAMETER	VALUE
ADC supply voltage	4.75 V (minimum)
RTD sensor type	3-wire Pt100
RTD resistance range	20 Ω to 400 Ω
RTD lead resistance range	0 Ω to 10 Ω
RTD self heating	1 mW
Accuracy ⁽¹⁾	$\pm 0.05 \Omega$

(1) $T_A = 25^\circ\text{C}$. After offset and full-scale calibration.

10.2.2 Detailed Design Procedure

The key considerations in the design of a 3-wire RTD circuit are the accuracy, the lead wire compensation, and the sensor self-heating. As the design values of [Table 45](#) show, several values of excitation currents are available. The resolution is expressed in units of noise-free resolution (NFR). Noise-free resolution is resolution with no code flicker. The selection of excitation currents trades off resolution against sensor self-heating. In general, measurement resolution improves with increasing excitation current. Increasing the excitation current beyond 1000 μA results in no further improvement in resolution for this example circuit. The design procedure is based on a 500- μA excitation current, because this level of current results in very low sensor self-heating (0.4 mW).

Table 45. RTD Circuit Design Parameters

I_{DAC} (μA)	NFR (bits)	P_{RTD} (mW)	V_{RTD} (V)	Gain (V/V)	$V_{\text{REFMIN}}^{(1)}$ (V)	$V_{\text{REF}}^{(2)}$ (V)	R_{REF} ($\text{k}\Omega$)	$V_{\text{AINNLM}}^{(3)}$ (V)	$V_{\text{AINPLM}}^{(4)}$ (V)	R_{BIAS} ($\text{k}\Omega$)	$V_{\text{RTDN}}^{(5)}$ (V)	$V_{\text{RTDP}}^{(6)}$ (V)	$V_{\text{IDAC1}}^{(7)}$ (V)
50	16.8	0.001	0.02	32	0.64	0.70	18	0.6	4.1	7.10	0.7	0.7	1.9
100	17.8	0.004	0.04	32	1.28	1.41	14.1	0.9	3.8	5.10	1.0	1.1	2.8
250	18.8	0.025	0.10	16	1.60	1.76	7.04	1.1	3.7	2.30	1.2	1.3	3.3
500	19.1	0.100	0.20	8	1.60	1.76	3.52	1.0	3.8	1.10	1.1	1.3	3.4
750	18.9	0.225	0.30	4	1.20	1.32	1.76	0.8	4.0	0.57	0.9	1.2	2.8
1000	19.3	0.400	0.40	4	1.60	1.76	1.76	0.9	3.9	0.50	1.0	1.4	3.5
1500	19.1	0.900	0.60	2	1.20	1.32	0.88	0.6	4.2	0.23	0.7	1.3	3.0
2000	18.3	1.600	0.80	1	0.80	0.90	0.45	0.3	4.5	0.10	0.4	1.2	2.4

- (1) V_{REFMIN} is the minimum reference voltage required by the design.
- (2) V_{REF} is the design target reference voltage allowing for 10% overrange.
- (3) V_{AINNLM} is the absolute minimum input voltage required by the ADC.
- (4) V_{AINPLM} is the absolute maximum input voltage required by the ADC.
- (5) V_{RTDN} is the design target negative input voltage.
- (6) V_{RTDP} is the design target positive input voltage.
- (7) V_{IDAC1} is the design target IDAC1 loop voltage.

Initially, R_{LEAD1} and R_{LEAD2} are considered to be 0 Ω . Route the IDAC1 current through the external reference resistor, R_{REF} . IDAC1 generates the ADC reference voltage, V_{REF} , across the reference resistor. This voltage is defined by [Equation 12](#):

$$V_{\text{REF}} = I_{\text{IDAC1}} \cdot R_{\text{REF}} \quad (12)$$

Route the second current (IDAC2) to the second RTD lead.

Program the IDAC value by using the IDACMAG register; however, only the IDAC1 current flows through the reference resistor and RTD. The IDAC1 current excites the RTD to produce a voltage proportional to the RTD resistance. The RTD voltage is defined by [Equation 13](#):

$$V_{\text{RTD}} = R_{\text{RTD}} \cdot I_{\text{IDAC1}} \quad (13)$$

The ADC amplifies the RTD signal voltage (V_{RTD}) and measures the resulting voltage against the reference voltage to produce a proportional digital output code, as shown in [Equation 14](#) through [Equation 16](#).

$$\text{Code} \propto V_{\text{RTD}} \cdot \text{Gain} / V_{\text{REF}} \quad (14)$$

$$\text{Code} \propto (R_{\text{RTD}} \cdot I_{\text{IDAC1}}) \cdot \text{Gain} / (I_{\text{IDAC1}} \cdot R_{\text{REF}}) \quad (15)$$

$$\text{Code} \propto (R_{\text{RTD}} \cdot \text{Gain}) / R_{\text{REF}} \quad (16)$$

As shown in [Equation 16](#), the RTD measurement depends on the value of the RTD, the PGA gain, and the reference resistor R_{REF} , but not on the IDAC1 value. Therefore, the absolute accuracy and temperature drift of the excitation current does not matter.

The second excitation current (IDAC2) provides a second voltage drop across the second RTD lead resistance, R_{LEAD2} . The second voltage drop compensates the voltage drop caused by I_{IDAC1} and R_{LEAD1} . The leads of a 3-wire RTD typically have the same length; therefore, the lead resistance is typically identical. Taking the lead resistance into account ($R_{\text{LEADX}} \neq 0$), the differential voltage (V_{IN}) across ADC inputs AIN8 and AIN9 is shown in [Equation 17](#):

$$V_{\text{IN}} = I_{\text{IDAC1}} \cdot (R_{\text{RTD}} + R_{\text{LEAD1}}) - I_{\text{IDAC2}} \cdot R_{\text{LEAD2}} \quad (17)$$

If $R_{\text{LEAD1}} = R_{\text{LEAD2}}$ and $I_{\text{IDAC1}} = I_{\text{IDAC2}}$, the expression for V_{IN} reduces to [Equation 18](#):

$$V_{\text{IN}} = I_{\text{IDAC1}} \cdot R_{\text{RTD}} \quad (18)$$

In other words, the measurement error resulting from the voltage drop across the RTD lead resistance is compensated as long as the lead resistance values and the IDAC values are matched.

Using Equation 13, the value of RTD resistance (400 Ω, maximum) and the excitation current (500 μA) yields an RTD voltage of $V_{RTD} = 500 \mu A \cdot 400 \Omega = 0.2 \text{ V}$. Use the maximum gain of 8 in order to limit the corresponding loop voltage of IDAC1. Gain = 8 requires a minimum reference voltage $V_{REFMIN} = 0.2 \text{ V} \cdot 8 = 1.6 \text{ V}$. To provide margin for the ADC operating range, increase the target reference voltage by 10% ($V_{REF} = 1.6 \text{ V} \cdot 1.1 = 1.76 \text{ V}$). Calculate the value of the reference resistor, as shown in Equation 19:

$$R_{REF} = V_{REF} / I_{IDAC1} = 1.76 \text{ V} / 500 \mu A = 3.52 \text{ k}\Omega \quad (19)$$

For this example application, 3.5 kΩ is chosen for R_{REF} . For best results, use a precision reference resistor R_{REF} with a low temperature drift (< 10 ppm/°C). Any change in R_{REF} is reflected in the measurement as a gain error.

The next step in the design is determining the value of the R_{BIAS} resistor, in order to level shift the RTD voltage to meet the ADC absolute input-voltage specification. The required level-shift voltage is determined by calculating the minimum absolute voltage (V_{AINNLM}) as shown in Equation 20:

$$AVSS + 0.15 + V_{RTDMAX} \cdot (\text{Gain} - 1) / 2 \leq V_{AINNLM}$$

where

- V_{RTDMAX} = maximum differential RTD voltage = 0.2 V
 - Gain = 8
 - AVSS = 0 V
- (20)

The result of the equation requires a minimum absolute input voltage (V_{RTDN}) > 0.85 V. Therefore, the RTD voltage must be level shifted by a minimum of 0.85 V. To meet this requirement, a target level-shift value of 1 V is chosen to provide extra margin. Calculate the value of R_{BIAS} as shown in Equation 21:

$$R_{BIAS} = V_{AINN} / (I_{IDAC1} + I_{IDAC2}) = 1 \text{ V} / (2 \cdot 500 \mu A) = 1 \text{ k}\Omega \quad (21)$$

After the level-shift voltage is determined, verify that the positive RTD voltage (V_{RTDP}) is less than the maximum absolute input voltage ($V_{AINPLIM}$), as shown in Equation 22:

$$V_{AINPLIM} \leq AVDD - 0.15 - V_{RTDMAX} \cdot (\text{Gain} - 1) / 2$$

where

- V_{RTDMAX} = maximum differential RTD voltage = 0.2 V
 - Gain = 8
 - AVDD = 4.75 V (minimum)
- (22)

Solving Equation 22 results in a required V_{RTDP} of less than 3.9 V. Calculate the V_{RTDP} input voltage by Equation 23:

$$V_{AINP} = V_{RTDN} + I_{IDAC1} \cdot (R_{RTD} + R_{LEAD1}) = 1 \text{ V} + 500 \mu A \cdot (400 \Omega + 10 \Omega) = 1.2 \text{ V} \quad (23)$$

Because 1.2 V is less than the 3.9-V maximum input voltage limit, the absolute positive and negative RTD voltages are within the ADC specified input range.

The next step in the design is to verify that the IDACs have enough voltage headroom (compliance voltage) to operate. The loop voltage of the excitation current must be less than the supply voltage minus the specified IDAC compliance voltage. Calculate the voltage drop developed across each IDAC current path to AVSS. In this circuit, IDAC1 has the largest voltage drop developed across its current path. The IDAC1 calculation is sufficient to satisfy IDAC2 because the IDAC2 voltage drop is always less than IDAC1 voltage drop. The sum of voltages in the IDAC1 loop is shown in Equation 24:

$$V_{IDAC1} = [(I_{IDAC1} + I_{IDAC2}) \cdot (R_{LEAD3} + R_{BIAS})] + [I_{IDAC1} \cdot (R_{RTD} + R_{LEAD1} + R_{REF})] + V_D$$

where

- V_D = external blocking diode voltage
- (24)

The equation results in a loop voltage of $V_{IDAC1} = 3.0 \text{ V}$. The worst-case current source compliance voltage is: $(AVDD - 0.4 \text{ V}) = (4.75 \text{ V} - 0.4 \text{ V}) = 4.35 \text{ V}$. The V_{IDAC1} loop voltage is less than the specified current source compliance voltage ($3.0 \text{ V} < 4.35 \text{ V}$).

Many applications benefit from using an analog filter at the inputs to remove noise and interference from the signal. Filter components are placed on the ADC inputs (R_{F1} , R_{F2} , C_{DIF1} , C_{CM1} , and C_{CM2}), as well as on the reference inputs (R_{F3} , R_{F4} , C_{DIF2} , C_{CM3} , and C_{CM4}). The filters remove both differential and common-mode noise. The application shows a differential input noise filter formed by R_{F1} , R_{F2} and C_{DIF1} , with additional differential mode capacitance provided by the common-mode filter capacitors, C_{CM1} and C_{CM2} . Calculate the differential –3-dB cutoff frequency as shown in [Equation 25](#):

$$f_{DIF} = 1 / [2\pi \cdot (R_{F1} + R_{F2}) \cdot (C_{DIF1} + C_{CM1} || C_{CM2})] \quad (25)$$

The common-mode noise filter is formed by components R_{F1} , R_{F2} , C_{CM1} , and C_{CM2} . Calculate the common-mode signal –3-dB cutoff frequency, as shown in [Equation 26](#):

$$f_{CM} = 1 / (2\pi \cdot R_{F1} \cdot C_{CM1}) = 1 / (2\pi \cdot R_{F2} \cdot C_{CM2}) \quad (26)$$

Mismatches in the common-mode filter components convert common-mode noise into differential noise. To reduce the effect of mismatch, use a differential mode filter with a corner frequency that is at least 10 times lower than the common-mode filter corner frequency. The low-frequency differential filter removes the common-mode converted noise. The filter resistors (R_{FX}) also serve as current-limiting resistors. These resistors limit the current into the analog inputs (AINx) of the device to safe levels when an overvoltage occurs on the inputs.

Filter resistors lead to an offset voltage error due to the dc input current leakage flowing into and out of the device. Remove this voltage error by system offset calibration. Resistor values that are too large generate excess thermal noise and degrade the overall noise performance. The recommended range of the filter resistor values is 100 Ω to 10 k Ω . The properties of the capacitors are important because the capacitors are connected to the signal; use high-quality COG ceramics or film-type capacitors.

For consistent noise performance across the full range of RTD measurements, match the corner frequencies of the input and reference filter. See the [RTD Ratiometric Measurements and Filtering Using the ADS1148 and ADS1248 Application Report](#) (SBAA201) for detailed information on matching the input and reference filter.

10.2.2.1 Register Settings

The register settings for this design are shown in [Table 46](#).

Table 46. Register Settings

REGISTER	NAME	SETTING	DESCRIPTION
02h	INPMUX	12h	Select AIN _P = AIN1 and AIN _N = AIN2
03h	PGA	0Bh	PGA enabled, PGA Gain = 8
04h	DATARATE	14h	Continuous conversion mode, low-latency filter, 20-SPS data rate
05h	REF	06h	Positive and negative reference buffers enabled, REFP1 and REFN1 reference inputs selected, internal reference always on
06h	IDACMAG	05h	IDAC magnitude set to 500 μ A
07h	IDACMUX	35h	IDAC2 set to AIN3, IDAC1 set to AIN5
08h	VBIAS	00h	
09h	SYS	10h	
0Ah	OFCAL0 ⁽¹⁾	xxh	
0Bh	OFCAL1	xxh	
0Ch	OFCAL2	xxh	
0Dh	FSCAL0 ⁽¹⁾	xxh	
0Eh	FSCAL1	xxh	
0Fh	FSCAL2	xxh	
10h	GPIO DAT	00h	
11h	GPIO CON	00h	

(1) A two-point offset and gain calibration removes errors from the R_{REF} tolerance. The results are used for the OFC and FSC registers.

10.2.3 Application Curves

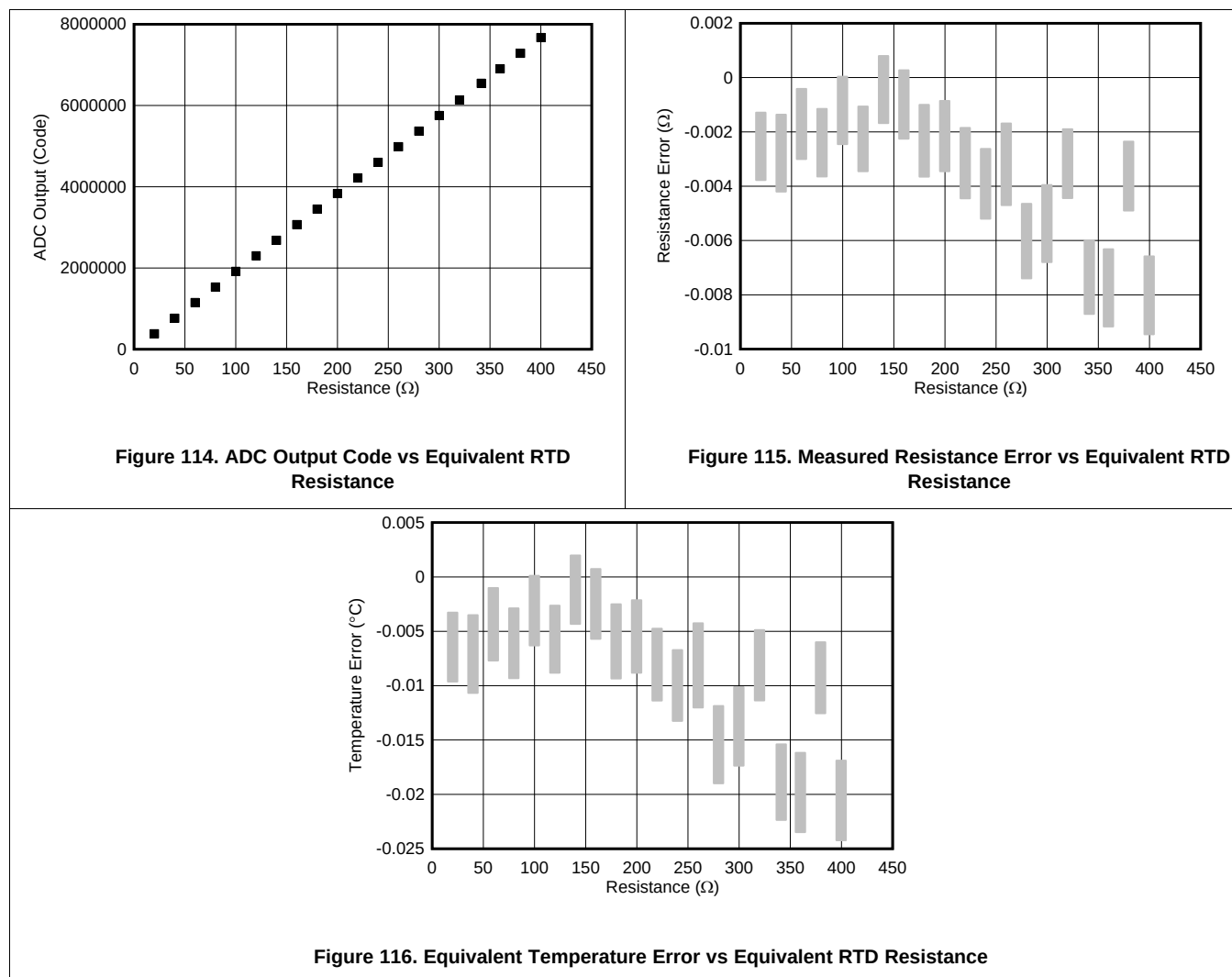
To test the accuracy of the acquisition circuit, a series of calibrated high-precision discrete resistors are used as an input to the system. Measurements are taken at $T_A = 25^\circ\text{C}$. Figure 114 displays the resistance measurement over an input span from $20\ \Omega$ to $400\ \Omega$. Any offset error is generally attributed to the offset of the ADC, and the gain error can be attributed to the accuracy of the R_{REF} resistor and the ADC. The R_{REF} value is also calibrated to reduce the gain error contribution.

Precision temperature measurement applications are typically calibrated to remove the effects of gain and offset errors that generally dominate the total system error. The simplest calibration method is a linear, or two-point calibration that applies an equal and opposite gain and offset term to cancel the measured system gain and offset error. In this particular tested application, the gain and offset error was very small, and did not require additional calibration other than the self offset and gain calibration provided by the device. The resulting measured resistance error is shown in Figure 115.

The results in Figure 115 are converted to temperature accuracy by dividing the results by the RTD sensitivity (α) at the measured resistance. Over the full resistance input range, the maximum total measured error is $\pm 0.00929\ \Omega$. Equation 27 uses the measured resistance error and the RTD sensitivity at 0°C to calculate the measured temperature accuracy.

$$\text{Error } (^\circ\text{C}) = \text{Error } (\Omega) / \alpha_{@0^\circ\text{C}} = \pm 0.00929\ \Omega / 0.39083\ \Omega / ^\circ\text{C} = \pm 0.024^\circ\text{C} \quad (27)$$

Figure 116 displays the calculated temperature accuracy of the circuit assuming a linear RTD resistance to temperature response. This figure does not include any linearity compensation of the RTD.



10.3 Do's and Don'ts

- Do partition the analog, digital, and power-supply circuitry into separate sections on the PCB.
- Do use a single ground plane for analog and digital grounds.
- Do place the analog components close to the ADC pins using short, direct connections.
- Do keep the SCLK pin free of glitches and noise.
- Do verify that the analog input voltages are within the specified PGA input voltage range under all input conditions.
- Do float unused analog input pins to minimize input leakage current on all other analog inputs. Connecting unused pins to AVDD is the next best option.
- Do provide current limiting to the analog inputs in case overvoltage faults occur.
- Do use a low-dropout linear regulator (LDO) to reduce ripple voltage generated by switch-mode power supplies. Reducing ripple is especially important for AVDD where the supply noise can affect the performance.
- Don't cross analog and digital signals.
- Don't allow the analog and digital power supply voltages to exceed 5.5 V under any condition, including during power-up and power-down.

Do's and Don'ts (continued)

Figure 117 shows the do's and don'ts of the ADC circuit connections.

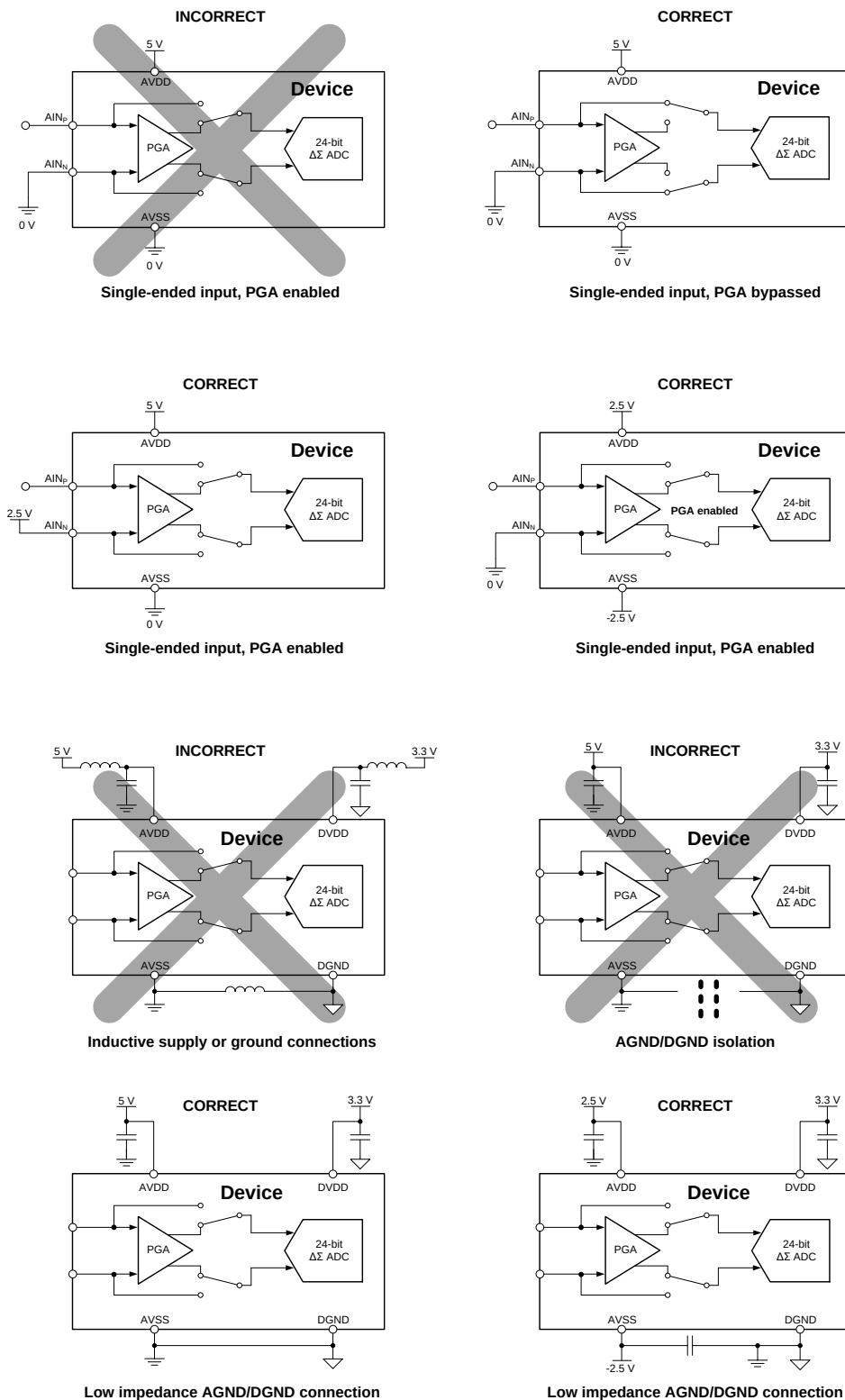


Figure 117. Do's and Don'ts Circuit Connections

11 Power Supply Recommendations

11.1 Power Supplies

The ADS124S0x requires three power supplies: analog (AVDD, AVSS), digital core (DVDD, DGND), and digital I/O (IOVDD, DGND). The analog power supply can be bipolar (for example, AVDD = 2.5 V, AVSS = –2.5 V) or unipolar (for example, AVDD = 3.3 V, AVSS = 0 V) and is independent of the digital power supplies. DVDD is used to power the digital circuits of the devices. IOVDD sets the digital I/O levels (with the exception of the GPIO levels that are set by the analog supply of AVDD and AVSS). IOVDD must be equal to or larger than DVDD.

11.2 Power-Supply Sequencing

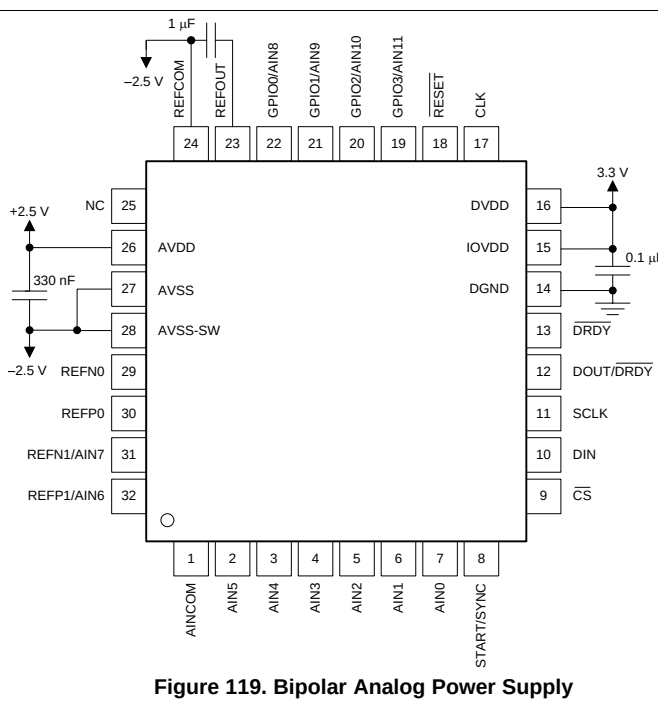
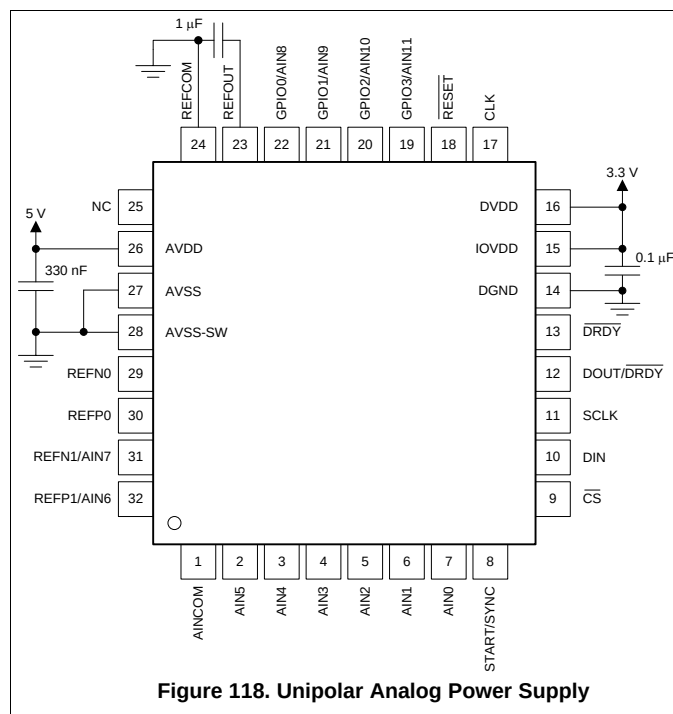
AVDD and DVDD may be powered up in any order. However, IOVDD is recommended to be powered up before or at the same time as DVDD. If DVDD comes up before IOVDD, a reset of the device using the **RESET** pin or the **RESET** command may be required.

11.3 Power-On Reset

An internal POR is released after all three supplies exceed approximately 1.65 V. Each supply has an individual POR circuit. A brownout condition on any of the three supplies triggers a reset of the complete device.

11.4 Power-Supply Decoupling

Good power-supply decoupling is important to achieve best performance. AVDD must be decoupled with at least a 330-nF capacitor to AVSS. DVDD and IOVDD (when not connected to DVDD) must be decoupled with at least a 0.1-μF capacitor to DGND. [Figure 118](#) and [Figure 119](#) show typical power-supply decoupling examples for unipolar and bipolar analog supplies, respectively. Place the bypass capacitors as close to the power-supply pins of the device as possible using low-impedance connections. Use multi-layer ceramic chip capacitors (MLCCs) that offer low equivalent series resistance (ESR) and inductance (ESL) characteristics for power-supply decoupling purposes. To reduce inductance on the supply pins, avoid the use of vias for connecting the capacitors to the supply pins. The use of multiple vias in parallel lowers the overall inductance and is beneficial for connections to ground planes. Connect analog and digital grounds together as close to the device as possible.



12 Layout

12.1 Layout Guidelines

Employing best design practices is recommended when laying out a printed-circuit board (PCB) for both analog and digital components. This recommendation generally means that the layout separates analog components [such as ADCs, amplifiers, references, digital-to-analog converters (DACs), and analog MUXs] from digital components [such as microcontrollers, complex programmable logic devices (CPLDs), field-programmable gate arrays (FPGAs), radio frequency (RF) transceivers, universal serial bus (USB) transceivers, and switching regulators]. An example of good component placement is shown in [Figure 120](#). Although [Figure 120](#) provides a good example of component placement, the best placement for each application is unique to the geometries, components, and PCB fabrication capabilities employed. That is, there is no single layout that is perfect for every design and careful consideration must always be used when designing with any analog component.

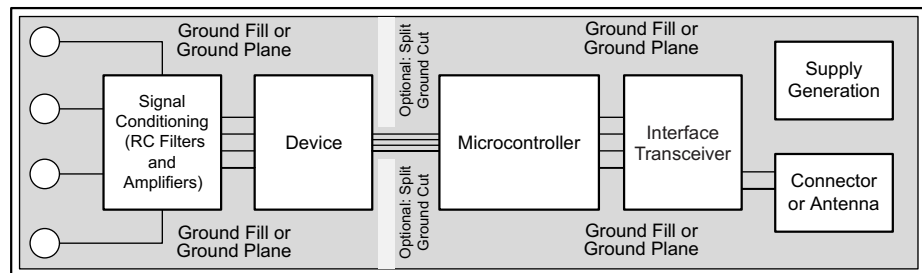


Figure 120. System Component Placement

The following basic recommendations for layout of the ADS124S0x help achieve the best possible performance of the ADC. A good design can be ruined with a bad circuit layout.

- Separate analog and digital signals. To start, partition the board into analog and digital sections where the layout permits. Route digital lines away from analog lines. This prevents digital noise from coupling back into analog signals.
- The ground plane can be split into an analog plane (AGND) and digital plane (DGND), but this (splitting) is not necessary. Place digital signals over the digital plane, and analog signals over the analog plane. As a final step in the layout, the split between the analog and digital grounds must be connected to together at the ADC.
- Fill void areas on signal layers with ground fill.
- Provide good ground return paths. Signal return currents will flow on the path of least impedance. If the ground plane is cut or has other traces that block the current from flowing right next to the signal trace, another path must be found to return to the source and complete the circuit. If forced into a larger path, the chance that the signal radiates increases. Sensitive signals are more susceptible to EMI interference.
- Use bypass capacitors on supplies to reduce high-frequency noise. Do not place vias between bypass capacitors and the active device. Placing the bypass capacitors on the same layer as close to the active device yields the best results.
- Consider the resistance and inductance of the routing. Often, traces for the inputs have resistances that react with the input bias current and cause an added error voltage. Reducing the loop area enclosed by the source signal and the return current reduces the inductance in the path. Reducing the inductance reduces the EMI pickup and reduces the high-frequency impedance at the input of the device.
- Watch for parasitic thermocouples in the layout. Dissimilar metals going from each analog input to the sensor can create a parasitic thermocouple that can add an offset to the measurement. Differential inputs must be matched for both the inputs going to the measurement source.
- Analog inputs with differential connections must have a capacitor placed differentially across the inputs. Best input combinations for differential measurements use adjacent analog input lines (such as AIN0, AIN1 and AIN2, AIN3). The differential capacitors must be of high quality. The best ceramic chip capacitors are COG (NPO) that have stable properties and low noise characteristics.

12.2 Layout Example

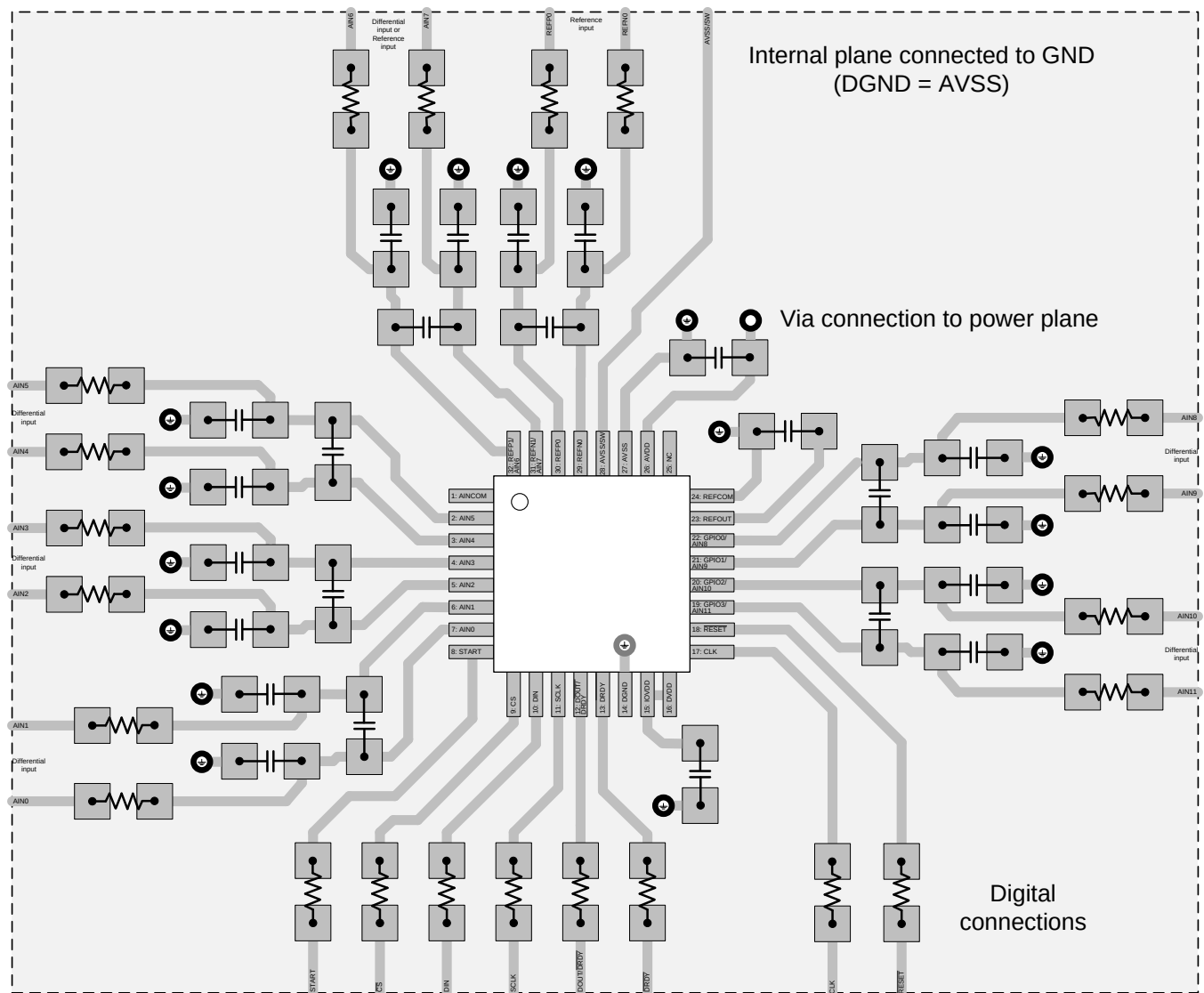


Figure 121. ADS124S0x Layout Example

13 Device and Documentation Support

13.1 Device Support

13.1.1 Development Support

[ADS1x4S0x Design Calculator](#)

13.2 Documentation Support

13.2.1 Related Documentation

For related documentation see the following:

- [REF50xx Low-Noise, Very Low Drift, Precision Voltage Reference](#)
- [RTD Ratiometric Measurements and Filtering Using the ADS1148 and ADS1248 Application Report](#)
- [3-Wire RTD Measurement System Reference Design, -200°C to 850°C](#)

13.3 Related Links

The table below lists quick access links. Categories include technical documents, support and community resources, tools and software, and quick access to order now.

Table 47. Related Links

PARTS	PRODUCT FOLDER	ORDER NOW	TECHNICAL DOCUMENTS	TOOLS & SOFTWARE	SUPPORT & COMMUNITY
ADS124S06	Click here	Click here	Click here	Click here	Click here
ADS124S08	Click here	Click here	Click here	Click here	Click here

13.4 Receiving Notification of Documentation Updates

To receive notification of documentation updates, navigate to the device product folder on ti.com. In the upper right corner, click on *Alert me* to register and receive a weekly digest of any product information that has changed. For change details, review the revision history included in any revised document.

13.5 Community Resources

The following links connect to TI community resources. Linked contents are provided "AS IS" by the respective contributors. They do not constitute TI specifications and do not necessarily reflect TI's views; see TI's [Terms of Use](#).

TI E2E™ Online Community *TI's Engineer-to-Engineer (E2E) Community*. Created to foster collaboration among engineers. At e2e.ti.com, you can ask questions, share knowledge, explore ideas and help solve problems with fellow engineers.

Design Support *TI's Design Support* Quickly find helpful E2E forums along with design support tools and contact information for technical support.

13.6 Trademarks

E2E is a trademark of Texas Instruments.
All other trademarks are the property of their respective owners.

13.7 Electrostatic Discharge Caution



This integrated circuit can be damaged by ESD. Texas Instruments recommends that all integrated circuits be handled with appropriate precautions. Failure to observe proper handling and installation procedures can cause damage.

ESD damage can range from subtle performance degradation to complete device failure. Precision integrated circuits may be more susceptible to damage because very small parametric changes could cause the device not to meet its published specifications.

13.8 Glossary

[SLYZ022](#) — *TI Glossary*.

This glossary lists and explains terms, acronyms, and definitions.

14 Mechanical, Packaging, and Orderable Information

The following pages include mechanical packaging and orderable information. This information is the most current data available for the designated devices. This data is subject to change without notice and revision of this document. For browser-based versions of this data sheet, refer to the left-hand navigation.

PACKAGING INFORMATION

Orderable part number	Status (1)	Material type (2)	Package Pins	Package qty Carrier	RoHS (3)	Lead finish/ Ball material (4)	MSL rating/ Peak reflow (5)	Op temp (°C)	Part marking (6)
ADS124S06IPBS	Active	Production	TQFP (PBS) 32	250 JEDEC TRAY (10+1)	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S06
ADS124S06IPBS.A	Active	Production	TQFP (PBS) 32	250 JEDEC TRAY (10+1)	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S06
ADS124S06IPBS.B	Active	Production	TQFP (PBS) 32	250 JEDEC TRAY (10+1)	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S06
ADS124S06IPBSG4	Active	Production	TQFP (PBS) 32	250 JEDEC TRAY (10+1)	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S06
ADS124S06IPBSG4.A	Active	Production	TQFP (PBS) 32	250 JEDEC TRAY (10+1)	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S06
ADS124S06IPBSG4.B	Active	Production	TQFP (PBS) 32	250 JEDEC TRAY (10+1)	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S06
ADS124S06IPBSR	Active	Production	TQFP (PBS) 32	1000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S06
ADS124S06IPBSR.A	Active	Production	TQFP (PBS) 32	1000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S06
ADS124S06IPBSR.B	Active	Production	TQFP (PBS) 32	1000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S06
ADS124S06IRHBR	Active	Production	VQFN (RHB) 32	3000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S06
ADS124S06IRHBR.A	Active	Production	VQFN (RHB) 32	3000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S06
ADS124S06IRHBR.B	Active	Production	VQFN (RHB) 32	3000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S06
ADS124S06IRHBT	Active	Production	VQFN (RHB) 32	250 SMALL T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S06
ADS124S06IRHBT.A	Active	Production	VQFN (RHB) 32	250 SMALL T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S06
ADS124S06IRHBT.B	Active	Production	VQFN (RHB) 32	250 SMALL T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S06
ADS124S08IPBS	Active	Production	TQFP (PBS) 32	250 JEDEC TRAY (10+1)	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S08
ADS124S08IPBS.A	Active	Production	TQFP (PBS) 32	250 JEDEC TRAY (10+1)	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S08
ADS124S08IPBS.B	Active	Production	TQFP (PBS) 32	250 JEDEC TRAY (10+1)	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S08

Orderable part number	Status (1)	Material type (2)	Package Pins	Package qty Carrier	RoHS (3)	Lead finish/ Ball material (4)	MSL rating/ Peak reflow (5)	Op temp (°C)	Part marking (6)
ADS124S08IPBSR	Active	Production	TQFP (PBS) 32	1000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S08
ADS124S08IPBSR.A	Active	Production	TQFP (PBS) 32	1000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S08
ADS124S08IPBSR.B	Active	Production	TQFP (PBS) 32	1000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S08
ADS124S08IPBSRG4	Active	Production	TQFP (PBS) 32	1000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S08
ADS124S08IPBSRG4.A	Active	Production	TQFP (PBS) 32	1000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S08
ADS124S08IPBSRG4.B	Active	Production	TQFP (PBS) 32	1000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	124S08
ADS124S08IRHBR	Active	Production	VQFN (RHB) 32	3000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S08
ADS124S08IRHBR.A	Active	Production	VQFN (RHB) 32	3000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S08
ADS124S08IRHBR.B	Active	Production	VQFN (RHB) 32	3000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S08
ADS124S08IRHBGR4	Active	Production	VQFN (RHB) 32	3000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S08
ADS124S08IRHBGR4.A	Active	Production	VQFN (RHB) 32	3000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S08
ADS124S08IRHBGR4.B	Active	Production	VQFN (RHB) 32	3000 LARGE T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S08
ADS124S08IRHBT	Active	Production	VQFN (RHB) 32	250 SMALL T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S08
ADS124S08IRHBT.A	Active	Production	VQFN (RHB) 32	250 SMALL T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S08
ADS124S08IRHBT.B	Active	Production	VQFN (RHB) 32	250 SMALL T&R	Yes	NIPDAU	Level-3-260C-168 HR	-50 to 125	ADS 124S08

⁽¹⁾ **Status:** For more details on status, see our [product life cycle](#).

⁽²⁾ **Material type:** When designated, preproduction parts are prototypes/experimental devices, and are not yet approved or released for full production. Testing and final process, including without limitation quality assurance, reliability performance testing, and/or process qualification, may not yet be complete, and this item is subject to further changes or possible discontinuation. If available for ordering, purchases will be subject to an additional waiver at checkout, and are intended for early internal evaluation purposes only. These items are sold without warranties of any kind.

⁽³⁾ **RoHS values:** Yes, No, RoHS Exempt. See the [TI RoHS Statement](#) for additional information and value definition.

(4) **Lead finish/Ball material:** Parts may have multiple material finish options. Finish options are separated by a vertical ruled line. Lead finish/Ball material values may wrap to two lines if the finish value exceeds the maximum column width.

(5) **MSL rating/Peak reflow:** The moisture sensitivity level ratings and peak solder (reflow) temperatures. In the event that a part has multiple moisture sensitivity ratings, only the lowest level per JEDEC standards is shown. Refer to the shipping label for the actual reflow temperature that will be used to mount the part to the printed circuit board.

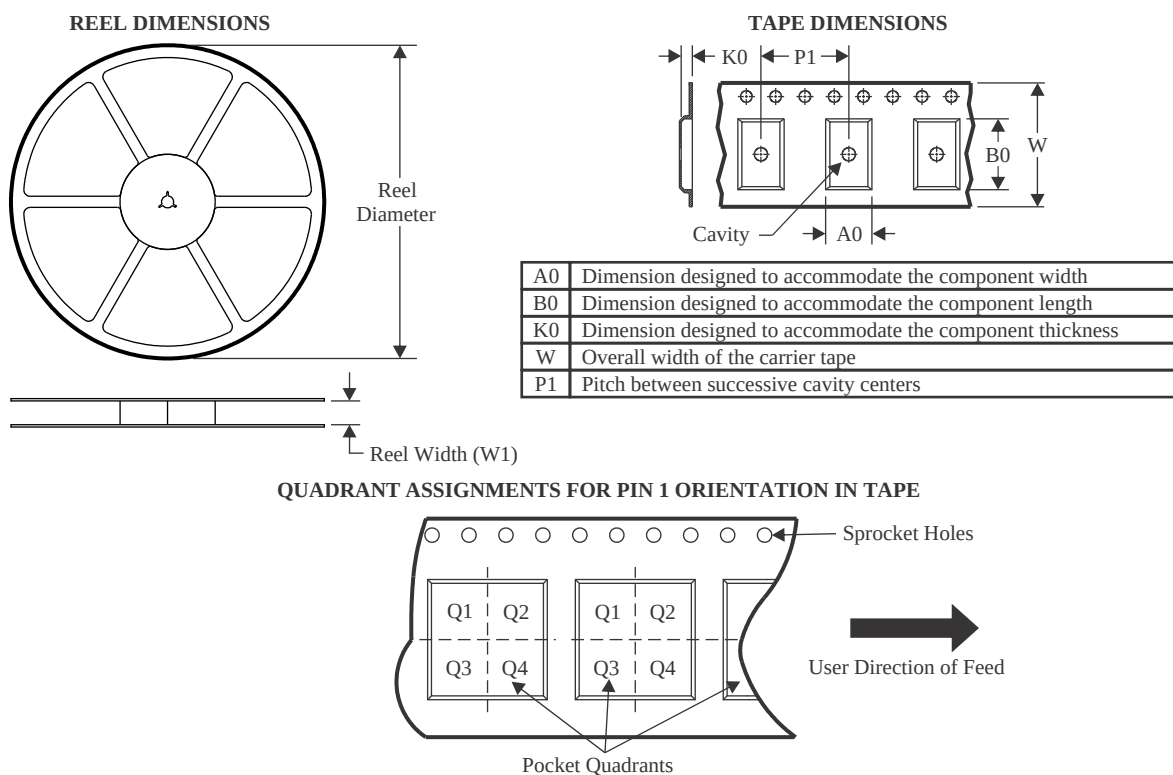
(6) **Part marking:** There may be an additional marking, which relates to the logo, the lot trace code information, or the environmental category of the part.

Multiple part markings will be inside parentheses. Only one part marking contained in parentheses and separated by a "~" will appear on a part. If a line is indented then it is a continuation of the previous line and the two combined represent the entire part marking for that device.

Important Information and Disclaimer: The information provided on this page represents TI's knowledge and belief as of the date that it is provided. TI bases its knowledge and belief on information provided by third parties, and makes no representation or warranty as to the accuracy of such information. Efforts are underway to better integrate information from third parties. TI has taken and continues to take reasonable steps to provide representative and accurate information but may not have conducted destructive testing or chemical analysis on incoming materials and chemicals. TI and TI suppliers consider certain information to be proprietary, and thus CAS numbers and other limited information may not be available for release.

In no event shall TI's liability arising out of such information exceed the total purchase price of the TI part(s) at issue in this document sold by TI to Customer on an annual basis.

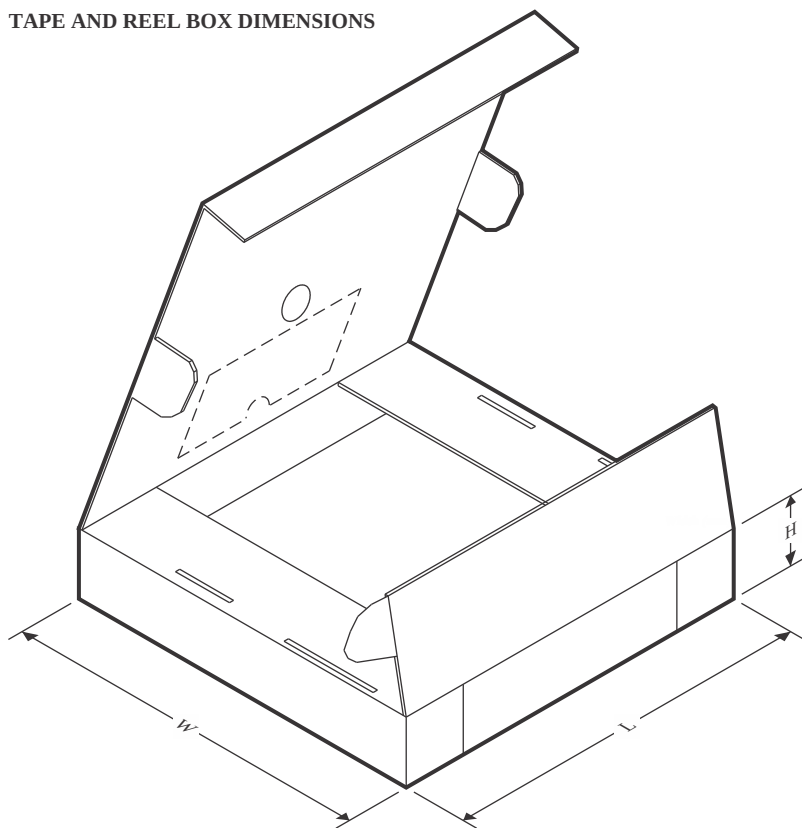
TAPE AND REEL INFORMATION



*All dimensions are nominal

Device	Package Type	Package Drawing	Pins	SPQ	Reel Diameter (mm)	Reel Width W1 (mm)	A0 (mm)	B0 (mm)	K0 (mm)	P1 (mm)	W (mm)	Pin1 Quadrant
ADS124S06IPBSR	TQFP	PBS	32	1000	330.0	16.4	7.2	7.2	1.5	12.0	16.0	Q2
ADS124S06IRHBR	VQFN	RHB	32	3000	330.0	12.4	5.3	5.3	1.1	8.0	12.0	Q2
ADS124S06IRHBT	VQFN	RHB	32	250	180.0	12.4	5.3	5.3	1.1	8.0	12.0	Q2
ADS124S08IPBSR	TQFP	PBS	32	1000	330.0	16.4	7.2	7.2	1.5	12.0	16.0	Q2
ADS124S08IPBSRG4	TQFP	PBS	32	1000	330.0	16.4	7.2	7.2	1.5	12.0	16.0	Q2
ADS124S08IRHBR	VQFN	RHB	32	3000	330.0	12.4	5.3	5.3	1.1	8.0	12.0	Q2
ADS124S08IRHBRG4	VQFN	RHB	32	3000	330.0	12.4	5.3	5.3	1.1	8.0	12.0	Q2
ADS124S08IRHBT	VQFN	RHB	32	250	180.0	12.4	5.3	5.3	1.1	8.0	12.0	Q2

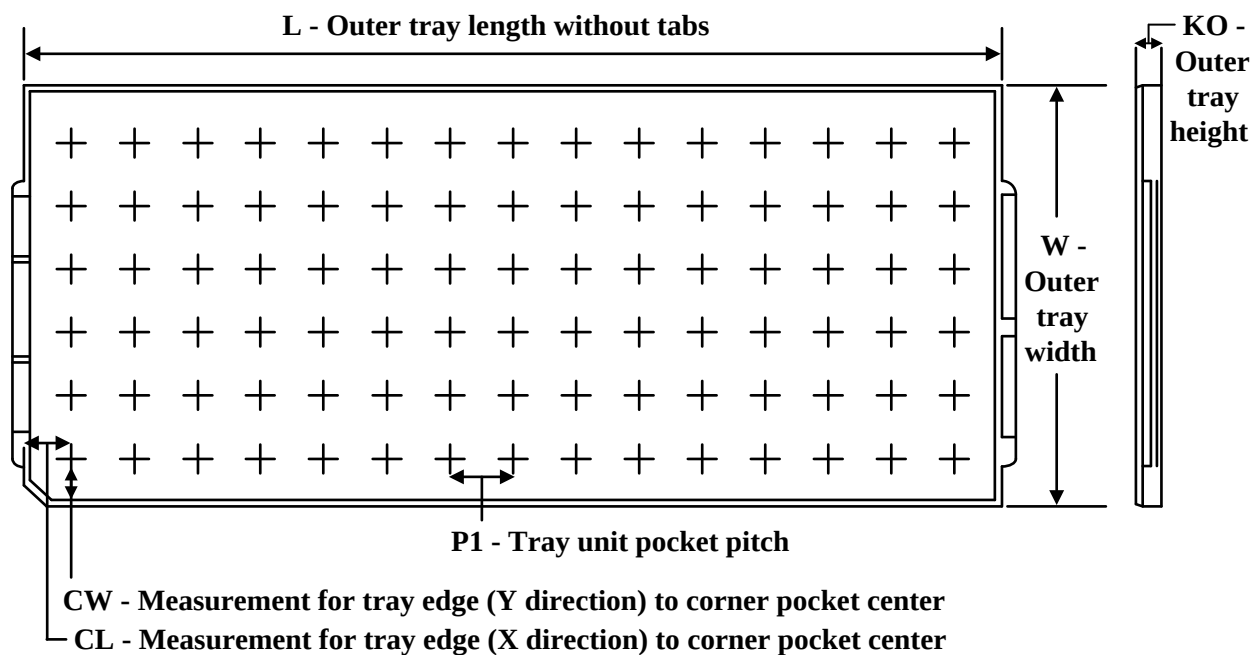
TAPE AND REEL BOX DIMENSIONS



*All dimensions are nominal

Device	Package Type	Package Drawing	Pins	SPQ	Length (mm)	Width (mm)	Height (mm)
ADS124S06IPBSR	TQFP	PBS	32	1000	350.0	350.0	43.0
ADS124S06IRHBR	VQFN	RHB	32	3000	346.0	346.0	33.0
ADS124S06IRHBT	VQFN	RHB	32	250	210.0	185.0	35.0
ADS124S08IPBSR	TQFP	PBS	32	1000	350.0	350.0	43.0
ADS124S08IPBSRG4	TQFP	PBS	32	1000	350.0	350.0	43.0
ADS124S08IRHBR	VQFN	RHB	32	3000	346.0	346.0	33.0
ADS124S08IRHBRG4	VQFN	RHB	32	3000	346.0	346.0	33.0
ADS124S08IRHBT	VQFN	RHB	32	250	210.0	185.0	35.0

TRAY



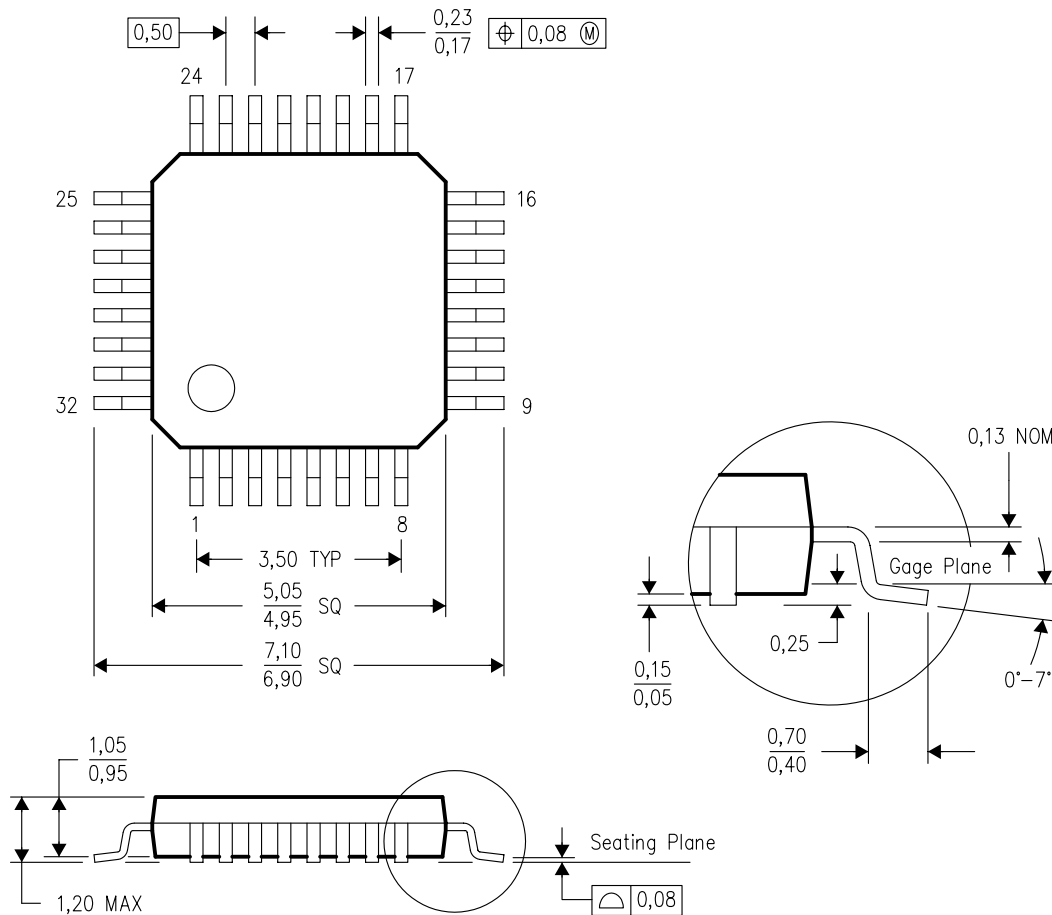
Chamfer on Tray corner indicates Pin 1 orientation of packed units.

*All dimensions are nominal

Device	Package Name	Package Type	Pins	SPQ	Unit array matrix	Max temperature (°C)	L (mm)	W (mm)	K0 (μm)	P1 (mm)	CL (mm)	CW (mm)
ADS124S06IPBS	PBS	TQFP	32	250	10 X 25	150	315	135.9	7620	12.2	11.1	11.25
ADS124S06IPBS.A	PBS	TQFP	32	250	10 X 25	150	315	135.9	7620	12.2	11.1	11.25
ADS124S06IPBS.B	PBS	TQFP	32	250	10 X 25	150	315	135.9	7620	12.2	11.1	11.25
ADS124S06IPBSG4	PBS	TQFP	32	250	10 X 25	150	315	135.9	7620	12.2	11.1	11.25
ADS124S06IPBSG4.A	PBS	TQFP	32	250	10 X 25	150	315	135.9	7620	12.2	11.1	11.25
ADS124S06IPBSG4.B	PBS	TQFP	32	250	10 X 25	150	315	135.9	7620	12.2	11.1	11.25
ADS124S08IPBS	PBS	TQFP	32	250	10 X 25	150	315	135.9	7620	12.2	11.1	11.25
ADS124S08IPBS.A	PBS	TQFP	32	250	10 X 25	150	315	135.9	7620	12.2	11.1	11.25
ADS124S08IPBS.B	PBS	TQFP	32	250	10 X 25	150	315	135.9	7620	12.2	11.1	11.25

PBS (S-PQFP-G32)

PLASTIC QUAD FLATPACK



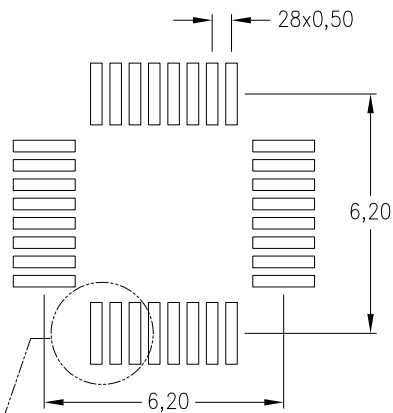
4087735/B 07/05

- NOTES:
- A. All linear dimensions are in millimeters.
 - B. This drawing is subject to change without notice.

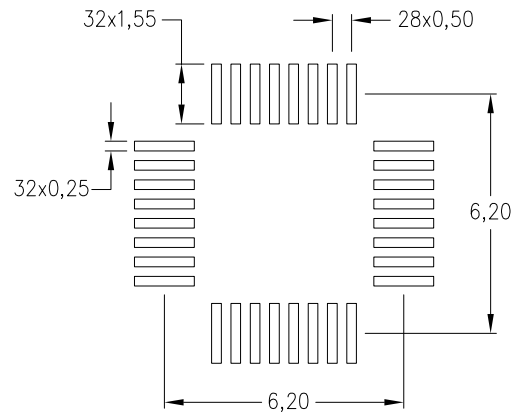
PBS (S-PQFP-G32)

PLASTIC QUAD FLATPACK

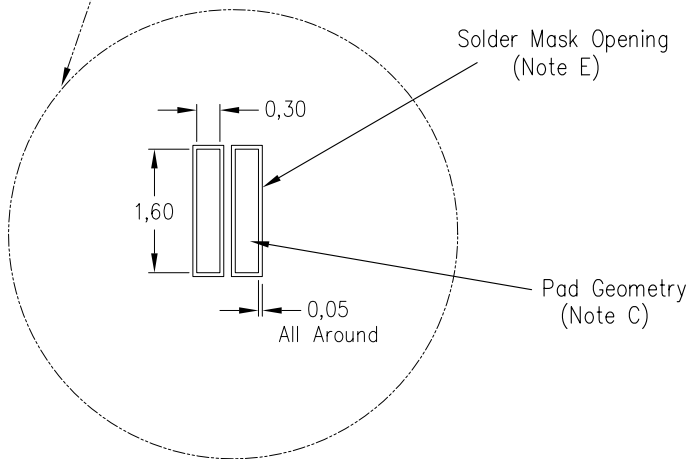
Example Board Layout
(Note C)



0,127mm Thick Stencil Design Example
(Note D)



Non Solder Mask
Defined Pad



4212229/A 10/11

- NOTES:
- A. All linear dimensions are in millimeters.
 - B. This drawing is subject to change without notice.
 - C. Publication IPC-7351 is recommended for alternate designs.
 - D. Laser cutting apertures with trapezoidal walls and also rounding corners will offer better paste release. Customers should contact their board assembly site for stencil design recommendations. Refer to IPC 7525 for stencil design considerations.
 - E. Customers should contact their board fabrication site for recommended solder mask tolerances between and around signal pads.

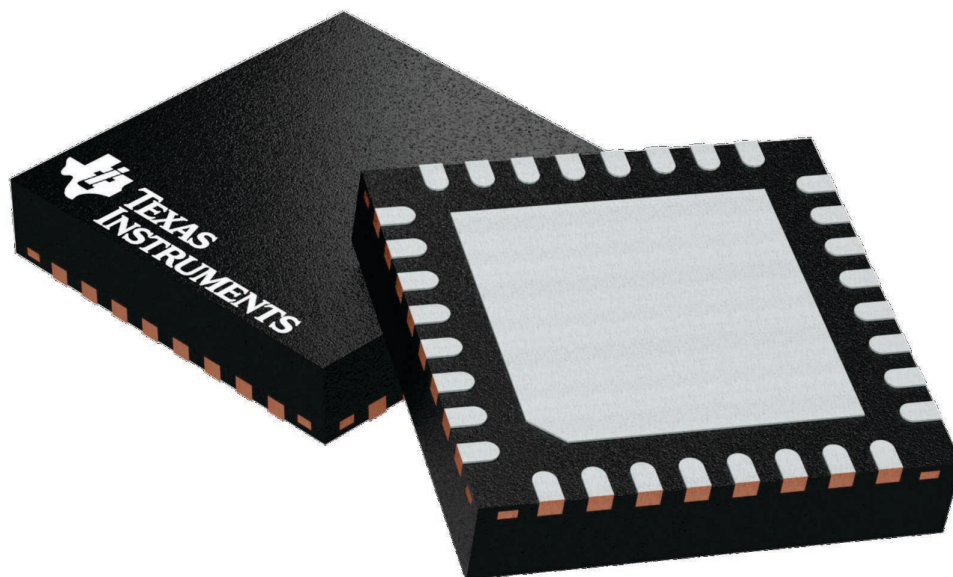
GENERIC PACKAGE VIEW

RHB 32

VQFN - 1 mm max height

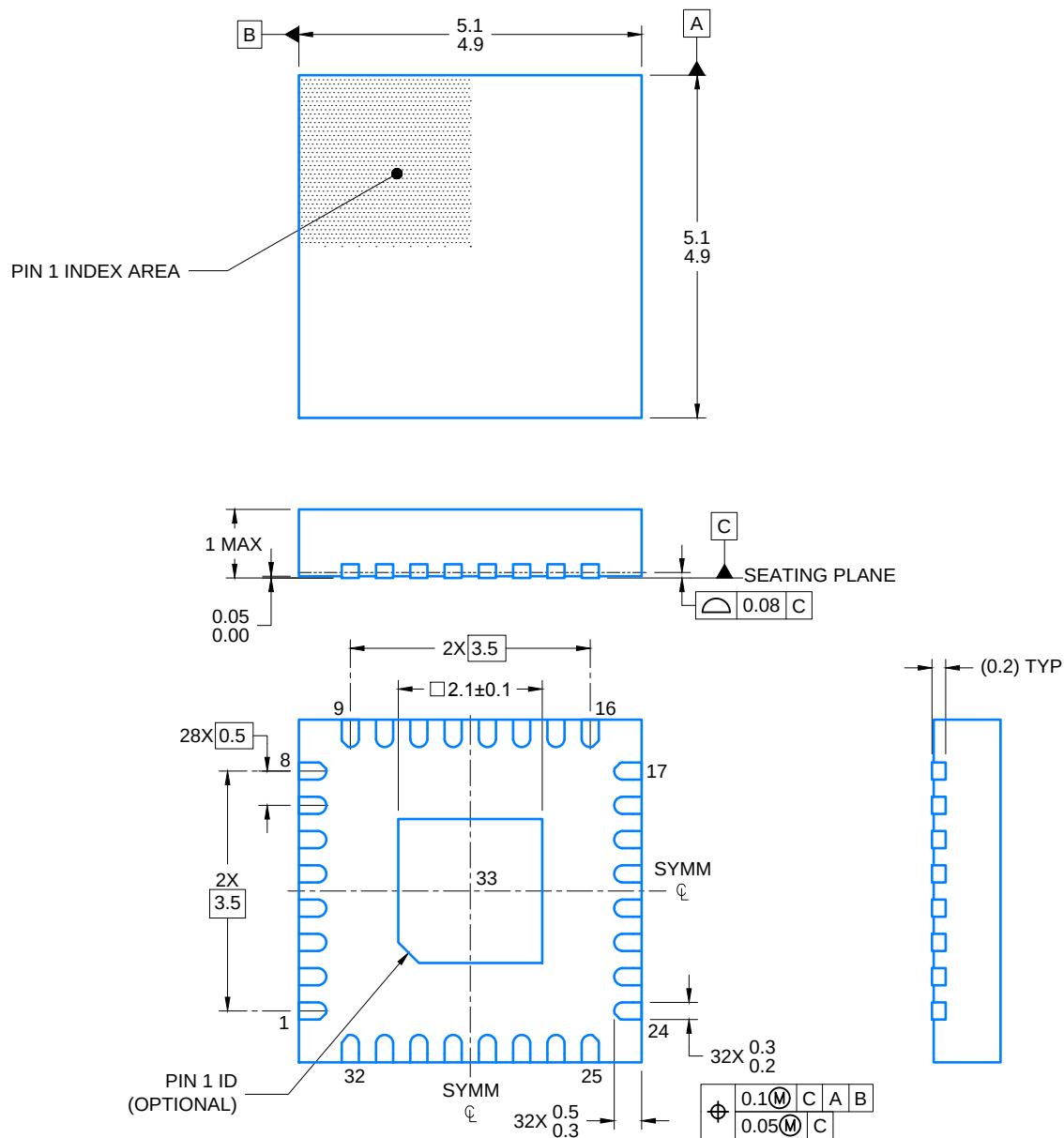
5 x 5, 0.5 mm pitch

PLASTIC QUAD FLATPACK - NO LEAD



Images above are just a representation of the package family, actual package may vary.
Refer to the product data sheet for package details.

4224745/A



4223725/A 08/2017

NOTES:

1. All linear dimensions are in millimeters. Any dimensions in parenthesis are for reference only. Dimensioning and tolerancing per ASME Y14.5M.
2. This drawing is subject to change without notice.
3. The package thermal pad must be soldered to the printed circuit board for optimal thermal and mechanical performance.

VQFN - 1 mm max height

This diagram provides a detailed view of the top surface of the PCB, highlighting various dimensions and component placements. Key features include:

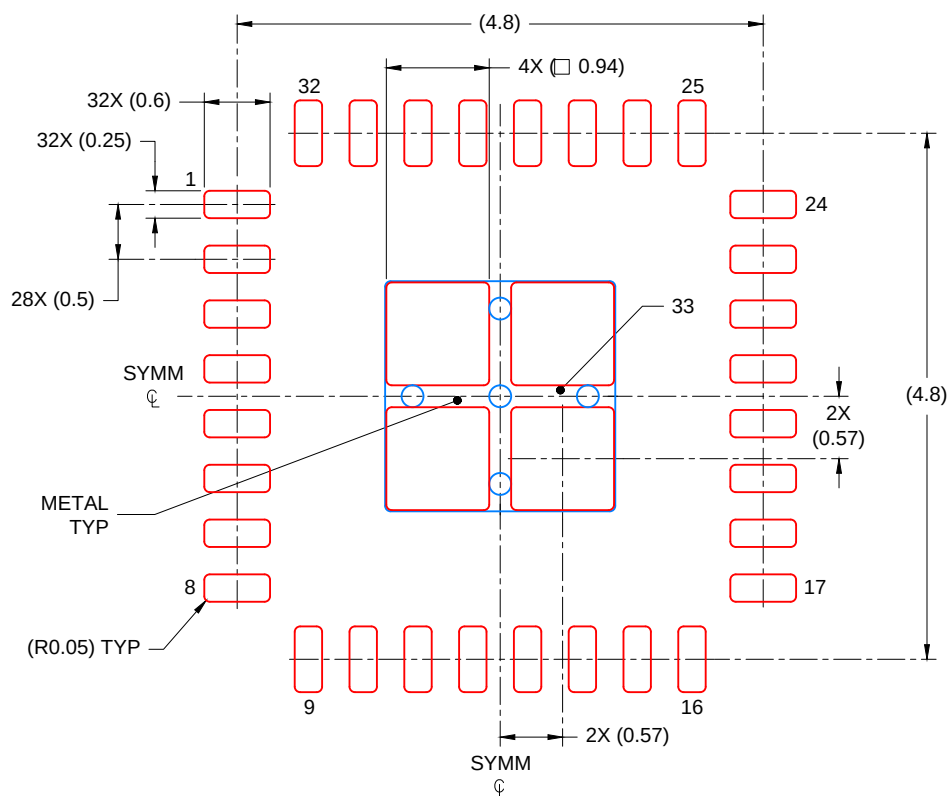
- Dimensions:**
 - A horizontal dimension of (4.8) spans the width of the central area.
 - A vertical dimension of (4.8) spans the height of the central area.
 - A horizontal distance of $32X(0.6)$ is indicated from the left edge to the first column of components.
 - A vertical distance of $32X(0.25)$ is indicated from the top edge to the first row of components.
 - A horizontal distance of $28X(0.5)$ is indicated from the left edge to the second column of components.
 - A horizontal distance of $(\square 2.1)$ is indicated between two columns of components.
 - A horizontal distance of $2X(0.8)$ is indicated at the bottom right.
- Component Locations and Counts:**
 - Top row: 32 components on the left, 25 components on the right.
 - Middle row: 24 components on the right.
 - Bottom row: 9 components on the left, 16 components on the right.
 - Center: A square area containing 33 components, with four specific components highlighted by circles.
- Other Labels:**
 - "SYMM" (Symmetry) is labeled twice, indicating symmetrical layouts.
 - " $(\varnothing 0.2)$ VIA TYP" points to a circular feature.
 - " $(R0.05)$ TYP" points to a corner radius on a component footprint.
 - "1" indicates a small gap or offset.
 - "8" indicates a count or identifier near a component.

The diagram compares two PCB pad designs for a through-hole component:

- NON SOLDER MASK DEFINED (PREFERRED):** This design shows a pad where the metal is exposed. The dimensions are specified as **0.07 MAX ALL AROUND**. Labels include **EXPOSED METAL**, **SOLDER MASK OPENING**, and **METAL**.
- SOLDER MASK DEFINED:** This design shows a pad where the metal is covered by solder mask. The dimensions are specified as **0.07 MIN ALL AROUND**. Labels include **EXPOSED METAL**, **SOLDER MASK OPENING**, and **METAL UNDER SOLDER MASK**.

4223725/A 08/2017

4. This package is designed to be soldered to a thermal pad on the board. For more information, see Texas Instruments literature number SLUA271 (www.ti.com/lit/sl原因271).
5. Vias are optional depending on application, refer to device data sheet. If any vias are implemented, refer to their locations shown on this view. It is recommended that vias under paste be filled, plugged or tented.



SOLDER PASTE EXAMPLE
 BASED ON 0.125 mm THICK STENCIL

EXPOSED PAD
 80% PRINTED COVERAGE BY AREA
 SCALE: 15X

4223725/A 08/2017

NOTES: (continued)

6. Laser cutting apertures with trapezoidal walls and rounded corners may offer better paste release. IPC-7525 may have alternate design recommendations.

IMPORTANT NOTICE AND DISCLAIMER

TI PROVIDES TECHNICAL AND RELIABILITY DATA (INCLUDING DATA SHEETS), DESIGN RESOURCES (INCLUDING REFERENCE DESIGNS), APPLICATION OR OTHER DESIGN ADVICE, WEB TOOLS, SAFETY INFORMATION, AND OTHER RESOURCES "AS IS" AND WITH ALL FAULTS, AND DISCLAIMS ALL WARRANTIES, EXPRESS AND IMPLIED, INCLUDING WITHOUT LIMITATION ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT OF THIRD PARTY INTELLECTUAL PROPERTY RIGHTS.

These resources are intended for skilled developers designing with TI products. You are solely responsible for (1) selecting the appropriate TI products for your application, (2) designing, validating and testing your application, and (3) ensuring your application meets applicable standards, and any other safety, security, regulatory or other requirements.

These resources are subject to change without notice. TI grants you permission to use these resources only for development of an application that uses the TI products described in the resource. Other reproduction and display of these resources is prohibited. No license is granted to any other TI intellectual property right or to any third party intellectual property right. TI disclaims responsibility for, and you will fully indemnify TI and its representatives against, any claims, damages, costs, losses, and liabilities arising out of your use of these resources.

TI's products are provided subject to [TI's Terms of Sale](#) or other applicable terms available either on [ti.com](https://www.ti.com) or provided in conjunction with such TI products. TI's provision of these resources does not expand or otherwise alter TI's applicable warranties or warranty disclaimers for TI products.

TI objects to and rejects any additional or different terms you may have proposed.

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2025, Texas Instruments Incorporated

app\adc_ads124s08.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      adc_ads124s08.py
4  @brief     Driver pour l'ADC ADS124S08 avec communication SPI.
5  @details   Ce module fournit une interface complète pour la communication avec
6             l'ADC ADS124S08 via SPI en mode 1.0. Supporte la mesure de résistance
7             et température avec configuration automatique des registres.
8             Séquence: Reset → Configuration → Mesure bloquante avec DRDY.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 from __future__ import annotations
18 # Bibliothèque pour la communication SPI
19 import spidev
20 # Module pour la gestion du temps et délais
21 import time
22 # Bibliothèque pour la gestion des GPIO
23 from periphery import GPIO
24 # Gestion des signaux système pour interruption propre
25 import signal
26 # Gestion des threads pour les opérations non-bloquantes
27 import threading
28 # Importation des constantes matérielles CMS
29 from pins_cm5 import SPI1_BUS, SPI1_DEV0, SPI_ADC_SPEED_HZ, GPIO_CHIP_PATH, ADC_DRDY
30 # Fonction de conversion résistance vers température
31 from app.temperature_conversion import resistance_to_temperature_dynamic
32 # Tables de conversion des différents capteurs
33 from app.sensor_profiles import SENSOR_TABLES
34
35 # -----
36 # Variables globales pour la gestion d'interruption
37 # -----
38
39 # Flag global pour indiquer une demande d'arrêt
40 _stop_requested = False
41 # Lock pour la synchronisation thread-safe
42 _stop_lock = threading.Lock()
43
44 def request_stop():
45     """
46     @brief Demande l'arrêt global du programme.
47     @details Active le flag global d'arrêt de manière thread-safe.
48     """
49     global _stop_requested
50     with _stop_lock:
51         _stop_requested = True
52         print("[ADC] Arrêt demandé - interruption des opérations en cours...")
53
54 def is_stop_requested():
55     """
56     @brief Vérifie si un arrêt a été demandé.
57     @details Retourne l'état du flag d'arrêt de manière thread-safe.
58     @return True si arrêt demandé, False sinon.
59     """
60     global _stop_requested
61     with _stop_lock:
62         return _stop_requested
63
64 def reset_stop_flag():
65     """
66     @brief Remet à zéro le flag d'arrêt.
67     @details Permet de redémarrer les opérations après un arrêt.
68     """
69     global _stop_requested
70     with _stop_lock:
71         _stop_requested = False
72
73 # Gestionnaire de signal pour Ctrl+C
74 def signal_handler(signum, frame):
75     """
76     @brief Gestionnaire pour les signaux système (Ctrl+C).
77     @details Capture SIGINT et SIGTERM pour arrêter proprement le programme.
78     """
79     print(f"\n[Signal] Signal {signum} reçu - arrêt en cours...")
80     request_stop()
81
82 # Installation du gestionnaire de signal
83 signal.signal(signal.SIGINT, signal_handler)
84 signal.signal(signal.SIGTERM, signal_handler)

```

```

85
86
87 # -----
88 # Constantes des registres ADC ADS124S08
89 # -----
90
91 # Registre d'identification de l'ADC
92 REG_ID      = 0x00
93 # Registre de statut et flags
94 REG_STATUS  = 0x01
95 # Registre de configuration du multiplexeur d'entrée
96 REG_INPMUX  = 0x02
97 # Registre de configuration de l'amplificateur programmable
98 REG_PGA     = 0x03
99 # Registre de configuration du taux d'échantillonnage
100 REG_DATARATE = 0x04
101 # Registre de configuration de la référence de tension
102 REG_REF     = 0x05
103 # Registre de configuration du multiplexeur IDAC
104 REG_IDACMUX  = 0x07
105 # Registre de configuration système générale
106 REG_SYS     = 0x09
107 # Registre de configuration de la magnitude IDAC
108 REG_IDACMAG  = 0x06
109
110
111 # -----
112 # Constantes des commandes ADC ADS124S08
113 # -----
114
115 # Commande de réinitialisation complète de l'ADC
116 CMD_RESET   = 0x06
117 # Commande de démarrage d'une conversion
118 CMD_START   = 0x08
119 # Commande d'arrêt de conversion
120 CMD_STOP    = 0x0A
121 # Commande de lecture des données de conversion
122 CMD_RDATA   = 0x12
123
124
125 # -----
126 # Constantes de calcul et mapping
127 # -----
128
129 # Valeur maximale pour un code 24 bits signé (pleine échelle)
130 FS = (1 << 23) - 1
131
132 # Mapping des canaux physiques vers les entrées analogiques de l'ADC
133 CHANNELS = {
134     # Canal 1: entrée positive AIN1, négative AIN2
135     1: {"ainp_idx": 1, "ainn_idx": 2},
136     # Canal 2: entrée positive AIN4, négative AIN5
137     2: {"ainp_idx": 4, "ainn_idx": 5},
138     # Canal 3: entrée positive AIN7, négative AIN8
139     3: {"ainp_idx": 7, "ainn_idx": 8},
140     # Canal 4: entrée positive AIN10, négative AIN11
141     4: {"ainp_idx": 10, "ainn_idx": 11}
142 }
143
144
145 # -----
146 # Fonctions d'encodage des paramètres ADC
147 # -----
148
149 def encode_gain(pga_gain):
150     """
151     @brief      Encode le gain PGA en code binaire pour le registre.
152     @details    Convertit une valeur de gain décimale en code binaire 3 bits
153                 pour la configuration du registre PGA de l'ADS124S08.
154
155     @param pga_gain Valeur du gain PGA (1, 2, 4, 8, 16, 32, 64, 128).
156
157     @return     Code binaire 3 bits correspondant au gain.
158
159     @exception  ValueError si le gain fourni n'est pas supporté.
160     @see        configure_channel
161     """
162     # Gain unitaire (0 dB)
163     if pga_gain == 1:
164         return 0
165     # Gain x2 (6 dB)
166     if pga_gain == 2:
167         return 1
168     # Gain x4 (12 dB)
169     if pga_gain == 4:
170         return 2
171     # Gain x8 (18 dB)

```

```

172     if pga_gain == 8:
173         return 3
174     # Gain x16 (24 dB)
175     if pga_gain == 16:
176         return 4
177     # Gain x32 (30 dB)
178     if pga_gain == 32:
179         return 5
180     # Gain x64 (36 dB)
181     if pga_gain == 64:
182         return 6
183     # Gain x128 (42 dB)
184     if pga_gain == 128:
185         return 7
186     # Erreur si gain non supporté
187     raise ValueError("PGA gain invalide: " + str(pga_gain))
188
189
190 def encode_idac_uA(idac_uA):
191     """
192     @brief      Encode la magnitude IDAC en code binaire pour le registre.
193     @details    Convertit une valeur de courant IDAC en microampères en code
194                 binaire 4 bits pour la configuration du registre IDACMAG.
195
196     @param idac_uA    Courant IDAC en microampères (10, 50, 100, 250, 500, 1000, 1500, 2000).
197
198     @return        Code binaire 4 bits correspondant au courant.
199
200     @exception    ValueError si le courant fourni n'est pas supporté.
201     @see          configure_channel
202     """
203     # Courant IDAC de 10 µA
204     if idac_uA == 10:
205         return 1
206     # Courant IDAC de 50 µA
207     if idac_uA == 50:
208         return 2
209     # Courant IDAC de 100 µA
210     if idac_uA == 100:
211         return 3
212     # Courant IDAC de 250 µA
213     if idac_uA == 250:
214         return 4
215     # Courant IDAC de 500 µA
216     if idac_uA == 500:
217         return 5
218     # Courant IDAC de 1000 µA (1 mA)
219     if idac_uA == 1000:
220         return 6
221     # Courant IDAC de 1500 µA (1.5 mA)
222     if idac_uA == 1500:
223         return 7
224     # Courant IDAC de 2000 µA (2 mA)
225     if idac_uA == 2000:
226         return 8
227     # Erreur si courant non supporté
228     raise ValueError("IDAC µA non supporté: " + str(idac_uA))
229
230
231 def sign_extend_24(b0, b1, b2):
232     """
233     @brief      Étend le signe d'un code ADC 24 bits vers 32 bits.
234     @details    Convertit un code ADC 24 bits signé en entier Python 32 bits
235                 en préservant le signe selon la représentation en complément à 2.
236
237     @param b0    Octet de poids fort (MSB) du code 24 bits.
238     @param b1    Octet du milieu du code 24 bits.
239     @param b2    Octet de poids faible (LSB) du code 24 bits.
240
241     @return      Valeur entière signée 32 bits correspondante.
242
243     @note        Nécessaire car Python gère automatiquement les entiers de taille variable.
244     @see         read_code24
245     """
246     # Reconstruction du mot 24 bits
247     raw = (b0 << 16) | (b1 << 8) | b2
248     # Test du bit de signe (bit 23)
249     if (raw & 0x800000) != 0:
250         # Extension du signe vers 32 bits
251         value = raw | 0xFF000000
252         # Conversion en entier signé Python
253         value = value - (1 << 32)
254         return value
255     # Valeur positive, pas d'extension nécessaire
256     return raw
257
258

```

```

259 # -----
260 # Classe principale du driver ADC
261 # -----
262
263 class Ads124s08:
264     """
265     @brief      Driver pour l'ADC ADS124S08 avec interface SPI.
266     @details    Cette classe encapsule toutes les fonctionnalités nécessaires
267                 pour communiquer avec l'ADC ADS124S08 via SPI en mode 1.0.
268                 Supporte la configuration automatique, la mesure de résistance
269                 et la conversion en température.
270
271     @note       L'ADC doit être connecté via SPI1 et le signal DRDY sur GPIO.
272     @see        configure_channel, measure_resistance, measure_temperature
273     """
274
275     def __init__(self):
276         """
277         @brief      Constructeur de la classe Ads124s08.
278         @details    Initialise la communication SPI, configure le GPIO DRDY,
279                 effectue un reset de l'ADC et vérifie la communication
280                 en lisant le registre d'identification.
281
282         @exception  Peut lever des exceptions SPI ou GPIO en cas d'échec.
283         @post       L'ADC est prêt pour la configuration et les mesures.
284         """
285         # Initialisation de l'interface SPI avec les paramètres du CM5
286         # Création de l'objet SPI
287         self.spi = spidev.SpiDev()
288         # Ouverture du bus SPI1, device 0
289         self.spi.open(SPI1_BUS, SPI1_DEV0)
290         # Configuration en mode SPI 1.0 (CPOL=0, CPHA=1)
291         self.spi.mode = 1
292         # Configuration de la vitesse SPI selon CM5
293         self.spi.max_speed_hz = SPI_ADC_SPEED_HZ
294         # Configuration de la taille des mots SPI à 8 bits
295         self.spi.bits_per_word = 8
296
297         # Initialisation du GPIO pour le signal DRDY (Data Ready)
298         # Configuration du GPIO DRDY en entrée
299         self.gpio_drdy = GPIO(GPIO_CHIP_PATH, ADC_DRDY, "in")
300
301         # Séquence de réinitialisation de l'ADC
302         # Envoi de la commande RESET via SPI
303         self.spi.xfer2([CMD_RESET])
304         # Délai d'attente post-reset (2 ms minimum)
305         time.sleep(0.002)
306
307         # Purge des flags de statut après reset
308         # Effacement des flags du registre STATUS
309         self._wreg(REG_STATUS, [0x00])
310
311         # Vérification de la communication avec l'ADC via lecture du registre ID
312         # Lecture du registre d'identification
313         adc_id = self.read_id()
314         # Test de validité de la réponse
315         if adc_id is None:
316             # Erreur de communication
317             print("[ADC] Erreur: aucune réponse sur le registre ID (0x00)")
318         else:
319             # Affichage de l'ID pour vérification
320             print("[ADC] ID (0x00) = 0x" + format(adc_id, "02X"))
321
322     def read_id(self):
323         """
324         @brief      Lit le registre d'identification de l'ADC.
325         @details    Effectue une lecture du registre ID (0x00) pour vérifier
326                 la communication SPI et identifier le type d'ADC connecté.
327
328         @return     Valeur du registre ID (0x00-0xFF) ou None en cas d'erreur.
329
330         @note       Utilisé principalement pour la vérification de communication.
331         @see        __init__
332         """
333         # Commande RREG pour registre 0x00, 1 byte
334         rx = self.spi.xfer2([0x20, 0x00, 0x00])
335         # Vérification de la longueur de réponse
336         if len(rx) >= 3:
337             # Retour de la valeur du registre ID
338             return rx[2]
339         # Erreur de communication SPI
340         return None
341
342     def close(self):
343         """
344         @brief      Ferme les interfaces SPI et GPIO proprement.
345         @details    Libère les ressources système utilisées par le driver.

```

```
346             Méthode à appeler avant la destruction de l'objet.
347
348     @return      Rien.
349
350     @note        Ignore les erreurs de fermeture pour éviter les exceptions.
351     @see         __init__
352     """
353     try:
354         # Fermeture de l'interface SPI
355         self.spi.close()
356     # Ignore les erreurs de fermeture SPI
357     except Exception:
358         pass
359     try:
360         # Fermeture de l'interface GPIO DRDY
361         self.gpio_drdy.close()
362     # Ignore les erreurs de fermeture GPIO
363     except Exception:
364         pass
365
366 def _rreg(self, addr, nbytes):
367     """
368     @brief        Lit un ou plusieurs registres de l'ADC via SPI.
369     @details      Utilise la commande RREG de l'ADS124S08 pour lire
370                   des registres consécutifs starting à l'adresse donnée.
371
372     @param addr   Adresse du premier registre à lire (0x00-0x1F).
373     @param nbytes Nombre d'octets à lire.
374
375     @return       Liste des valeurs lues depuis les registres.
376
377     @note         Méthode privée pour les opérations internes du driver.
378     @see          _wreg
379     """
380     # Construction de la commande RREG
381     cmd = 0x20 | (addr & 0x1F)
382     # Calcul du compteur (nbytes - 1)
383     count = nbytes - 1
384     # Initialisation du buffer de transmission
385     tx = [cmd, count]
386     # Index de boucle pour les octets dummy
387     index = 0
388     # Ajout des octets dummy pour la lecture
389     while index < nbytes:
390         # Octet dummy pour chaque byte à lire
391         tx.append(0x00)
392         # Incréméntation de l'index
393         index = index + 1
394     # Transmission SPI bidirectionnelle
395     rx = self.spi.xfer2(tx)
396     # Extraction des données (après cmd et count)
397     data = rx[2:]
398     # Retour des données lues
399     return data
400
401 def _wreg(self, addr, data_bytes):
402     """
403     @brief        Écrit dans un ou plusieurs registres de l'ADC via SPI.
404     @details      Utilise la commande WREG de l'ADS124S08 pour écrire
405                   des données dans des registres consécutifs.
406
407     @param addr   Adresse du premier registre à écrire (0x00-0x1F).
408     @param data_bytes Liste des octets à écrire dans les registres.
409
410     @return       Rien.
411
412     @note         Méthode privée pour les opérations internes du driver.
413     @see          _rreg
414     """
415     # Construction de la commande WREG
416     cmd = 0x40 | (addr & 0x1F)
417     # Calcul du compteur (nbytes - 1)
418     count = len(data_bytes) - 1
419     # Initialisation du buffer de transmission
420     tx = [cmd, count]
421     # Ajout des données à transmettre
422     tx = tx + list(data_bytes)
423     # Transmission SPI (pas de données retour)
424     self.spi.xfer2(tx)
425
426 def _sclk_nudge(self):
427     """
428     @brief        Envoie un coup de pouce sur SCLK pour synchroniser DRDY.
429     @details      Effectue une lecture factice du registre STATUS pour générer
430                   des transitions sur SCLK et forcer DRDY à l'état HIGH.
431
432     @return       Rien.
```

```

433
434     @note          Utilisé quand DRDY reste bloqué à LOW de manière inattendue.
435     @see           _ensure_drdy_high
436     """
437     # Lecture factice pour générer des transitions SCLK
438     _ = self._rreg(REG_STATUS, 1)
439     # Délai court pour la stabilisation
440     time.sleep(0.0001)
441
442 def _ensure_drdy_high(self, timeout_ms):
443     """
444     @brief          Assure que le signal DRDY est à l'état HIGH.
445     @details        Attend que DRDY passe à HIGH avec possibilité de coup de pouce
446                     sur SCLK si nécessaire. Timeout configurable.
447
448     @param timeout_ms Timeout en millisecondes pour l'attente.
449
450     @return          True si DRDY est HIGH, False en cas de timeout.
451
452     @note           Méthode critique pour la synchronisation des conversions.
453     @see            wait_drdy_falling_edge
454     """
455     # Timestamp de début d'attente
456     t0 = time.time()
457     # Flag pour éviter les multiples coups de pouce
458     kicked = False
459     # Boucle d'attente infinie avec timeout
460     while True:
461         # Lecture de l'état actuel de DRDY
462         val = self.gpio_drdy.read()
463         # Test si DRDY est passé à HIGH
464         if val is True:
465             # Succès: DRDY est HIGH
466             return True
467         # Premier passage avec DRDY LOW
468         if kicked is False:
469             # Coup de pouce sur SCLK
470             self._sclk_nudge()
471             # Marquer le coup de pouce comme effectué
472             kicked = True
473         # Test du timeout en secondes
474         if time.time() - t0 > float(timeout_ms) / 1000.0:
475             # Échec: timeout atteint
476             return False
477         # Délai court entre les lectures
478         time.sleep(0.001)
479
480 def wait_drdy_falling_edge(self, timeout_s):
481     """
482     @brief          Attend un front descendant sur DRDY (HIGH -> LOW) avec timeout.
483     @details        Surveille le signal DRDY pour détecter une transition de HIGH
484                     vers LOW, indiquant qu'une nouvelle conversion est disponible.
485                     Peut être interrompu par Ctrl+C ou signal d'arrêt.
486
487     @param timeout_s Timeout en secondes pour l'attente du front.
488
489     @return          True si front descendant détecté, False en cas de timeout ou interruption.
490
491     @note           Méthode cruciale pour la synchronisation des mesures.
492     @see            measure_resistance
493     """
494     # Timestamp de début d'attente
495     t0 = time.time()
496     # État initial de DRDY
497     prev = self.gpio_drdy.read()
498
499     # Affichage de l'état initial pour diagnostic
500     print(f"[ADC] Attente front descendant DRDY, état initial: {prev}, timeout: {timeout_s}s")
501     print("[ADC] Appuyez sur Ctrl+C pour interrompre l'attente...")
502
503     # Compteur pour les messages de debug périodiques
504     debug_counter = 0
505
506     # Boucle d'attente infinie avec timeout
507     while True:
508         # Vérification du flag d'arrêt global
509         if is_stop_requested():
510             print("[ADC] Interruption demandée - arrêt de l'attente DRDY")
511             return False
512
513         # Lecture de l'état actuel de DRDY
514         val = self.gpio_drdy.read()
515
516         # Affichage périodique pour diagnostic (toutes les 500 itérations)
517         debug_counter += 1
518         if debug_counter % 500 == 0:
519             elapsed = time.time() - t0

```

```

520         print(f"[ADC] Attente DRDY... État: {val}, Temps écoulé: {elapsed:.1f}s (Ctrl+C pour arrêter)")
521
522     # Détection du front descendant
523     if prev is True and val is False:
524         # Message de confirmation
525         print("[ADC] Front descendant DRDY détecté")
526         # Succès: front descendant détecté
527         return True
528     # Mémorisation de l'état précédent
529     prev = val
530     # Test du timeout
531     if time.time() - t0 > float(timeout_s):
532         # Message d'erreur timeout avec état final
533         print(f"[ADC] Timeout DRDY front descendant ! État final: {val}")
534         print("[ADC] Vérifiez: connexion DRDY, configuration ADC, horloge SPI")
535         # Échec: timeout atteint
536         return False
537     # Délai court entre les lectures
538     time.sleep(0.001)
539
540 def set_single_shot_lowlatency(self, dr_nibble):
541     """
542     @brief      Configure le mode single-shot low-latency de l'ADC.
543     @details    Configure le registre DATARATE pour utiliser le mode
544                 single-shot avec filtre low-latency et data rate spécifié.
545
546     @param dr_nibble  Code de data rate (4 bits, 0x00-0x0F).
547
548     @return        Rien.
549
550     @note          Mode optimisé pour des mesures rapides et ponctuelles.
551     @see           configure_channel
552     """
553     # Initialisation de la valeur du registre
554     value = 0
555     # MODE=1 pour single-shot
556     value = value | (1 << 5)
557     # FILTER=1 pour low-latency
558     value = value | (1 << 4)
559     # DR=data rate (4 bits bas)
560     value = value | (dr_nibble & 0x0F)
561     # Écriture dans le registre DATARATE
562     self._wreg(REG_DATARATE, [value])
563
564 def configure_channel(self, channel_index, pga_gain, idac_uA):
565     """
566     @brief      Configure un canal ADC avec ses paramètres de mesure.
567     @details    Configure complètement un canal ADC avec le gain PGA,
568                 le courant IDAC et le routage des entrées analogiques.
569                 Effectue une configuration séquentielle de tous les registres.
570
571     @param channel_index  Index du canal physique (1-4).
572     @param pga_gain       Gain de l'amplificateur programmable (1-128).
573     @param idac_uA        Courant IDAC en microampères (10-2000).
574
575     @return        Rien.
576
577     @exception      ValueError si le canal ou les paramètres sont invalides.
578     @see           encode_gain, encode_idac_uA
579     """
580     # Validation du canal demandé
581     # Test de validité du canal
582     if channel_index not in CHANNELS:
583         # Exception si canal invalide
584         raise ValueError("Canal ADC inconnu: " + str(channel_index))
585     # Récupération des paramètres du canal
586     ch = CHANNELS[channel_index]
587     # Index de l'entrée positive
588     ainp = ch["ainp_idx"]
589     # Index de l'entrée négative
590     ainm = ch["ainm_idx"]
591     print("[ADC] Configuration canal " + str(channel_index) + " gain=" + str(pga_gain) + " IDAC=" + str(idac_uA) + "uA")
592
593     # Configuration du multiplexeur d'entrée (INPMUX)
594     # Construction de la valeur INPMUX
595     inpmux_val = ((ainp & 0x0F) << 4) | (ainm & 0x0F)
596     # Affichage pour debug
597     print("[ADC] INPMUX=0x" + format(inpmux_val, "02X"))
598     # Écriture dans le registre INPMUX
599     self._wreg(REG_INPMUX, [inpmux_val])
600
601     # Configuration de l'amplificateur programmable (PGA)
602     # Encodage du gain en code binaire
603     gain_code = encode_gain(pga_gain)
604     # Initialisation de la valeur PGA
605     pga_val = 0
606     # PGA_EN = 1 (activation du PGA)

```

```

607     pga_val = pga_val | (1 << 3)
608     # GAIN = code du gain (3 bits)
609     pga_val = pga_val | (gain_code & 0x07)
610     # Affichage pour debug
611     print("[ADC] PGA=0x" + format(pga_val, "02X"))
612     # Écriture dans le registre PGA
613     self._wreg(REG_PGA, [pga_val])
614     # Lecture de vérification
615     val = self._rreg(REG_PGA, 1)
616     # Affichage de la valeur lue
617     print("[ADC DEBUG] PGA readback: 0x" + format(val[0], "02X"))
618
619     # Configuration du mode de conversion (single-shot low-latency)
620     # DR=0x04 pour data rate optimisé
621     self.set_single_shot_lowlatency(0x04)
622
623     # Configuration de la référence de tension (REF)
624     # REFSEL=00 (REFP0/REFN0), REFCON=10 (ref interne ON)
625     print("[ADC] REF=0x12")
626     # Écriture dans le registre REF
627     self._wreg(REG_REF, [0x12])
628     # Attente pour stabilisation de la référence interne
629     time.sleep(0.006)
630     # Lecture de vérification
631     val = self._rreg(REG_REF, 1)
632     # Affichage de la valeur lue
633     print("[ADC DEBUG] REF readback: 0x" + format(val[0], "02X"))
634
635     # Configuration de la magnitude IDAC
636     # Encodage du courant IDAC
637     mag_code = encode_idac_uA(idac_uA)
638     # Affichage pour debug
639     print("[ADC] IDACMAG=0x" + format(mag_code & 0x0F, "02X"))
640     # Écriture dans le registre IDACMAG
641     self._wreg(REG_IDACMAG, [mag_code & 0x0F])
642     # Lecture de vérification
643     val = self._rreg(REG_IDACMAG, 1)
644     # Affichage de la valeur lue
645     print("[ADC DEBUG] IDACMAG readback: 0x" + format(val[0], "02X"))
646
647     # Configuration du routage IDAC (IDACMUX)
648     # Calcul du routage IDAC1: 0, 3, 6, 9
649     idac1_route = 0 + (channel_index - 1) * 3
650     # Limitation basse du routage
651     if idac1_route < 0:
652         idac1_route = 0
653     # Limitation haute du routage
654     if idac1_route > 15:
655         idac1_route = 15
656     # IDAC2 déconnecté (0x0F)
657     idac2_route = 0x0F
658     # Construction de la valeur IDACMUX
659     idacmux_val = ((idac2_route & 0x0F) << 4) | (idac1_route & 0x0F)
660     # Affichage pour debug
661     print("[ADC] IDACMUX=0x" + format(idacmux_val, "02X"))
662     # Écriture dans le registre IDACMUX
663     self._wreg(REG_IDACMUX, [idacmux_val])
664     # Lecture de vérification
665     val = self._rreg(REG_IDACMUX, 1)
666     # Affichage de la valeur lue
667     print("[ADC DEBUG] IDACMUX readback: 0x" + format(val[0], "02X"))
668
669     # Délai de stabilisation après configuration complète
670     # Attente pour stabilisation des paramètres
671     time.sleep(0.001)
672
673     # Lecture de tous les registres clés pour debug final
674     reg_map = {
675         "INPMUX": REG_INPMUX,
676         "PGA": REG_PGA,
677         "DATARATE": REG_DATARATE,
678         "REF": REG_REF,
679         "IDACMAG": REG_IDACMAG,
680         "IDACMUX": REG_IDACMUX
681     }
682     # Parcours de tous les registres importants
683     for name, addr in reg_map.items():
684         # Lecture de chaque registre
685         val = self._rreg(addr, 1)
686         # Affichage de l'état final
687         print(f"[ADC] {name} {0x{addr:02X}} = 0x{val[0]:02X}")
688
689     def start(self):
690         """
691         @brief      Démarre une conversion ADC.
692         @details    Assure que DRDY est à HIGH puis envoie la commande START
693                     pour initier une nouvelle conversion sur le canal configuré.

```



```

694         Peut être interrompu par Ctrl+C ou signal d'arrêt.
695
696     @return      True si START envoyé avec succès, False en cas d'erreur ou interruption.
697
698     @note        DRDY doit être HIGH avant le START pour une conversion valide.
699     @see         stop, wait_drdy_falling_edge
700     """
701     # Attente que DRDY remonte à HIGH avant le démarrage
702     # Timestamp de début d'attente
703     t0 = time.time()
704     print("[ADC] Attente que DRDY soit HIGH avant START... (Ctrl+C pour arrêter)")
705
706     # Boucle tant que DRDY n'est pas HIGH
707     while self.gpio_drdy.read() is not True:
708         # Vérification du flag d'arrêt global
709         if is_stop_requested():
710             print("[ADC] Interruption demandée - arrêt de l'attente DRDY HIGH")
711             return False
712
713         # Lecture du registre STATUS pour kick SCLK
714         self._rreg(REG_STATUS, 1)
715         # Délai court entre les tentatives
716         time.sleep(0.001)
717         # Timeout de 5 secondes (augmenté pour diagnostic)
718         if time.time() - t0 > 5.0:
719             # Message d'erreur détaillé
720             print("[ADC] ERREUR: DRDY n'est pas remonté HIGH avant START !")
721             print("[ADC] Vérifiez la connexion DRDY et l'alimentation ADC")
722             # Retour d'erreur sans envoyer START
723             return False
724
725     # Vérification finale du flag d'arrêt avant d'envoyer START
726     if is_stop_requested():
727         print("[ADC] Interruption demandée - annulation de START")
728         return False
729
730     # DRDY est maintenant HIGH, on peut envoyer START
731     print("[ADC] DRDY HIGH détecté, envoi de la commande START")
732     # Envoi de la commande START pour débiter la conversion
733     # Transmission de la commande START via SPI
734     self.spi.xfer2([CMD_START])
735     return True
736
737 def stop(self):
738     """
739     @brief        Arrête une conversion ADC en cours.
740     @details      Envoie la commande STOP pour interrompre toute conversion
741                   en cours et remettre l'ADC en état idle.
742
743     @return       Rien.
744
745     @note         Utilisé après lecture des données ou en cas d'erreur.
746     @see          start, read_code24
747     """
748     # Transmission de la commande STOP via SPI
749     self.spi.xfer2([CMD_STOP])
750
751 def read_code24(self):
752     """
753     @brief        Lit un code de données 24 bits depuis l'ADC.
754     @details      Utilise la commande RDATA pour lire les 3 octets de données
755                   de conversion et les convertit en entier signé.
756
757     @return       Valeur signée 24 bits ou None en cas d'erreur SPI.
758
759     @note         Doit être appelé après détection du front descendant DRDY.
760     @see          sign_extend_24, measure_resistance
761     """
762     # Commande RDATA + 3 octets dummy
763     rx = self.spi.xfer2([CMD_RDATA, 0x00, 0x00, 0x00])
764     # Vérification de la longueur de réponse
765     if len(rx) < 4:
766         # Message d'erreur
767         print("[ADC] Erreur: réponse SPI trop courte " + str(rx))
768         # Retour d'erreur
769         return None
770     # Octet de poids fort (MSB)
771     b0 = rx[1]
772     # Octet du milieu
773     b1 = rx[2]
774     # Octet de poids faible (LSB)
775     b2 = rx[3]
776     # Extension du signe vers 32 bits
777     value = sign_extend_24(b0, b1, b2)
778     # Retour de la valeur signée
779     return value
780

```

```

781 def measure_resistance(self, rref_ohm, pga_gain, timeout_s):
782     """
783     @brief      Mesure la résistance d'une sonde via l'ADC.
784     @details    Effectue une mesure complète: démarrage de conversion,
785                 attente DRDY, lecture du code et calcul de la résistance
786                 selon la formule ratiométrique.
787
788     @param rref_ohm  Résistance de référence en ohms.
789     @param pga_gain  Gain PGA utilisé pour la mesure.
790     @param timeout_s Timeout en secondes pour la conversion.
791
792     @return        Résistance mesurée en ohms ou None en cas d'erreur.
793
794     @note          Utilise la formule:  $R = |code| / FS * (Rref / gain)$ .
795     @see           measure_temperature, configure_channel
796     """
797     print("[ADC] Mesure résistance: rref=" + str(rref_ohm) + " gain=" + str(pga_gain) + " timeout=" + str(timeout_s))
798
799     # Démarrage de la conversion ADC
800     # Envoi de la commande START
801     start_success = self.start()
802     if not start_success:
803         print("[ADC] Erreur: Impossible de démarrer la conversion, DRDY non disponible")
804         return None
805
806     # Attente du signal DRDY indiquant la fin de conversion
807     # Attente du front descendant DRDY
808     ok = self.wait_drdy_falling_edge(timeout_s)
809     # Test du résultat de l'attente
810     if ok is False:
811         # Arrêt de la conversion en cas de timeout
812         self.stop()
813         # Message d'erreur
814         print("[ADC] Erreur: DRDY non détecté, mesure annulée")
815         # Retour d'erreur
816         return None
817
818     # Lecture du code de conversion 24 bits
819     # Lecture des données de conversion
820     code = self.read_code24()
821     # Arrêt de la conversion
822     self.stop()
823
824     # Validation du code lu
825     # Test de validité du code
826     if code is None:
827         # Message d'erreur
828         print("[ADC] Erreur: code ADC non lu")
829         # Retour d'erreur
830         return None
831
832     # Affichage des paramètres de debug
833     # Affichage du code brut
834     print(f"[ADC DEBUG] Code brut: {code}")
835     # Affichage de la pleine échelle
836     print(f"[ADC DEBUG] FS: {FS}")
837     # Affichage des paramètres de calcul
838     print(f"[ADC DEBUG] Rref: {rref_ohm} / Gain: {pga_gain}")
839
840     # Prise de la valeur absolue du code pour le calcul
841     # Test du signe du code
842     if code < 0:
843         # Prise de la valeur absolue
844         code = -code
845
846     # Calcul de la résistance selon la formule ratiométrique
847     # Calcul du ratio code/pleine échelle
848     ratio = float(code) / float(FS)
849     # Calcul de la résistance de la sonde
850     r_sonde = ratio * (float(rref_ohm) / float(pga_gain))
851
852     # Affichage du résultat
853     print(f"[ADC] Résistance mesurée: {r_sonde:.1f} ohms")
854     # Retour de la résistance mesurée
855     return r_sonde
856
857 def measure_temperature(self, sensor_name, rref_ohm, pga_gain, timeout_s):
858     """
859     @brief      Mesure la température d'un capteur via l'ADC.
860     @details    Effectue une mesure de résistance puis utilise les tables
861                 de conversion de capteurs pour obtenir la température
862                 correspondante en degrés Celsius.
863
864     @param sensor_name  Nom du capteur (doit exister dans SENSOR_TABLES).
865     @param rref_ohm      Résistance de référence en ohms.
866     @param pga_gain      Gain PGA utilisé pour la mesure.
867     @param timeout_s     Timeout en secondes pour la conversion.

```

```
868
869     @return          Température mesurée en °C ou None si erreur.
870
871     @exception        Retourne None si capteur inconnu ou hors plage.
872     @see              mesure_resistance, resistance_to_temperature_dynamic
873     """
874     # Mesure de la résistance du capteur
875     # Appel de la mesure de résistance
876     resistance = self.mesure_resistance(rref_ohm, pga_gain, timeout_s)
877     # Test de validité de la mesure
878     if resistance is None:
879         # Message d'erreur
880         print("[ADC] Erreur: Résistance non mesurée ou saturation détectée.")
881         # Retour d'erreur
882         return None
883
884     # Vérification de l'existence de la table de conversion
885     # Test de présence du capteur
886     if sensor_name not in SENSOR_TABLES:
887         # Message d'erreur
888         print(f"[ADC] Erreur: Table de conversion introuvable pour le capteur {sensor_name}.")
889         # Retour d'erreur
890         return None
891
892     # Conversion de la résistance en température
893     # Appel de la conversion
894     temperature = resistance_to_temperature_dynamic(resistance, sensor_name)
895     # Test de validité de la conversion
896     if temperature is None:
897         # Message d'erreur
898         print("[ADC] Erreur: Température non calculable (hors plage de la table).")
899         # Retour d'erreur
900         return None
901
902     # Affichage du résultat
903     print(f"[ADC] Température mesurée: {temperature:.2f} °C")
904     # Retour de la température mesurée
905     return temperature
906
907 def read_gain(self):
908     """
909     @brief            Lit le registre de gain PGA pour diagnostic.
910     @details          Effectue une lecture du registre PGA et affiche sa valeur
911                      pour vérification de la configuration du gain.
912
913     @return           Valeur brute du registre PGA (0x00-0xFF).
914
915     @note             Utilisé principalement pour le debug et la vérification.
916     @see              configure_channel
917     """
918     # Lecture du registre PGA
919     val = self._rreg(REG_PGA, 1)
920     # Affichage de la valeur lue
921     print(f"[ADC DEBUG] PGA Gain Register: 0x{val[0]:02X}")
922     # Retour de la valeur du registre
923     return val[0]
924
925 def read_ref(self):
926     """
927     @brief            Lit le registre de configuration de la référence.
928     @details          Effectue une lecture du registre REF et affiche sa valeur
929                      pour vérification de la configuration de la référence.
930
931     @return           Valeur brute du registre REF (0x00-0xFF).
932
933     @note             Utilisé principalement pour le debug et la vérification.
934     @see              configure_channel
935     """
936     # Lecture du registre REF
937     val = self._rreg(REG_REF, 1)
938     # Affichage de la valeur lue
939     print(f"[ADC DEBUG] Reference Register: 0x{val[0]:02X}")
940     # Retour de la valeur du registre
941     return val[0]
942
943 def read_inpmux(self):
944     """
945     @brief            Lit le registre de configuration du multiplexeur d'entrée.
946     @details          Effectue une lecture du registre INPMUX et affiche sa valeur
947                      pour vérification du routage des entrées analogiques.
948
949     @return           Valeur brute du registre INPMUX (0x00-0xFF).
950
951     @note             Utilisé principalement pour le debug et la vérification.
952     @see              configure_channel
953     """
954     # Lecture du registre INPMUX
```

```
955     val = self._rreg(REG_INPMUX, 1)
956     # Affichage de la valeur lue
957     print(f"[ADC DEBUG] INPMUX Register: 0x{val[0]:02X}")
958     # Retour de la valeur du registre
959     return val[0]
```

webui\webui_server.py

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  @file      webui_server.py
5  @brief     Serveur web pour interface utilisateur de configuration.
6  @details   Interface web Flask monopage pour lecture/écriture de la configuration
7             sensors.json et affichage de l'état last_state.json. Fournit API REST
8             complète pour gestion configuration canaux et monitoring système.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 # Import du module système pour gestion des chemins
18 import os
19 # Import du module JSON pour sérialisation des données
20 import json
21 # Import pour gestion des horodatages
22 from datetime import datetime
23 # Imports Flask pour serveur web et API REST
24 from flask import Flask, request, jsonify, send_from_directory
25
26
27 # -----
28 # Configuration des chemins et constantes
29 # -----
30
31 # Répertoire de base adapté pour Windows - chemin relatif au script
32 BASE_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
33 # Chemin vers le fichier de configuration des capteurs
34 CONFIG_PATH = os.path.join(BASE_DIR, "config_module", "sensors.json")
35 # Chemin vers le fichier d'état du système
36 STATE_PATH = os.path.join(BASE_DIR, "state", "last_state.json")
37 # Répertoire des fichiers statiques de l'interface web
38 WEBUI_DIR = os.path.dirname(__file__)
39
40 # Création de l'instance Flask avec configuration des fichiers statiques
41 app = Flask(__name__, static_folder=WEBUI_DIR, static_url_path='/static')
42
43
44 # -----
45 # Routes de l'interface web
46 # -----
47
48 @app.route("/", methods=["GET"])
49 def serve_index():
50     """
51     @brief   Sert la page principale de l'interface web.
52     @details Route racine qui retourne le fichier index.html depuis
53             le répertoire webui pour affichage de l'interface utilisateur.
54
55     @return  Fichier HTML de l'interface principale.
56     """
57     # Retour du fichier index.html depuis le répertoire webui
58     return send_from_directory(WEBUI_DIR, "index.html")
59
60
61 # -----
62 # API REST pour gestion de la configuration
63 # -----
64
65 @app.route("/api/config", methods=["GET"])
66 def get_config():
67     """
68     @brief   Récupère la configuration actuelle des capteurs.
69     @details Lecture du fichier sensors.json et retour des données
70             de configuration en format JSON. Gère les erreurs de
71             lecture et affiche des informations de débogage.
72
73     @return  Configuration JSON ou message d'erreur avec code HTTP.
74     """
75     # Bloc de traitement avec gestion d'exceptions
76     try:
77         # Log de l'opération de lecture avec chemin complet
78         print(f"[WEBUI] Lecture configuration depuis {CONFIG_PATH}")
79         # Ouverture du fichier de configuration en UTF-8
80         with open(CONFIG_PATH, "r", encoding="utf-8") as f:
81             # Chargement des données JSON depuis le fichier
82             data = json.load(f)
83             # Log du nombre de canaux chargés pour validation
84             print(f"[WEBUI] Configuration chargée: {len(data.get('channels', []))} canaux")
```

```

85         # Retour des données avec code HTTP 200 (succès)
86         return jsonify(data), 200
87     except Exception as e:
88         # Log de l'erreur rencontrée lors de la lecture
89         print(f"[WEBUI] Erreur lecture config: {e}")
90         # Retour d'erreur avec code HTTP 500 (erreur serveur)
91         return jsonify({"error": str(e)}), 500
92
93 @app.route("/api/config", methods=["PUT"])
94 def put_config():
95     """
96     @brief Sauvergarde une nouvelle configuration des capteurs.
97     @details Reçoit des données JSON, valide le contenu et sauvegarde
98             dans sensors.json avec sauvegarde atomique via fichier temporaire.
99             Affiche un résumé des changements principaux.
100
101     @return Statut de sauvegarde ou message d'erreur avec code HTTP.
102     """
103     # Bloc de traitement avec gestion d'exceptions
104     try:
105         # Récupération des données JSON depuis la requête HTTP
106         payload = request.get_json(force=True, silent=False)
107         # Log du nombre de canaux à sauvegarder
108         print(f"[WEBUI] Sauvegarde configuration: {len(payload.get('channels', []))} canaux")
109
110         # Section d'affichage des changements principaux de configuration
111         if 'mac_address' in payload:
112             # Log de la nouvelle adresse MAC si présente
113             print(f"[WEBUI] MAC: {payload['mac_address']}")
114         if 'auto_sequence' in payload:
115             # Log du mode séquence automatique si présent
116             print(f"[WEBUI] Auto séquence: {payload['auto_sequence']}")
117
118         # Filtrage et affichage des canaux activés seulement
119         enabled_channels = [ch for ch in payload.get('channels', []) if ch.get('enabled')]
120         # Log des numéros de canaux activés pour validation
121         print(f"[WEBUI] Canaux activés: {[ch['channel'] for ch in enabled_channels]}")
122
123         # Création du chemin de fichier temporaire pour sauvegarde atomique
124         tmp_path = CONFIG_PATH + ".tmp"
125         # Ouverture du fichier temporaire en mode écriture UTF-8
126         with open(tmp_path, "w", encoding="utf-8") as f:
127             # Écriture des données JSON avec formatage lisible
128             json.dump(payload, f, ensure_ascii=False, indent=2)
129             # Force l'écriture vers le disque (buffer flush)
130             f.flush()
131             # Synchronisation système pour garantir l'écriture physique
132             os.fsync(f.fileno())
133         # Remplacement atomique du fichier de configuration
134         os.replace(tmp_path, CONFIG_PATH)
135
136         # Log de confirmation de sauvegarde réussie
137         print(f"[WEBUI] Configuration sauvegardée dans {CONFIG_PATH}")
138         # Retour de confirmation avec code HTTP 200 (succès)
139         return jsonify({"status": "ok"}), 200
140     except Exception as e:
141         # Log de l'erreur rencontrée lors de la sauvegarde
142         print(f"[WEBUI] Erreur sauvegarde config: {e}")
143         # Retour d'erreur avec code HTTP 400 (requête invalide)
144         return jsonify({"error": str(e)}), 400
145
146 # -----
147 # API REST pour monitoring de l'état système
148 # -----
149
150
151 @app.route("/api/last", methods=["GET"])
152 def get_last():
153     """
154     @brief Récupère le dernier état du système de mesure.
155     @details Lecture du fichier last_state.json contenant les dernières
156             mesures, températures et statuts. Gère le cas où le fichier
157             n'existe pas encore et retourne un état par défaut.
158
159     @return État JSON du système ou structure par défaut si fichier absent.
160     """
161     # Bloc de traitement avec gestion spécifique des exceptions
162     try:
163         # Ouverture du fichier d'état en mode lecture UTF-8
164         with open(STATE_PATH, "r", encoding="utf-8") as f:
165             # Chargement des données d'état depuis le fichier JSON
166             data = json.load(f)
167         # Log conditionnel seulement si données significatives présentes
168         if data.get('last_channel') is not None:
169             # Affichage du canal et température de la dernière mesure
170             print(f"[WEBUI] Données canal {data['last_channel']}: {data.get('last_temperature_c', 'N/A')}°C")
171         # Retour des données d'état avec code HTTP 200 (succès)

```

```

172         return jsonify(data), 200
173     except FileNotFoundError:
174         # Gestion spécifique du cas où le fichier d'état n'existe pas
175         print(f"[WEBUI] Fichier état non trouvé: {STATE_PATH}")
176         # Retour d'une structure d'état par défaut avec tous les champs
177         return jsonify({
178             "last_timestamp": None,
179             "last_channel": None,
180             "last_resistance_ohm": None,
181             "last_temperature_c": None,
182             "last_temperature_sim_c": None,
183             "last_error": "state file not found",
184             "channels": {}
185         }), 200
186     except Exception as e:
187         # Gestion de toute autre erreur de lecture d'état
188         print(f"[WEBUI] Erreur lecture état: {e}")
189         # Retour d'erreur avec code HTTP 500 (erreur serveur)
190         return jsonify({"error": str(e)}), 500
191
192
193 # -----
194 # API REST pour contrôle système
195 # -----
196
197 @app.route("/api/force-reload", methods=["POST"])
198 def force_reload():
199     """
200     @brief Force le rechargement de la configuration au prochain cycle.
201     @details Crée un fichier flag force_reload.flag avec horodatage
202             pour déclencher le rechargement de configuration par
203             le processus principal lors du prochain cycle de mesure.
204
205     @return Confirmation de demande ou message d'erreur avec code HTTP.
206     """
207     # Bloc de traitement avec gestion d'exceptions
208     try:
209         # Construction du chemin vers le fichier flag de rechargement
210         flag_path = os.path.join(BASE_DIR, "state", "force_reload.flag")
211         # Création du répertoire de destination si inexistant
212         os.makedirs(os.path.dirname(flag_path), exist_ok=True)
213         # Création du fichier flag avec horodatage ISO
214         with open(flag_path, "w") as f:
215             # Écriture de l'horodatage de la demande de rechargement
216             f.write(datetime.now().isoformat())
217
218         # Log de confirmation de demande de rechargement
219         print(f"[WEBUI] Rechargement config demandé")
220         # Retour de confirmation avec code HTTP 200 (succès)
221         return jsonify({"status": "reload requested"}), 200
222     except Exception as e:
223         # Log de l'erreur rencontrée lors de la création du flag
224         print(f"[WEBUI] Erreur demande rechargement: {e}")
225         # Retour d'erreur avec code HTTP 500 (erreur serveur)
226         return jsonify({"error": str(e)}), 500
227
228
229 # -----
230 # Point d'entrée principal du serveur
231 # -----
232
233 if __name__ == "__main__":
234     # Affichage des chemins de configuration pour débogage
235     print(f"BASE_DIR: {BASE_DIR}")
236     print(f"CONFIG_PATH: {CONFIG_PATH}")
237     print(f"STATE_PATH: {STATE_PATH}")
238     print(f"WEBUI_DIR: {WEBUI_DIR}")
239     # Démarrage du serveur Flask sur adresse IP fixe et port 8080
240     app.run(host="192.168.1.109", port=8080)

```

```
app\test_lm70_compatible.py
```

```
1  # -*- coding: utf-8 -*-
2  #!/usr/bin/env python3
3  """
4  Test LM70 avec support automatique pour simulateur et hardware réel.
5  Ce script fonctionnera aussi bien sur un PC que sur un Raspberry Pi.
6
7  Connexions matérielles spécifiques :
8  - LM70 MISO → GPIO 19 (SPI_MISO_ADC_GPIO)
9  - LM70 MOSI → GPIO 20 (SPI_MOSI_ADC_GPIO)
10 - LM70 SCLK → GPIO 21 (SPI_SCLK_ADC_GPIO)
11 - LM70 CS → GPIO 18 (GPIO libre)
12 """
13
14 import logging
15 import os
16 import sys
17 import time
18 import spidev
19
20 # Configuration du logging
21 logging.basicConfig(
22     level=logging.INFO,
23     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
24     handlers=[
25         logging.StreamHandler()
26     ]
27 )
28 logger = logging.getLogger("LM70-Test")
29
30 # Définition des GPIO SPI1
31 LM70_CS_GPIO = 18 # GPIO 18 utilisé comme CS
32 SPI_MOSI_ADC_GPIO = 20
33 SPI_MISO_ADC_GPIO = 19
34 SPI_SCLK_ADC_GPIO = 21
35
36 # Déterminer le chemin absolu du script
37 current_dir = os.path.dirname(os.path.abspath(__file__))
38 src_dir = os.path.dirname(current_dir) if "src" in current_dir else current_dir
39
40 # Importer le module de mock automatique
41 sys.path.insert(0, src_dir)
42 try:
43     from mock_imports import GPIO
44     logger.info("Module d'importation automatique chargé avec succès")
45 except ImportError:
46     logger.error("Impossible d'importer le module mock_imports.py")
47     logger.error("Assurez-vous que le fichier mock_imports.py est présent dans le répertoire src")
48     sys.exit(1)
49
50 # Configuration SPI pour le LM70
51 spi = spidev.SpiDev()
52 spi.open(1, 0) # Bus 1, Device 0 (SPI1)
53 spi.max_speed_hz = 100000 # 100 kHz
54 spi.mode = 0 # Mode 0: CPOL=0, CPHA=0
55 spi.bits_per_word = 8
56
57 def test_lm70():
58     """Test de communication SPI avec le capteur LM70"""
59     try:
60         # Configuration GPIO
61         GPIO.setmode(GPIO.BCM)
62         GPIO.setwarnings(False)
63
64         # Configuration du GPIO CS pour le LM70
65         GPIO.setup(LM70_CS_GPIO, GPIO.OUT, initial=GPIO.HIGH)
66
67         logger.info("Début du test LM70")
68
69         while True: # Boucle infinie pour tester en continu
70             # Active CS
71             GPIO.output(LM70_CS_GPIO, GPIO.LOW)
72
73             # Envoi de données arbitraires sur MOSI
74             resp = spi.xfer2([0x00, 0x00])
75
76             # Désactive CS
77             GPIO.output(LM70_CS_GPIO, GPIO.HIGH)
78
79             logger.info(f"Données SPI reçues : {resp}")
80
81             # Vérification des données reçues
82             if len(resp) == 2:
83                 # Combine les 2 octets en un entier 16 bits
84                 raw_temp = (resp[0] << 8) | resp[1]
```



```
85
86     # Ignore les 5 bits les moins significatifs
87     raw_temp = raw_temp >> 5
88
89     # Convertir les 11 bits en entier signé
90     if raw_temp & 0x0400: # Si le bit de signe est 1
91         raw_temp -= 0x0800 # Appliquer le complément à 2
92
93     # Calculer la température en °C
94     temp_celsius = raw_temp * 0.25
95
96     logger.info(f"Données brutes combinées : {raw_temp}")
97     logger.info(f"Température calculée : {temp_celsius:.2f}°C")
98 else:
99     logger.warning("Données SPI invalides - vérifiez les connexions")
100
101     # Délai pour éviter de saturer le bus SPI
102     time.sleep(0.5)
103
104 except KeyboardInterrupt:
105     logger.info("Test interrompu par l'utilisateur")
106 except Exception as e:
107     logger.error(f"Erreur lors du test LM70 : {e}")
108
109 finally:
110     GPIO.cleanup()
111     spi.close()
112
113 if __name__ == "__main__":
114     test_lm70()
115
```